

PiPER + LeRobot

完整教程

从硬件连接到模仿学习部署的端到端指南

PiPER 六轴协作机械臂 × HuggingFace LeRobot 框架

版本: 1.0

2025 年 6 月

目录

- 1 第一章 系统总览
- 2 第二章 硬件基础
- 3 第3章 PiPER SDK 深度解析
- 4 第 4 章 LeRobot 框架深度解析
- 5 第五章 PiPER 电机总线适配层: PiperMotorsBus 完全解析
- 6 第六章 Robot 与 Teleoperator 封装详解
- 7 第七章 PC 中继遥操作
- 8 第八章 数据录制: 从示教到数据集
- 9 第九章 模仿学习与 Action Chunking Transformer (ACT)
- A 附录 A: 脚本参考手册
- B 附录 B CAN ID 快速参考
- C 附录 C 故障排除指南

正文

第一章 系统总览

本章是教程的起点。你将了解 PiPER 双臂协作机器人 + LeRobot 模仿学习流水线的全貌——从"人是怎么教的"到"机器是怎么学的", 从最底层的 CAN 总线信号到最上层的神经网络策略, 建立一套完整的心智模型。

学完本章后, 你应该能够回答以下问题:

- 这套系统的输入是什么, 输出是什么?
- 硬件是怎么连在一起的 (拓线图、CAN 拓扑、相机布局)?
- 数据从机械臂编码器流到训练数据集的每一步发生了什么?
- 归一化到底是干什么的? 为什么需要它?
- 代码仓库里每个目录是做什么的?

如果某些术语不理解, 可以直接翻到 1.5 节查询术语表。如果想知道"这段代码到底在哪里", 可以翻到 1.6 节的代码仓库结构图。

1.1 项目是什么

1.1.1 名字的含义

本项目由两个核心组件组成:

组件	全称	在本项目中的角色
PiPER	松灵机器人 (AgileX) 出品的桌面级 6 轴协作机械臂	硬件平台: 执行动作、反馈状态
LeRobot	HuggingFace 开源的一套模仿学习框架	软件框架: 采集数据、训练策略、部署推理

两者结合在一起, 构成了一条完整的**模仿学习流水线** (Imitation Learning Pipeline)。

机器人学关键概念: 连接 PiPER 的设计选择

要理解 PiPER + LeRobot 流水线中的每一个设计决定, 我们需要先建立一套机器人学的基础词汇。下面这些概念不是枯燥的教科书定义——每一个都会直接映射到你在本项目代码中看到的某个参数、某个 CAN 命令、或者某个策略输出维度。

1. 自由度 (DOF) 与机械臂构型

什么是自由度

在机器人学中，一个自由度（Degree of Freedom, DOF）就是一个独立的运动方向。想象一扇门：它只能绕铰链轴旋转（打开或关上），所以它有 1 个自由度。想象你的手指：每个指节可以弯曲和伸展，不同关节可以独立运动，所以一只手有 20+ 个自由度。

对于机械臂而言，每一个可以独立控制的关节就是一个自由度。关节越多，机械臂能到达的姿态就越丰富——但同时也意味着控制更复杂，结构更昂贵。

为什么 PiPER 是 6 自由度机械臂

这不是随意选择的。一个刚体在三维空间中的位姿（pose）需要 **6 个独立参数** 来描述：

3 个位置参数 (translation): x, y, z — 描述"在哪"
3 个姿态参数 (rotation): roll, pitch, yaw — 描述"朝向"

因此，要让机械臂的末端执行器（夹爪）能够以任意姿态到达工作空间中的任意一点，**至少需要 6 个自由度**。少一个都不行——5 轴机械臂无法到达某些姿态（这是数学事实，不是工程限制）。

3-DOF arm:	只能到达空间中的一个点，姿态固定
4-DOF arm:	可以改变位置 + 一个旋转方向
5-DOF arm:	可以改变位置 + 两个旋转方向
6-DOF arm:	可以到达任意位置 + 任意姿态（最小完备）
7-DOF arm:	冗余自由度，可以做避障等优化

PiPER 选择 6 轴是"刚好够用"的设计哲学——它在功能完备性（能到达任意姿态）和成本/复杂度之间取得了平衡。多一个轴（变成 7 轴）会增加一个电机、一个减速器、一个编码器、一个 CAN 节点，成本显著增加。

冗余机械臂 vs 非冗余机械臂

7 自由度机械臂（如 KUKA LBR iiwa、Franka Emika Panda）被称为**冗余（redundant）机械臂**。冗余的意思是：对于同一个末端位姿，存在无穷多种关节配置可以达到它。这个"多余"的自由度非常有用——你可以在满足末端位置要求的同时，优化其他指标，比如：

- 避开障碍物（关节空间的避障规划）
- 避免关节极限（让关节远离 0° 或 180° 的死区）
- 优化力矩分布（让大关节承担更多负载，小关节保持灵活）

PiPER 作为 6 轴非冗余臂，对每个可达的末端位姿只有有限组解（通常是多组，但不是无穷组），这就丧失了上述优化空间。但对于桌面级操作任务（抓取、放置、推拉），6 轴完全够用。

PiPER 的关节布局

PiPER 的 6 个旋转关节的排列模仿了人类手臂的拓扑结构：

关节编号	功能类比	运动类型
Joint 1	腰部旋转	绕基座垂直轴旋转（整个臂左右摆动）
Joint 2	肩部俯仰	大臂前后摆动（决定末端的前后范围）
Joint 3	肘部俯仰	小臂前后摆动（决定末端的高低）
Joint 4	前臂旋转	小臂绕自身轴旋转
Joint 5	腕部俯仰	手腕上下摆动
Joint 6	腕部旋转	手腕绕自身轴旋转（决定夹爪的最终朝向）

关节 4、5、6 的轴线交于一点（腕部中心），这种设计使得腕部的姿态控制与手臂的位置控制可以部分解耦——这是工业机械臂的标准设计模式（称为"球腕"或 spherical wrist）。

为什么这对 LeRobot 很重要

PiPER 是 6 自由度 + 1 个夹爪开合 → **7 维动作空间**。这个数字不是随便设定的——它直接来自硬件的机械自由度。你在代码中看到的每一条数据：

```
# 动作向量：6 个关节角度 + 1 个夹爪开度
action = [j1_pos, j2_pos, j3_pos, j4_pos, j5_pos, j6_pos, gripper_pos]
```

对应的是 PiPER 硬件上 6 个伺服电机 + 1 个夹爪舵机的目标位置。策略网络（ACT）的输入和输出维度之所以是 7，根本原因在这里。

一个深层含义：**策略控制的是关节角度，而不是末端的笛卡尔坐标（xyz + 姿态）**。这是一个刻意的设计选择——我们将在下一节详细展开。

2. 关节空间 vs 笛卡尔空间控制

这是机器人控制中最基本的一个二分法，直接决定了策略的输入/输出形式和泛化能力。

两种控制模式



直接发给电机执行（或经过 PID 内环）

特点：不需要 IK，没有奇异性问题，但策略与具体机械臂绑定

笛卡尔空间 (Cartesian Space) 控制

策略输出: $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}, \text{gripper}]$

↓

逆运动学 (IK) 求解器 $\rightarrow [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]$

↓

发给电机执行

特点：策略与机械臂解耦，但 IK 引入了奇异性、多解性、计算开销

在笛卡尔空间控制的流程中，**逆运动学 (Inverse Kinematics, IK)** 是一个关键步骤。它的任务是把末端目标位姿（6 维）映射到关节角度（6 维）。听起来是解 6 个方程找 6 个未知数，但实际上这 6 个方程是高度非线性的三角函数方程组：

- **多解性**：同一个末端位姿通常对应多组关节角度（比如"肘在上"和"肘在下"两种配置都能到达同一个位置）
- **奇异性 (Singularity)**：在某些关节配置下，末端在某些方向上失去运动能力（Jacobian 矩阵降秩），此时 IK 无解或需要无穷大的关节速度
- **计算开销**：数值 IK 需要迭代求解（如 Newton-Raphson 法），不适合高频控制

为什么 LeRobot/PiPER 选择关节空间

这是本项目最重要的设计选择之一。以下是完整理由：

理由 1：不需要 IK 求解器。 直接从演示数据中学习关节角度到关节角度的映射，完全绕过了 IK 的奇异性 and 多解性问题。人类演示时移动主臂，主臂编码器读出的就是关节角度；策略训练和推理时，输出的也是关节角度——整个流水线中没有任何一个环节需要做运动学计算。

理由 2：神经网络隐式学习视觉→关节的映射。 视觉编码器 (ResNet-18) 从相机图像中提取特征，策略网络将这些视觉特征映射到关节角度。神经网络不需要显式地"知道"机械臂的正向运动学方程——它从数据中隐式地学到了"看到杯子在左边 $\rightarrow$ 关节 1 朝左转"这样的映射。这避免了显式建模桌面、物体、机械臂之间的几何关系。

理由 3：PiPER SDK 原生使用关节角度。 PiPER 机械臂的 CAN 通信协议中，位置数据以**毫度 (milli-degree)** 为单位传输，而不是以毫米为单位的笛卡尔坐标。PiPER SDK 的 `ModeCtrl` 接口

的 Joint 模式（0x01）直接接受关节目标角度。如果使用笛卡尔空间控制，需要在 PC 端或固件端引入 IK 求解——增加一层复杂度和潜在的故障点。

```
# PiPER SDK 的位置控制模式：原生的关节角度接口
arm.Set_Joint_Position_Control_Mode() # 设置各关节为位置控制模式
arm.MotionCtrl(
    tcp_pose=[0]*6,    # 在 Joint 模式下不使用
    joint=j1_j6_pos,   # 直接传关节角度（毫度）
    gripper=2000,      # 夹爪开度
    ctrl_mode=0x01,    # 0x01 = Joint 模式
)
```

理由 4：数据维度大幅降低。动作空间只有 7 维，而不是 $640 \times 480 \times 3 = 921,600$ 维的像素空间或 6 维的笛卡尔空间（加上 IK 的不确定性）。低维动作空间使得学习问题更容易——策略只需要拟合 7 个连续值，而不是在百万维空间中搜索。

这个选择的代价和应对

关节空间控制的最大代价是**策略与机械臂几何构型强耦合**。一个在 PiPER 上训练的 ACT 策略，无法直接部署到 KUKA 或 Franka 上——即使任务完全相同（比如都是"抓取杯子"），因为不同机械臂的关节编码方式、零点定义、运动范围都不同。

LeRobot 通过 **归一化（normalization）** 部分缓解了这个问题：

```
原始关节角度（毫度，PiPER 特有） → 归一化到 [-100, 100] → 策略输入
策略输出（归一化值） → 反归一化 → 目标关节角度（毫度，PiPER 特有）
```

归一化后的动作表示与具体硬件的数值范围解耦，但**不解决关节映射关系不同的根本问题**。完全解决这个问题需要走到 VLA 路线——用大规模、多机器人、多任务的数据来训练一个"硬件无关"的策略，这也是 LeRobot 选择插件式架构的动机之一。

3. 位置控制 vs 扭矩控制

两种控制模式的定义

在伺服电机的层面，有两种根本不同的控制方式：

位置控制 (Position Control)	
你告诉电机："转到 45.0°"	
电机的 PID 内环自己搞定：加速 → 接近目标 → 减速 → 停在目标	

接口：目标角度 → 电机（内部有位置-电流-电压三级 PID 级联）	
优点：简单、稳定、对负载变化不敏感	
缺点：无法控制接触力（一碰就停或一直推）	

扭矩控制 (Torque Control)	
你告诉电机："输出 1.5 N·m 的扭矩"	
电机电流环直接控制输出力矩	
接口：目标扭矩 → 电机	
优点：柔顺、可以精确控制接触力（装配、抛光等任务必需）	
缺点：需要精确的动力学模型，对负载变化敏感，安全性更难保证	

用一个比喻来理解： - **位置控制**像开车时告诉方向盘"转到 30 度"——不管路面阻力多大，方向盘会尽力转到那个角度。 - **扭矩控制**像告诉方向盘"输出 5 N·m 的力"——在光滑路面上转得多，在粗糙路面上转得少。

为什么 PiPER 使用位置控制

PiPER 的 `ModeCtrl` 中使用的 Joint 模式（0x01）是位控模式。这是模仿学习中非常自然的选择：

理由 1：人类教的是位置，策略学的也是位置。 在遥操作演示中，人类移动主臂，记录的是关节角度（位置）；策略训练时，监督信号是"预测的关节角度应接近人类演示的关节角度"；推理时，策略输出的也是关节角度。从演示到部署，信息始终在位置域流动，不需要在位置和力之间做域转换。

理由 2：更安全。 位置控制天然有"到位就停"的特性——电机到达目标角度后，PID 内环会主动减小电流以维持位置。扭矩控制则不同：如果策略错误地输出了一个过大的扭矩，机械臂可能会猛烈撞击桌面或障碍物。在神经网络策略还不够可靠的现阶段，位置控制是一个更安全的选择。

理由 3：与示教再现 (kinesthetic teaching) 兼容。 PiPER 双臂设计中的"主臂"不需要驱动——人类直接用手拖拽主臂完成演示。主臂的关节是被动的，编码器读取当前位置即可。如果使用扭矩控制，主臂需要具备反向驱动能力 (backdrivability)，这需要力矩传感器和更复杂的控制算法。

理由 4：不需要动力学模型。 位置控制的 PID 内环不依赖机械臂的动力学模型（质量矩阵、科里奥利力、摩擦力模型等）。扭矩控制通常需要前馈动力学补偿（计算力矩控制，Computed Torque Control），这需要精确辨识机械臂的动力学参数——对桌面级机械臂来说，工程成本高且辨识精度有限。

夹爪的部分例外

PiPER 的夹爪支持 **effort（力/扭矩）控制模式**——这是合理的，因为抓取任务天然需要力控：

硬抓取（位置控制）：夹爪合到固定宽度 → 软物体压扁或滑落
柔顺抓取（力控）：夹爪以恒定力闭合 → 自适应物体形状和硬度

夹爪的力控是通过电流限制实现的（舵机电流 $\approx$ 输出力矩），精度有限但对于桌面级抓取任务足够。

4. 遥操作范式

模仿学习的一个核心前提是**遥操作（Teleoperation）**——人类如何控制机器人来采集演示数据。不同的遥操作方案直接决定了硬件拓扑和软件架构。

三种遥操作方案

方案 A：双边力反馈遥操作（Bilateral Teleoperation）

人类操作主端 $\longleftrightarrow$ 力反馈通道 $\longleftrightarrow$ 从端机器人
|
|
└── 主端感受从端的接触力（"你能感觉到物体有多硬"）

这是最先进的方案，用于 da Vinci 手术机器人等场景。主端和执行端之间存在双向的力/位置信号流——操作者不仅能控制机器人，还能"感觉"到机器人正在接触的东西。缺点是硬件极其昂贵（需要高精度的力矩传感器和力反馈执行器）。

方案 B：单边遥操作（Unilateral Teleoperation）——PiPER 的方式

人类拖拽主臂 → 主臂编码器读取关节角度 → PC 中继 → 从臂复现角度
|
插入安全检查、数据记录

主臂关节是被动的（没有电机驱动），只负责感知。操作者直接用手移动主臂，编码器读取关节位置，PC 处理后将目标位置发给从臂的电机。信息流是单向的：主臂 → PC → 从臂。

方案 C：基于视觉的遥操作

相机拍摄人手 → 手部姿态估计 → 重定向到机械臂关节角 → 执行

不使用物理主臂，而是用相机跟踪人手。这是 Elon Musk 的 Optimus 机器人演示中使用的方案。优点是不需要额外硬件；缺点是需要解决人-机运动学重定向（retargeting）问题（人的手臂和机械臂的关节拓扑不同）。

为什么 PiPER 选择了方案 B（单边）

成本考量： 双边系统需要每个关节配备力矩传感器和力反馈电机，硬件成本是单边的 2-3 倍。PiPER 的定价定位是"桌面级"——采购两套双臂做一个模仿学习实验台的预算在几万元以内，而不是几十万元的工业级遥操作台。

总线拓扑更简单： CAN 总线本来就可以支持多节点双向通信。如果使用双边方案，CAN 总线上需要同时传输"主臂编码器读数 → 从臂"和"从臂力矩传感器 → 主臂力反馈"两路高频数据，总线负载和固件复杂度都会大幅增加。单边方案中，CAN 总线只需传输"从臂关节角度"一种实时数据。

PC 中继提供了灵活性： 因为数据经过了 PC（而不是主臂 CAN 直连从臂 CAN），可以在 PC 端插入各种逻辑：

主臂 CAN 帧 → PC 接收 → [安全检查] → [关节变换] → [数据记录] → PC 发送 → 从臂 CAN 帧

[未来：VLA 模型在线推理]

特别是 **数据记录** 这一环——PC 中继使得采集 (observation, action) 对变得非常简单。从臂的实际关节角度 (observation.state) 和主臂的目标角度 (action) 在 PC 上同时可用，直接写入 LeRobot 数据集即可。如果使用硬件直连方案，你需要额外在 CAN 总线上挂一个监听节点来抓取数据。

"教学"与"数据采集"的微妙区别

这里有一个重要的概念区分：

示教 (Teaching / Demonstration)	
- 目标：让人做一遍任务，机器人"看"	
- 采集的是：(observation, human_action) 对	
- 策略学习：observation → human_action	
- 部署时：策略替代人类	
遥操作数据采集 (Teleoperation for Data Collection)	
- 目标：人在各种场景下操作机器人，收集多样化数据	
- 采集的是：(observation, robot_action) 对	
- 策略学习：observation → robot_action	

- 部署时：策略复现学到的行为

在 PiPER 的双臂方案中，这两者统一了：主臂的关节角度（人类动作）直接映射为从臂的关节角度（机器人动作），所以 $\text{human_action} \approx \text{robot_action}$ 。这简化了数据标注——你不需要解决"人的动作"到"机器人的动作"之间的映射问题。

5. 仿真到现实 (Sim-to-Real) 的差距

什么是 Sim-to-Real Gap

Sim-to-Real gap 指的是在仿真环境中训练的策略部署到真实机器人上时性能下降的现象。它的根源在于仿真永远无法完美复制真实物理世界：

仿真环境 (Simulation)

- 完美的刚体碰撞
- 统一的摩擦系数
- 精确的光照模型
- 零延迟的传感器
- 确定的物理规律

↓

策略在仿真中表现完美

真实世界 (Real World)

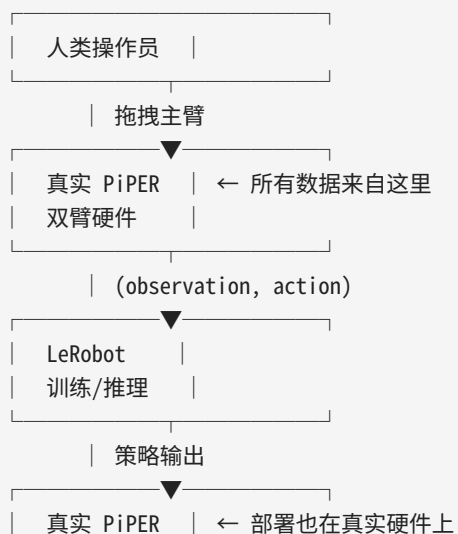
- 物体的弹性形变
- 表面微观纹理导致的不均匀摩擦
- 环境光变化、阴影、反射
- 相机延迟、运动模糊、噪声
- 各种未建模的物理效应

↓

部署到真实世界 → 策略失败

为什么 PiPER + LeRobot 选择了"全真机"路线

本项目的核心设计哲学是：不依赖仿真，直接从真实硬件上采集数据。



这个选择直接避免了 sim-to-real gap，但引入了另一个成本：**每个人类演示都需要真实的物理时间**。一个 episode 可能需要 30-120 秒，收集 50 个 episode 就需要 25-100 分钟的人工操作。这就是为什么本项目非常重视数据效率（如 ACT 的 action chunking 减少了对数据量的需求）和操作效率（如语音提示引导、快速重置 workflows 等自定义修改）。

这也解释了为什么"好的工作流"如此重要

如果你的每次数据采集都需要：1. 手动重置场景（把方块放回原位）2. 打开录制程序 3. 执行任务 4. 停止录制 5. 检查数据是否完整 6. 重复...

那么 50 个 episode 可能需要几个小时。本项目中的语音提示、自动命名、一键重置等自定义修改，本质上是在**降低 sim-to-real avoidance 策略的人工成本**——用工程自动化换取研究者的时间。

VLA (Vision-Language-Action) 深入

1. 从视觉策略到语言条件策略

基础形式：视觉模仿学习策略

一个传统的模仿学习策略（如 ACT）可以形式化为：

$$\pi: (\text{image}_t, \text{state}_t) \rightarrow \text{action}_t$$

输入是当前时刻的视觉观测（相机图像）和本体感受状态（关节角度），输出是下一步的动作。策略是一个从感知到动作的"反射弧"——它不"理解"任务，只是复现训练数据中的视觉-动作模式。

这意味着：**每个任务需要一个独立的策略**。"抓杯子"和"抓方块"需要两个不同的训练集，训练出两个不同的模型。如果需要执行 10 个任务，就需要 10 个模型（或一个多任务模型，但每个任务仍需要单独标注和训练）。

进阶形式：语言条件策略 (VLA)

$$\pi_{\text{vla}}: (\text{language_instruction}, \text{image}_t, \text{state}_t) \rightarrow \text{action}_t$$

关键变化：**引入自然语言作为条件信号**。语言描述了"要做什么"，策略在理解语言语义后生成对应的动作。

```
# 传统 IL 策略
action = policy(image) # 策略只知道"看到什么", 不知道"要做什么"

# VLA 策略
action = policy(language="拿起红色的杯子", image=current_view)
# 策略同时知道"目标"和"当前状态", 可以完成任务切换
```

这个变化看似简单, 但能力提升是质的飞跃:

同一个 VLA 模型, 不同语言指令 → 不同行为 (零样本任务切换)	
"拿起红色方块" → 抓取动作指向红色方块	
"把杯子推到左边" → 推动动作指向杯子, 方向向左	
"打开夹爪" → 纯粹的夹爪动作	
"放下物体" → 移动到放置位置后释放	

语言作为条件的本质

从信息论的角度看, 语言提供了一个"意图编码"——它将策略的输入空间从一个"无上下文的感知空间"扩展到了"感知 + 意图空间":

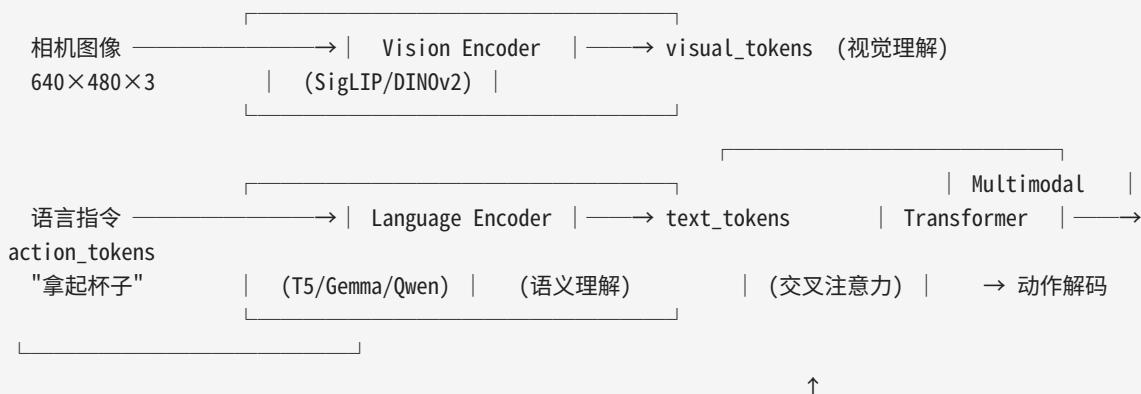
没有语言: 策略看到桌子上的杯子和方块, 不知道目标是什么
→ 可能输出一个"平均"动作 (接近杯子也接近方块, 但对两者都不精确)

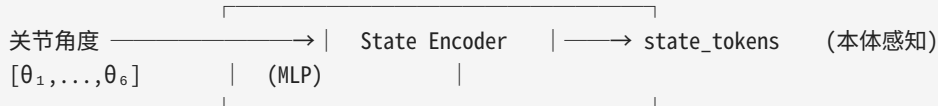
有语言: 策略知道"目标是杯子", 可以忽略方块,
所有视觉特征提取都围绕"找到杯子并规划抓取动作"

2. VLA 的典型架构

三塔结构

当前主流的 VLA 架构通常采用"三塔 + 融合"的设计模式:





各组件的作用： - **Vision Encoder**（视觉编码器）：将高维图像映射到紧凑的视觉 token 序列。常用 SigLIP（Google）或 DINOv2（Meta），参数量通常在 300M-1B 级别。 - **Language Encoder**（语言编码器）：将自然语言指令编码为语义向量或 token 序列。常用 T5（Google）或 Gemma（Google），也可以复用 VLM（视觉语言模型）的语言分支。 - **Multimodal Transformer**（多模态融合器）：通过自注意力和交叉注意力机制融合视觉、语言、状态三种模态的信息，输出动作 token。 - **Action Decoder**（动作解码器）：将融合后的 token 解码为具体的动作向量（关节角度 + 夹爪开度）。

与 ACT 的架构对比

ACT 架构（本项目当前使用）	VLA 架构（未来演进方向）
● Vision backbone: ResNet-18 (约 11M 参数) ● 无语言输入 ● Transformer encoder + decoder ● CNN 特征图 → patch embedding ● 输出: 30 步关节动作序列	● Vision backbone: SigLIP/DINOv2/... (300M-1B+ 参数) ● 语言编码器分支 ● 更大的多模态 Transformer ● ViT 风格的 patch embedding ● 输出: 可变长度的动作序列

核心差异在于规模和模态：ACT 是"小型化 + 纯视觉"方案，VLA 是"大规模 + 多模态"方案。

LeRobot 的策略接口：为 VLA 预留的空间

LeRobot 框架中最有远见的设计之一是 **统一的策略接口**：

```

# LeRobot 的策略抽象（简化表示）
class Policy:
    def predict(self, observation: dict) -> dict:
        """
        Args:
            observation: {
                "observation.state": Tensor,      # 关节角度
                "observation.images.wrist": Tensor, # 腕部图像
                "observation.images.global": Tensor, # 全局图像
                "observation.language": str,        # (可选) 语言指令 — 为 VLA 预留
            }
        Returns:
            action: {
                "action": Tensor, # 动作向量
            }
        """
  
```

这个接口的巧妙之处在于： - ACT、Diffusion Policy、Pi0 (VLA) 等不同策略都实现了同一个接口 - `observation dict` 的 key 可以按需扩展——加上 `observation.language` 不会破坏现有策略 - 从 ACT 迁移到 VLA 只需要替换 Policy 的实现类，数据采集和部署流水线完全不变

3. VLA 为什么需要大量数据

数据需求的量级差异

任务数量	单任务 IL (如 ACT)	VLA
1 个任务	50-200 episodes	需要预训练模型
5 个任务	5 × 50-200 episodes	500-2000 episodes
50 个任务	不可行	5000-20000+ episodes
开放词汇任务	不可行	100K+ episodes

VLA 需要海量数据的原因：

原因 1：多任务学习需要覆盖任务分布。 语言指令的空间是开放的（自然语言），策略必须学会将不同的语言表达映射到正确动作。这需要数据覆盖足够多的任务和语言变体（"拿起杯子"、"把杯子抓起来"、"grasp the cup"——都是同一个任务的不同表达）。

原因 2：视觉泛化需要多样性。 同一个"拿起红色方块"任务，在不同的桌面背景、光照条件、相机角度下看起来完全不同。策略需要在足够的视觉多样性中训练才能泛化。

原因 3：预训练模型的规模。 VLA 通常使用大参数量模型（1B+ 参数），大模型的容量需要大量数据来填充，否则会过拟合。

这解释了 LeRobot 为什么强调数据集共享

LeRobot 选择将数据集托管在 HuggingFace Hub 上，使用标准化的 LeRobot 数据集格式（`datasets.Dataset` + 特定的 feature 结构），不是偶然的。这直接回应了 VLA 时代的数据需求：

愿景：100 个 PiPER 用户 → 每人贡献 100 episodes → 10,000 episodes 共享数据 → 在共享数据上训练一个通用 VLA → 每个用户都可以用自己的语言指令驱动
--

你的 PC 中继 + 录制工作流使单个 episode 的采集时间降到 30-120 秒，大大降低了数据贡献的门槛。

World Model 深入

1. 什么是世界模型：形式化定义

数学定义

一个世界模型（World Model）是一个**状态转移的预测器**：

$$f: (s_t, a_t) \rightarrow s_{t+1}$$

其中： - s_t 是 t 时刻的状态（可以是关节角度 + 图像，或一个学习的隐状态） - a_t 是 t 时刻执行的动作 - s_{t+1} 是模型预测的 $t+1$ 时刻的状态

在视觉机器人学习的情境下：

$$f: (\text{image}_t, \text{state}_t, \text{action}_t) \rightarrow (\text{image}_{t+1_pred}, \text{state}_{t+1_pred})$$

训练方式

世界模型的训练数据与行为克隆完全相同—— (o_t, a_t, o_{t+1}) 三元组：

```
从数据集中采样：
o_t      = (image_t, joint_angles_t)      # 当前观测
a_t      = (joint_targets_t)              # 执行的动作
o_{t+1} = (image_{t+1}, joint_angles_{t+1})# 下一帧观测（真实值）

训练世界模型：
o_{t+1}_pred = world_model(o_t, a_t)
loss = MSE(o_{t+1}_pred, o_{t+1}) # 使预测逼近真实
```

注意：**LeRobot** 的数据集格式天然支持世界模型训练。每个 episode 保存了完整的 (observation, action, next_observation) 序列，不需要额外的数据标注或格式转换。

关键区分：预测未来 vs 分类现在

```
分类模型（如 ImageNet）： image → "这是猫"          （理解现在）
检测模型（如 YOLO）：    image → (bbox, "杯子")      （定位现在）
世界模型：                (image, action) → next_image （预测未来）
```

世界模型的难点正在于"预测未来"——像素级别的未来预测极其困难。一个微小的预测误差在下一帧会被放大，导致预测质量迅速退化（这就是著名的"compounding prediction error"问题）。

2. 世界模型 vs 端到端策略

两种范式



为什么端到端目前更实用

可视化预测的挑战: 训练一个能准确预测下一帧 RGB 图像 ($640 \times 480 \times 3$) 的世界模型极其困难。即使是 Sora 级别的视频生成模型, 在长时间预测中也会出现物体变形、物理不一致等问题。对于一个桌面级机器人来说, 世界模型的质量要求非常高——如果预测的画面中杯子位置偏了 5 个像素, 抓取就会失败。

像素空间 vs 隐空间: 现代的模型基于方法 (如 Dreamer、TD-MPC) 不在像素空间而是**隐空间** (latent space) 中预测。隐空间维度远小于像素空间 (如 1024 维 vs 921,600 维), 预测更稳

定。但这引入了另一个问题：隐空间中的预测误差如何影响最终的决策？这是当前研究的前沿问题。

ACT 的折中方案：隐式世界模型。 ACT 的 action chunking——一次性预测未来 30 步的动作——可以被理解成一种**学习到的隐式世界模型**：

```
| ACT 不显式地预测"下一帧图像长什么样" |
| 但它隐式地编码了"从现在到未来 30 步，机械臂应该怎么动" |
| |
| 这 30 步动作序列本身就蕴含了关于环境变化的"理解": |
| - 如果杯子在前方，动作序列会包含接近→抓取→抬起 |
| - 动作序列的时间结构编码了任务的物理时序约束 |
| |
| 这是一种"廉价版世界模型"——不需要预测像素，只预测动作 |
```

3. 世界模型如何解释 LeRobot 的设计选择

数据集的远见：保存 (o_t, a_t, o_{t+1}) 完整转移

LeRobot 的 episode 格式：

```
episode = {
  "observation.state": [s_0, s_1, s_2, ..., s_T], # 关节角度序列
  "observation.images.wrist": [img_w_0, img_w_1, ...], # 腕部图像序列
  "observation.images.global": [img_g_0, img_g_1, ...], # 全局图像序列
  "action": [a_0, a_1, a_2, ..., a_T], # 动作序列
}
```

注意这个格式中，**每一帧都同时保留了 observation 和 action**。这意味着： - 对于相邻的帧 i 和 $i+1$ ，你可以提取出 (o_i, a_i, o_{i+1}) 三元组 - 这正是训练世界模型所需的完整转移数据

如果 LeRobot 只保存 actions ("因为行为克隆只用 action")，你今天就无法用这些数据训练世界模型。这个设计选择反映了数据集的"存档价值"——今天用行为克隆，明天可能用世界模型，数据本身的格式应该保持完整和通用。

为什么多相机很重要：3D 理解的隐式途径

PiPER 的双相机配置（腕部 + 全局）在当前 ACT 训练中看起来只是"两张图 vs 一张图"的区别。但从世界模型的角度看，双相机提供了**隐式的 3D 信息**：

```
单相机世界模型的困境：
2D 图像 → 预测 2D 未来图像
```

问题：深度方向的变化（物体靠近或远离相机）在 2D 图像中只表现为微弱的尺度变化。世界模型很难准确预测。

多相机世界模型的优势：

(view_1, view_2) → 预测 (future_view_1, future_view_2)

两个视角提供了视差信息，世界模型可以隐式地学到物体的 3D 位置和运动，不需要显式的深度传感器。

这就是为什么 LeRobot 在设计时就将多相机支持作为一等公民——不是"先支持单相机，以后再扩展"，而是从一开始就在 observation dict 中用 `observation.images.*` 的命名空间支持任意数量的相机。

从 ACT 到世界模型的演进路径

```
graph LR
    A[今天: ACT 行为克隆] --> B[明天: ACT + 世界模型辅助训练]
    B --> C[后天: 纯世界模型规划]

    A --> |"端到端，简单可靠  
30-step action chunking  
数据: 50-200 episodes"| A
    B --> |"世界模型生成增强数据  
在世界模型中做数据增广  
数据: 原始 episodes + 合成 transitions"| B
    C --> |"在隐空间中规划  
Model Predictive Control  
数据: 需要更多、更多样化的真实数据"| C
```

LeRobot 的统一接口设计使得这个演进路径成为可能：你可以保持数据采集和部署流水线不变，只是把 Policy 从 ACT 换成基于世界模型的规划器。

设计选择的完整总结

下面的表格将本教程中讨论的每一个概念映射到 PiPER + LeRobot 中的具体体现，帮助你建立"理论 → 实现"的对应关系：

概念	PiPER / LeRobot 中的具体体现	代码/配置位置
6-DOF 机械臂	7 维动作空间 (6 关节 + 1 夹爪)。策略输入输出的维度直接来自硬件自由度。	<code>ACTION_KEYS = ["joint1.pos", ..., "gripper.pos"]</code>
关节空间控制	策略直接输出关节角度，不做 IK。PiPER SDK 的 <code>ModeCtrl</code> 以毫度为单位的 Joint 模式 (0x01)。	<code>arm.MotionCtrl(ctrl_mode=0x01)</code>

概念	PiPER / LeRobot 中的具体体现	代码/配置位置
位置控制	通过 CAN 总线向每个关节发送目标位置（毫度），电机的内嵌 PID 自行完成位置跟踪。	CAN 帧: 关节 ID + 目标角度
单边遥操作	主臂被动（无电机）+ PC 中继（安全/记录/变换）+ 从臂执行。信息流为主臂→PC→从臂。	PC relay 程序
模仿学习	Record → Train → Eval 三段式流水线。人类演示作为监督信号。	lerobot/scripts/ 下的 CLI 工具
行为克隆 (BC)	ACT 使用 L2 loss 最小化预测动作与人类演示动作之间的均方误差。	<code>mse_loss = F.mse_loss(pred_actions, gt_actions)</code>
Action Chunking	ACT 的 Transformer decoder 一次预测未来 30 步的关节目标值，缓解累积误差。	<code>chunk_size = 30</code>
Sim-to-Real	所有数据在真实 PiPER 硬件上采集，不使用仿真环境，彻底规避 sim-to-real gap。	数据采集都在物理硬件上进行
VLA	LeRobot 的 <code>Policy.predict(observation)</code> → <code>action</code> 统一接口， <code>observation</code> dict 预留 <code>language</code> 字段。	Policy 抽象基类
World Model	数据集格式保留完整 (<code>o_t, a_t, o_{t+1}</code>) 转移三元组，支持未来训练世界模型。	<code>episode["observation.*"]</code> + <code>episode["action"]</code>
归一化	原始毫度值 → [-100, 100] 归一化空间。策略与具体硬件数值范围解耦。	Normalization statistics 计算
安全检查	<code>max_relative_target</code> 限幅单步最大位移， <code>max_step_deg</code> 限制每步角度增量。策略输出不可信任，必须加硬约束。	SafeRollout 配置参数
CAN 总线	6 轴同步控制、高频状态反馈（30-100Hz）。工业级实时性保证动作序列的时间一致性。	PiPER SDK 底层 CAN 通信
双相机	腕部相机提供近距离操作视角，全局相机提供场景上下文。多视图为未来的 3D 世界模型提供视差信息。	<code>observation.images.wrist</code> , <code>observation.images.global</code>
PC 中继	解耦主臂和从臂，允许软件层插入安全逻辑、数据记录、关节变换、未来 VLA 在线推理。	中继程序代码

1.1.3 模仿学习（本项目的核心范式）

这里用一个不严谨但直观的比喻：

这里用一个不严谨但直观的比喻：

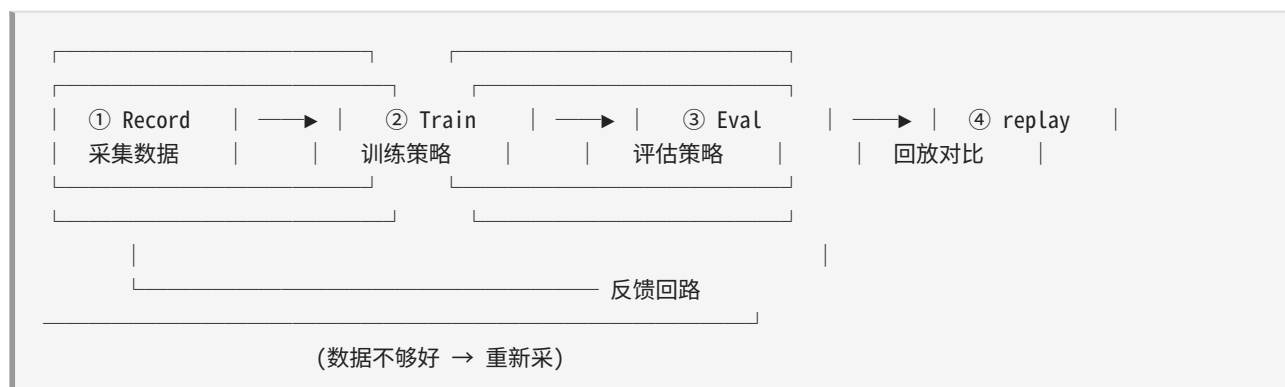
你是一位陶艺老师。你手把手地握着学徒的手，做出一个花瓶。这个过程就是**演示**（demonstration）。学徒不是“理解”了怎么做花瓶，而是**记住了**你做了哪些动作、在什么姿态下做了这些动作。下一次你给他一块陶土，他就能凭记忆复现出差不多的花瓶。这个过程就是**行为克隆**（Behavior Cloning），是模仿学习最经典的一类方法。

把这个比喻翻译成技术语言：

- **老师** = 人（操作者）
- **学徒** = 神经网络策略（Policy）
- **手把手教** = 人手动拖拽主臂（leader arm），同时记录下每一个时刻的关节角度和相机图像
- **记住动作** = 从记录的数据中训练一个神经网络，让它学会“给定观测 → 输出动作”
- **自己做花瓶** = 训练完成后，策略模型根据相机图像和当前关节状态，自主输出动作序列，驱动从臂（follower arm）复现任务

1.1.4 核心 workflow

整个项目围绕着以下 4 个步骤循环进行：



① Record（数据采集）

- **人做什么**：用手拖动主臂（leader arm），像牵木偶一样完成一次任务（比如“抓起一个方块，放到碗里”）。
- **电脑做什么**：同时记录以下信息，帧率约 30 FPS：
 - 从臂（follower arm）的当前关节角度（6 个关节 + 1 个夹爪）
 - 腕部相机的 RGB 图像（640x480）
 - 全局相机的 RGB 图像（640x480）
 - 来自主臂的动作命令（这些动作正在被 PC 中继转发给从臂执行）
- **产出**：一个 **episode**（回合），也就是一次完整任务尝试的所有帧数据。

命令行入口：

```
lerobot-record \  
  --robot.type=piper_follower \  
  --robot.port=can1 \  
  --robot.cameras="{wrist: {type: realsense, serial: 238222076529, width: 640, height: 480, fps: 30}, \  
                    global: {type: realsense, serial: 142122071524, width: 640, height: 480, fps: 30}}" \  
 \  
  --dataset.repo_id=my_user/my_task \  
  --dataset.num_episodes=10 \  
  --dataset.single_task="Pick the cube and place it in the bowl"
```

注意：在 `record` 模式中，遥操作是由 `piper_pc_relay_teleop.py` 在另一个终端中独立运行的。LeRobot 的 `record` 脚本本身不负责将主臂动作转发给从臂——它只负责采集数据。这就是 PC 中继方案的架构特点（见 1.2.2 节）。

② Train（训练策略）

- **输入：**采集到的所有 episode 数据（存储为 LeRobotDataset 格式，见 1.4 节）。
- **过程：**用模仿学习算法（本项目默认使用 **ACT**，Action Chunking Transformer）训练一个神经网络，输入是"观测（observation）"，输出是"动作（action）"。
- **输出：**一个 PyTorch checkpoint 文件（包含了神经网络的权重）。

命令行入口：

```
lerobot-train \  
  --dataset.repo_id=my_user/my_task \  
  --policy.type=act
```

③ Eval（评估策略）

- **过程：**加载训练好的策略模型，让它自主控制从臂完成任务。人不需要再碰主臂——模型直接输出动作。
- **评估标准：**成功率（任务完成次数 / 总尝试次数）。

命令行入口：

```
lerobot-eval \  
  --robot.type=piper_follower \  
  --robot.port=can1 \  
  --robot.cameras="{wrist: ..., global: ...}" \  
  --policy.path=my_user/my_policy
```

④ Replay (回放对比)

- **过程**: 把之前记录的某个 episode 的原始动作数据, 逐帧"播放"给从臂执行。用于对比"人演示的效果"和"模型复现的效果"。

命令行入口:

```
lerobot-replay \  
  --robot.type=piper_follower \  
  --robot.port=can1 \  
  --dataset.repo_id=my_user/my_task \  
  --dataset.episode=0
```

1.1.4 你在整个流程中扮演的角色

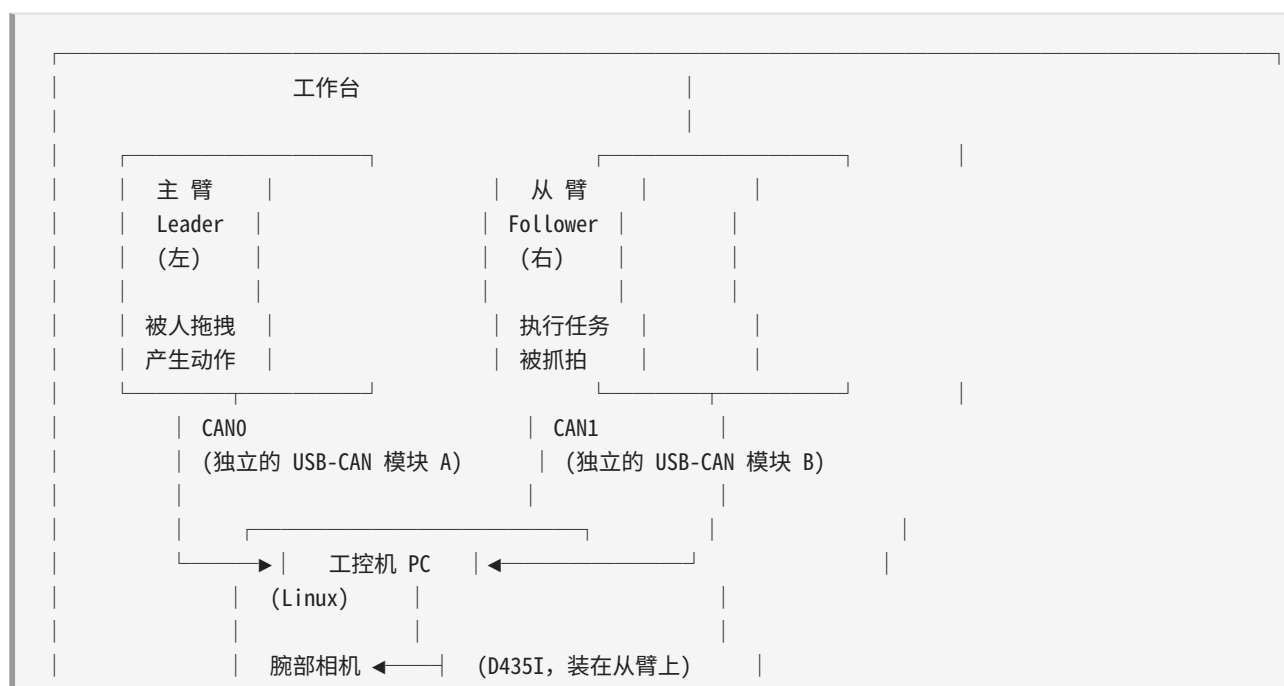
作为"AI 辅助系统搭建者"——也就是你——的工作不是手动编写控制程序, 而是:

1. **把硬件正确连好、软件正确装好** (这是各种 shell 脚本和 ROS 工具在帮你做的事情)
2. **高质量地演示任务** (用主臂完成足够多次的示范)
3. **让 LeRobot 框架替你完成训练和部署的工程细节**

1.2 硬件拓扑

1.2.1 "双臂"到底是什么

本项目使用的是**两台 PiPER 六轴机械臂**, 分别扮演两个不同的角色:





关键区别：

属性	主臂 (Leader)	从臂 (Follower)
英文名	Leader / Teleoperator	Follower / Robot
CAN 接口	can0	can1
是否装相机	否	是（腕部相机装在从臂上）
人的操作	用手拖动	不碰
主要作用	生成动作命令	执行动作，被相机观测
LeRobot 类	PiperLeader	PiperFollower

1.2.2 PC 中继方案：为什么用两台独立 CAN 总线

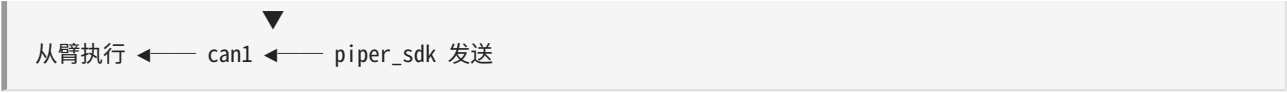
PIPER SDK 原生支持**硬件主从模式**（Hardware Master-Slave）：将两台臂的 CAN 线并联到同一根总线上，一台配置为 Master，另一台为 Slave。Master 发出的命令通过 CAN 总线直接控制 Slave。

但本项目采用了**PC 中继方案**（PC Relay），原因如下：

- 数据记录需求：** LeRobot 框架需要同时读取从臂的编码器反馈值作为 `observation`，也需要读取主臂的动作作为 `action`。如果两只臂在同一根 CAN 总线上，带宽会被平分，且帧 ID 管理变复杂。
- 安全控制需求：** PC 可以在转发过程中做安全限幅（rate limiting）、符号变换（sign mapping）、偏移校准（offset calibration）——这些都是纯硬件主从中做不到或不好做的。
- 调试便利性：** 可以先用 `--execute` 不传的方式做 dry-run（打印主臂读数但不实际驱动从臂），确保数据流正确再实际执行。

PC 中继的数据流：





对应的代码是 `scripts/piper_pc_relay_teleop.py`（311 行 Python 脚本），后面 1.4 节会详细追溯它的每一步。

1.2.3 双 RealSense 相机配置

属性	腕部相机	全局相机
型号	Intel RealSense D435I	Intel RealSense D435
安装位置	固定在从臂腕部（随臂运动）	固定在三脚架上（俯瞰工作区）
序列号	238222076529	142122071524
分辨率	640 × 480	640 × 480
帧率	30 FPS	30 FPS
用途	提供"手眼协调"视角	提供"上帝视角"的全局场景理解
D435I 的 IMU	可用但当前未启用	N/A（D435 没有 IMU）

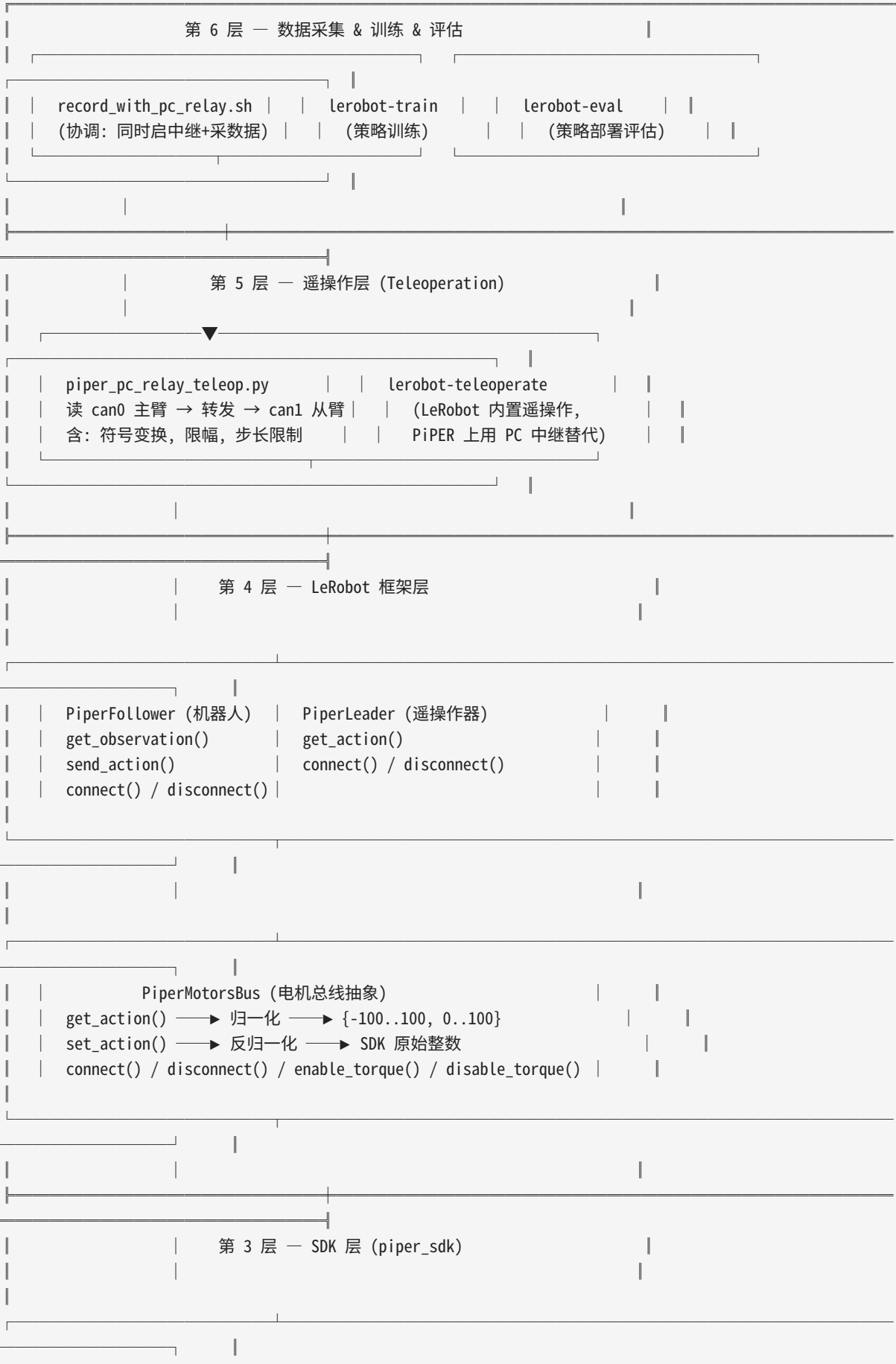
两台相机通过 USB 3.0 接口连接到工控机。LeRobot 框架通过 `realsense` 相机后端（位于 `src/lerobot/cameras/realsense/`）与之通信。

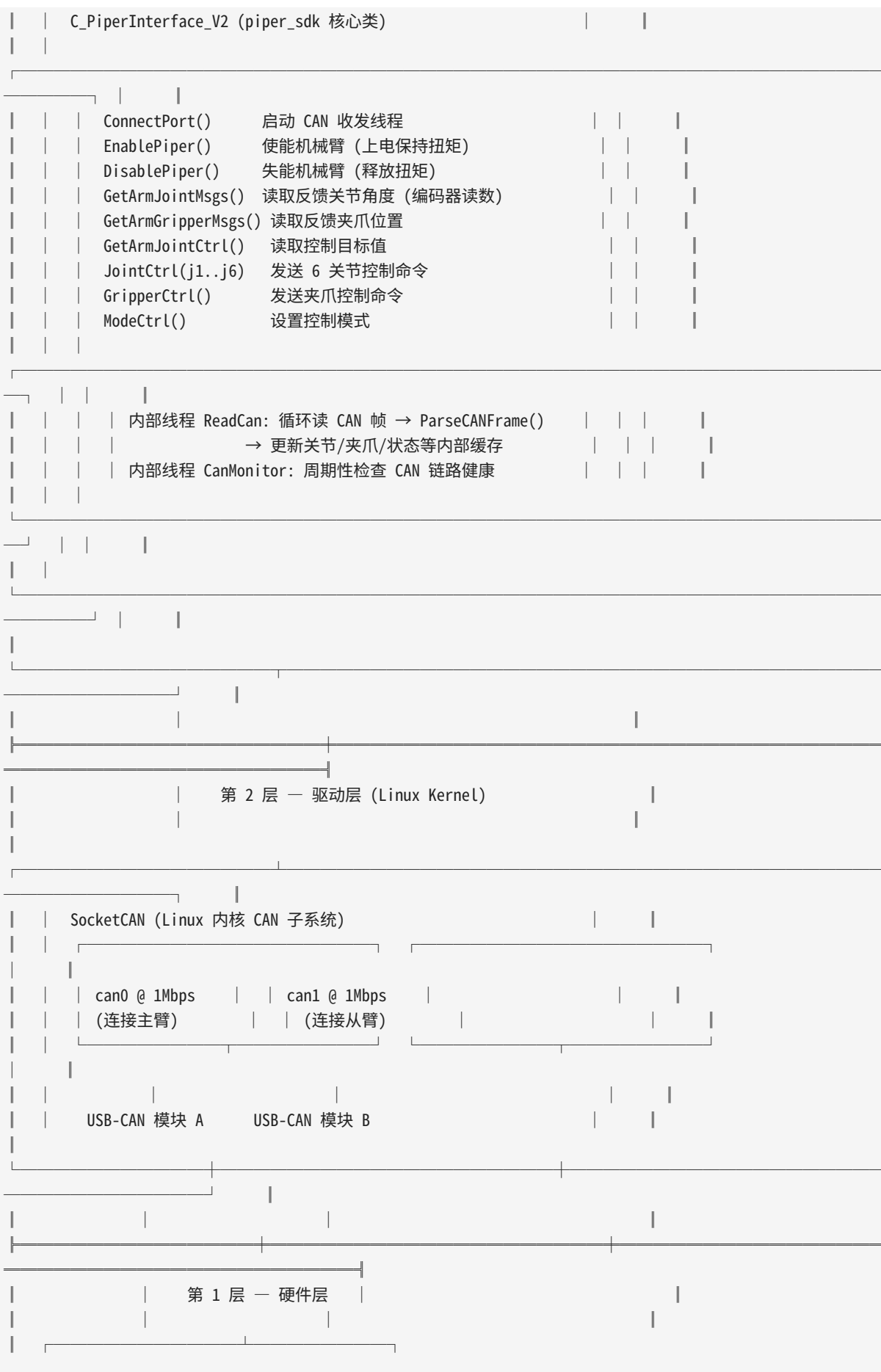
1.2.4 工控机规格

- **操作系统**: Linux (Ubuntu)
- **CAN 接口**: 两路 USB-CAN 模块（USB 转 CAN FD），分别对应 `can0` 和 `can1`
- **CAN 协议**: SocketCAN（Linux 内核原生 CAN 子系统），`can0` 和 `can1` 是标准的 SocketCAN 网络接口名
- **Python 版本**: 3.10+
- **CUDA**: 如果有 NVIDIA GPU，用于加速策略训练

1.3 软件栈全貌

下图展示了从硬件到应用层的完整软件栈。越往下越接近硬件，越往上越接近"业务逻辑"。请花 5 分钟仔细看这张图——后面每一节都在解释它的某一个层面。





PiPER 主臂 (Leader)	PiPER 从臂 (Follower)
6 关节 + 1 夹爪	6 关节 + 1 夹爪
CAN ID: 0x2A1~0x2A7	CAN ID: 0x2A1~0x2A7
电机型号: HTDW-5047	电机型号: AGILEX-M/S

各层职责简述

第 1 层 — 硬件层：物理机械臂本体。包含伺服电机、编码器、CAN 控制器、电源板等。每个关节电机通过 CAN 总线报告当前编码器位置，接收目标位置指令。不需要写代码——上电就行。

第 2 层 — 驱动层 (SocketCAN)：Linux 内核原生的 CAN 总线子系统。当你把 USB-CAN 模块插入电脑后，Linux 会创建一个名为 `can0`（或 `can1`）的网络接口。应用程序通过标准的 Socket API（`socket()`，`bind()`，`send()`，`recv()`）读写 CAN 帧。你可以用 `ifconfig can0` 或 `ip link show can0` 查看它的状态。

第 3 层 — SDK 层 (piper_sdk)：松灵机器人官方提供的 Python SDK，核心类是 `C_PiperInterface_V2`。它的主要工作是：- 封装 SocketCAN 的收发细节（你不需要直接操作 socket）- 解析 CAN 原始字节流 → 结构化的 Python 对象（`ArmMsgFeedbackJointStates` 等）- 启动后台 `ReadCan` 线程，以约 200Hz 的速率持续从 CAN 总线拉取机械臂反馈 - 提供高层 API：`GetArmJointMsgs()`，`JointCtrl()`，`EnablePiper()` 等

第 4 层 — LeRobot 框架层：这是你日常打交道最多的层。它包含三个核心类：- **PiperMotorsBus**：把 `piper_sdk` 的不统一接口包装成 LeRobot 的标准接口。最核心的工作是**归一化和反归一化**——把 SDK 返回的原始整数（如 50000）映射到模型友好的范围（如 `[-100, 100]`）。- **PiperFollower**：代表"从臂"这一侧的完整抽象，包含电机总线 + 相机。提供 `get_observation()`（返回关节角度 + 图像）和 `send_action()`（发送动作命令）。- **PiperLeader**：代表"主臂"这一侧的抽象，只包含电机总线（没有相机）。提供 `get_action()`（读取主臂当前位置）。

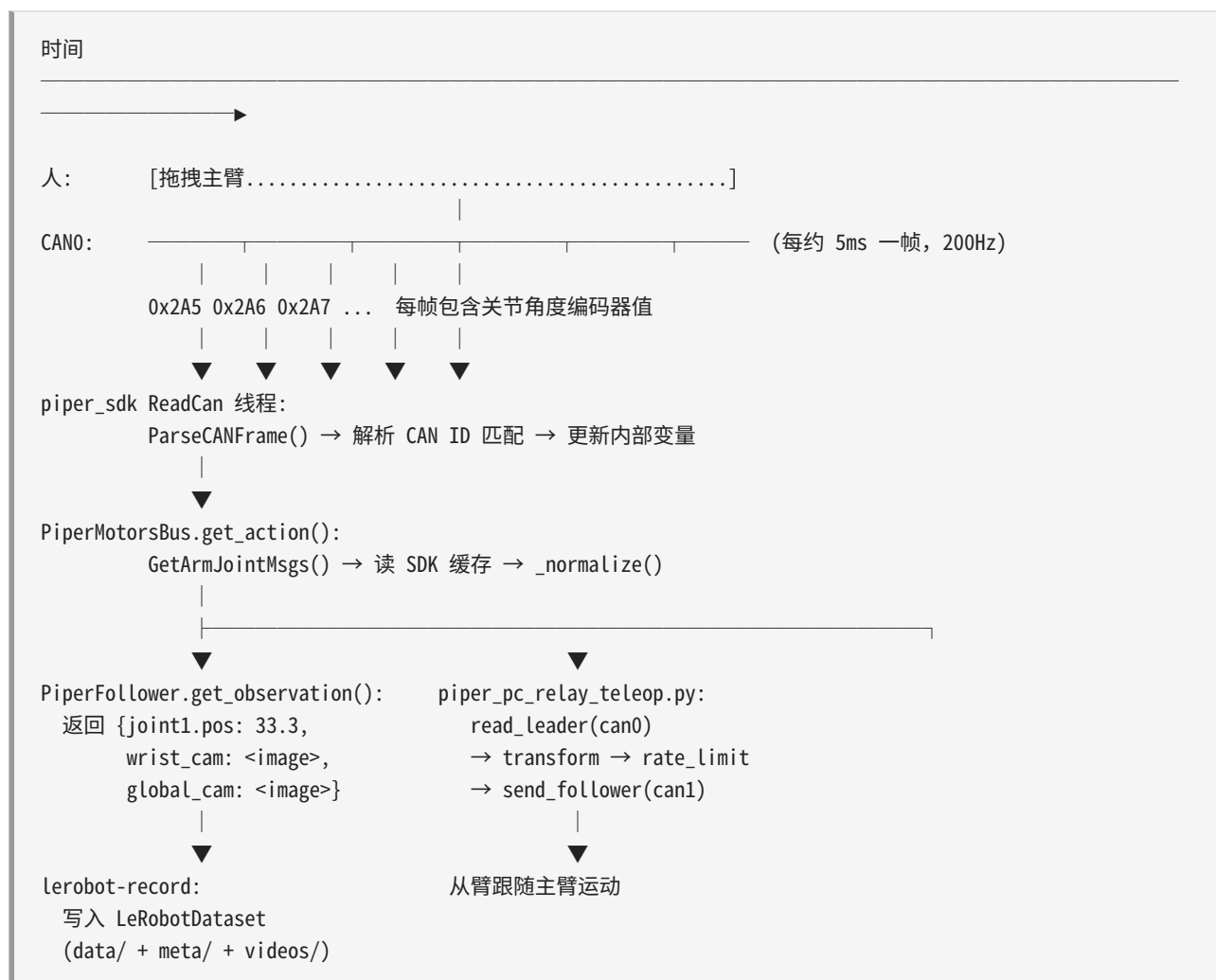
第 5 层 — 遥操作层：在 LeRobot 标准架构中，遥操作（Teleoperation）是指"人操作主臂 → 框架自动转发给从臂"。但在 PiPER 上，由于采用 PC 中继方案，这个转发不是通过 LeRobot 的 `lerobot-teleoperate` 完成的，而是通过独立的 `scripts/piper_pc_relay_teleop.py` 脚本。它直接调用 `piper_sdk` 的 API（不经过 `PiperMotorsBus`），因为需要更底层的控制（限幅、符号变换、模式设置等）。

第 6 层 — 数据采集 & 训练 & 评估：用户直接使用的命令行工具层。- `lerobot-record`：记录数据（需要同时运行遥操作脚本）- `lerobot-train`：训练策略（纯软件，不需要连机械臂）- `lerobot-eval`：用训练好的策略实时控制机械臂 - `lerobot-replay`：回放之前录制的数据来"重复历史"

1.4 数据流向：从动作到数据集

这是本章最需要认真理解的一节——你不仅要知道"发生了什么事"，还要知道"每件事在哪一行代码发生"。

1.4.1 整体时序图



1.4.2 逐步追踪

步骤 1: 人移动主臂 → 电机编码器读数

主臂（leader）处于**示教模式**（teaching mode）。当你用手抓住主臂末端并推动它时：

- 主臂的伺服电机不主动抵抗（电机扭矩使能但设置为柔性模式）
- 电机内部的**编码器**（encoder）持续测量转子的实际角度

- 编码器值以毫度（millidegree, 1/1000 度）为单位，例如 joint1 旋转了 45 度 → 编码器值为 45000

步骤 2：CAN 总线帧上报

PiPER 机械臂通过 CAN 总线以约 **200Hz** 的频率主动上报关节状态。关键 CAN 帧 ID：

CAN ID	内容	频率
0x2A5	关节 1、关节 2 反馈（高字节）	~200Hz
0x2A6	关节 3、关节 4 反馈	~200Hz
0x2A7	关节 5、关节 6 反馈	~200Hz
...	夹爪反馈、状态信息、末端位姿等	不同频率

每帧是一个标准的 CAN 2.0 数据帧：8 字节 payload 中编码了两个关节的 16 位有符号整数值。

步骤 3：piper_sdk ReadCan 线程缓存

这段逻辑位于 `piper_sdk/piper_sdk/interface/piper_interface_v2.py` 的 `ConnectPort()` 方法中。

当 `C_PiperInterface_V2("can0").ConnectPort()` 被调用时，SDK 内部启动了**两个后台守护线程**：

1. **ReadCan 线程**（`__can_deal_th`）：以 ~200Hz 的速率循环调用 `self.__arm_can.ReadCanMessage()` 从 SocketCAN 接口读取 CAN 帧。每读到一帧，调用 `ParseCANFrame()` 进行解析和缓存。
2. **CanMonitor 线程**（`__can_monitor_th`）：以较低频率（20Hz）检查 CAN 链路健康状态。

`ParseCANFrame()` 的解析逻辑（简化版）：

```
def ParseCANFrame(self, rx_message: can.Message):
    msg = PiperMessage()
    if self.__parser.DecodeMessage(rx_message, msg):
        # 根据 CAN ID 把消息内容写入对应的内部变量
        self.__UpdateArmJointState(msg)      # → self.__joint_msgs
        self.__UpdateArmGripperState(msg)    # → self.__gripper_msgs
        self.__UpdateArmJointCtrl(msg)       # → self.__joint_ctrl_msgs
        # ... 更多更新 ...
```

之后，当你调用 `GetArmJointMsgs()` 时，它直接返回 `self.__joint_msgs` 的最新值——不需要等待下一帧到达。

步骤 4：PiperMotorsBus.get_action() 读 SDK + 归一化

这段逻辑位于 `src/lerobot/motors/piper/piper.py` 的 `get_action()` 方法。

```
def get_action(self) -> dict[str, Any]:
    # ① 从 SDK 读取原始反馈值 (int 类型)
    msg_joint = self.piper.GetArmJointMsgs()      # 关节编码器
    msg_gripr = self.piper.GetArmGripperMsgs()    # 夹爪编码器

    # ② 从消息对象提取各字段
    rlt = {
        "joint1": float(msg_joint.joint_state.joint_1), # 如: -50000
        "joint2": float(msg_joint.joint_state.joint_2), # 如: 90000
        "joint3": float(msg_joint.joint_state.joint_3),
        "joint4": float(msg_joint.joint_state.joint_4),
        "joint5": float(msg_joint.joint_state.joint_5),
        "joint6": float(msg_joint.joint_state.joint_6),
        "gripper": float(msg_gripr.gripper_state.grippers_angle),
    }

    # ③ 归一化: 原始整数 → [-100, 100] / [0, 100]
    rlt = self._normalize(rlt)
    return rlt
```

归一化的数学公式 (详见 1.5 节"归一化"词条) :

- 关节 (MotorNormMode.RANGE_M100_100): $\text{norm} = ((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 200 - 100$
- 夹爪 (MotorNormMode.RANGE_0_100): $\text{norm} = ((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 100$

举例: joint1 的 $\text{min}=-150000$, $\text{max}=150000$, 如果 $\text{raw}=75000$, 则 $\text{norm} = ((75000 - (-150000)) / (150000 - (-150000))) * 200 - 100 = (225000/300000)*200 - 100 = 150 - 100 = 50.0$ 。

步骤 5: lerobot-record 记录 observation + action → LeRobotDataset

这段逻辑位于 `src/lerobot/record.py` (LeRobot 框架的通用数据记录脚本)。

每帧 (约 30fps), `record.py` 执行以下循环:

```
while True:
    observation = robot.get_observation() # → PiperFollower
    # observation = {"joint1.pos": 33.3, ..., "wrist": <img>, "global": <img>}

    action = leader.get_action() # → PiperLeader (当使用 LeRobot 内置遥操作时)
    # 但在 PC 中继方案中, action 也是从 follower.get_observation() 中获取的

    dataset.add_frame(observation, action) # 写入 LeRobotDataset
```

LeRobotDataset 是 HuggingFace `datasets` 库的封装。最终产出是以下目录结构:

```
datasets/local/my_user/my_task/
├── data/
│   └── chunk-000/
```



```

|   |   | episode_000000.parquet # 第 0 个 episode 的关节/夹爪/动作数据
|   |   | episode_000001.parquet
|   |   | ...
|   | meta/
|   |   | info.json # 数据集元信息 (机器人类型, fps, 任务描述等)
|   |   | stats.json # 归一化统计量 (mean, std, min, max, quantile)
|   |   | episodes.json # 每个 episode 的帧数, 时长, 文件名
|   |   | tasks.json # 任务描述与 episode 的对应关系
|   | videos/
|   |   | observation.images.wrist/
|   |   |   | episode_000000.mp4 # 腕部相机录制的视频 (H.264 编码)
|   |   |   | episode_000001.mp4
|   |   | observation.images.global/
|   |   |   | episode_000000.mp4
|   | images/ # (可选) 逐帧图像文件

```

关键文件格式说明:

文件	格式	内容
*.parquet	Apache Parquet (列式存储)	每列是一个字段: <code>observation.state</code> (关节角度), <code>action</code> (动作), <code>timestamp</code>
info.json	JSON	<code>fps</code> , <code>total_episodes</code> , <code>total_frames</code> , <code>robot_type</code> , <code>features</code> 等
stats.json	JSON	用于训练时对数据进行标准化 (z-score normalization), 防止不同量纲的特征对损失函数产生不成比例的影响
episode_*.mp4	H.264 MP4	从记录过程中逐帧收集的 JPEG → FFmpeg 编码的连续视频

步骤 6: piper_pc_relay_teleop.py 读主臂位置 → can1 控制从臂

这是与步骤 5 并行运行的一个独立进程。它的核心循环（简化后）如下：

```

# 位置: scripts/piper_pc_relay_teleop.py
leader = C_PiperInterface_V2(can_name="can0")
follower = C_PiperInterface_V2(can_name="can1")

leader.ConnectPort()
follower.ConnectPort()

while not stop:
    # ① 读主臂动作
    leader_sample = read_leader(leader, source="auto")
    # → ArmSample(joints=[...], gripper=..., ...)

    # ② 读从臂当前位置 (用于限幅计算)
    follower_sample = feedback_sample(follower)

    # ③ 符号变换: 主臂关节方向 → 从臂关节方向
    raw_target = transform_joints(leader_sample.joints, signs, offsets_mdeg)

```

```
# 例如 signs=[-1, 1, -1, 1, -1, 1] 表示奇数关节反向

# ④ 关节极限钳位
bounded_target = bounded_joints(raw_target)
# joint1 限制在 [-150000, 150000] 毫度以内

# ⑤ 步长限幅：防止单帧跳变过大
limited_target = rate_limit(follower_sample.joints, bounded_target, max_step_mdeg)
# 单步最多移动 max_step_mdeg 毫度（默认 1000 毫度 = 1 度）

# ⑥ 发送给从臂
follower.JointCtrl(*limited_target)
if send_gripper:
    follower.GripperCtrl(gripper, 1000, 0x03, 0)
```

安全机制总结：PC 中继方案有三层安全保护——(a) 符号变换确保方向一致，(b) 关节极限钳位防止超出物理范围，(c) 步长限幅防止主臂抖动传到从臂。如果在步骤 5 中需要暂停从臂，可以通过 `--pause-file` 创建一个标记文件。

步骤 7：双相机捕获 RGB 图像

相机采集与关节读取在同一个 `get_observation()` 调用中并行执行。代码位于 `src/lerobot/robots/piper_follower/piper_follower.py`：

```
def get_observation(self) -> dict[str, Any]:
    obs_dict = {}

    # 读关节角度（同步，等待 CAN 帧）
    obs_dict = self.bus.get_action()
    obs_dict = {f"{motor}.pos": val for motor, val in obs_dict.items()}

    # 逐相机采集图像（异步读取最新帧）
    for cam_key, cam in self.cameras.items():
        obs_dict[cam_key] = cam.async_read() # 非阻塞读

    return obs_dict
```

`async_read()` 方法不会等待新帧——它直接返回相机内部缓冲区中最新的完整帧。这种设计保证了即使某个相机的帧率波动，也不会阻塞关节数据的采集。

步骤 8：最终产出

经过多次 `record` 会话后，数据集目录看起来像这样：

```
datasets/local/<user>/<task_name>/
|—— data/chunk-000/
|   |—— episode_000000.parquet    # 第一次采集：40 秒，1200 帧
```

```

|   |   | episode_000001.parquet    # 第二次采集: 35 秒, 1040 帧
|   |   | episode_000002.parquet    # ...
|   |   | ...
|   |   | meta/
|   |   |   | info.json
|   |   |   | stats.json
|   |   |   | episodes.json
|   |   |   | tasks.json
|   |   |   | videos/
|   |   |   |   | observation.images.wrist/
|   |   |   |   |   | episode_000000.mp4
|   |   |   |   |   | ...
|   |   |   |   | observation.images.global/
|   |   |   |   |   | episode_000000.mp4
|   |   |   |   |   | ...

```

1.5 关键术语表

以下是本教程中使用频率最高的一组术语。每个术语同时给出英文和中文解释，以及在本项目上下文中的特定含义。

CAN (Controller Area Network)

英文	CAN (Controller Area Network)
中文	CAN 总线 / 控制器局域网
定义	一种用于实时控制系统的串行通信协议，由博世公司于 1986 年发布。广泛应用于汽车、工业自动化、机器人领域。
本项目中的含义	PiPER 机械臂的伺服电机和工控机之间通过 CAN 总线通信。每个机械臂占用一条独立的 CAN 总路——这是"双臂使用两条独立 CAN"的硬件根源。
关键特性	多主架构、优先仲裁、硬件 CRC 校验、最高 1Mbps (CAN 2.0) 或 8Mbps (CAN FD)。本项目使用标准 CAN 2.0 @ 1Mbps。

SocketCAN

英文	SocketCAN
中文	SocketCAN (无通用中文译名，直译为"套接字 CAN")
定义	

英文	SocketCAN
	Linux 内核自 2.6 版本起内置的 CAN 总线标准子系统。它将 CAN 总线抽象为一个网络接口（如 <code>can0</code> ），用户程序通过标准的 socket API（ <code>socket()</code> ， <code>bind()</code> ， <code>sendto()</code> ， <code>recvfrom()</code> ）来收发 CAN 帧。
本项目中的含义	<code>can0</code> 和 <code>can1</code> 就是两个 SocketCAN 网络接口。你可以用 <code>candump can0</code> 命令查看总线上的原始 CAN 帧，用 <code>cansend</code> 发送单帧——这两个命令是调试 CAN 通信的第一个工具。
常用命令	<code>sudo ip link set can0 type can bitrate 1000000</code> → 设置 CAN 0 为 1Mbps 并启动

SDK (Software Development Kit)

英文	SDK (Software Development Kit)
中文	软件开发工具包
定义	设备厂商提供的、封装了底层硬件通信细节的软件库，给上层应用程序提供干净的高级 API。
本项目中的含义	<code>piper_sdk</code> （装在 <code>piper_sdk/</code> 目录下）是松灵机器人为 PiPER 机械臂提供的 Python SDK。它的核心类是 <code>C_PiperInterface_V2</code> ，负责 CAN 帧收发、协议解析、线程管理等。你永远不需要在应用层直接读写 CAN 字节——SDK 替你做了这一切。

Motor / Servo（电机 / 伺服）

英文	Motor / Servo
中文	电机 / 伺服电机
定义	伺服电机是一种可以精确控制位置、速度、扭矩的电机。它内部有一个编码器实时反馈转子的当前位置，控制器根据目标值和反馈值的误差来调节电流。
本项目中的含义	PiPER 机械臂的每个关节内部都是一个独立的伺服电机。本项目涉及 7 个电机：6 个转动关节（ <code>joint1 ~ joint6</code> ）+ 1 个夹爪（ <code>gripper</code> ）。从臂使用 AGILEX-M（大扭矩关节）和 AGILEX-S（小扭矩关节）两种型号。

MotorsBus（电机总线）

英文	MotorsBus
中文	电机总线（抽象基类）
定义	LeRobot 框架定义的一个抽象类（ <code>src/lerobot/motors/motors_bus.py</code> ），封装了"通过某种通信总线连接的一组电机"的通用操作：读位置、写目标、归一化、标定。
本项目中的含义	<code>PiperMotorsBus</code> 是 <code>MotorsBus</code> 的 PiPER 专用子类（ <code>src/lerobot/motors/piper/piper.py</code> ），约 670 行代码。一个机械臂对应一个 <code>PiperMotorsBus</code> 实例。对于其他品牌的机械臂，LeRobot 有对应的 <code>DynamixelMotorsBus</code> 、 <code>FeetechMotorsBus</code> 等子类。

Robot vs Teleoperator（机器人 vs 遥操作器）

英文	Robot	Teleoperator
中文	机器人（从臂）	遥操作器（主臂）
定义	执行任务、被观测、受策略控制的设备。	被人操控、产生动作命令、训练时不直接出现的设备。
本项目中的含义	<code>PiperFollower</code> （类名，继承自 <code>Robot</code> ）	<code>PiperLeader</code> （类名，继承自 <code>Teleoperator</code> ）
关联的硬件	从臂 (CAN1) + 两个 RealSense 相机	主臂 (CAN0)，无相机
关键方法	<code>get_observation()</code> 返回 {关节, 图像}, <code>send_action()</code> 发送动作	<code>get_action()</code> 返回 {关节}

Normalization（归一化）

英文	Normalization
中文	归一化
定义	将不同量纲、不同量级的数据映射到统一的范围（如 <code>[-100, 100]</code> 或 <code>[0, 100]</code> ），让模型训练更稳定、收敛更快。
本项目中的含义	<code>PiperMotorsBus._normalize()</code> 把 SDK 返回的原始编码器整数范围（如 <code>[-150000, 150000]</code> ）映射到 <code>[-100, 100]</code> ； <code>_unnormalize()</code> 则反向映射。归一化后，所有关节值的区间统一了——模型不需要自己学习"joint1 的数据大概在 -15 万到 15 万之间，joint5 的数据大概在 -6.5 万到 6.5 万之间"。

英文	Normalization
数学公式	关节: $\text{norm} = ((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 200 - 100$ 夹爪: $\text{norm} = ((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 100$

Calibration（校准 / 标定）

英文	Calibration
中文	校准 / 标定
定义	确定电机可运动范围的物理极限，以及当前位置与零位的偏移关系。
本项目中的含义	<code>MotorCalibration</code> 是一个数据类（dataclass），包含 <code>id</code> ， <code>drive_mode</code> ， <code>homing_offset</code> ， <code>range_min</code> ， <code>range_max</code> 四个字段。 <code>range_min</code> 和 <code>range_max</code> 定义了该关节的 0% 和 100% 对应哪个原始编码器值——它是归一化公式的核心参数。PiPER 的标定值是硬编码的（定义在 <code>PiperFollower.__init__()</code> 中），不需要从硬件读取。
从臂标定值	joint1: [-150000, 150000], joint2: [0, 180000], joint3: [-170000, 0], joint4: [-100000, 100000], joint5: [-65000, 65000], joint6: [-100000, 130000], gripper: [0, 68000]

Observation vs Action（观测 vs 动作）

英文	Observation	Action
中文	观测	动作
定义	在时刻 t ，智能体从环境中接收到的所有信息。	在时刻 t ，智能体输出的控制命令。
本项目中的含义	关节角度（6 个归一化值） + 夹爪位置（1 个归一化值） + 腕部 RGB 图像 ($640 \times 480 \times 3$) + 全局 RGB 图像 ($640 \times 480 \times 3$)	7 个归一化目标值 （6 关节 + 1 夹爪）
存储字段	<code>observation.state</code> （浮点数组） <code>observation.images.wrist</code> （图像） <code>observation.images.global</code> （图像）	<code>action</code> （浮点数组）

Policy（策略）

英文	Policy
中文	策略
定义	强化学习 / 模仿学习中的核心概念。数学上表示为 $\pi(a$
本项目中的含义	训练产生的 <code>.pt</code> 或 <code>.safetensors</code> checkpoint 文件，加载后可以接受 <code>observation</code> 并输出 <code>action</code> 。评估（eval）阶段就是这样运行的。

Imitation Learning（模仿学习）

英文	Imitation Learning (IL)
中文	模仿学习
定义	机器学习的一个子领域：智能体通过观察人类（或专家）的演示来学习某项策略，而不是通过尝试-错误（试错学习，即强化学习）来学习。
本项目中的含义	人手动拖拽主臂完成任务 → 记录为数据 → 训练神经网络 → 神经网络复现任务。这是行为克隆（Behavior Cloning, BC）——模仿学习最基础的范式。

ACT (Action Chunking Transformer)

英文	ACT (Action Chunking Transformer)
中文	动作分块 Transformer
定义	一种专门为精细操控任务设计的模仿学习算法。核心思想是：一次性预测未来 K 个时间步的动作序列（"动作块"），而不是逐帧预测单一动作。这能产生更平滑、更稳定的运动轨迹。
本项目中的含义	LeRobot 框架内置的 ACT 实现位于 <code>src/lerobot/policies/act/</code> 。使用时通过 <code>--policy.type=act</code> 指定。模型结构基于 Transformer Encoder-Decoder，用 ResNet 作为视觉编码器。
关键参数	<code>n_action_steps</code> （预测多少步）， <code>chunk_size</code> （动作块大小）， <code>n_obs_steps</code> （回溯多少帧观测）

Episode (回合 / 片段)

英文	Episode
中文	回合 / 片段 / 回合
定义	一次完整的任务尝试，从"开始采集"到"停止采集"之间的所有连续帧。
本项目中的含义	<code>dataset.num_episodes=10</code> 表示采集 10 个 episode。每个 episode 是一个独立的 <code>.parquet</code> 文件和一个独立的 <code>.mp4</code> 视频文件。在 <code>LeRobotDataset</code> 中，episode 是最小的可索引单元——训练时随机采样的是 episode 内的某个连续帧段。

FPS (Frames Per Second)

英文	FPS (Frames Per Second)
中文	帧率 / 每秒帧数
定义	数据采集或动作执行的频率。
本项目中的含义	相机和关节数据的记录频率约为 30 FPS（通过 <code>--dataset.fps=30</code> 控制）。CAN 总线上的报告频率约为 200 FPS（硬件限制），但 <code>record</code> 脚本会以指定的 <code>fps</code> 做降采样。控制周期的倒数——30 FPS 意味着每帧约 33ms 的时间窗口。

1.6 代码仓库结构

本章最后，我们快速浏览一下 `lerobot_piper-piper/` 内部的关键目录。你不需要现在就打开每个文件，但建立这个"文件地图"能让你在后续章节中快速定位。

<code>lerobot_piper-piper/</code>	
<code> </code>	
<code> —— src/lerobot/</code>	<code># ★ 核心 Python 包</code>
<code> </code>	
<code> —— motors/</code>	<code># 电机总线层 (MotorsBus)</code>
<code> —— motors_bus.py</code>	<code># MotorsBus 基类 (抽象类, ~1220 行)</code>
<code> </code>	<code># 定义: Motor, MotorCalibration,</code>
<code> </code>	<code># MotorNormMode, _normalize, _unnormalize</code>
<code> —— piper/</code>	<code># PiPER 专用电机总线</code>
<code> —— piper.py</code>	<code># PiperMotorsBus 类 (~670 行)</code>
<code> </code>	<code># connect, disconnect, parking,</code>
<code> </code>	<code># get_action, set_action,</code>
<code> </code>	<code># get_control, enable_torque,</code>
<code> </code>	<code># _normalize, _unnormalize</code>
<code> —— tables.py</code>	<code># 常量表: 型号参数, 初始位姿</code>
<code> —— __init__.py</code>	


```

|   |   |—— dynamixel/          # Dynamixel 舵机支持 (本项目未使用)
|   |   |—— feetech/            # Feetech 舵机支持 (本项目未使用)
|
|   |—— robots/                # 机器人层 (Robot)
|   |   |—— robot.py            # Robot 基类 (抽象类)
|   |   |—— config.py          # RobotConfig 基类 (配置类)
|   |   |—— piper_follower/    # PiPER 从臂
|   |   |   |—— piper_follower.py # PiperFollower 类 (~187 行)
|   |   |   |   |               # connect, get_observation, send_action
|   |   |   |—— config_piper_follower.py # PiperFollowerConfig (配置类)
|   |   |   |—— __init__.py
|   |   |—— ...                # 其他型号机器人 (本项目未使用)
|
|   |—— teleoperators/         # 遥操作器层 (Teleoperator)
|   |   |—— teleoperator.py    # Teleoperator 基类 (抽象类)
|   |   |—— config.py          # TeleoperatorConfig 基类 (配置类)
|   |   |—— piper_leader/     # PiPER 主臂
|   |   |   |—— piper_leader.py # PiperLeader 类 (~111 行)
|   |   |   |   |               # connect, get_action
|   |   |   |—— config_piper_leader.py # PiperLeaderConfig (配置类)
|   |   |   |—— __init__.py
|   |   |—— ...                # 其他型号遥操作器
|
|   |—— cameras/              # 相机层
|   |   |—— camera.py          # 相机基类
|   |   |—— realsense/        # RealSense 相机后端
|   |   |   |—— camera_realsense.py # async_read(), connect(), disconnect()
|   |   |   |—— configuration_realsense.py
|   |   |—— opencv/           # OpenCV 相机后端 (USB 摄像头)
|   |   |—— ...
|
|   |—— policies/             # 策略层 (Policy)
|   |   |—— act/               # ACT (Action Chunking Transformer)
|   |   |   |—— modeling_act.py  # 神经网络模型定义
|   |   |   |—— configuration_act.py # 配置类
|   |   |   |—— processor_act.py # 数据预处理
|   |   |—— diffusion/        # 扩散策略 (Diffusion Policy)
|   |   |—— pi0/              # Pi0 (Physical Intelligence 模型)
|   |   |—— ...
|
|   |—— datasets/             # 数据集层
|   |   |—— lerobot_dataset.py # LeRobotDataset 类 (核心)
|   |   |   |               # 读取/写入 parquet, 管理 meta/
|   |   |—— image_writer.py    # 异步图像写入器
|   |   |—— video_utils.py     # 视频编解码 (FFmpeg 封装)
|   |   |—— compute_stats.py   # 计算归一化统计量 → stats.json
|   |   |—— ...
|
|   |—— processor/            # 数据处理器层
|   |   |—— normalize_processor.py # 标准化处理器 (z-score)
|   |   |—— observation_processor.py # 观测处理器
|   |   |—— ...
|
|   |—— configs/              # 全局配置

```



```

|       |—— feedback/      # 反馈消息结构体
|       |—— transmit/      # 发送消息结构体
|—— protocol/              # 协议解析器

```

提醒：阅读代码时，请从 `piper.py` (PiperMotorsBus) 开始。它是整个系统的"翻译器"，连接了 `piper_sdk` 的底层 API 和 LeRobot 的上层框架。理解了它，就理解了一半的系统。

1.7 本章总结

恭喜你完成了系统总览。让我们回顾一下关键信息：

层次	核心组件	掌握程度检查
硬件	主臂 (can0) + 从臂 (can1) + 双相机	能画出硬件拓扑图
拓扑	PC 中继方案：电脑读 can0 → 处理后发 can1	能解释为什么不使用硬件主从模式
数据流	人 → 主臂编码器 → CAN 帧 → SDK 缓存 → 归一化 → LeRobotDataset	能从步骤 1 数到步骤 8 不卡壳
归一化	$\text{norm} = ((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 200 - 100$	能手算 joint1 的 raw=75000 对应的 norm 值 (=50.0)
软件栈	SDK → MotorsBus → Robot/Teleoperator → Scripts	能说出每层的一个代表性类名
工作流	record → train → eval → replay	能写出每个步骤对应的 lerobot 命令

下一章预告：我们将从逐行阅读 `piper_pc_relay_teleop.py` 开始，深入理解 PC 中继方案的实现细节——包括如何配置 CAN 口、如何选择数据源（feedback vs control）、以及三层安全限幅的数学原理。

第二章 硬件基础

"想要控制好机器人，首先要理解它的神经系统。"

本章导览

在正式编写模仿学习代码之前，你必须理解 PiPER 机器人硬件是如何工作的。本章从最底层的 CAN 总线开始，逐层向上介绍伺服电机、PiPER 专有 CAN 协议、RealSense 深度相机，最后解释主臂与从臂的协作架构。读完本章，你将能够：

- 用 `candump` 命令直接观机械臂的实时数据流
- 理解每一帧 CAN 报文背后代表的物理含义
- 知道自己写的代码最终是如何转化为电机的物理运动的
- 排查通信故障时，能精准定位问题出在哪一层

2.1 CAN 总线基础

2.1.1 什么是 CAN 总线

CAN (Controller Area Network, 控制器局域网) 是一种为工业现场设计的串行通信协议，最初由 Bosch 公司在 1986 年为汽车电子系统开发。如今 CAN 已广泛应用在工业机器人、医疗设备、航空航天等领域。

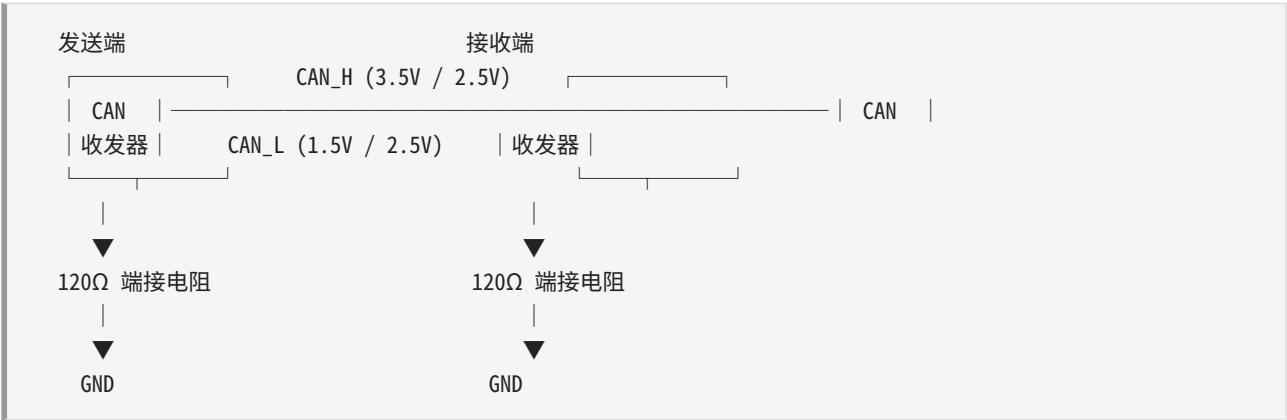
核心特征一：多主架构 (Multi-Master)

传统的 RS-232/RS-485 总线采用主从架构——只有一台主机可以发起通信，其余设备只能被动应答。CAN 总线则采用多主架构：总线上任何一个节点都可以在任何时刻主动发送消息。当两个节点同时尝试发送时，CAN 通过**仲裁机制**自动决定优先级——ID 越小的帧优先级越高，低优先级的节点会自动退让，不会引起数据碰撞。这一特性使得 CAN 的实时性远优于需要轮询的主从总线。

核心特征二：差分信号传输

CAN 使用两根线传输数据：CAN_H 和 CAN_L。发送器将逻辑信号转换为差分电压——逻辑 0（显性位）时，CAN_H $\approx$ 3.5V，CAN_L $\approx$ 1.5V，压差约 2V；逻辑 1（隐性位）时，两线电压都约为 2.5V，压差接近 0V。接收器通过检测这两根线的**电压差**来判断信号，而不是单根线的绝对电压。

差分传输的最大优势是**抗共模干扰**：外部电磁噪声通常同时耦合到两根线上（共模），而接收端只看差分信号，噪声自然被抵消。这就是为什么在充满电机 PWM 噪声的机器人内部，CAN 依然能稳定通信。



两端各有一个 120Ω 终端电阻，用于阻抗匹配，防止信号反射。总线上任一电阻开路都会导致通信失败。

2.1.2 为什么机器人选择 CAN

在 PiPER 的工控机箱内，你会看到 USB-to-CAN 适配器通过 USB 线连接到工控机，另一端是 CAN 总线连接到机械臂的电机驱动器。机器人选择 CAN 而不是 PWM、RS-485 或 EtherCAT，有以下关键原因：

特性	CAN	RS-485	PWM	EtherCAT
拓扑结构	多主总线	主从总线	点对点	主从（逻辑环）
实时性	微秒级仲裁	轮询依赖波特率	无延迟但无反馈	微秒级（专用硬件）
抗干扰	差分 + 硬件 CRC	差分，无硬件 CRC	易受干扰	差分 + 硬件 CRC
错误检测	5 种机制（位/填充/CRC/格式/应答）	仅奇偶校验	无	硬件 CRC
多节点能力	一条总线挂上百节点	理论 32 节点	一对一接线	支持多节点
成本	低（单片机内置控制器）	低	极低	高（需专用 ASIC）
数据载荷	0-8 字节（经典 CAN）	可变长	无（模拟信号）	可变长

对于 PiPER 这样的 6+1 自由度机械臂，CAN 在**实时性、可靠性和成本**之间取得了最佳平衡。100 元人民币级别的 USB-CAN 适配器 + Linux 内核原生 SocketCAN 支持，让整个通信栈极为轻量。

2.1.3 CAN 帧结构

一个完整的标准 CAN 数据帧（11-bit ID）如下：



各字段详解：

- **SOF (Start Of Frame, 1 bit)**：帧起始标志，一个显性位（逻辑 0）。所有节点以此同步时钟。
- **仲裁字段 (Arbitration Field)**：包含 11 位标准 ID（或 29 位扩展 ID）和 1 位 RTR（远程帧请求位）。数据帧的 RTR 为隐性，表明这是一帧“我在发数据”。
- **控制字段 (Control Field)**：IDE 位区分标准/扩展帧，**DLC (Data Length Code, 4 bit)** 标明后面的数据有多少字节（0~8）。PiPER 中所有 CAN 帧都用满 8 字节（DLC=8）。
- **数据字段 (Data Field)**：实际载荷，0 到 8 字节。在 PiPER 中，两字节可存一个 int16，四字节可存一个 int32。
- **CRC 字段 (15 bit + 1 bit 定界符)**：发送方计算的多项式校验值，接收方独立验算。不匹配则立即标记错误。
- **ACK 字段 (2 bit)**：发送方发隐性位，任何正确收到此帧的接收站必须在 ACK slot 回复显性位。发送方检测到这个显性位就知道“至少有一个节点正确收到了”。
- **EOF (End Of Frame, 7 bit)**：7 个隐性位标记帧结束。
- **IFS (Interframe Space, ≥3 bit)**：帧间间隔，给控制器处理时间。

PiPER 使用的全部是**标准格式 (11-bit ID)**，**DLC=8 (8 字节数据)**。没有用扩展 ID，也没有用 CAN FD（可变速率/大载荷）。

2.1.4 SocketCAN —— Linux 内核原生的 CAN 支持

SocketCAN 是 Linux 内核自 2.6 版本起内置的 CAN 子系统。它将 CAN 总线抽象为网络接口——你可以像操作网卡一样操作 CAN 口。can0、can1 出现在 ifconfig 的输出中，用 ip link 命令配置，用标准 socket API 编程。

使用 SocketCAN 意味着你不需要任何用户态的 CAN 驱动。USB-to-CAN 适配器（通常基于 gs_usb 芯片）插入后，内核自动加载 gs_usb 模块，创建 canX 网络接口。配置好波特率并 ip link set up 后，就可以像读写普通网络 socket 一样收发 CAN 帧。

2.1.5 CAN 配置关键命令

PiPER 的 CAN 总线工作速率是 **1 Mbps (1,000,000 bps)**。这是一个在传输距离和数据速率之间的工程平衡点——1 Mbps 下 CAN 总线的最大长度约为 40 米，远超机械臂线缆长度（通常 1-2 米）。

最简配置（单臂）：

```
# 查看系统中有哪些 CAN 接口
ip link show type can

# 设置波特率并启用（PiPER 固定在 1 Mbps）
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 验证是否成功启用
ip -details link show can0
# 输出中应能看到：bitrate 1000000 且 state UP

# 实时查看 CAN 总线上的数据（裸数据抓取）
candump can0
# 输出示例：
# can0 2A5 [8] 00 00 00 00 FF FF A3 D8
# can0 2A6 [8] 00 00 00 00 FF FF 7D 90
# can0 2A7 [8] 00 00 00 00 00 00 00 00
# can0 2A8 [8] 00 00 00 00 00 00 00 00
```

PiPER 专用初始化脚本（双臂自动识别）：

项目中 can_activate.sh 脚本封装了上述步骤，并增加了自动识别和重命名功能。它使用 ethtool -i 查询每个 CAN 口背后 USB 适配器的物理位置（bus-info，如 3-1.1:1.0），从而在多个适配器同时插入时可靠地区分谁是谁。

```
# 单臂：激活 can0，波特率 1 Mbps
bash can_activate.sh can0 1000000

# 双臂：激活 can_left 和 can_right，通过 USB 物理位置映射
# can_config.sh 中预设映射：
# 物理位置 3-2:1.0 → can_left （主臂）
# 物理位置 3-1:1.0 → can_right （从臂）
bash can_config.sh
```

2.1.6 CAN ID 的双重作用

CAN ID 不是"地址"。在 CAN 协议层面，CAN ID 有两个作用：

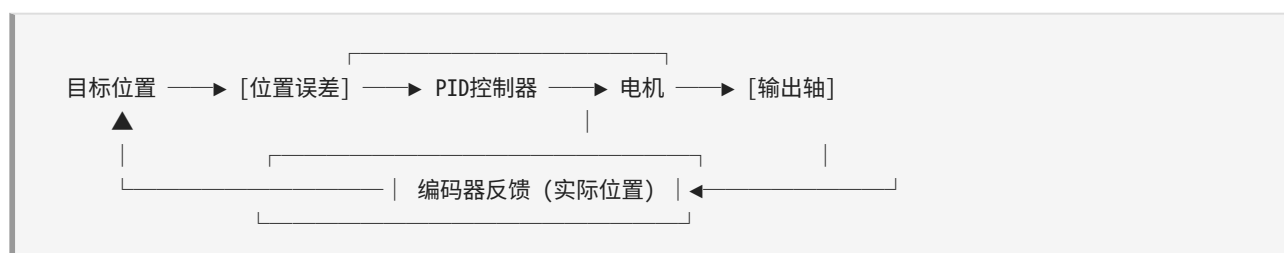
1. **优先级 (Priority)**：ID 越小优先级越高。当两个节点同时开始发送，在仲裁阶段，发送 ID=0x150 的节点会"赢过"发送 ID=0x2A1 的节点（因为 $0x150 < 0x2A1$ ），后者自动退让，等下一轮重发。因此 PiPER 的控制指令 ID（0x15x 范围）优先级高于状态反馈 ID（0x2Ax 范围）——控制指令优先发出。
2. **消息类型标识 (Message Type Identifier)**：听的人通过 ID 知道这帧内容是什么。ID=0x2A5 表示"这是关节 1 和关节 2 反馈数据"，ID=0x155 表示"这是关节 1 和关节 2 控制指令"。PiPER 协议定义了约 50 个不同的 CAN ID，每个代表一类特定的消息。

PiPER 使用**静态 ID 分配**——每个 CAN ID 与消息类型的对应关系是固定的、在协议文档中预定义的，不随运行状态动态变化。这与某些动态 ID 分配的协议（如 J1939）不同。

2.2 伺服电机基础

2.2.1 什么是伺服电机

普通的直流电机通电就转，断电就停。你无法精确控制它转了多少度——想知道位置，需要外挂编码器；想停在某个位置，需要外挂控制器。**伺服电机 (Servo Motor)** 将电机、编码器和控制器集成在一起，形成了一个带反馈的闭环位置控制系统。



闭环控制的基本原理：1. 你给出目标位置（例如"转到 45° "）2. 编码器读取当前位置（例如"当前在 43.2° "）3. 控制器计算误差 = $45.0^\circ - 43.2^\circ = 1.8^\circ$ 4. 控制器根据 PID 算法决定给电机多少电流来缩小误差 5. 电机转动 → 编码器重新读数 → 循环直到误差趋近于 0

伺服电机的"锁定"特性也来源于闭环：即使外力推动输出轴，编码器检测到位置偏移，控制器立即反向驱动电机来"抵抗"，所以正常使能的伺服电机很难被人手推动。

2.2.2 编码器原理

PiPER 使用的编码器基于磁性原理。在电机后轴上安装有一个多极磁环，当轴转动时，磁环产生的磁场方向随之旋转。贴近磁环的霍尔传感器阵列检测磁场角度，将其转换为数字位置信号。

两种主流编码器技术对比：

特性	光电编码器	磁性编码器
原理	光栅盘 + 光电对管	磁环 + 霍尔传感器
分辨率	可达数万线	通常 4096 线 (12 bit)
抗污染	差 (怕灰尘/油污)	好 (磁场不受灰尘影响)
抗震动	一般	好
成本	高 (精密光栅加工)	低 (半导体工艺)
温度稳定性	好	一般 (磁环温度漂移)

PiPER 选择磁性编码器是因为工业机器人的工作环境可能存在粉尘和油污，磁性方案更鲁棒。4096 线的分辨率对机械臂应用而言足够——将一圈 360° 量化为 4096 份，每份约 0.088° ，远小于机械重复定位误差 (通常 0.5° 级别)。

2.2.3 位置编码单位：毫度 (milli-degree)

PiPER 的 CAN 协议中，所有角度值均以毫度 (milli-degree, 简称 **mdeg**) 为单位传输。1 毫度 = 0.001 度。

这意味着：- 关节转到 $90^\circ \rightarrow$ CAN 上传输的整数值为 90000 - 关节转到 $-150^\circ \rightarrow$ CAN 上传输的整数值为 -150000 - 关节转到 $0^\circ \rightarrow$ CAN 上传输的整数值为 0

为什么用固定的整数单位而不是浮点数？

- 避免精度损失：**IEEE 754 浮点数在表示大角度小增量时会出现精度不足。例如在 1e6 附近加 1，float32 无法区分 (最小可分辨差值约 0.125)。但 int32 能精确表示 ± 2147483647 mdeg 范围内的每一个整毫度值，远超机械臂需要的 ± 180000 mdeg 范围。
- CAN 帧效率：**4 字节直接存一个 int32 (大端)，不需要编码浮点数的符号位/指数/尾数。
- 协议简单可靠：**收方只需要知道"单位是毫度"一个规则，不需要协商浮点格式 (float16? float32? double?)。

在实际代码中，SDK 向 LeRobot 框架暴露的是**归一化值**（关节 $[-100, 100]$ ，夹爪 $[0, 100]$ ），转换公式将在 2.3.4 中详细介绍。

2.2.4 PiPER 使用的电机型号

PiPER 双臂系统使用了三种不同型号的伺服电机，对应不同的关节负载需求：

AGILEX-M（大电机，Model Number 1190） - 安装位置：从臂（follower）关节 1、2、3（基座区大关节） - 定位：高扭矩、驱动基座旋转和肩部俯仰 - 编码器分辨率：4096 线/转（12 bit） - 工作模式：支持模式 0、1、3、4、5、16

AGILEX-S（小电机，Model Number 1191） - 安装位置：从臂（follower）关节 4、5、6（腕部区小关节）+ 夹爪 - 定位：低扭矩、驱动腕部精细运动和夹爪开合 - 编码器分辨率：4096 线/转（12 bit） - 工作模式：支持模式 0、1、3、4、5、16

HTDW-5047（示教电机） - 安装位置：主臂（leader）全部 6 个关节 + 夹爪 - 定位：高反驱透明性（backdrivability），被人拖拽时阻力极小 - 编码器分辨率：4096 线/转（12 bit） - 特点：与 AGILEX 系列不同的力矩-速度特性，侧重于被拖动时的柔顺性和低阻尼

在代码中的定义（`tables.py` 和 `piper_leader.py` / `piper_follower.py`）：

```
# 从臂（follower）的电机分配
motors = {
    "joint1": Motor(1, "AGILEX-M", MotorNormMode.RANGE_M100_100),
    "joint2": Motor(2, "AGILEX-M", MotorNormMode.RANGE_M100_100),
    "joint3": Motor(3, "AGILEX-M", MotorNormMode.RANGE_M100_100),
    "joint4": Motor(4, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    "joint5": Motor(5, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    "joint6": Motor(6, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    "gripper": Motor(7, "AGILEX-S", MotorNormMode.RANGE_0_100),
}

# 主臂（leader）的电机分配
motors = {
    "joint1": Motor(1, "HTDW-5047", MotorNormMode.RANGE_M100_100),
    "joint2": Motor(2, "HTDW-5047", MotorNormMode.RANGE_M100_100),
    "joint3": Motor(3, "HTDW-5047", MotorNormMode.RANGE_M100_100),
    "joint4": Motor(4, "HTDW-5047", MotorNormMode.RANGE_M100_100),
    "joint5": Motor(5, "HTDW-5047", MotorNormMode.RANGE_M100_100),
    "joint6": Motor(6, "HTDW-5047", MotorNormMode.RANGE_M100_100),
    "gripper": Motor(7, "HTDW-5047", MotorNormMode.RANGE_0_100),
}
```

2.2.5 扭矩控制与夹爪力反馈

夹爪不仅要控制位置（开合角度），还要控制输出力矩（抓握力）。在 PiPER 的 CAN 协议中，夹爪反馈帧（0x2A8）和控制帧（0x159）都包含一个 2 字节的 `gripper_effort` 字段，单位为 **0.001 N/m（毫牛米）**。

这意味着：- `gripper_effort = 500` 表示 0.5 N/m 的力矩 - `gripper_effort = 2000` 表示 2.0 N/m 的力矩

通过力反馈，你可以实现柔顺抓取——检测到力矩上升说明夹爪正在紧握物体，到达预设阈值后停止增加力，避免夹碎物体。

2.3 PiPER 的 CAN 协议详解

本节是本章最核心的内容。你将逐帧了解 PiPER 机械臂的完整 CAN 通信协议。

2.3.1 CAN ID 总览

PiPER 的 CAN ID 按功能分为四大类：

功能类别	ID 范围	方向	说明
反馈消息 (Feedback)	0x2A1 ~ 0x2A8	机械臂 → PC	编码器位置、状态、位姿
驱动器信息	0x251 ~ 0x266	驱动器 → PC	电机转速、电流、温度
控制消息 (Control)	0x150 ~ 0x15F	PC → 机械臂	关节/夹爪目标、模式设置
配置消息 (Config)	0x470 ~ 0x47E	双向	主从配置、使能、参数查询
关节速度反馈	0x481 ~ 0x486	机械臂 → PC	各关节末端速度/加速度
灯光控制	0x121	PC → 机械臂	指示灯控制
固件升级	0x422 / 0x4AF	双向	静默模式、固件读取

2.3.2 核心 CAN 帧：完整数据表

以下是 PiPER 遥操作和模仿学习中**最核心的 14 条 CAN 消息**的完整定义。这些消息覆盖了读取关节状态、发送控制命令、配置主从模式等全部基本操作：

CAN ID	方向	消息名称	数据布局（字节 0..7）	编码说明
0x2A5	反馈		<code>joint_1[4]</code> <code>joint_2[4]</code>	int32 大端, 毫度

CAN ID	方向	消息名称	数据布局（字节 0..7）	编码说明
		关节 1-2 反馈		
0x2A6	反馈	关节 3-4 反馈	joint_3[4] joint_4[4]	int32 大端, 毫度
0x2A7	反馈	关节 5-6 反馈	joint_5[4] joint_6[4]	int32 大端, 毫度
0x2A8	反馈	夹爪 反馈	angle[4] effort[2] status[1] rsv[1]	angle: int32 mdeg, effort: uint16 0.001N/m, status: uint8
0x2A1	反馈	机械 臂状 态	ctrl_mode[1] arm_status[1] mode_feed[1] teach_status[1] motion_status[1] trajectory_num[1] err_code[2]	各字段均为 uint8/int16, 详见下文
0x155	控制	关节 1-2 目 标	target_1[4] target_2[4]	int32 大端, 毫度
0x156	控制	关节 3-4 目 标	target_3[4] target_4[4]	int32 大端, 毫度
0x157	控制	关节 5-6 目 标	target_5[4] target_6[4]	int32 大端, 毫度
0x159	控制	夹爪 控制	angle[4] effort[2] status_code[1] set_zero[1]	angle: int32 mdeg, effort: uint16, code: uint8, zero: uint8
0x151	控制	运动 模式 控制	ctrl_mode[1] move_mode[1] speed[1] mit_mode[1] residence[1] install_pos[1] rsv[2]	各 1 字节, 见下表
0x470	配置	主从 模式 配置	linkage[1] fb_offset[1] ctrl_offset[1] linkage_offset[1] rsv[4]	0xFA = 主臂, 0xFC = 从臂

CAN ID	方向	消息名称	数据布局 (字节 0..7)	编码说明
0x471	控制	电机使能/失能	motor_num[1] enable_flag[1] rsv[6]	motor_num=7 表示全部电机, enable_flag=0x02 使能
0x472	配置	查询电机限位	motor_num[1] search_content[1] rsv[6]	search=0x01 查角度/速度, 0x02 查加速度
0x477	配置	参数查询设置	param_enquiry[1] param_setting[1] data_feedback[1] end_load_effective[1] set_end_load[1] rsv[3]	参数查询: 0x01=末端速度, 0x02=碰撞, 0x04=夹爪

表注: [N] 表示该字段占 N 字节。| 分隔不同的字段, 字段所在字节位用偏移表示。所有多字节整型均为大端字节序 (Big Endian, Motorola 格式) ——高位字节存在低地址。例如 int32 值 0x000186A0 (=100000 mdeg = 100°), 在 CAN 数据字节中排列为 [0x00, 0x01, 0x86, 0xA0]。

2.3.3 反馈消息详解 (0x2A1 ~ 0x2A8)

反馈消息由机械臂的主控板周期性广播 (约 200 Hz), PC 端只听不发。

关节反馈帧族 (0x2A5, 0x2A6, 0x2A7)

这三帧将 6 个关节的编码器读数成对打包 (省下三个 CAN ID):

帧 0x2A5: [joint_1 高字节...低字节] [joint_2 高字节...低字节]
 帧 0x2A6: [joint_3 高字节...低字节] [joint_4 高字节...低字节]
 帧 0x2A7: [joint_5 高字节...低字节] [joint_6 高字节...低字节]

每个关节值是有符号 int32, 范围取决于该关节的物理限位。以从臂为例:

关节	最小角度 (mdeg)	最大角度 (mdeg)	对应角度范围
Joint 1	-150,000	+150,000	-150° ~ +150°
Joint 2	0	+180,000	0° ~ +180°
Joint 3	-170,000	0	-170° ~ 0°
Joint 4	-100,000	+100,000	-100° ~ +100°

关节	最小角度 (mdeg)	最大角度 (mdeg)	对应角度范围
Joint 5	-70,000	+70,000	-70° ~ +70°
Joint 6	-120,000	+120,000	-120° ~ +120°

夹爪反馈帧 (0x2A8)

字节 0-3: grippers_angle (int32, 毫度, 0~68000)
 字节 4-5: grippers_effort (uint16, 0.001 N/m)
 字节 6: status_code (uint8, 夹爪状态码)
 字节 7: 保留

夹爪的角度范围是 0 ~ 68000 mdeg (0° ~ 68°)，不同于关节的正负对称范围。status_code 可以指示夹爪当前是否正在运动、是否到达目标、是否有故障等信息。

机械臂状态帧 (0x2A1)

字节 0: ctrl_mode — 当前控制模式 (0=待机, 1=CAN指令模式)
 字节 1: arm_status — 机械臂整体状态
 字节 2: mode_feed — 模式反馈
 字节 3: teach_status — 示教状态
 字节 4: motion_status — 运动状态 (0=静止, 非0=运动中)
 字节 5: trajectory_num — 当前轨迹点编号
 字节 6-7: err_code — 错误码 (uint16, 0=正常)

SDK 中 GetArmStatus() 返回的就是解析后的这一帧。代码中常检查 motion_status 来判断机械臂是否到达目标位置：

```
status = piper.GetArmStatus()
if status.arm_status.motion_status == 0:
    print("机械臂已停在目标位置")
```

2.3.4 归一化：从 CAN 毫度到模型输入

CAN 总线上传输的是毫度 (mdeg) 整数，但 LeRobot 框架和神经网络期望的是归一化到统一范围的浮点值。这个转换由 PiperMotorsBus._normalize() 和 _unnormalize() 完成。

关节归一化 ([-100, 100])：

将任意关节范围的原始毫度值线性映射到 [-100, 100] 区间：

```
norm = ((raw - min) / (max - min)) * 200 - 100
```

以 Joint 1 为例（范围 [-150000, 150000] mdeg）：

原始值 (mdeg)	计算过程	归一化值
0	$((0 - (-150000)) / 300000) * 200 - 100 = 0.5 * 200 - 100$	0.0
150000	$((150000 - (-150000)) / 300000) * 200 - 100 = 1.0 * 200 - 100$	100.0
-150000	$((-150000 - (-150000)) / 300000) * 200 - 100 = 0.0 * 200 - 100$	-100.0
75000	$((75000 - (-150000)) / 300000) * 200 - 100 = 0.75 * 200 - 100$	50.0

夹爪归一化 ([0, 100])：

夹爪只有单向运动（从开到闭），所以映射到 [0, 100]：

$$\text{norm} = ((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 100$$

以夹爪为例（范围 [0, 68000] mdeg）：

原始值 (mdeg)	归一化值	含义
0	0.0	完全张开
34000	50.0	半开
68000	100.0	完全闭合

反归一化（模型输出 → CAN 命令）：

推理时的逆过程：

$$\begin{aligned} \text{关节: raw} &= \text{round}(((\text{norm} + 100) / 200) * (\text{max} - \text{min}) + \text{min}) \\ \text{夹爪: raw} &= \text{round}((\text{norm} / 100) * (\text{max} - \text{min}) + \text{min}) \end{aligned}$$

函数中还对输入值做了**安全钳位**（clamping），防止模型输出超出预期范围导致机械臂收到非法位置命令。

2.3.5 控制消息详解 (0x150 ~ 0x15F)

控制消息由 PC 向机械臂**主动发起**，设置目标位置和运动参数。

关节控制帧族 (0x155, 0x156, 0x157)

结构与反馈帧完全对称——同样是每帧打包两个关节的四字节 int32 毫度值。编码逻辑在 SDK 的 `EncodeMessage()` 中：

```
# 发送 Joint 1,2 的目标位置
tx_can_frame.arbitration_id = 0x155
tx_can_frame.data = (
    ConvertToList_32bit(joint_1_target) + # 4 bytes, big-endian
    ConvertToList_32bit(joint_2_target)   # 4 bytes, big-endian
)
```

`JointCtrl(j1, j2, j3, j4, j5, j6)` 内部连续发送三帧 (0x155, 0x156, 0x157)，将 6 个目标关节角一次性发出。在高频控制循环中（如 PC 中继的 100 Hz 模式），每个周期都会调用 `JointCtrl` 更新目标。

夹爪控制帧 (0x159)

```
字节 0-3: grippers_angle (int32, 毫度)
字节 4-5: grippers_effort (uint16, 0.001 N/m)
字节 6:   status_code    (uint8, 控制码)
字节 7:   set_zero       (uint8, 0=正常控制, 1=设当前位置为零点)
```

LeRobot 集成层的每次 `set_action()` 调用都会发送夹爪命令：

```
self.piper.GripperCtrl(
    abs(int(action_denormalized["gripper"])), # 角度位置
    1000,                                     # 速度参数
    0x03,                                     # 控制模式
    0                                         # 非阻塞
)
```

运动模式控制帧 (0x151)

```
字节 0: ctrl_mode      — 控制模式
        0x00 = 待机 (Standby)
        0x01 = CAN 指令控制模式
字节 1: move_mode      — 运动模式
        0x00 = MOVE P (位置)
        0x01 = MOVE J (关节空间)
        0x02 = MOVE L (直线)
        0x03 = MOVE C (圆弧)
字节 2: move_spd_rate_ctrl — 速度百分比 (0~100)
字节 3: mit_mode       — MIT 模式标志
        0x00 = 普通位置-速度模式
        0xAD = MIT 力控模式
字节 4: residence_time  — 在目标点的停留时间
字节 5: installation_pos — 安装姿态
字节 6-7: 保留
```


在实际使用中，`ModeCtrl()` 是对 `MotionCtrl_2()` 的薄封装：

```
# Set joint-position control mode, 30% speed (safe default)
piper.ModeCtrl(ctrl_mode=0x01, move_mode=0x01, move_spd_rate_ctrl=30, is_mit_mode=0x00)
```

速度百分比是一个重要的安全参数。PC 中继遥操作默认使用 100% 速度（追求响应性），而 ACT 策略推理使用 30% 速度（安全优先，避免策略探索时动作过大）。

2.3.6 反馈消息与控制消息的方向性区别

理解 CAN 消息的方向性是调试通信问题的关键：

- **反馈消息 (0x2A1~0x2A8)**：由**机械臂主控板**主动周期性广播。这些消息不需要 PC 请求，只要机械臂上电、CAN 总线启用，数据就在总线上流动。你可以用 `candump can0` 直接看到这些帧，无需运行任何 Python 程序。
- **控制消息 (0x150~0x15F)**：由**PC** 按需发送。只有当你调用 `JointCtrl()` 或 `GripperCtrl()` 时，PC 才会在 CAN 总线上发送这些帧。它们不是周期性的——控制频率完全由应用代码决定。
- **配置消息 (0x470~0x47E)**：双向。PC 发送查询请求 → 机械臂回复应答。例如 `EnablePiper()` 发送 0x471 → 机械臂使能电机后回复状态。

一个常见的调试场景：机械臂不动，先 `candump can0` 看有没有 0x2A5~0x2A7 在飞。如果有，说明机械臂本身工作正常——是 PC 没发控制指令或网络连接有问题。如果连反馈帧都没有，检查机械臂电源和 CAN 物理链路。

2.3.7 硬件主从模式下 CAN ID 的"位移"

当通过 `MasterSlaveConfig(0xFA/0xFC)` 启用**硬件主从模式**时，CAN 消息的行为发生了根本变化。

正常模式（无主从配置）： - 主臂（can0）：仅广播自己的反馈帧（0x2A5~0x2A8） - 从臂（can1）：仅广播自己的反馈帧（0x2A5~0x2A8） - PC 分别从两个 CAN 口读取反馈，向从臂 CAN 口发送控制指令 - 两个 CAN 总线完全独立

硬件主从模式（两臂接同一 CAN 总线）： - 主臂被配置为 `Linkage_config=0xFA`（示教输入臂）： - 将自身反馈帧的 CAN ID 加上可配置的偏移值（默认偏移量决定新 ID） - **主动发送** 0x155~0x159 控制指令（就像 PC 一样），内容为自身当前关节角 - 不再响应 PC 的控制指令 - 从臂被配置为 `Linkage_config=0xFC`（运动输出臂）： - 接收到 CAN 总线上的 0x155~0x159 帧（由主臂发出的），直接执行 - 这是**固件级别的跟随**，延迟低于 PC 中继方案

硬件主从模式的核心优势是**极低延迟**——从臂接收主臂命令的延迟仅为 CAN 帧传输时间（约 100 微秒），无需经过 Linux 网络栈和 Python 解释器。缺点是**无法做变换、限速或记录数据**，因为数据不经过 PC。

在 PC 中继遥操作中，读主臂状态用的是 `GetArmJointCtrl()` 和 `GetArmGripperCtrl()` ——这些方法读的是主臂发送出来的**控制帧**（0x155~0x157），而不是反馈帧（0x2A5~0x2A7）。这就是为什么在 PC 中继模式中，即使主臂在示教模式（不发送反馈流），PC 也能读到它的位置。

2.4 RealSense D435 深度相机

2.4.1 硬件特性

Intel RealSense D435 是基于**主动立体红外（Active Stereo IR）**技术的深度相机。其核心工作原理：

1. 左侧红外相机和右侧红外相机从略微不同的视角拍摄同一场景
2. 红外点阵投影器向场景投射不可见的伪随机点阵图案
3. 立体匹配算法在左右红外图像中找到对应点，根据视差计算深度
4. RGB 相机提供彩色纹理，可与深度图对齐生成彩色点云

与传统双目相机相比，主动投影器解决了**弱纹理表面**（白墙、纯色桌面）的匹配困难——即使物体表面没有纹理，投影器投射的点阵提供了"人工纹理"。

D435 关键规格：

参数	数值
深度技术	主动立体红外
RGB 传感器	1920×1080 @ 30fps（全局快门）
深度视场角	87°(H) × 58°(V)
最小深度距离	~0.1 m（10 cm）
理想范围	0.3 ~ 3 m
快门类型	全局快门（Global Shutter）
接口	USB 3.0 Type-C
尺寸	90 × 25 × 25 mm

全局快门是关键特征——所有像素同时曝光，没有卷帘快门的"果冻效应"。当相机或物体在快速运动时（遥操作中的手臂挥动），全局快门确保每一帧图像是瞬时拍摄的，不会出现倾斜或扭曲。

2.4.2 D435 vs D435I

D435I 在 D435 的基础上增加了一颗 **IMU（惯性测量单元, Inertial Measurement Unit）**，包含 3 轴加速度计和 3 轴陀螺仪。可以向主机提供 200 Hz 的加速度和角速度数据。

对于 PiPER 模仿学习任务，目前**不需要 IMU**——训练数据只包含关节角（本体感知）和 RGB 图像（视觉感知），没有用到加速度或角速度信息。因此 D435 就是正确的选择，不必为 IMU 支付额外成本。

2.4.3 序列号系统与双相机识别

每台 RealSense 相机出厂时都烧录了**唯一的序列号**，是一个 12 位数字字符串（如 "238222076529"）。这个序列号在 USB 枚举时可以读取，是区分多台相机的唯一可靠标识。

在 PiPER 系统中，通常配置两台相机：

- **腕部相机（Wrist Camera）**：安装在从臂的末端（通常在第 5 或第 6 关节附近），随机械臂运动。固定焦距，近距离拍摄操作台面。
- **全局相机（Global Camera）**：安装在三脚架或固定支架上，不随机械臂运动。覆盖整个工作空间。

在 camera_config_realsense.env 配置文件中，通过序列号将物理角色分配给具体设备：

```
# 腕部相机序列号（安装在从臂上）
export WRIST_REALSENSE_SERIAL="238222076529"

# 全局相机序列号（安装在固定位置）
export GLOBAL_REALSENSE_SERIAL="142122071524"

# 完整相机配置
export ROBOT_CAMERAS='{
  wrist: {
    type: intelrealsense,
    serial_number_or_name: "238222076529",
    width: 640, height: 480, fps: 15
  },
  global: {
    type: intelrealsense,
    serial_number_or_name: "142122071524",
    width: 640, height: 480, fps: 15
  }
}'
```

如果你交换了两个相机的物理位置，只需交换序列号即可——代码完全不需要改动。

2.4.4 硬件同步策略

PiPER 系统**不使用** genlock（硬件帧同步）或外部触发信号来同步两台相机。同步策略是软件层面的：

1. 每台相机独立以指定的 fps（如 15 或 30）自由运行
2. 数据采集循环在每个周期内同时拉取两台相机的最新帧
3. `lerobot-record` 框架通过时间戳对齐：取最接近目标采集时刻的那一帧

这种"软同步"方案在 15~30 fps 的数据采集速率下完全足够——因为机械臂的运动是连续的，相邻两帧之间的姿态差异很小。训练时，模型学习的是"从当前图像和关节角到下一时刻动作"的映射，对相机间的微秒级时间对齐不敏感。

2.4.5 相机参数配置

PiPER 数据采集和推理使用的相机分辨率为 640×480 ，采样率 **30 fps**（与 LeRobot 数据集帧率对齐）。

640×480 是模仿学习的标准分辨率，原因： - 更高的分辨率（如 1280×720 ）会使 CNN 特征提取的计算量增加 4 倍，而额外细节对抓取等任务并无实质帮助 - 更低的 fps 会丢失快速运动时的时序信息，30 fps 在"够用"和"计算量"之间取得了平衡 - 所有公开的 LeRobot 预训练模型都使用 640×480 输入，换分辨率意味着无法复用预训练权重

相机的 fps 必须与数据集的 `DATASET_FPS` 参数一致。`record_with_pc_relay.sh` 中有一段验证逻辑：

```
# 验证相机 fps 与 DATASET_FPS 是否一致
python - <<'PY'
camera_cfg = yaml.safe_load(os.environ["ROBOT_CAMERAS"])
dataset_fps = int(os.environ["DATASET_FPS"])
mismatches = {
    name: cfg.get("fps")
    for name, cfg in camera_cfg.items()
    if cfg.get("fps") is not None and int(cfg.get("fps")) != dataset_fps
}
if mismatches:
    raise SystemExit(
        f"ERROR: camera fps must match DATASET_FPS. "
        f"DATASET_FPS={dataset_fps}, mismatches={mismatches}"
    )
PY
```

2.5 主臂 vs 从臂的角色定义

2.5.1 角色分工

PiPER 双臂系统的核心设计哲学是**人机协作模仿学习**：一个人拖动"教学臂"（主臂）演示任务，一台"作业臂"（从臂）通过观察和模仿来学习。两台机械臂在硬件上是完全独立的——各有各的 CAN 总线和 USB 连接。

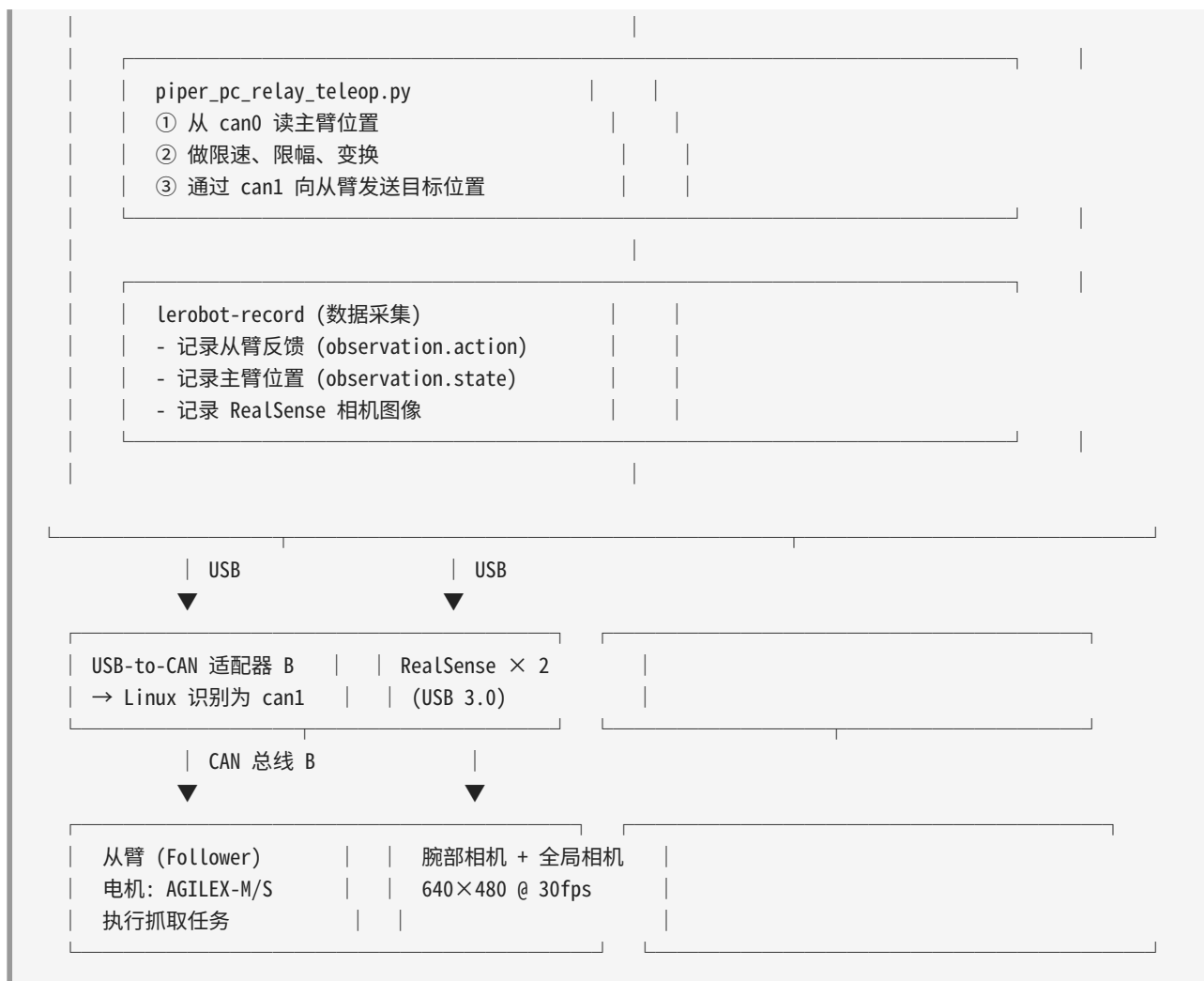
角色	CAN 接口	电机型号	工作模式	操纵方式
主臂 (Leader)	can0	HTDW-5047 × 7	示教输入模式（教学）	人手直接拖拽各关节
从臂 (Follower)	can1	AGILEX-M × 3 + AGILEX-S × 4	位置伺服模式（执行）	PC 通过 CAN 发送目标位置

主臂 (Leader, can0)： - 由人手直接拖拽，不施加驱动力矩 - 电机处于"软"状态（或使用具有高反驱透明性的 HTDW-5047 电机），被推动时阻力极小 - 编码器持续读取关节角度并通过 CAN 发给 PC - PC 中继模式下，PC 读取主臂的 `GetArmJointCtrl()` 来获取位置 - 不执行任何 PC 发来的位置控制命令

从臂 (Follower, can1)： - 接收 PC 发来的关节控制命令 (`0x155~0x157 + 0x159`) - 电机处于"硬"锁定状态（伺服使能），精确跟踪目标位置 - 执行任务操作（抓取、移动、放置等） - 编码器反馈帧 (`0x2A5~0x2A8`) 被 LeRobot 框架记录为 `observation` 数据 - 是策略模型训练和推理的直接操纵对象

2.5.2 硬件连接拓扑





2.5.3 两种遥操作模式对比

PiPER 系统支持两种不同的遥操作架构:

模式一：硬件主从 (MasterSlaveConfig)

主臂 — CAN 总线 —> 从臂 (固件级直接跟随, 不经过 PC)

配置方法:

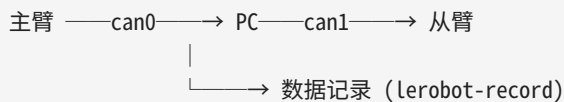
```
# 配置主臂
master = C_PiperInterface_V2("can0")
master.ConnectPort()
master.MasterSlaveConfig(0xFA, 0, 0, 0) # 0xFA = 示教输入臂

# 配置从臂
slave = C_PiperInterface_V2("can0") # 注意: 同一个 CAN 总线
```

```
slave.ConnectPort()
slave.MasterSlaveConfig(0xFC, 0, 0, 0) # 0xFC = 运动输出臂
```

- **上电顺序**：先从臂上电，后主臂上电（约等 5 秒再拖拽主臂测试）
- **linkage_config=0xFA**：将主臂设置为"示教输入模式"——主臂在此模式下主动在 CAN 总线上发送控制帧（0x155~0x159），内容为自身关节位置
- **linkage_config=0xFC**：将从臂设置为"运动输出模式"——从臂在此模式下监听 CAN 总线上的控制帧并执行
- 参数中的三个偏移量（`feedback_offset`，`ctrl_offset`，`linkage_offset`）用于在同一总线上挂多对主从时的 ID 隔离。默认值 0 表示使用标准 ID
- **优点**：极低延迟（~100 微秒），无需 PC 处理器参与实时控制循环
- **缺点**：无法做坐标变换、限速、限位、数据记录；主从必须同型号、同固件版本

模式二：PC 中继（PC Relay Teleop）



配置方法（`record_with_pc_relay.sh`）：

```
# 启动 PC 中继进程（后台运行）
python piper_pc_relay_teleop.py \
  --leader can0 \
  --follower can1 \
  --source control \      # 从主臂读控制帧（GetArmJointCtrl）
  --hz 20 \               # 20 Hz 中继循环
  --speed 100 \           # 跟随速度 100%
  --max-step-deg 10 \     # 每步最大跟随角度 10°
  --include-gripper \     # 同步夹爪
  --execute               # 实际执行（不加就是 dry-run）
```

- PC 作为"信号中继站"：从 `can0` 读取主臂控制帧，处理后通过 `can1` 发送给从臂
- 主臂和从臂分别连接在不同的 CAN 总线上（`can0` ≠ `can1`），完全隔离
- 从臂通过 `lerobot-record` 记录自己的反馈位置和相机图像作为模仿学习数据集
- **优点**：可以在中继过程中施加安全限制（`--max-step-deg` 限速）、坐标变换、自动回零；支持主从不同型号；可以同时记录数据
- **缺点**：多了 Linux 网络栈 + Python 解释器两层延迟（通常 5~50 ms，取决于控制频率）

PC 中继模式读取主臂的逻辑

在 PC 中继的 `read_leader()` 函数中，默认的行为是**优先读控制帧**（`GetArmJointCtrl`），后备读反馈帧（`GetArmJointMsgs`）：

```
def read_leader(leader, source="auto"):
    if source == "control":
        # 读主臂主动发出的控制帧（硬件主从模式下主臂会发送）
        return leader.GetArmJointCtrl(), leader.GetArmGripperCtrl()
    elif source == "feedback":
        # 读主臂的编码器反馈（像正常关节读数一样）
        return leader.GetArmJointMsgs(), leader.GetArmGripperMsgs()
    elif source == "auto":
        # 智能模式：先试控制帧，没有数据流就切到反馈帧
        ...
```

这一设计的背景是：当主臂使用 HTDW-5047 示教电机并被拖拽时，它内部的控制器实际上是在生成"目标位置=当前位置"的伪控制指令，这些控制指令的 CAN ID（0x155~0x159）与 PC 发送给从臂的是相同的 ID 族。PC 中继过程就是在转发这些 ID。

2.6 硬件初始化流程

整套 PiPER 系统从开机到可以开始遥控操作，需要经历明确的初始化步骤。每个步骤都有对应的自动化脚本。

2.6.1 第一步：CAN 口初始化

```
# 脚本名称: 1__init_can.sh
bash ~/piper_sdk/piper_sdk/can_activate.sh can0 1000000
```

此脚本执行以下操作（由 `can_activate.sh` 具体实现）：1. 检查 `ethtool` 和 `can-utils` 是否已安装（不满足则提示安装）2. 检测系统中已识别的 CAN 接口数量 3. 如果多于一个，列出所有接口及其 USB 物理位置，报错退出 4. 如果正好一个，或用户指定了 USB 地址，匹配并激活 5. 执行 `sudo ip link set <iface> type can bitrate 1000000` 设置波特率 6. 执行 `sudo ip link set <iface> up` 启用接口 7. 如果需要则重命名为 `can0`

双臂初始化（`can_config.sh`）：

```
# 预设映射：
# USB bus-info 3-2:1.0 → can_left （主臂 can0）
# USB bus-info 3-1:1.0 → can_right （从臂 can1）
bash can_config.sh
```


2.6.2 第二步：查找相机

```
# 脚本名称: 2__find_camera.sh
python ~/lerobot/src/lerobot/find_cameras.py
```

`lerobot-find-cameras` 使用 `pyrealsense2` 库枚举所有连接的 RealSense 设备，打印出序列号、USB 路径和固件版本。输出示例：

```
Found 2 RealSense device(s):
[0] Serial: 238222076529, USB: 3-4, Firmware: 05.14.00.00
[1] Serial: 142122071524, USB: 3-3, Firmware: 05.14.00.00
```

记下这两个序列号，填入 `camera_config_realsense.env`。哪个是腕部、哪个是全局，可以通过逐个遮挡相机镜头来判断。

2.6.3 第三步：配置相机参数

```
# 脚本名称: 3__set_camera.sh
python ./src/lerobot/camera_prop.py
```

`camera_prop.py` 将相机参数（分辨率、fps、曝光等）固化到 RealSense 固件的持久存储中。设定后即使重启设备，参数也保持不变。

2.6.4 第四步：机械臂使能与模式设置

机械臂上电后不会自动进入伺服模式。需要通过 PiPER SDK 执行以下流程：

```
# 1. 连接到 CAN 口（启动收发线程、查询固件和电机限位）
piper = C_PiperInterface_V2("can0") # 或 "can1"
piper.ConnectPort(can_init=True, piper_init=True, start_thread=True)

# 2. 使能所有电机（发送 0x471 使能命令，最多重试 10 次）
piper.EnablePiper()
# EnablePiper 内部: EnableArm(motor_num=7 → 全部电机)

# 3. 设置控制模式为关节位置控制
piper.ModeCtrl(
    ctrl_mode=0x01,      # CAN 指令控制模式
    move_mode=0x01,      # MOVE J (关节空间)
    move_spd_rate_ctrl=30, # 30% 速度（安全默认）
    is_mit_mode=0x00     # 普通位置-速度模式
)
```

在 LeRobot 框架中，这些步骤被封装在 `PiperFollower.connect()` 中：

```
robot = PiperFollower(config)          # 创建机器人对象
robot.connect(calibrate=True)          # ① 打开 CAN 口
                                       # ② 使能扭矩
                                       # ③ 回零 (parking to INITIALIZE_POSITION)
```

使能到底发生了什么？

当 PC 发送 `EnableArm(7)` 时，CAN 总线上出现：

```
can1  471  [8]  07 02 00 00 00 00 00 00
```

- 07： `motor_num=7`，表示"对所有 7 个电机执行"
- 02： `enable_flag=0x02`，表示"使能"（如果发 `0x00` 则是失能）

机械臂主控板收到此帧后，依次向 6 个关节电机驱动器发送使能指令。每个驱动器启动位置闭环，电机立即从"松软"状态切换为"锁定"状态——你可以听到轻微的电磁锁定声，此时不应再用手拖拽从臂。

失能 (DisablePiper) 释放电机扭矩，关节变松。务必在失能前确保机械臂有支撑（或已到达安全姿态），否则重力会导致机械臂下坠。

2.7 总结：从 CAN 帧到物理运动的全链路

让我们用一张完整的全链路图来串联本章所有概念：





关键对称性： - 数据采集时，`action` = 主臂控制值 (= 从臂收到的目标值) = 从臂反馈值（在足够跟随精度的前提下） - 策略推理时，`action` = 模型输出（归一化 [-100,100]）→ 反归一化 → SDK 原始整数 → CAN 帧 → 从臂电机执行

理解了这一全链路，你就知道了自己写的每一行 Python 代码最终是如何影响物理世界的。

自测题

1. 用 `candump can0` 抓到的 `2A5 [8] 00 01 86 A0 00 00 00 00` 中，`joint_1` 的值是多少毫度？换算成角度是多少？
2. 为什么 PiPER 的控制消息 CAN ID (0x15x) 比反馈消息 CAN ID (0x2Ax) 小？这个设计有什么好处？
3. 如果 `candump can0` 能看到 0x2A5-0x2A7 的反馈帧，但机械臂不响应 Python 代码的 `JointCtrl` 命令，可能是哪些原因？
4. 一个从臂关节的范围是 [-150000, 150000] mdeg。模型输出了一个归一化值 42.5。反归一化后的 SDK 原始整数值是多少？
5. PC 中继模式和硬件主从模式相比，各有什么优劣势？在什么场景下你会选择硬件主从而不是 PC 中继？

第3章 PiPER SDK 深度解析

本章深入剖析 PiPER 机械臂 Python SDK 的软件架构、通信协议和核心 API。我们将从硬件入口 `C_PiperInterface_V2` 出发，逐层解读 CAN 通信、线程模型、数据编解码、控制指令发送等关键机制。阅读本章后，你将具备直接通过 SDK 底层接口操控机械臂的能力，为后续的模仿学习数据采集和策略部署奠定坚实基础。

3.1 C_PiperInterface_V2 概述

3.1.1 文件位置与地位

整个 PiPER SDK 的核心入口类为 `C_PiperInterface_V2`，定义在：

```
piiper_sdk/piiper_sdk/interface/piiper_interface_v2.py (3216 行)
```

这个类是整个软件栈的中枢——所有对机械臂的操作（读关节角度、写目标位置、使能/失能电机、主从配置等）都必须经过这个类。它内部持有三个核心组件，分别负责硬件通信、协议编解码和运动学计算。

3.1.2 类架构总览

`C_PiperInterface_V2` 内部包含以下核心成员：

成员	类型	职责
<code>__arm_can</code>	<code>C_STD_CAN</code>	底层 SocketCAN 硬件端口，负责 CAN 帧的实际收发
<code>__parser</code>	<code>C_PiperParserV2</code>	CAN 帧编解码器，实现 Python 对象与 8 字节 CAN data 之间的双向转换
<code>__piiper_fk</code>	<code>C_PiperForwardKinematics</code>	前向运动学求解器，根据 6 个关节角度计算末端位姿及各连杆位姿

此外，还有参数管理器 `__piiper_param_mag`（`C_PiperParamManager`）用于管理 SDK 层面的软限位参数。

3.1.3 单例模式

`C_PiperInterface_V2` 实现了基于 `can_name` 的单例模式，确保同一个 CAN 口只有一个连接实例。其核心机制如下：

```

_instances = {} # 类级别字典, 以 can_name 为 key 缓存实例
_lock = threading.Lock()

def __new__(cls, can_name="can0", ...):
    key = (can_name)
    with cls._lock:
        if key not in cls._instances:
            instance = super().__new__(cls)
            instance._initialized = False
            cls._instances[key] = instance
    return cls._instances[key]

```

`__init__` 方法通过 `_initialized` 标志位防止重复初始化:

```

def __init__(self, ...):
    if getattr(self, "_initialized", False):
        return # 避免重复初始化
    # ... 执行一次性的初始化逻辑
    self._initialized = True

```

注意: 单例的 `key` 仅由 `can_name` 决定。如果以相同 `can_name` 但不同其他参数 (如 `start_sdk_joint_limit`) 再次调用构造函数, 将返回已缓存的实例, 新参数**不会**生效。这是 SDK 使用中需要特别注意的设计约束。

3.1.4 构造参数详解

```

def __init__(self,
              can_name: str = "can0",          # SocketCAN 接口名
              judge_flag: bool = True,         # 是否在初始化时检查 CAN 口是否正常工作
              can_auto_init: bool = True,      # 是否自动初始化 CAN 口
              dh_is_offset: int = 0x01,        # DH 参数中关节 1-2 是否有 2 度偏置
              start_sdk_joint_limit: bool = False, # 是否启用 SDK 软限位
              start_sdk_gripper_limit: bool = False, # 是否启用夹爪软限位
              start_sdk_fk_cal: bool = False,    # 是否启用前向运动学计算
              logger_level: LogLevel = LogLevel.WARNING,
              log_to_file: bool = False,
              log_file_path = None)

```

各参数说明:

- **can_name**: Linux SocketCAN 接口名称, 通常为 "can0"。对于多臂系统, 可能为 "can1" 等。
- **judge_flag**: 若为 `True`, 构造时会调用 `JudgeCanInfo()` 检查 CAN 口是否存在且已 `ip link set up`。当使用 PCIe 转 CAN 模块时, 由于硬件枚举时机问题, 可能需要设为 `False`。
- **can_auto_init**: 若为 `True`, 构造时自动调用 `C_STD_CAN.Init()` 创建 `can.interface.Bus` 实例 (基于 `python-can` 库)。

- `dh_is_offset`：PiPER 机械臂的 DH 参数中，关节 1-2 存在 2 度的装配偏置。`0x01` 表示应用偏置（`_theta` 使用 172.22 和 102.78 度），`0x00` 表示不应用（使用 174.22 和 100.78 度）。
- `start_sdk_joint_limit`：启用后，所有读取和写入的关节角度值会被 clamp 到 SDK 参数管理器中的软限位范围。这是一种纯软件层面的安全保护。
- `start_sdk_gripper_limit`：与上类似，对夹爪角度值施加软限位。
- `start_sdk_fk_cal`：启用后，每次收到 CAN 帧会额外计算前向运动学（6 个关节各自的位姿矩阵）。计算开销较大，仅在需要各连杆位姿时才启用。

3.2 后台线程详解

3.2.1 为什么需要后台线程

PiPER 机械臂的 CAN 通信采用**推模型（push model）**：机械臂主控板以固定频率（约 200Hz）主动向 CAN 总线上报关节角度、末端位姿、驱动器状态等反馈数据。SDK 必须持续从 SocketCAN 读取这些帧，否则内核缓冲区会溢出导致数据丢失。

为此，SDK 在 `ConnectPort()` 中启动两个 daemon 线程。

3.2.2 ReadCan 线程

`ReadCan` 是核心读取线程，在 `ConnectPort()` 中以内联函数定义并启动：

```
def ReadCan():
    self.logger.info("[ReadCan] ReadCan Thread started")
    while not self.__read_can_stop_event.is_set():
        self.__fps_counter.increment("CanMonitor")
        try:
            read_status = self.__arm_can.ReadCanMessage()
        except can.CanOperationError:
            self.logger.error("[ReadCan] CAN is closed, stop ReadCan thread")
            break
        except Exception as e:
            self.logger.error("[ReadCan] 'error: %s'", e)
            break
```

关键流程：

1. **循环条件**：通过 `threading.Event`（`__read_can_stop_event`）控制线程退出。
2. **FPS 计数**：每次迭代递增 `CanMonitor` 计数器，供帧率监测使用。

3. `self.__arm_can.ReadCanMessage()`：调用底层 `C_STD_CAN` 的读取方法。该方法内部调用 `self.bus.recv(timeout=0.001)` 从 SocketCAN 读取一帧。读取成功后，自动调用注册的回调函数 `self.ParseCANFrame`（在 `C_STD_CAN` 构造时通过 `callback_function` 参数传入）。

4. **异常处理**：若 CAN 口被关闭（`CanOperationError`），线程优雅退出。

线程属性：`daemon=True`，这意味着主程序退出时该线程会随进程终止，不会阻止程序退出。

3.2.3 CanMonitor 线程

`CanMonitor` 是 CAN 帧率监测线程：

```
def CanMonitor():
    self.logger.info("[ReadCan] CanMonitor Thread started")
    while not self.__can_monitor_stop_event.is_set():
        try:
            self.__CanMonitor()
        except Exception as e:
            self.logger.error("CanMonitor() exception: %s", e)
            break
    self.__can_monitor_stop_event.wait(0.05) # 每 50ms 检查一次
```

核心逻辑在 `__CanMonitor()` 中：

```
def __CanMonitor(self):
    if self.__q_can_fps.full():
        self.__q_can_fps.get() # 队列满时丢弃最旧数据
    self.__q_can_fps.put(self.GetCanFps()) # 压入当前帧率
    with self.__is_ok_mtx:
        if self.__q_can_fps.full() and all(x == 0 for x in self.__q_can_fps.queue):
            self.__is_ok = False # 连续 5 次帧率均为 0 → CAN 总线卡死
        else:
            self.__is_ok = True
```

机制说明：

- 使用一个大小为 5 的 `Queue` 缓存最近 5 次帧率采样（每次间隔 50ms，共 250ms 窗口）。
- 若队列满且 5 次帧率**全部为零**，判定 CAN 总线卡死（`__is_ok = False`）。
- 上层可通过 `isOk()` 方法查询此状态，决定是否触发错误处理。

3.2.4 线程安全机制

SDK 为每个数据缓存配备了独立的 `threading.Lock`：

```
self.__arm_joint_msgs_mtx = threading.Lock() # 关节角度缓存锁
self.__arm_gripper_msgs_mtx = threading.Lock() # 夹爪缓存锁
self.__arm_status_mtx = threading.Lock() # 状态缓存锁
# ... 共 20+ 个锁
```

这种**细粒度锁**设计的优势： - ReadCan 线程在回调中写数据时，只锁住正在更新的那个缓存，不影响其他缓存的读写。 - 用户线程调用 getter 方法时，只需获取对应缓存的锁，不会因为其他缓存的更新而阻塞。

3.3 核心读方法详解

本节详细介绍 C_PiperInterface_V2 提供的各类数据读取方法。所有 getter 方法均遵循统一的线程安全模式：获取对应锁 → 更新 Hz 字段 → 返回数据引用。

3.3.1 GetArmJointMsgs() — 关节角度反馈

```
def GetArmJointMsgs(self):
    with self.__arm_joint_msgs_mtx:
        self.__arm_joint_msgs.Hz = self.__fps_counter.cal_average(
            self.__fps_counter.get_fps('ArmJoint_12'),
            self.__fps_counter.get_fps('ArmJoint_34'),
            self.__fps_counter.get_fps('ArmJoint_56'))
    return self.__arm_joint_msgs
```

返回值类型： self.ArmJoint —— 定义在 C_PiperInterface_V2 内部的嵌套类。

```
class ArmJoint():
    def __init__(self):
        self.time_stamp: float = 0    # epoch 秒级时间戳
        self.Hz: float = 0            # 数据流频率
        self.joint_state = ArmMsgFeedBackJointStates()
```

joint_state (ArmMsgFeedBackJointStates) 的字段：

字段	类型	单位	说明
joint_1	int	0.001 度（毫度）	关节 1 当前角度
joint_2	int	0.001 度	关节 2 当前角度
joint_3	int	0.001 度	关节 3 当前角度
joint_4	int	0.001 度	关节 4 当前角度
joint_5	int	0.001 度	关节 5 当前角度
joint_6	int	0.001 度	关节 6 当前角度

数据来源：CAN 帧 0x2A5 (joint_1, joint_2)、0x2A6 (joint_3, joint_4)、0x2A7 (joint_5, joint_6)。每帧 8 字节，每关节占 4 字节小端 int32。

编解码细节（在 C_PiperParserV2.DecodeMessage() 中）：

```
# 以 0x2A5 为例
msg.arm_joint_feedback.joint_1 = self.ConvertToNegative_32bit(
    self.ConvertBytesToInt(can_data, 0, 4)) # Byte 0-3 → int32
msg.arm_joint_feedback.joint_2 = self.ConvertToNegative_32bit(
    self.ConvertBytesToInt(can_data, 4, 8)) # Byte 4-7 → int32
```

ConvertToNegative_32bit 处理有符号整数的补码表示，确保负角度值正确解码。

使用示例：

```
joint_msgs = piper.GetArmJointMsgs()
print(f"关节角度: J1={joint_msgs.joint_state.joint_1 * 0.001:.2f}°")
print(f"数据频率: {joint_msgs.Hz:.1f} Hz")
```

3.3.2 GetArmGripperMsgs() — 夹爪反馈

```
def GetArmGripperMsgs(self):
    with self.__arm_gripper_msgs_mtx:
        self.__arm_gripper_msgs.Hz = self.__fps_counter.get_fps('ArmGripper')
    return self.__arm_gripper_msgs
```

返回值类型： self.ArmGripper 。

```
class ArmGripper():
    def __init__(self):
        self.time_stamp: float = 0
        self.Hz: float = 0
        self.gripper_state = ArmMsgFeedBackGripper()
```

gripper_state (ArmMsgFeedBackGripper) 的字段：

字段	类型	单位	说明
grippers_angle	int	0.001 度	夹爪当前角度
grippers_effort	int	0.001 N/m	夹爪当前力矩
status_code	int	位掩码	夹爪状态码（见下文）

status_code 是一个位掩码字段，通过 foc_status 属性提供分解后的布尔标志：

位	属性名	说明
bit[0]	voltage_too_low	电源电压是否过低
bit[1]	motor_overheating	电机是否过温
bit[2]	driver_overcurrent	驱动器是否过流
bit[3]	driver_overheating	驱动器是否过温
bit[4]	sensor_status	传感器状态
bit[5]	driver_error_status	驱动器错误状态
bit[6]	driver_enable_status	驱动器使能状态（1=使能）
bit[7]	homing_status	回零状态（1=已回零）

状态码的设置通过 `@status_code.setter` 自动解析：

```
@status_code.setter
def status_code(self, value: int):
    self._status_code = value
    self.foc_status.voltage_too_low      = bool(value & (1 << 0))
    self.foc_status.motor_overheating    = bool(value & (1 << 1))
    # ...
```

数据来源：CAN 帧 0x2A8。

CAN 帧结构（共 8 字节）：

Byte	内容	类型
0-3	grippers_angle	int32, 小端
4-5	grippers_effort	int16, 小端
6	status_code	uint8
7	保留	—

3.3.3 GetArmJointCtrl() — 控制指令读取（主从模式专用）

重要概念区分： `GetArmJointMsgs()` 读取的是机械臂**实际反馈**的关节角度，而 `GetArmJointCtrl()` 读取的是 **CAN 总线上的控制指令帧**。在主从模式下，教学臂（主臂）会主动发送关节控制指令（CAN ID 0x155/0x156/0x157），运动臂（从臂）可以通过此方法**监听**主臂发送的目标角度。这是实现主从跟随的关键机制。

```
def GetArmJointCtrl(self):
    with self.__arm_joint_ctrl_msgs_mtx:
        self.__arm_joint_ctrl_msgs.Hz = self.__fps_counter.cal_average(
            self.__fps_counter.get_fps('ArmJointCtrl_12'),
            self.__fps_counter.get_fps('ArmJointCtrl_34'),
            self.__fps_counter.get_fps('ArmJointCtrl_56'))
    return self.__arm_joint_ctrl_msgs
```

返回值类型： `self.ArmJointCtrl`，其 `joint_ctrl` 字段为 `ArmMsgJointCtrl`，结构同 `ArmMsgFeedBackJointStates`（`joint_1` 到 `joint_6`，单位 0.001 度）。

数据来源： CAN 帧 `0x155`、`0x156`、`0x157`。注意这两个 ID 系列在不同场景下的含义：

CAN ID	发送方	含义
0x155/156/157	主臂（教学臂）	主臂发送的控制指令
0x2A5/2A6/2A7	任意臂	当前关节的实际反馈角度

3.3.4 GetArmGripperCtrl() — 夹爪控制指令读取

与 `GetArmJointCtrl()` 类似，读取主臂发送的夹爪控制指令。

```
def GetArmGripperCtrl(self):
    with self.__arm_gripper_ctrl_msgs_mtx:
        self.__arm_gripper_ctrl_msgs.Hz = self.__fps_counter.get_fps("ArmGripperCtrl")
    return self.__arm_gripper_ctrl_msgs
```

返回值类型： `self.ArmGripperCtrl`，其 `gripper_ctrl` 字段为 `ArmMsgGripperCtrl`，包含 `grippers_angle`（int32）、`grippers_effort`（uint16）、`status_code`（uint8）、`set_zero`（uint8）。

数据来源： CAN 帧 `0x159`。

3.3.5 其他 Getter 方法一览

GetArmEndPoseMsgs()

获取机械臂末端位姿（欧拉角表示）。返回 `self.ArmEndPose`。

字段	单位
<code>end_pose.X_axis</code>	0.001 mm
<code>end_pose.Y_axis</code>	0.001 mm
<code>end_pose.Z_axis</code>	0.001 mm

字段	单位
end_pose.RX_axis	0.001 度
end_pose.RY_axis	0.001 度
end_pose.RZ_axis	0.001 度

数据来源：CAN 帧 0x2A2 / 0x2A3 / 0x2A4（每帧传 2 个字段）。

GetArmStatus()

获取机械臂综合状态。返回 self.ArmStatus，其 arm_status 字段 (ArmMsgFeedbackStatus) 包含：

子字段	类型	说明
ctrl_mode	CtrlMode 枚举	控制模式（待机/CAN指令/示教/以太网/WiFi/遥控/联动输入/离线轨迹）
arm_status	ArmStatus 枚举	机械臂状态（正常/急停/无解/奇异点/超限/通信异常/碰撞等）
mode_feed	ModeFeed 枚举	当前运动模式（P/J/L/C/M）
teach_status	TeachingState 枚举	示教状态
motion_status	MotionStatus 枚举	是否到达目标位置
err_code	int (16位)	故障码（包含各关节超限位和各关节通信异常标志）

数据来源：CAN 帧 0x2A1。

GetArmHighSpdInfoMsgs()

获取 6 个关节电机的高速反馈信息。返回 self.ArmMotorDriverInfoHighSpd，包含 motor_1 到 motor_6（每个为 ArmMsgFeedbackHighSpd 实例）。

每电机字段：

字段	说明
motor_speed	电机转速
current	当前电流
pos	当前位置

字段	说明
effort	计算力矩（通过 <code>cal_effort()</code> 方法）

数据来源：CAN 帧 0x251 - 0x256。

GetArmLowSpdInfoMsgs()

获取 6 个关节电机的低速反馈信息。返回 `self.ArmMotorDriverInfoLowSpd`。

每电机字段：

字段	单位	说明
vol	0.1 V	驱动器电压
foc_temp	1 °C	驱动器温度
motor_temp	1 °C	电机温度
foc_status_code	位掩码	驱动器状态（含电源/过温/过流/碰撞/使能/堵转标志）
bus_current	0.001 A	母线电流

数据来源：CAN 帧 0x261 - 0x266。

foc_status 各 bit 含义：

bit	属性名	说明
0	voltage_too_low	电压过低
1	motor_overheating	电机过温
2	driver_overcurrent	驱动器过流
3	driver_overheating	驱动器过温
4	collision_status	碰撞保护状态
5	driver_error_status	驱动器错误
6	driver_enable_status	使能状态（1=已使能）
7	stall_status	堵转保护状态

此方法在 `GetArmEnableStatus()` 中被用于检查当前使能状态。

GetFK()

获取前向运动学计算结果。返回 6 个 6 元素列表的列表 `[[x,y,z,rx,ry,rz]_1 ... [x,y,z,rx,ry,rz]_6]`，分别表示 1-6 号关节坐标系相对于 base_link 的位姿。需要 `start_sdk_fk_cal=True` 才会更新。

```
def GetFK(self, mode: Literal["feedback", "control"] = "feedback"):
    if mode == "feedback":
        with self.__piper_feedback_fk_mtx:
            return self.__link_feedback_fk
    elif mode == "control":
        with self.__piper_ctrl_fk_mtx:
            return self.__link_ctrl_fk
```

- "feedback" 模式：基于实际反馈的关节角度计算。
- "control" 模式：基于控制指令中的目标关节角度计算（主从模式专用）。

3.4 核心写方法详解

发送控制指令遵循统一的流程：**构造消息对象** → **编码为 CAN 帧** → **调用 SendCanMessage** → **检查发送状态**。

3.4.1 JointCtrl() — 关节角度控制

```
def JointCtrl(self,
    joint_1: int, joint_2: int,
    joint_3: int, joint_4: int,
    joint_5: int, joint_6: int):
```

参数：6 个关节的目标角度，单位均为 **0.001 度（毫度）**。例如 `joint_1=45000` 表示目标 45.0 度。

关节限位参考：

关节	最小角度（度）	最大角度（度）	对应毫度范围
Joint 1	-150.0	150.0	-150000 ~ 150000
Joint 2	-2.0	180.0	-2000 ~ 180000
Joint 3	-170.0	2.0	-170000 ~ 2000
Joint 4	-100.0	100.0	-100000 ~ 100000
Joint 5	-70.0	70.0	-70000 ~ 70000

关节	最小角度（度）	最大角度（度）	对应毫度范围
Joint 6	-120.0	120.0	-120000 ~ 120000

内部执行流程：

1. **SDK 软限位 clamp**：若启用了 `start_sdk_joint_limit`，先通过 `__CalJointSDKLimit()` 将每个角度 clamp 到软限位范围内。
2. 分发到三个私有方法：
3. `__JointCtrl_12(joint_1, joint_2)` → CAN ID `0x155`
4. `__JointCtrl_34(joint_3, joint_4)` → CAN ID `0x156`
5. `__JointCtrl_56(joint_5, joint_6)` → CAN ID `0x157`
6. **每帧编码格式**（以 `__JointCtrl_12` 为例）：

```
def __JointCtrl_12(self, joint_1, joint_2):
    tx_can = Message()
    joint_ctrl = ArmMsgJointCtrl(joint_1=joint_1, joint_2=joint_2)
    msg = PiperMessage(type_=ArmMsgType.PiperMsgJointCtrl_12, arm_joint_ctrl=joint_ctrl)
    self.__parser.EncodeMessage(msg, tx_can) # Python 对象 → 8 字节 CAN data
    feedback = self.__arm_can.SendCanMessage(tx_can.arbitration_id, tx_can.data)
    if feedback is not self.__arm_can.CAN_STATUS.SEND_MESSAGE_SUCCESS:
        self.logger.error("JointCtrl_J12 send failed: SendCanMessage(%s)", feedback)
```

CAN 帧编码细节（在 `C_PiperParserV2.EncodeMessage()` 中）：

```
elif(msg_type_ == ArmMsgType.PiperMsgJointCtrl_12):
    tx_can_frame.data = self.ConvertToList_32bit(msg.arm_joint_ctrl.joint_1) + \
        self.ConvertToList_32bit(msg.arm_joint_ctrl.joint_2)
```

`ConvertToList_32bit` 将 `int32` 转为 4 字节小端列表。因此 `0x155` 的 8 字节布局为：

```
[joint_1 byte0] [joint_1 byte1] [joint_1 byte2] [joint_1 byte3]
[joint_2 byte0] [joint_2 byte1] [joint_2 byte2] [joint_2 byte3]
```

注意事项：

- `JointCtrl()` 会发送 3 个 CAN 帧（`0x155`, `0x156`, `0x157`），每个帧独立发送。
- 发送前必须通过 `ModeCtrl()` 设置为关节控制模式（`move_mode=0x01`）。
- 发送前必须通过 `EnablePiper()` 使能电机。

3.4.2 GripperCtrl() — 夹爪控制

```
def GripperCtrl(self,
    gripper_angle: int = 0,          # 单位 0.001 度
    gripper_effort: int = 0,         # 单位 0.001 N/m, 范围 0-5000
    gripper_code: Literal[0x00, 0x01, 0x02, 0x03] = 0,
    set_zero: Literal[0x00, 0xAE] = 0):
```

参数详解:

参数	取值	含义
<code>gripper_code=0x00</code>	失能	夹爪电机停止输出力矩
<code>gripper_code=0x01</code>	使能	夹爪电机使能
<code>gripper_code=0x02</code>	失能并清除错误	先清除错误标志再失能
<code>gripper_code=0x03</code>	使能并清除错误	先清除错误标志再使能
<code>set_zero=0x00</code>	无效	不执行零点设置
<code>set_zero=0xAE</code>	设零点	将当前位置设为夹爪零点

CAN ID: 0x159

CAN 帧编码 (8 字节) :

```
tx_can_frame.data = self.ConvertToList_32bit(gripper_angle) + \
    self.ConvertToList_16bit(gripper_effort, False) + \
    self.ConvertToList_8bit(gripper_code, False) + \
    self.ConvertToList_8bit(set_zero, False)
```

Byte	内容
0-3	<code>gripper_angle</code> (int32, 小端)
4-5	<code>gripper_effort</code> (uint16, 小端)
6	<code>gripper_code</code> (uint8)
7	<code>set_zero</code> (uint8)

3.4.3 ModeCtrl() — 模式控制

```
def ModeCtrl(self,
    ctrl_mode: Literal[0x00, 0x01] = 0x01,
    move_mode: Literal[0x00, 0x01, 0x02, 0x03] = 0x01,
```



```
move_spd_rate_ctrl: int = 50,
is_mit_mode: Literal[0x00, 0xAD, 0xFF] = 0x00):
```

参数详解:

参数	取值	含义
ctrl_mode=0x00	待机模式	机械臂不响应控制指令
ctrl_mode=0x01	CAN 指令控制模式	通过 CAN 总线发送控制指令
move_mode=0x00	MOVE P	末端位姿控制（笛卡尔空间）
move_mode=0x01	MOVE J	关节空间控制
move_mode=0x02	MOVE L	直线路径控制
move_mode=0x03	MOVE C	圆弧路径控制
move_spd_rate_ctrl	0-100	运动速度百分比
is_mit_mode=0x00	位置速度模式	标准 PID 位置控制
is_mit_mode=0xAD	MIT 模式	阻抗/导纳控制模式

ModeCtrl() 实际是 MotionCtrl_2() 的简化封装，内部直接调用：

```
self.MotionCtrl_2(ctrl_mode, move_mode, move_spd_rate_ctrl, is_mit_mode)
```

CAN ID: 0x151

CAN 帧编码（8 字节）：

```
tx_can_frame.data = [ctrl_mode, move_mode, move_spd_rate_ctrl, is_mit_mode, 0x00, 0x00, 0x00, 0x00]
```

3.4.4 MasterSlaveConfig() — 主从模式配置

```
def MasterSlaveConfig(self,
    linkage_config: int,
    feedback_offset: int,
    ctrl_offset: int,
    linkage_offset: int):
```

参数详解:

参数	取值	含义
linkage_config=0xFA	示教输入臂	该臂作为教学臂，主动发送控制指令
linkage_config=0xFC	运动输出臂	该臂作为运动臂，接收并执行控制指令
feedback_offset=0x00	—	反馈 ID 不偏移（基 ID 为 0x2Ax）
feedback_offset=0x10	—	反馈 ID 偏移到 0x2Bx
feedback_offset=0x20	—	反馈 ID 偏移到 0x2Cx
ctrl_offset=0x00	—	控制指令基 ID 不偏移（0x15x）
ctrl_offset=0x10	—	控制指令基 ID 偏移到 0x16x
ctrl_offset=0x20	—	控制指令基 ID 偏移到 0x17x
linkage_offset=0x00	—	联动目标地址不偏移
linkage_offset=0x10	—	联动目标地址偏移到 0x16x
linkage_offset=0x20	—	联动目标地址偏移到 0x17x

CAN ID: 0x470

偏移机制的作用：在多臂系统中，避免不同机械臂的 CAN ID 冲突。教学臂的控制指令（原本 0x155-0x157）在加上偏移后变为 0x165-0x167，运动臂的反馈 ID 也随之偏移。

3.4.5 其他写方法一览

EndPoseCtrl()

笛卡尔空间末端位姿控制，发送欧拉角位姿指令。内部拆分为 3 帧：

调用	CAN ID	发送内容
__CartesianCtrl_XY(X, Y)	0x152	X, Y 坐标 ($\text{int32} \times 2$)
__CartesianCtrl_ZRX(Z, RX)	0x153	Z, RX 坐标 ($\text{int32} \times 2$)
__CartesianCtrl_RYRZ(RY, RZ)	0x154	RY, RZ 坐标 ($\text{int32} \times 2$)

使用前需通过 ModeCtrl(move_mode=0x00) 切换为 MOVE P 模式。

EmergencyStop()

发送急停/恢复指令（CAN ID 0x150）。

```
def EmergencyStop(self, emergency_stop=0x01): # 0x01=急停, 0x02=恢复
    self.MotionCtrl_1(emergency_stop, 0x00, 0x00)
```

3.5 ConnectPort 和初始化流程

3.5.1 ConnectPort 完整流程

ConnectPort() 是机械臂连接的入口函数，完整的调用链为：

```
ConnectPort(can_init, piper_init, start_thread)
├── [can_init 或 首次连接] → __arm_can.Init()
│   └── C_STD_CAN.Init() → can.interface.Bus(channel, bustype, bitrate)
│       └── 创建 SocketCAN 总线实例
├── [start_thread] → 启动 ReadCan 线程 (daemon)
├── [start_thread] → 启动 CanMonitor 线程 (daemon)
├── [start_thread] → __fps_counter.start() (启动帧率统计)
└── [piper_init] → PiperInit()
    ├── SearchAllMotorMaxAngleSpd() → 查询全部电机角度/速度限制
    ├── SearchAllMotorMaxAccLimit() → 查询全部电机加速度限制
    └── SearchPiperFirmwareVersion() → 查询固件版本
```

```
def ConnectPort(self, can_init=False, piper_init=True, start_thread=True):
    # 步骤 1: CAN 初始化
    if(can_init or not self.__connected):
        init_status = self.__arm_can.Init()

    # 步骤 2: 设置连接状态
    with self.__lock:
        if self.__connected:
            return # 防止重复连接
        self.__connected = True
        self.__read_can_stop_event.clear()
        self.__can_monitor_stop_event.clear()

    # 步骤 3: 启动后台线程
    if start_thread:
        self.__can_deal_th = threading.Thread(target=ReadCan, daemon=True)
        self.__can_deal_th.start()
        self.__can_monitor_th = threading.Thread(target=CanMonitor, daemon=True)
        self.__can_monitor_th.start()
        self.__fps_counter.start()

    # 步骤 4: 机械臂初始化查询
    if piper_init and self.__arm_can is not None:
        self.PiperInit()
```

3.5.2 PiperInit — 初始化查询序列

```
def PiperInit(self):
    self.SearchAllMotorMaxAngleSpd()      # 查询 6 个电机各 1 次
    self.SearchAllMotorMaxAccLimit()      # 查询 6 个电机各 1 次
    self.SearchPiperFirmwareVersion()     # 查询固件版本
```

`SearchAllMotorMaxAngleSpd()` 和 `SearchAllMotorMaxAccLimit()` 分别对电机 1-6 逐一发送查询指令（CAN ID 0x472），区别在于 `search_content` 参数：

```
def SearchAllMotorMaxAngleSpd(self):
    for motor_num in [1, 2, 3, 4, 5, 6]:
        self.SearchMotorMaxAngleSpdAccLimit(motor_num, 0x01) # 查询角度/速度限制

def SearchAllMotorMaxAccLimit(self):
    for motor_num in [1, 2, 3, 4, 5, 6]:
        self.SearchMotorMaxAngleSpdAccLimit(motor_num, 0x02) # 查询加速度限制
```

这三个查询之所以必要，是因为：

- **角度限制**：获取每个电机的最大/最小角度限制和最大关节速度，用于后续的软限位保护。
- **加速度限制**：了解每个电机的最大加速度能力，用于运动规划。
- **固件版本**：获取机械臂主控板的固件版本字符串（如 "S-V1.7.4"），用于兼容性检查。

3.5.3 DisconnectPort — 断开流程

```
def DisconnectPort(self, thread_timeout=0.1):
    with self.__lock:
        self.__connected = False
        self.__read_can_stop_event.set() # 通知 ReadCan 线程退出

    # 等待 ReadCan 线程退出（超时 0.1 秒）
    if self.__can_deal_th and self.__can_deal_th.is_alive():
        self.__can_deal_th.join(timeout=thread_timeout)

    # 关闭 CAN 端口
    self.__arm_can.Close()
```

CanMonitor 线程通过 `__can_monitor_stop_event` 自然退出（每 50ms 检查一次），不需要显式 join。

3.6 Enable/Disable 流程

3.6.1 EnablePiper()

```
def EnablePiper(self) -> bool:
    enable_list = self.GetArmEnableStatus() # 读取当前使能状态
    self.EnableArm(7)                       # 发送使能指令
    return all(enable_list)                 # 返回使能前的状态
```

调用链：

1. `GetArmEnableStatus()` → 从 `GetArmLowSpdInfoMsgs()` 读取 6 个电机的 `foc_status.driver_enable_status`，返回布尔值列表。
2. `EnableArm(7)` → 构造 `ArmMsgMotorEnableDisableConfig(motor_num=7, enable_flag=0x02)`，编码后发送 CAN 帧 0x471。

3.6.2 DisablePiper()

```
def DisablePiper(self) -> bool:
    enable_list = self.GetArmEnableStatus()
    self.DisableArm(7)
    return any(enable_list) # 如果有任何电机之前是使能的，返回 True
```

`DisableArm(7)` 发送 `enable_flag=0x01`，其他逻辑与 `EnableArm` 相同。

3.6.3 motor_num=7 的含义

CAN 协议中，`motor_num` 取值 1-6 分别代表单独控制 1-6 号关节电机，取值 7 代表全部电机。这是一个 SDK 层面的约定（不是标准 CANopen 协议）。对应的 CAN 帧编码：

```
# EnableArm(7):
tx_can_frame.data = [0x07, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00]
#               ↑      ↑
#               motor_num=7 enable_flag=0x02 (使能)
```

3.7 消息类型系统

3.7.1 系统分层架构

PIPER SDK 的消息处理分为四个层次：

C_PiperInterface_V2	← 应用层（读写 API）
PiperMessage	← 消息容器（统一消息格式）
C_PiperParserV2.EncodeMessage/DecodeMessage	← 编解码层（Python ↔ CAN bytes）
C_STD_CAN.SendCanMessage/ReadCanMessage	← 硬件层（SocketCAN）

3.7.2 PiperMessage — 通用消息容器

`PiperMessage` 类（`piper_msgs/msg_v2/arm_messages.py`）是整个 SDK 的统一消息容器。它聚合了所有可能的反馈消息和发送消息类型：

```
class PiperMessage:
    def __init__(self, type_=None, time_stamp=0.0,
                 arm_status_msgs=None,          # 状态反馈
                 arm_joint_feedback=None,        # 关节角度反馈
                 gripper_feedback=None,          # 夹爪反馈
                 arm_end_pose=None,              # 末端位姿反馈
                 arm_high_spd_feedback=None,     # 高速驱动器反馈 ×6
                 arm_low_spd_feedback=None,      # 低速驱动器反馈 ×6
                 arm_motion_ctrl_1=None,         # 运动控制指令 1
                 arm_motion_ctrl_2=None,         # 运动控制指令 2
                 arm_joint_ctrl=None,            # 关节控制指令
                 arm_gripper_ctrl=None,          # 夹爪控制指令
                 arm_motor_enable=None,          # 电机使能/失能指令
                 # ... 共 30+ 种消息类型
                ):
```

在解码阶段，`ParseCANFrame` 回调用同一个 `PiperMessage` 实例承载解码后的数据，然后通过 `type_` 字段路由到对应的 `__Update*` 方法更新内部缓存。

3.7.3 ArmMsgType — 消息类型枚举

`ArmMsgType`（`piper_msgs/msg_v2/arm_msg_type.py`）是一个枚举类，定义了所有可能的消息类型。其枚举值从 `auto()` 递增：

```
class ArmMsgType(Enum):
    PiperMsgUnkonwn = 0x00
    PiperMsgStatusFeedback = auto()      # → 1
    PiperMsgEndPoseFeedback_1 = auto()    # → 2
    PiperMsgEndPoseFeedback_2 = auto()    # → 3
    # ...
    PiperMsgJointFeedBack_12 = auto()     # → 5
    PiperMsgJointFeedBack_34 = auto()     # → 6
    PiperMsgJointFeedBack_56 = auto()     # → 7
```

```
PiperMsgGripperFeedBack = auto()      # → 8
# ... (transmit)
PiperMsgMotionCtrl_1 = auto()          # → ...
PiperMsgJointCtrl_12 = auto()
# ... 共 60+ 个枚举值
```

3.7.4 CanIDPiper 和 ArmMessageMapping — CAN ID 映射

CanIDPiper（`piper_msgs/msg_v2/can_id.py`）定义了所有 CAN ID 常量：

```
class CanIDPiper(Enum):
    ARM_STATUS_FEEDBACK = 0x2A1        # 状态反馈
    ARM_END_POSE_FEEDBACK_1 = 0x2A2     # 末端位姿反馈 1
    ARM_JOINT_FEEDBACK_12 = 0x2A5       # 关节反馈 12
    ARM_GRIPPER_FEEDBACK = 0x2A8        # 夹爪反馈
    ARM_MOTION_CTRL_1 = 0x150           # 运动控制 1
    ARM_MOTION_CTRL_2 = 0x151           # 运动控制 2
    ARM_JOINT_CTRL_12 = 0x155           # 关节控制 12
    ARM_GRIPPER_CTRL = 0x159            # 夹爪控制
    ARM_MOTOR_ENABLE_DISABLE_CONFIG = 0x471 # 使能/失能
    ARM_FIRMWARE_READ = 0x4AF           # 固件读取
    # ... 共 40+ 个
```

ArmMessageMapping 提供 CAN ID 与 ArmMsgType 的双向映射（`id_to_type_mapping` 和 `type_to_id_mapping`）：

```
class ArmMessageMapping:
    id_to_type_mapping = {
        0x2A1: ArmMsgType.PiperMsgStatusFeedback,
        0x2A5: ArmMsgType.PiperMsgJointFeedBack_12,
        0x155: ArmMsgType.PiperMsgJointCtrl_12,
        # ...
    }
    type_to_id_mapping = {v: k for k, v in id_to_type_mapping.items()}
```

这种设计使得编解码时可以通过 CAN ID 快速定位消息类型，反之亦然。

3.7.5 反馈消息 vs 发送消息

消息类型在代码组织上分为两个目录：

目录	内容
<code>piper_msgs/msg_v2/feedback/</code>	机械臂 → 主控的反馈消息
<code>piper_msgs/msg_v2/transmit/</code>	主控 → 机械臂的控制/配置指令

部分关键文件对照：

反馈类	CAN ID	发送类	CAN ID
ArmMsgFeedBackJointStates	0x2A5-7	ArmMsgJointCtrl	0x155-7
ArmMsgFeedBackGripper	0x2A8	ArmMsgGripperCtrl	0x159
ArmMsgFeedbackStatus	0x2A1	ArmMsgMotionCtrl_1	0x150
ArmMsgFeedBackEndPose	0x2A2-4	ArmMsgMotionCtrl_2	0x151

3.7.6 编解码完整流程

解码（CAN → Python 对象）

```
C_STD_CAN.ReadCanMessage()  
↳ bus.recv(timeout=0.001) → can.Message (arbitration_id + data)  
↳ callback_function = ParseCANFrame  
    ↳ parser.DecodeMessage(rx_can_frame, msg)  
        ↳ 根据 arbitration_id 匹配 if/elif 分支  
        ↳ 通过 ConvertBytesToInt + ConvertToNegative_XXbit 解码各字段  
        ↳ 设置 msg.type_  
        ↳ 返回 True (ID 已识别) / False (未知 ID)  
    ↳ UpdateArmJointState(msg) [以关节状态为例]  
        ↳ 加锁 → 更新 __arm_joint_msgs.joint_state.joint_X → 释放锁
```

编码（Python 对象 → CAN）

```
C_PiperInterface_V2.JointCtrl(j1..j6)  
↳ ArmMsgJointCtrl(joint_1=j1, joint_2=j2) # 创建消息对象  
↳ PiperMessage(type_=PiperMsgJointCtrl_12, arm_joint_ctrl=joint_ctrl)  
↳ parser.EncodeMessage(msg, tx_can_frame)  
    ↳ 根据 msg.type_ 匹配 if/elif 分支  
    ↳ ArmMessageMapping.get_mapping(msg_type=type_) → arbitration_id  
    ↳ ConvertToList_32bit / 16bit / 8bit → 8 字节 data list  
    ↳ 返回 True  
↳ __arm_can.SendCanMessage(tx_can.arbitration_id, tx_can.data)  
    ↳ bus.send(Message(arbitration_id=id, data=data))
```

3.8 SDK 数值单位总结

PiPER SDK 广泛使用**整数表示法**来避免浮点精度问题。下表列出所有关键物理量在 SDK 内部的表示单位：

物理量	SDK 内部单位	换算公式	示例
关节角度	0.001 度（毫度）	<code>angle_deg = raw / 1000</code>	<code>joint_1=45000 = 45.0 deg</code>
夹爪角度	0.001 度	<code>gripper_deg = raw / 1000</code>	<code>gripper_angle=30000 = 30.0 deg</code>
夹爪力矩	0.001 N/m	<code>effort_nm = raw / 1000</code>	<code>effort=1000 = 1.0 N/m</code>
末端位置 (X, Y, Z)	0.001 mm	<code>pos_mm = raw / 1000</code>	<code>x=500000 = 500 mm</code>
末端姿态 (RX, RY, RZ)	0.001 度	<code>euler_deg = raw / 1000</code>	<code>rx=90000 = 90.0 deg</code>
电机限制角度	0.1 度	<code>limit_deg = raw / 10</code>	<code>max_angle=1500 = 150.0 deg</code>
关节最大速度	0.001 rad/s	<code>vel_rads = raw / 1000</code>	<code>max_spd=3000 = 3.0 rad/s</code>
关节最大加速度	0.01 rad/s ²	<code>acc_rads2 = raw / 100</code>	<code>max_acc=500 = 5.0 rad/s²</code>
末端线速度	0.001 m/s	<code>vel_ms = raw / 1000</code>	<code>lin_vel=500 = 0.5 m/s</code>
末端角速度	0.001 rad/s	<code>ang_vel = raw / 1000</code>	<code>ang_vel=1000 = 1.0 rad/s</code>
末端线加速度	0.001 m/s ²	<code>lin_acc = raw / 1000</code>	
末端角加速度	0.001 rad/s ²	<code>ang_acc = raw / 1000</code>	
驱动器电压	0.1 V	<code>vol = raw / 10</code>	<code>vol=240 = 24.0 V</code>
驱动器/电机温度	1 degC	—	<code>foc_temp=45 = 45 degC</code>
母线电流	0.001 A	<code>current_A = raw / 1000</code>	<code>bus_current=1500 = 1.5 A</code>

关键记忆口诀： - 角度类（关节/夹爪/末端姿态）→ /1000 得到度 - 位置类（末端 X/Y/Z）→ /1000 得到 mm - 力矩类（夹爪力矩）→ /1000 得到 N/m - 驱动器电压 → /10 得到 V，这是少数不按千分之一的特例

3.9 使用示例

下面展示一个最小但完整的 Python 脚本，演示如何使用 SDK 连接、使能、控制 PiPER 机械臂。

```
#!/usr/bin/env python3
"""PiPER SDK 最小使用示例 —— 关节空间位置控制"""

import time
import math
from piper_sdk.interface.piper_interface_v2 import C_PiperInterface_V2

# —— 步骤 1: 创建接口实例 ——
```

```

# can_name 为 SocketCAN 接口名, 通常为 "can0"
# 使用 PCIe CAN 模块时 judge_flag=False
piper = C_PiperInterface_V2(
    can_name="can0",
    judge_flag=True,
    can_auto_init=True,
    start_sdk_joint_limit=True,      # 启用 SDK 软限位保护
    start_sdk_gripper_limit=True,
    start_sdk_fk_cal=False,         # 不需要 FK 时可关闭以节省计算
)

# ----- 步骤 2: 连接并使能 -----
print("[1] 正在连接 CAN 口并启动后台线程...")
piper.ConnectPort(can_init=True, piper_init=True, start_thread=True)
time.sleep(0.5) # 等待初始化查询完成和首帧数据到达

print("[2] 正在使能机械臂...")
piper.EnablePiper()
time.sleep(0.3)

# 确认使能状态
enable_status = piper.GetArmEnableStatus()
print(f"    各关节使能状态: {enable_status}")

# ----- 步骤 3: 设置控制模式 -----
print("[3] 切换到关节控制模式 (MOVE J)...")
piper.ModeCtrl(
    ctrl_mode=0x01,      # CAN 指令控制模式
    move_mode=0x01,      # MOVE J (关节空间)
    move_spd_rate_ctrl=30 # 30% 速度
)
time.sleep(0.1)

# ----- 步骤 4: 读取当前关节位置 -----
joint_msgs = piper.GetArmJointMsgs()
current_joints = [
    joint_msgs.joint_state.joint_1,
    joint_msgs.joint_state.joint_2,
    joint_msgs.joint_state.joint_3,
    joint_msgs.joint_state.joint_4,
    joint_msgs.joint_state.joint_5,
    joint_msgs.joint_state.joint_6,
]
print(f"[4] 当前关节角度 (毫度): {current_joints}")
print(f"    当前关节角度 (度):  {[j * 0.001 for j in current_joints]}")

# ----- 步骤 5: 发送目标位置 -----
# 注意: 实际使用中应设置合理的运动范围, 不要超出关节限位
print("[5] 发送目标关节角度...")
target_j1 = 45000 # 45.0 度
target_j2 = 90000 # 90.0 度
target_j3 = -90000 # -90.0 度
target_j4 = 30000 # 30.0 度
target_j5 = 45000 # 45.0 度

```

```

target_j6 = 60000 # 60.0 度

piper.JointCtrl(target_j1, target_j2, target_j3,
                target_j4, target_j5, target_j6)

# 等待机械臂运动到位 (简单实现: 固定等待; 生产环境应检查 motion_status)
time.sleep(3.0)

# ----- 步骤 6: 循环读取反馈 -----
print("[6] 循环读取反馈 (按 Ctrl+C 停止)...")
try:
    for i in range(100):
        joint_msgs = piper.GetArmJointMsgs()
        gripper_msgs = piper.GetArmGripperMsgs()
        status = piper.GetArmStatus()

        joints_deg = [
            joint_msgs.joint_state.joint_1 * 0.001,
            joint_msgs.joint_state.joint_2 * 0.001,
            joint_msgs.joint_state.joint_3 * 0.001,
            joint_msgs.joint_state.joint_4 * 0.001,
            joint_msgs.joint_state.joint_5 * 0.001,
            joint_msgs.joint_state.joint_6 * 0.001,
        ]

        print(f"[{i:03d}] 关节角度: {[f'{j:.2f}' for j in joints_deg]} 度 | "
              f"状态: {status.arm_status.motion_status} | "
              f"帧率: {joint_msgs.Hz:.1f} Hz")

        # 检查 CAN 是否正常
        if not piper.isOk():
            print("警告: CAN 总线可能已卡死!")

        time.sleep(0.05) # 约 20Hz 打印

except KeyboardInterrupt:
    print("\n用户中断")

# ----- 步骤 7: 失能并断开 -----
print("[7] 失能机械臂并断开连接...")
piper.DisablePiper()
time.sleep(0.2)
piper.DisconnectPort()
print(" 已断开连接。")

```

示例关键说明

1. **ConnectPort** 的参数:
2. **can_init=True**: 强制执行 CAN 口初始化 (首次或重新连接时必需)。
3. **piper_init=True**: 自动查询电机参数和固件版本。
4. **start_thread=True**: 启动 ReadCan 和 CanMonitor 后台线程。

5. **模式切换顺序**：必须先 `EnablePiper()` 使能，再 `ModeCtrl()` 设置控制模式，最后才能发送 `JointCtrl()` 指令。
6. `ModeCtrl(move_spd_rate_ctrl=30)`：设置速度为满速的 30%，在调试阶段建议使用较低速度，降低碰撞风险。
7. **读取顺序无关**：由于后台线程持续更新缓存，`GetArmJointMsgs()` 等读方法随时返回最新的缓存值，无需担心阻塞。但需要注意首次调用前给足初始化时间（至少等待 0.3-0.5 秒）。
8. **线程安全**：所有 getter 方法内部使用独立锁，可以在主线程中安全调用，不受 ReadCan 线程影响。
9. **错误检查**：生产代码中应检查各方法的返回值、`isOk()` 状态以及 `arm_status` 中的错误码，实现健壮的错误处理。

进阶：主从模式设置

```
# 在教学臂（主臂）上执行
master = C_PiperInterface_V2(can_name="can0")
master.ConnectPort()
master.EnablePiper()
master.ModeCtrl(ctrl_mode=0x01, move_mode=0x01)
# 配置为示教输入臂，控制指令偏移 0x16x
master.MasterSlaveConfig(
    Linkage_config=0xFA,      # 教学输入臂
    feedback_offset=0x00,
    ctrl_offset=0x10,        # 控制指令 ID 偏移 +0x10
    linkage_offset=0x10      # 联动目标地址偏移 +0x10
)

# 在运动臂（从臂）上执行
slave = C_PiperInterface_V2(can_name="can1")
slave.ConnectPort()
slave.EnablePiper()
# 配置为运动输出臂，读取偏移后的控制指令
slave.MasterSlaveConfig(
    Linkage_config=0xFC,      # 运动输出臂
    feedback_offset=0x10,    # 反馈 ID 也偏移
    ctrl_offset=0x00,
    linkage_offset=0x00
)

# 从臂读取主臂的控制指令
ctrl = slave.GetArmJointCtrl()
print(f"主臂目标角度: J1={ctrl.joint_ctrl.joint_1 * 0.001:.2f}°")
```

本章小结

本章从源代码层面深入剖析了 PiPER SDK 的核心架构：

1. **C_PiperInterface_V2** 是整个系统的中枢，通过单例模式管理 CAN 连接，内部聚合了底层 CAN 硬件端口、协议编解码器和前向运动学求解器。
2. **多线程后台模型**（ReadCan + CanMonitor）解决了 CAN 总线推模型数据接收和总线健康监测的需求，通过细粒度锁机制保证线程安全。
3. **消息类型系统**（PiperMessage + ArmMsgType + CanIDPiper + ArmMessageMapping）提供了一套完整的 CAN 帧与 Python 对象之间的双向映射，所有数据以整数形式存储（毫度、毫牛米等），避免了浮点精度损失。
4. **统一的控制指令发送流程**：构造消息对象 → 编码为 CAN 帧 → 调用 SendCanMessage → 检查发送状态。
5. **主从模式**通过 CAN ID 偏移机制实现，教学臂主动发送控制指令，运动臂监听并执行，实现硬件级别的低延迟主从跟随。

掌握这些底层机制后，你可以自信地直接通过 SDK 接口操控 PiPER 机械臂，为后续的数据采集（第4章）和策略部署（第5章）做好准备。

第 4 章 LeRobot 框架深度解析

4.1 LeRobot 是什么

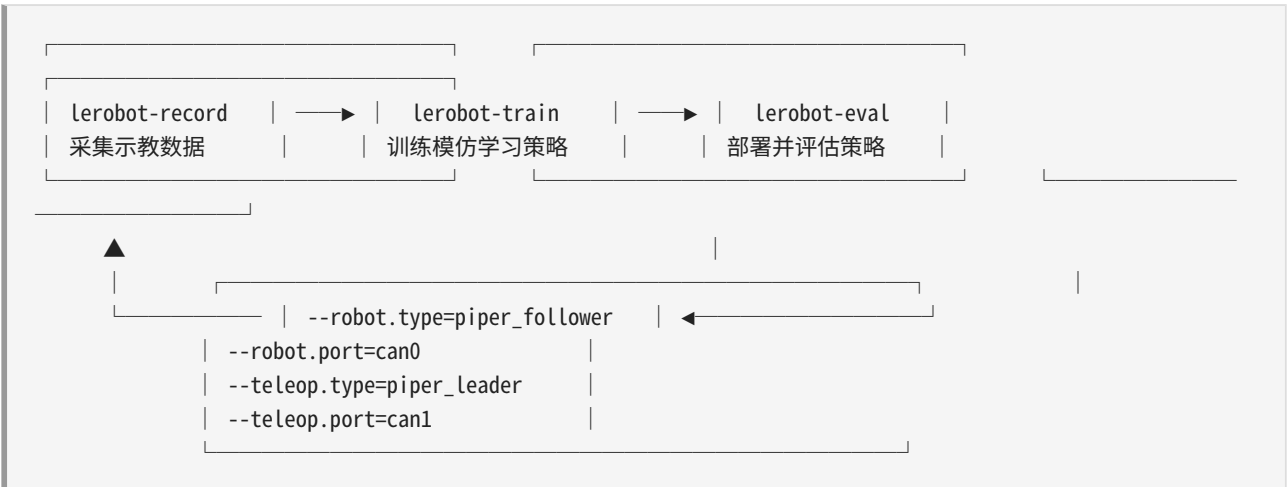
LeRobot 是 HuggingFace 于 2024 年开源的机器人学习框架，目标是为真实机器人数据采集、策略训练和部署评估提供一套统一的标准化流水线。其核心理念可以概括为三个关键词：

统一的数据格式 —— 所有机器人，无论机械结构如何不同，它们采集出来的数据都遵循同一套命名规范和存储结构。这使得在不同的机器人硬件之间迁移算法和数据集成为可能。

插件式硬件接入 —— 新增一种机器人型号，只需编写一个配置类和实现若干抽象方法，无需修改框架核心代码。框架通过 `draccus.ChoiceRegistry` 自动发现并加载新硬件。

模块化策略训练 —— 训练算法（ACT、Diffusion Policy、Pi0、VQ-BeT、TD-MPC 等）与机器人硬件完全解耦。策略只看到归一化后的动作值，与底层舵机或 CAN 总线无关。

LeRobot 提供了三大 CLI 工具，覆盖从数据采集到训练再到部署的完整生命周期：



所有这些 CLI 工具都是通过 `pyproject.toml` 中的 `[project.scripts]` 注册为系统命令的。当你执行 `pip install -e ".[piper]"` 时，`pip` 会自动在你的 `$PATH` 中创建可执行脚本。下面是从 `pyproject.toml` 中摘录的实际配置：

```
[project.scripts]
lerobot-calibrate="lerobot.calibrate:main"
lerobot-find-cameras="lerobot.find_cameras:main"
lerobot-find-port="lerobot.find_port:main"
lerobot-record="lerobot.record:main"
lerobot-replay="lerobot.replay:main"
lerobot-setup-motors="lerobot.setup_motors:main"
```

```
lerobot-teleoperate="lerobot.teleoperate:main"
lerobot-eval="lerobot.scripts.eval:main"
lerobot-train="lerobot.scripts.train:main"
```

每一行等号左边是用户敲的命令，等号右边的 `模块路径:函数名` 告诉 pip 把哪个 Python 函数包装成可执行入口。

`pip install -e ".[piper]"` 实际上做了三件事：

1. **安装 `piper_sdk` 依赖**——从 `[project.optional-dependencies]` 中找到 `piper` = `["piper_sdk>=0.4.1"]`，确保 PiPER 硬件的 CAN 通信库可用。
2. **注册 CLI 入口点**——将上表中的每个 `模块:函数` 映射创建为系统级可执行文件。
3. **触发模块导入**——`pip install -e` 以 editable 模式安装后，Python 在 `import lerobot` 时会遍历 `src/lerobot/` 下所有子包。由于每个子类配置文件中使用了 `@RobotConfig.register_subclass("piper_follower")` 装饰器，这些注册都在模块导入时执行完毕，`ChoiceRegistry` 的 `_subclasses` 字典在程序启动时就已经填好了。

4.2 LeRobot 的设计哲学：从硬件多样性到算法统一性

在理解了 LeRobot 的基本功能之后，我们有必要深入探讨一个问题：**为什么 LeRobot 要这样设计？**这不仅仅是一个工程选择，而是反映了机器人学习领域对“如何让 AI 走进物理世界”这个问题的深层思考。

4.2.1 机器学习的数据问题

深度学习的成功——从 ImageNet 到 GPT——很大程度上依赖于**大规模、高质量、标准化的数据集**。但机器人领域长期面临一个“鸡和蛋”的问题：

1. **没有统一的数据格式**：每个实验室、每个机器人平台都有自己的数据格式。MIT 的机器人可能用 ROS bag，Stanford 的可能用 HDF5 + 自定义 schema，工业机器人用专有格式。数据无法共享、无法复用。
2. **数据采集成本极高**：CV/NLP 的数据可以从互联网抓取（免费且海量），机器人数据需要真人操作物理硬件（昂贵且有限）。
3. **硬件碎片化**：机器人有 6 轴的、7 轴的、并联的、移动的，每种都有不同的自由度和控制接口。

LeRobot 的设计正是为了解决这三个问题：**- 统一格式** → 解决数据孤岛（类似于 ImageNet 统一了图像分类数据）**- 插件式架构** → 解决硬件碎片化（类似于 PyTorch 的 Module 统一了所有网络

层) - **标准化工具链** → 降低采集门槛 (类似于 HuggingFace 的 Transformers 让非专家也能训练模型)

4.2.2 为什么归一化是设计的核心

在第 1 章中我们介绍了 VLA 和 World Model 的概念。LeRobot 的归一化设计 (所有动作统一为 $[-100, 100]$ 或 $[0, 100]$) 不仅是为了工程便利——它是实现 **跨机器人迁移学习** 的必要前提。

考虑两种可能的发展路径:

路径 A (当前): PiPER 上的 ACT

PiPER 数据 → 训练 ACT → 部署到 PiPER

路径 B (未来的 VLA): 多种机器人数据联合训练

PiPER 数据 + So100 数据 + Koch 数据 + Reachy 数据 → 训练 VLA → 部署到任意机器人

路径 B 只有在所有机器人的动作空间被归一化到同一尺度时才有可能。一个 2 米长的工业机械臂和 30 厘米的桌面机械臂, 它们的关节运动范围差了近一个数量级。但归一化到 $[-100, 100]$ 后, 对策略模型来说它们都是"相同的动作空间", 模型只需要关注"在这个视觉场景中应该如何移动", 而不是"这个特定机械臂的物理极限是多少毫米"。

这引出了一个更深层的概念——**embodiment-agnostic policy (体态无关策略)**: 一个能理解"抓取"这个动作语义的策略, 应该能在不同大小、不同自由度的机械臂上执行, 就像人类既可以用左手也可以用右手拿杯子一样。LeRobot 的归一化框架正是通向这一愿景的一小步。

4.2.3 插件式设计如何支持 VLA 和 World Model 的演进

LeRobot 的三层架构 (详见 4.3 节) 看似只是经典的软件工程实践, 但它实际上有更深远的考虑:

MotorsBus 层的独立性 意味着: - 可以单独升级为**带有 World Model 的 MotorsBus**: 在执行动作前, 先用 World Model 预测该动作的后果, 如果预测的结果不安全, 降低速率或拒绝执行。- 可以替换为**仿真 MotorsBus**: 在仿真器中运行相同的策略, 用于大规模的离线训练和评估 (这就是 4.2.1 节提到的 3 条路径中的纯 RL + sim-to-real)。

Robot/Teleoperator 层的独立性 意味着: - 可以添加**语言接口**: `connect()` 变为 `connect("pick up the red cube")`, 将自然语言指令转换为任务上下文, 策略据此调整行为 (VLA 的前置需求)。- 可以引入**多模态观测**: 当前的 observation 包含 RGB 图像和关节角度。未来可以添加深度图、触觉力反馈、甚至麦克风音频——Robot 层的 `get_observation()` 接口天然支持这种扩展。

Policy 层的独立性 意味着： - ACT 可以替换为 Pi0（一个 VLA 模型）或带 World Model 的 TD-MPC，而硬件层完全不用修改。 - 策略可以不直接输出最终动作，而是输出一个 **planner 的中间表示**（如 sub-goal 坐标），再由一个下层的 motion planner 将 sub-goal 翻译为关节角度指令（分层策略，Hierarchical Policy）。

4.2.4 与更广泛的 AI 研究的关系

LeRobot 的设计受到了以下研究的深刻影响：

影响来源	对 LeRobot 的具体影响
ImageNet (2012)	证明了统一的大规模数据集可以催生算法革命。LeRobot 的统一数据格式意在创建"机器人的 ImageNet"。
HuggingFace Transformers (2019)	证明了统一的模型接口（AutoModel、AutoTokenizer）可以极大降低使用门槛。LeRobot 的 ChoiceRegistry 和 make_robot_from_config() 是同样的思想。
GPT 系列 (2018-2023)	证明了 scaling law——模型越大、数据越多，泛化能力越强。这推动了 VLA 的研究和 LeRobot 的标准化采集工具。
RT-2 / Octo / Pi0 (2023-2024)	证明了 VLA 模型可以在真实机器人上 zero-shot 执行指令。LeRobot 的设计就是为了让这类模型能方便地接入不同硬件。
JEPA / World Models (2022-2024)	证明了预测未来观测是学习世界知识的有效方式。LeRobot 保留 observation 和 action 的完整历史，为未来集成 World Model 提供了数据基础。

4.3 插件式设计：三层架构

LeRobot 采用了严格的**三层架构**，每层之间通过抽象基类定义契约，通过 ChoiceRegistry 实现自动发现。这种设计使得添加新硬件时，只需在对应的层实现具体子类，上层和数据流水线完全不受影响。

```
| CLI 层 |
| lerobot-record / lerobot-train / lerobot-eval / lerobot-teleoperate |
|
| Example: |
| $ lerobot-record --robot.type=piper_follower --robot.port=can0 |
| $ lerobot-record --robot.type=koch_follower --robot.port=/dev/tty |
| $ lerobot-record --robot.type=so100_follower --robot.port=... |
| $ lerobot-record --robot.type=bi_so100_follower |
|
| draccus.ChoiceRegistry 根据 --robot.type 的值自动查找对应子类 |
```

Robot / Teleoperator 层	
PiperFollower / PiperLeader / KochFollower / KochLeader /	
S0100Follower / S0100Leader / LeKiwi / Reachy2Robot / Stretch3 /	
BiS0100Follower / ViperX / HopeJr / HomunculusGlove / Phone / ...	
每个 Robot 持有 1 个 MotorsBus 实例 (bimanual 机器人持有 2 个)	
每个 Teleoperator 持有 1 个 MotorsBus 实例	
MotorsBus 层	
PiperMotorsBus / DynamixelMotorsBus / FeetechMotorsBus / ...	
每个 MotorsBus 持有:	
- port_handler: 串口/CAN 端口的打开/关闭/参数设置	
- 若干 Motor 对象: 电机的 id、型号、归一化模式	
- 若干 MotorCalibration 对象: 运动范围 min/max	
硬件层	
piper_sdk (CAN 总线) / serial (USB 串口) / CAN bus / ...	
真正的物理通信: CAN 帧收发、RS-485 协议、舵机控制表读写	

为什么 LeRobot 能自动发现 PiperFollower?

关键语句在 `src/lerobot/robots/piper_follower/config_piper_follower.py` 的第 26 行:

```
@RobotConfig.register_subclass("piper_follower")
@dataclass(kw_only=True)
class PiperFollowerConfig(RobotConfig):
    port: str
    ...
```

当 Python 执行 `import lerobot.robots.piper_follower` 时 (这发生在 `lerobot-record` 启动阶段的 `make_robot_from_config` 或通过 `draccus` 解析 `--robot.type=piper_follower` 参数时), 装饰器 `register_subclass("piper_follower")` 将字符串 `"piper_follower"` 与类 `PiperFollowerConfig` 的映射写入 `ChoiceRegistry` 的 `_subclasses` 字典中。

此后, 当用户在命令行输入 `--robot.type=piper_follower`, `draccus` 的序列化引擎会:

1. 查找 `RobotConfig._subclasses["piper_follower"]`
2. 发现是 `PiperFollowerConfig`

3. 用剩余的 CLI 参数（如 `--robot.port=can0`）实例化 `PiperFollowerConfig(port="can0")`
4. 将这个 config 对象传递给 `PiperFollower(config)`

整个过程对用户完全透明——用户只需要知道机器人类型名和端口名。

4.4 draccus ChoiceRegistry 自动类型发现

draccus 是什么

draccus 是 HuggingFace 团队开发的配置管理库（当前 LeRobot 固定依赖 `draccus==0.10.0`），它在标准 Python `dataclass` 的基础上增加了：

- **命令行参数自动解析**——`dataclass` 的字段自动映射为 CLI 参数（`--field=value` 或 `--field.subfield=value`）
- **ChoiceRegistry 子类注册机制**——基类维护一个全局注册表，子类通过装饰器注册，运行时按名称查找
- **嵌套配置支持**——一个 `dataclass` 可以包含另一个 `dataclass`，CLI 参数用 `.` 分隔嵌套层级
- **插件动态加载**——通过 `--xxx.discover_packages_path=my_package` 可以在运行时动态加载外部包中的子类

ChoiceRegistry 机制剖析

`ChoiceRegistry` 是 draccus 提供的一个混入类（Mixin），它的核心工作方式是维护一个类级别的 `_subclasses` 字典：

```
# draccus 内部逻辑的简化示意
class ChoiceRegistry:
    _subclasses: dict[str, type] = {}

    @classmethod
    def register_subclass(cls, name: str):
        """将子类注册到注册表中"""
        def decorator(subclass):
            cls._subclasses[name] = subclass
            return subclass
        return decorator

    @classmethod
    def get_choice_name(cls, subclass) -> str:
        """反向查找：给定子类，返回注册名"""
        for name, sub in cls._subclasses.items():
            if sub is subclass:
```

```
        return name
    raise KeyError(subclass)
```

在 LeRobot 中, `RobotConfig` 继承自 `draccus.ChoiceRegistry` :

```
# src/lerobot/robots/config.py 第 23 行
@dataclass(kw_only=True)
class RobotConfig(draccus.ChoiceRegistry, abc.ABC):
    id: str | None = None
    calibration_dir: Path | None = None
```

`TeleoperatorConfig` 也采用完全相同的方式:

```
# src/lerobot/teleoperators/config.py 第 23 行
@dataclass(kw_only=True)
class TeleoperatorConfig(draccus.ChoiceRegistry, abc.ABC):
    id: str | None = None
    calibration_dir: Path | None = None
```

完整调用链示例

假设用户在终端执行:

```
lerobot-record --robot.type=piper_follower --robot.port=can1 --robot.id=black --dataset.repo_id=my_user/
my_data
```

整个类型发现和实例化过程如下:

1. shell 调用 `lerobot-record` 可执行文件
2.
 - ↳ Python 执行 `lerobot.record:main()`
 - ↳ `@parser.wrap()` 装饰器分析 `main()` 的第一个参数类型 → `RecordConfig`
 - ↳ `RecordConfig` 的第一个字段是 `robot: RobotConfig`
3. parser 扫描 `sys.argv`, 发现 `--robot.type=piper_follower`
 - ↳ 查找 `RobotConfig._subclasses["piper_follower"]`
 - ↳ 找到 `PiperFollowerConfig`
4. parser 发现 `--robot.port=can1`
 - ↳ 匹配 `PiperFollowerConfig` 的 `port: str` 字段
 - ↳ 值 = "can1"
5. parser 发现 `--robot.id=black`
 - ↳ 匹配 `RobotConfig` 基类的 `id` 字段
 - ↳ 值 = "black"
6. parser 发现 `--dataset.repo_id=my_user/my_data`
 - ↳ 匹配 `RecordConfig.dataset` 的 `repo_id` 字段

```

        值 = "my_user/my_data"
7.  parser 用所有解析到的参数调用:
    RecordConfig(
        robot=PiperFollowerConfig(port="can1", id="black"),
        dataset=DatasetRecordConfig(repo_id="my_user/my_data"),
        teleop=None,
        policy=None,
    )
8.  main() 收到 cfg: RecordConfig 对象
    ↳ make_robot_from_config(cfg.robot)
    返回 PiperFollower(config=cfg.robot)
9.  PiperFollower.__init__() 创建 PiperMotorsBus(...)
    ↳ PiperMotorsBus.__init__() 调用 piper_sdk.C_PiperInterface_V2(can1)
10.  进入主循环: record_loop(robot=PiperFollower, ...)

```

已注册的所有机器人类型

LeRobot 生态中目前已注册的机器人类型（通过在代码库中搜索 `@RobotConfig.register_subclass` 所得）：

注册名	配置类	底层总线	说明
<code>piper_follower</code>	<code>PiperFollowerConfig</code>	<code>PiperMotorsBus</code> (<code>piper_sdk</code>)	PiPER 双臂中的执行臂，CAN 通信
<code>koch_follower</code>	<code>KochFollowerConfig</code>	<code>DynamixelMotorsBus</code>	Koch v1.1 低成本臂
<code>so100_follower</code>	<code>SO100FollowerConfig</code>	<code>FeetechMotorsBus</code>	SO-100 低成本臂
<code>so101_follower</code>	<code>SO101FollowerConfig</code>	<code>FeetechMotorsBus</code>	SO-101 改进版
<code>bi_so100_follower</code>	<code>BiSO100FollowerConfig</code>	<code>FeetechMotorsBus x2</code>	SO-100 双臂版
<code>lekiwi</code>	<code>LeKiwiConfig</code>	<code>FeetechMotorsBus</code>	LeKiwi 教育机器人
<code>lekiwi_client</code>	<code>LeKiwiClientConfig</code>	(远程)	LeKiwi 远程客户端
<code>reachy2</code>	<code>Reachy2Config</code>	(<code>reachy2_sdk</code>)	Pollen Robotics 的 Reachy2
<code>stretch3</code>	<code>Stretch3Config</code>	<code>DynamixelMotorsBus</code>	Hello Robot 的 Stretch 3
<code>viperx</code>	<code>ViperXConfig</code>	<code>DynamixelMotorsBus</code>	Trossen Robotics ViperX
<code>hope_jr_arm</code>	<code>HopeJrArmConfig</code>	<code>FeetechMotorsBus</code>	Hope Jr 手臂
<code>hope_jr_hand</code>	<code>HopeJrHandConfig</code>	<code>FeetechMotorsBus</code>	Hope Jr 灵巧手

对应的 Teleoperator 注册:

注册名	配置类	底层总线	说明
<code>piper_leader</code>	<code>PiperLeaderConfig</code>	<code>PiperMotorsBus</code> (<code>piper_sdk</code>)	PiPER 主臂 (示教臂)
<code>koch_leader</code>	<code>KochLeaderConfig</code>	<code>DynamixelMotorsBus</code>	Koch 主臂
<code>so100_leader</code>	<code>SO100LeaderConfig</code>	<code>FeetechMotorsBus</code>	SO-100 主臂
<code>so101_leader</code>	<code>SO101LeaderConfig</code>	<code>FeetechMotorsBus</code>	SO-101 主臂
<code>bi_so100_leader</code>	<code>BiSO100LeaderConfig</code>	<code>FeetechMotorsBus x2</code>	SO-100 双臂版主臂
<code>gamepad</code>	<code>GamepadConfig</code>	(HID)	游戏手柄遥操作
<code>keyboard</code>	<code>KeyboardConfig</code>	(<code>pynput</code>)	键盘遥操作
<code>keyboard_ee</code>	<code>KeyboardEEConfig</code>	(<code>pynput</code>)	键盘末端执行器控制
<code>widowx</code>	<code>WidowXConfig</code>	<code>DynamixelMotorsBus</code>	WidowX 主臂
<code>stretch3</code>	<code>Stretch3TeleopConfig</code>	—	Stretch 3 游戏手柄
<code>reachy2_teleoperator</code>	<code>Reachy2TeleoperatorConfig</code>	(<code>reachy2_sdk</code>)	Reachy2 遥操作
<code>homunculus_glove</code>	<code>HomunculusGloveConfig</code>	—	数据手套
<code>homunculus_arm</code>	<code>HomunculusArmConfig</code>	—	外骨骼臂
<code>phone</code>	<code>PhoneConfig</code>	(<code>hebi-py</code>)	手机遥操作

4.5 MotorsBus 基类详解

`MotorsBus` 是 LeRobot 框架中最接近物理硬件的抽象层, 定义在 `src/lerobot/motors/motors_bus.py` 中 (共 1220 行)。它封装了与一串串联电机 (daisy-chained motors) 通信的完整逻辑。

4.4.1 Motor 数据类

```
# motors_bus.py 第 95-99 行
@dataclass
class Motor:
    id: int          # 电机 ID (1-7)
    model: str       # 电机型号 ("AGILEX-M", "AGILEX-S", "HTDW-5047")
    norm_mode: MotorNormMode # 归一化模式
```

每个 `Motor` 对象描述一个物理电机的三要素元数据：

- **id**：电机在总线上的唯一标识。在 Dynamixel/Feetech 协议中，id 用于定址；在 PiPER 中，id 是逻辑编号（1=joint1, 2=joint2, ..., 7=gripper）。
- **model**：电机型号字符串。不同型号有不同的控制表（ctrl_table）、分辨率（resolution_table）和型号编码（model_number_table）。例如 AGILEX-M 是 PiPER 的大关节电机（1190），AGILEX-S 是小关节和夹爪电机（1191），HTDW-5047 是主臂示教电机的型号。
- **norm_mode**：归一化模式，决定了从原始编码器值映射到归一化值时的公式。

4.4.2 MotorNormMode 枚举

```
# motors_bus.py 第 80-83 行
class MotorNormMode(str, Enum):
    RANGE_0_100 = "range_0_100"
    RANGE_M100_100 = "range_m100_100"
    DEGREES = "degrees"
```

模式	归一化范围	使用场景	公式
RANGE_M100_100	[-100, 100]	旋转关节	$((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 200 - 100$
RANGE_0_100	[0, 100]	夹爪	$((\text{raw} - \text{min}) / (\text{max} - \text{min})) * 100$
DEGREES	角度制	旋转关节（以度为单位）	$(\text{raw} - \text{mid}) * 360 / \text{max_resolution}$

PiPER 执行臂的关节 1~6 使用 `RANGE_M100_100`，夹爪使用 `RANGE_0_100`。PiPER 不使用 `DEGREES` 模式（在主臂/执行臂的配置中均无此用法）。

4.4.3 MotorCalibration 数据类

```
# motors_bus.py 第 86-92 行
@dataclass
class MotorCalibration:
    id: int
    drive_mode: int # 驱动模式 (0=正向, 1=反向)
    homing_offset: int # 回零偏置
    range_min: int # 最小原始值
    range_max: int # 最大原始值
```

这是理解归一化系统的关键数据结构。`range_min` 和 `range_max` 定义了每个关节在 SDK 原始坐标系中的运动范围。由于不同关节的物理运动范围不同，必须为每个关节分别设置。

以 PiperFollower 的标定为例（来自 `piper_follower.py` 第 60-68 行）：

```
calibration={
    "joint1": MotorCalibration(1, 0, 0, -150000, 150000), # ±150k 对称
    "joint2": MotorCalibration(2, 0, 0, 0, 180000), # 只能正向 0~180°
    "joint3": MotorCalibration(3, 0, 0, -170000, 0), # 只能负向 -170°~0
    "joint4": MotorCalibration(4, 0, 0, -100000, 100000), # ±100k
    "joint5": MotorCalibration(5, 0, 0, -65000, 65000), # ±65k
    "joint6": MotorCalibration(6, 0, 0, -100000, 130000), # 不对称
    "gripper": MotorCalibration(7, 0, 0, 0, 68000), # 0~68k
}
```

如果不使用 `range_min/max` 而使用固定范围（如统一假设所有关节都是 `[0, 4096]`），那么 `joint2`（实际范围 `[0, 180000]`）的归一化会完全错误——一个微小的物理运动会被放大为巨大的归一化变化，反之亦然。`range_min/range_max` 确保每个关节的归一化都精确映射到它的真实物理运动范围。

4.4.4 MotorsBus 抽象类核心设计

`MotorsBus` 是一个 `abc.ABC` 抽象类（第 212 行），其构造函数接受：

```
def __init__(
    self,
    port: str, # "can1", "/dev/ttyUSB0" 等
    motors: dict[str, Motor], # {"joint1": Motor(1, "AGILEX-M", ...), ...}
    calibration: dict[str, MotorCalibration] | None = None,
):
```

构造函数执行后，内部会建立三个关键的映射字典（第 280-282 行）：

```
self._id_to_model_dict = {m.id: m.model for m in self.motors.values()}
self._id_to_name_dict = {m.id: motor for motor, m in self.motors.items()}
self._model_nb_to_model_dict = {v: k for k, v in self.model_number_table.items()}
```

这些映射使得框架可以通过电机 ID（整数）快速查找到电机名称（字符串）和型号。

核心方法清单：

方法	功能	在 PiPER 中
<code>connect()</code> / <code>disconnect()</code>	打开/关闭通信端口，可选握手	打开 CAN 口 / 关闭 CAN 口
<code>get_action()</code> → <code>_normalize()</code>	读取当前电机位置并归一化	调 <code>piper_sdk</code> 读编码器 → 归一化
<code>set_action()</code> → <code>_unnormalize()</code>	将归一化值反归一化后发送给电机	反归一化 → <code>JointCtrl</code> / <code>GripperCtrl</code>

方法	功能	在 PiPER 中
<code>sync_read()</code> / <code>sync_write()</code>	批量读/写多个电机的同一寄存器	PiPER 不使用（CAN 是广播协议）
<code>parking()</code>	机械臂回到预定义的安全位置	发送 INITIALIZE_POSITION
<code>enable_torque()</code> / <code>disable_torque()</code>	使能/失能电机扭矩	EnablePiper / DisablePiper
<code>read()</code> / <code>write()</code>	读/写单个电机的单个寄存器	PiPER 不使用（CAN 不走寄存器协议）

Dynamixel 特有方法（PiPER 中为空实现）：

这些方法存在于抽象接口中是因为 `DynamixelMotorsBus` 和 `FeetechMotorsBus` 需要它们，但 `PiperMotorsBus` 将其全部实现为空或直通（pass-through）：

```
# PiperMotorsBus 中
def _handshake(self):          # 不需要握手协议
    pass

def broadcast_ping(self, ...): # 不需要广播 ping
    pass

def _find_single_motor(self, ...): # 不需要逐电机扫描
    pass

def _get_half_turn_homing(self, ...): # 不需要半圈回零
    pass

def _encode_sign(self, data_name, ids_values):
    return ids_values          # 不需要符号编码，直通

def _decode_sign(self, data_name, ids_values):
    return ids_values          # 不需要符号解码，直通

def _split_into_byte_chunks(self, ...):
    pass                       # piper_sdk 内部处理 CAN 帧
```

这个设计说明了 LeRobot 抽象层次的权衡：`MotorsBus` 基类继承目标是覆盖 `Dynamixel` 和 `Feetech` 的通用需求，`PiperMotorsBus` 作为一个"非标准"实现，将不适用的方法空实现，而覆盖了核心方法（`_normalize`，`_unnormalize`，`connect`，`disconnect`，`enable_torque` 等）来接入自己的 SDK。

4.4.5 `_normalize` 和 `_unnormalize` 数学原理

`_normalize` 和 `_unnormalize` 是框架最核心的两个方法，定义在 `MotorsBus` 基类中（第 776-833 行），但 `PiperMotorsBus` 重写了它们以适配自己的 SDK 值传递方式。以下是它们的确切数学公式：

RANGE_M100_100 模式（关节）：

```
归一化: norm = ((raw - min) / (max - min)) * 200 - 100
反归一化: raw = round(((norm + 100) / 200) * (max - min) + min)

示例 joint1 [min=-150000, max=150000]:
raw = 0 → norm = 0.0 （中位）
raw = 150000 → norm = 100.0 （正向极限）
raw = -150000 → norm = -100.0 （负向极限）
```

RANGE_0_100 模式（夹爪）：

```
归一化: norm = ((raw - min) / (max - min)) * 100
反归一化: raw = round((norm / 100) * (max - min) + min)

示例 gripper [min=0, max=68000]:
raw = 0 → norm = 0.0 （全开）
raw = 68000 → norm = 100.0 （全闭）
```

两个方法都在开头和结尾做了**双层安全钳位**：

1. `_normalize` 中：`bounded_val = min(max_, max(min_, val))` ——防止编码器异常值越界
2. `_unnormalize` 中：`bounded_val = min(100.0, max(-100.0, val))` ——防止策略模型输出超出预期范围

当 `apply_drive_mode` 为 `True` 且 `calibration.drive_mode` 为 `True` 时（仅 Dynamixel 舵机使用），公式中会多一次符号反转。PiPER 中 `apply_drive_mode = False`，所以这条路径永远不会被触发。

4.6 Robot 基类详解

`Robot` 抽象类定义在 `src/lerobot/robots/robot.py`（共 185 行），是所有 LeRobot 兼容机器人的最顶层抽象。

4.5.1 类属性

```
# robot.py 第 42-44 行
config_class: builtins.type[RobotConfig] # 关联的配置类类型
name: str # 机器人名称，如 "piper_follower"
```

每个子类必须设置这两个属性。例如 PiperFollower:

```
class PiperFollower(Robot):
    config_class: PiperFollowerConfig # 类型注解
    name = "piper_follower"          # 实际值
```

`name` 被用于生成标定文件的存储路径: `HF_LEROBOT_HOME/calibration/robots/{name}/{id}.json`。

4.5.2 构造函数

```
def __init__(self, config: RobotConfig):
    self.robot_type = self.name
    self.id = config.id
    self.calibration_dir = (
        config.calibration_dir if config.calibration_dir
        else HF_LEROBOT_CALIBRATION / ROBOTS / self.name
    )
    self.calibration_dir.mkdir(parents=True, exist_ok=True)
    self.calibration_fpath = self.calibration_dir / f"{self.id}.json"
    self.calibration: dict[str, MotorCalibration] = {}
    if self.calibration_fpath.is_file():
        self._load_calibration()
```

构造函数负责: 1. 确定标定文件的存储目录 (可以自定义或使用默认的 `~/.cache/huggingface/lerobot/calibration/robots/{name}/`) 2. 如果已有标定文件, 自动加载 (`_load_calibration` 使用 `draccus` 的 JSON 反序列化, 可直接将 JSON 字典转换回 `dict[str, MotorCalibration]`)

4.5.3 抽象属性与方法

抽象成员	类型	说明
<code>observation_features</code>	property → dict	描述观测空间的 key 和类型。Key 如 <code>"joint1.pos"</code> , <code>"joint2.pos"</code> , <code>"wrist"</code> , <code>"top"</code> ; Value 为 <code>float</code> 或 (H, W, C) 形状元组
<code>action_features</code>	property → dict	描述动作空间的 key 和类型。Key 如 <code>"joint1.pos"</code> ... <code>"gripper.pos"</code> ; Value 通常全为 <code>float</code>
<code>is_connected</code>	property → bool	机器人当前是否已连接
<code>is_calibrated</code>	property → bool	机器人当前是否已标定
<code>connect(calibrate)</code>	方法	建立通信。 <code>calibrate=True</code> 时自动执行标定
<code>disconnect()</code>	方法	断开通信并清理资源

抽象成员	类型	说明
<code>calibrate()</code>	方法	执行标定过程
<code>configure()</code>	方法	设置电机参数、控制模式等一次性配置
<code>get_observation()</code>	方法	读取当前观测（电机位置 + 相机图像），返回 dict
<code>send_action(action)</code>	方法	发送动作命令给机器人，返回实际发出的动作 dict

4.5.4 以 PiperFollower 为例的完整实现

PiperFollower 定义在 `src/lerobot/robots/piper_follower/piper_follower.py` 中（共 187 行）。以下是关键实现：

构造函数（第 44-70 行）——创建 MotorsBus：

```
def __init__(self, config: PiperFollowerConfig):
    self.bus = PiperMotorsBus(
        id=config.id,
        port=config.port,
        motors={
            "joint1": Motor(1, "AGILEX-M", MotorNormMode.RANGE_M100_100),
            "joint2": Motor(2, "AGILEX-M", MotorNormMode.RANGE_M100_100),
            "joint3": Motor(3, "AGILEX-M", MotorNormMode.RANGE_M100_100),
            "joint4": Motor(4, "AGILEX-S", MotorNormMode.RANGE_M100_100),
            "joint5": Motor(5, "AGILEX-S", MotorNormMode.RANGE_M100_100),
            "joint6": Motor(6, "AGILEX-S", MotorNormMode.RANGE_M100_100),
            "gripper": Motor(7, "AGILEX-S", MotorNormMode.RANGE_0_100),
        },
        calibration={
            "joint1": MotorCalibration(1, 0, 0, -150000, 150000),
            # ... (见 4.4.3 节)
        }
    )
    self.cameras = make_cameras_from_configs(config.cameras)
```

注意 Joint 1-3 使用 AGILEX-M（大关节电机），Joint 4-6 使用 AGILEX-S（小关节电机），这与物理硬件的真实配置一致。

observation_features（第 86-88 行）：

```
@cached_property
def observation_features(self) -> dict:
    return {**self._motors_ft, **self._cameras_ft}
```

其中 `_motors_ft` 生成 `{"joint1.pos": float, "joint2.pos": float, ..., "gripper.pos": float}`, `_cameras_ft` 生成 `{"wrist": (480, 640, 3), "top": (480, 640, 3), ...}`。两者合并为完整的观测特征字典。

get_observation (第 133-153 行) :

```
def get_observation(self) -> dict[str, Any]:
    obs_dict = {}
    # 读电机位置 (已归一化到 [-100,100] / [0,100])
    obs_dict = self.bus.get_action()
    obs_dict = {f"{motor}.pos": val for motor, val in obs_dict.items()}
    # 读相机帧
    for cam_key, cam in self.cameras.items():
        obs_dict[cam_key] = cam.async_read()
    return obs_dict
```

`bus.get_action()` 返回的键是 `{"joint1": 33.5, ..., "gripper": 50.0}`，通过字典推导式加上 `.pos` 后缀变成 `{"joint1.pos": 33.5, ..., "gripper.pos": 50.0}`，符合 LeRobot 数据集规范。

send_action (第 155-170 行) :

```
def send_action(self, action: dict[str, Any]) -> dict[str, Any]:
    goal_pos = {key.removesuffix(".pos"): val
                 for key, val in action.items() if key.endswith(".pos")}
    # 可选的安全限幅
    if self.config.max_relative_target is not None:
        present_pos = self.bus.sync_read("Present_Position")
        goal_pos = ensure_safe_goal_position(
            {key: (g_pos, present_pos[key])
             for key, g_pos in goal_pos.items()}
            self.config.max_relative_target
        )
    rlt = self.bus.set_action(goal_pos)
    return {f"{motor}.pos": val for motor, val in rlt.items()}
```

这里有一个重要的**安全保护机制**——`max_relative_target`：如果模型输出的目标位置与当前位置的差值超过阈值，会自动钳位到阈值范围。这可以防止策略模型输出异常大的动作导致机械臂剧烈运动。

4.5.5 标定框架

基类提供了 `_load_calibration` 和 `_save_calibration` 两个方法 (第 125-145 行)，使用 draccus 的 JSON 序列化功能：

```
def _load_calibration(self, fpath: Path | None = None) -> None:
    fpath = self.calibration_fpath if fpath is None else fpath
```

```

with open(fpath) as f, draccus.config_type("json"):
    self.calibration = draccus.load(dict[str, MotorCalibration], f)

def _save_calibration(self, fpath: Path | None = None) -> None:
    fpath = self.calibration_fpath if fpath is None else fpath
    with open(fpath, "w") as f, draccus.config_type("json"):
        draccus.dump(self.calibration, f, indent=4)

```

PiperFollower 将这两个方法覆盖为空实现（第 120-124 行），因为 PiPER 的标定值是硬编码在 `__init__` 中的静态值，不需要存取到文件。

4.7 Teleoperator 基类详解

`Teleoperator` 抽象类定义在 `src/lerobot/teleoperators/teleoperator.py`（共 181 行）。它与 `Robot` 的结构非常相似，但有几个关键区别。

4.6.1 Robot vs Teleoperator 对比

Robot（执行臂）	Teleoperator（主臂/示教臂）
<code>observation_features</code> — 电机位置 + 相机图像	（没有 <code>observation_features</code> ）
<code>action_features</code> — 接收并执行动作	<code>action_features</code> — 输出动作（用户的示教动作）
<code>get_observation() → obs</code> — 读电机 + 读相机	<code>get_action() → action</code> — 读主臂关节状态
<code>send_action(action)</code> — 执行动作	<code>send_feedback(feedback)</code> — 可选的力反馈（PiPER 未实现）
<code>feedback_features</code> （无）	<code>feedback_features</code> — 力反馈特征的描述

4.6.2 核心属性

Teleoperator 成员	类型	说明
<code>config_class</code>	type	关联的 <code>TeleoperatorConfig</code> 类型

Teleoperator 成员	类型	说明
name	str	唯一名称，如 "piper_leader"
action_features	property → dict	主臂输出的动作特征，如 {"joint1.pos": float, ...}
feedback_features	property → dict	力反馈特征（PiperLeader 返回 {}，未实现）
is_connected	property → bool	主臂是否已连接
is_calibrated	property → bool	主臂是否已标定（PiperLeader 始终返回 True）

4.6.3 以 PiperLeader 为例

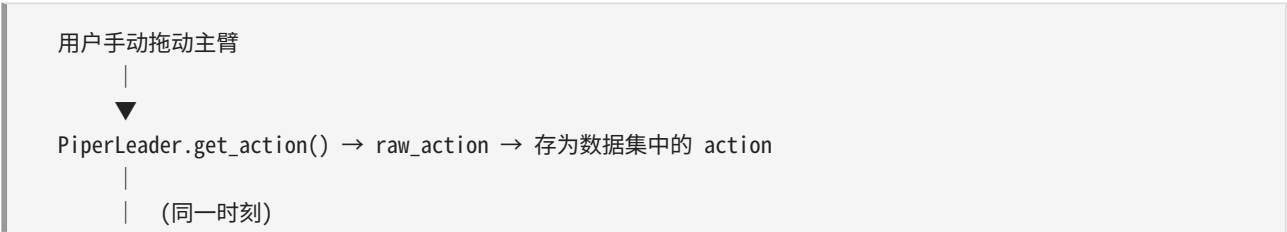
PiperLeader 定义在 `src/lerobot/teleoperators/piper_leader/piper_leader.py`（共 111 行）。与 PiperFollower 的对比：

对比维度	PiperFollower (执行臂)	PiperLeader (主臂)
电机型号	AGILEX-M / AGILEX-S	HTDW-5047
电机数量	6 + 1 (关节 + 夹爪)	6 + 1 (关节 + 夹爪)
标定信息	硬编码完整的 range_min/max	无标定
connect 行为	连接 → 使能扭矩 → parking 回零	连接 → 使能扭矩
状态读取	<code>get_observation()</code> 读编码器 + 相机	<code>get_action()</code> 只读编码器
动作发送	<code>send_action()</code> 调 <code>bus.set_action()</code>	不发送动作
力反馈	无	<code>send_feedback()</code> 空实现

关键区别在于**职责分离**：

- **PiperLeader** 的角色是"传感器"——把用户的示教动作转换成归一化数值输出给数据采集流水线。它不执行动作，只读状态。
- **PiperFollower** 的角色是"执行器"——接收策略模型的动作指令，转换成物理运动。它也提供传感器读数（编码器位置 + 相机图像）用于形成观测。

在 `lerobot-record` 的数据采集中，两者同时工作：



▼
PiperFollower.get_observation() → raw_obs → 存为数据集中的 observation

注意：在这种采集模式下，执行臂（PiperFollower）的 `send_action()` 并没有被调用——它在采集中充当的是被动传感器，记录“在执行这个动作之前，机械臂在什么位置”。动作的来源是主臂（PiperLeader），而不是执行策略模型。

4.8 pyproject.toml 注册机制

4.7.1 入口点注册

pyproject.toml 中的 `[project.scripts]` 部分定义了 CLI 入口点：

```
[project.scripts]
lerobot-calibrate="lerobot.calibrate:main"
lerobot-find-cameras="lerobot.find_cameras:main"
lerobot-find-port="lerobot.find_port:main"
lerobot-record="lerobot.record:main"
lerobot-replay="lerobot.replay:main"
lerobot-setup-motors="lerobot.setup_motors:main"
lerobot-teleoperate="lerobot.teleoperate:main"
lerobot-eval="lerobot.scripts.eval:main"
lerobot-train="lerobot.scripts.train:main"
```

每个等号左边是用户在 Shell 中敲的命令，右边是 `包.模块:函数` 的 Python 导入路径。当 `pip install` 执行时，`setuptools` 会为每个入口生成一个操作系统级的可执行脚本（在 `$PREFIX/bin/` 下），脚本的内容大致是：

```
from lerobot.record import main
if __name__ == "__main__":
    sys.exit(main())
```

4.7.2 可选依赖分组

```
[project.optional-dependencies]
# 电机驱动
feetech = ["feetech-servo-sdk>=1.0.0"]
dynamixel = ["dynamixel-sdk>=3.7.31"]
piper = ["piper_sdk>=0.4.1"]

# 机器人
gamepad = ["lerobot[pygame-dep]", "hidapi>=0.14.0"]
hopejr = ["lerobot[feetech]", "lerobot[pygame-dep]"]
```



```

lekiwi = ["lerobot[feetech]", "pyzmq>=26.2.1"]
reachy2 = ["reachy2_sdk>=1.0.14"]
# ...

# 策略
pi0 = ["lerobot[transformers-dep]"]
smolvla = ["lerobot[transformers-dep]", "num2words>=0.5.14", ...]
hilserl = ["lerobot[transformers-dep]", "gym-hil>=0.1.9", ...]

# 全部
all = ["lerobot[dynamixel]", "lerobot[gamepad]", "lerobot[hopejr]", ...]

```

这种分组设计的好处是： - `pip install lerobot[piper]` 只安装 PiPER 需要的 `piper_sdk`，不安装 `dynamixel-sdk` 或 `feetech-servo-sdk` - 在 Docker 镜像或 CI 环境中可以用 `pip install lerobot[all]` 安装全部依赖 - 依赖之间可以相互引用（如 `hopejr = ["lerobot[feetech]", "lerobot[pygame-dep]"]`）

4.7.3 完整的安装与发现流程

```

$ pip install -e ".[piper]"
|
├─> 1. 安装 piper_sdk>=0.4.1 (CAN 总线通信库)
|
├─> 2. setuptools 将 entry_points 注册到系统 PATH
|
└─> 3. 安装完成, Python 可以 import lerobot

$ lerobot-record --robot.type=piper_follower --robot.port=can0
|
├─> shell 执行 lerobot-record → lerobot/record.py:main()
|
├─> @parser.wrap() 装饰器触发参数解析
|
├─> import lerobot.robots 触发所有机器人模块的 __init__.py
|   └─> 执行 @RobotConfig.register_subclass("piper_follower")
|       → RobotConfig._subclasses["piper_follower"] = PiperFollowerConfig
|
├─> draccus 解析 --robot.type=piper_follower
|   └─> 查找 RobotConfig._subclasses["piper_follower"]
|       → 找到 PiperFollowerConfig → 用剩余 CLI 参数实例化
|
└─> make_robot_from_config(config) → PiperFollower(config)
    └─> PiperFollower.__init__() → PiperMotorsBus.__init__()
        └─> C_PiperInterface_V2("can0") → 开始通信

```

4.9 数据格式标准

4.8.1 LeRobot Dataset 目录结构

```
~/cache/huggingface/lerobot/my_user/my_dataset/
├── meta/
│   ├── info.json          # 数据集元信息
│   │   ├── robot_type: "piper_follower"
│   │   ├── features: {...} # observation.state + action 的格式定义
│   │   ├── fps: 30
│   │   ├── total_episodes: 50
│   │   └── total_frames: ...
├── data/
│   ├── episode_000000.parquet
│   ├── episode_000001.parquet
│   └── ...
└── videos/
    ├── observation.images.wrist/
    │   ├── episode_000000.mp4
    │   └── episode_000001.mp4
    ├── observation.images.top/
    │   ├── episode_000000.mp4
    │   └── episode_000001.mp4
```

4.8.2 Observation 和 Action 的 Key 命名规范

LeRobot 使用 `.` 作为层级分隔符，key 的命名遵循严格的语义约定：

<code>observation.state</code>	→ 电机状态（位置、速度等）的集合
<code>observation.images.wrist</code>	→ 腕部相机图像
<code>observation.images.global</code>	→ 全局相机图像
<code>observation.language</code>	→ 语言指令（多任务场景）
<code>observation.environment_state</code>	→ 环境状态（用于 Gym 环境）
<code>action</code>	→ 动作空间

在 PiperFollower 中，这些 key 是通过 `observation_features` 和 `action_features` 属性产生的：

observation_features 示例：

```
{
  "joint1.pos": float,      # 关节 1 位置
  "joint2.pos": float,      # 关节 2 位置
  "joint3.pos": float,      # 关节 3 位置
  "joint4.pos": float,      # 关节 4 位置
  "joint5.pos": float,      # 关节 5 位置
  "joint6.pos": float,      # 关节 6 位置
  "gripper.pos": float,     # 夹爪位置
  "wrist": (480, 640, 3),   # 腕部相机图像 (H, W, C)
```

```

    "top": (480, 640, 3),    # 顶部相机图像 (H, W, C)
}

```

action_features 示例:

```

{
    "joint1.pos": float,
    "joint2.pos": float,
    "joint3.pos": float,
    "joint4.pos": float,
    "joint5.pos": float,
    "joint6.pos": float,
    "gripper.pos": float,
}

```

注意: `action_features` 只包含电机位置（由策略模型输出），不包含图像。模型不需要输出图像。

4.8.3 特征类型定义规范

在 `src/lerobot/constants.py` 中定义了数据集中使用的标准 key 常量:

```

OBS_ENV_STATE = "observation.environment_state"
OBS_STATE = "observation.state"
OBS_IMAGE = "observation.image"
OBS_IMAGES = "observation.images"
OBS_LANGUAGE = "observation.language"
ACTION = "action"
REWARD = "next.reward"
TRUNCATED = "next.truncated"
DONE = "next.done"

```

PiperFollower 的 `get_observation()` 返回的是一个扁平字典 (flat dict)，每个 key 直接对应一个值或数组:

```

{
    "joint1.pos": 33.5,
    "joint2.pos": 50.0,
    ...
    "gripper.pos": 50.0,
    "wrist": np.ndarray(shape=(480, 640, 3), dtype=uint8),
    "top": np.ndarray(shape=(480, 640, 3), dtype=uint8),
}

```

LeRobot 数据集框架负责将这个扁平字典序列化到 `parquet` 文件（电机状态）和 `mp4` 视频文件（图像）。`parquet` 中每一行是一帧，包含所有标量观测和动作值；视频文件存储对应帧的图像帧。

4.10 LeRobot 支持的其他机器人

下表汇总了 LeRobot 代码库中已注册的所有机器人类型，展示 PiPER 在整个生态系统中的位置：

类型名	厂商/型号	电机总线	自由度	价格区间	特点
<code>piper_follower</code>	AgileX PiPER	CAN (piper_sdk)	6+1	中高端	双臂平台，工业级 CAN 通信
<code>koch_follower</code>	Koch v1.1	Dynamixel (RS-485)	6+1	低成本	开源低成本的前驱设计
<code>so100_follower</code>	SO-100	Feetech (RS-485)	6+1	低成本	Koch 的改进版
<code>so101_follower</code>	SO-101	Feetech (RS-485)	6+1	低成本	SO-100 的升级版
<code>bi_so100_follower</code>	SO-100 双臂	Feetech (RS-485) x2	2x(6+1)	低成本	双臂低成本方案
<code>lekiwi</code>	LeKiwi	Feetech + ZeroMQ	6	教育	面向教学的桌面机器人
<code>reachy2</code>	Pollen Robotics	reachy2_sdk	7+	高端	灵巧手 + 双臂 + 头部
<code>stretch3</code>	Hello Robot	Dynamixel	3+	中高端	移动操作平台
<code>viperx</code>	Trossen Robotics	Dynamixel	6+1	中端	Interbotix 系列
<code>hope_jr_arm</code>	Hope Jr.	Feetech	—	教育	教育平台的手臂部分
<code>hope_jr_hand</code>	Hope Jr.	Feetech	—	教育	教育平台的灵巧手部分

PiPER 在生态系统中的独特定位

与基于 Dynamixel 或 Feetech 舵机的低成本方案（Koch、SO-100）相比，PiPER 有以下几个独特特点：

- CAN 总线通信**——PiPER 使用 CAN (Controller Area Network) 总线而非 RS-485 串口。CAN 支持更高的带宽（最高 1 Mbps）和更强的抗干扰能力，适合工业环境。但这也意味着 PiPER 的 MotorsBus 实现不能复用 Dynamixel/Feetech 的 `sync_read` / `sync_write` 协议栈，而是直接通过 `piper_sdk` 的高层 API（`GetArmJointMsgs`、`JointCtrl` 等）完成通信。

2. **非标电机**——PiPER 使用 AgileX 定制的 AGILEX-M 和 AGILEX-S 电机，它们不兼容 Dynamixel 的控制表协议。因此 PiperMotorsBus 不需要 `_handshake`、`broadcast_ping`、`_find_single_motor`、`configure_motors` 等方法，这些在 Dynamixel/Feetech 上至关重要的方法在 PiPER 上都是空实现。
 3. **静态标定**——大多数 Dynamixel/Feetech 机器人需要执行动态标定过程（读取编码器、移动关节、计算范围）。PiPER 的关节运动范围在硬件层面是固定的，标定值直接硬编码在 `PiperFollower.__init__` 中。这简化了标定流程，但也意味着如果物理更换了一个运动范围不同的关节，需要手动修改源代码中的标定值。
 4. **双臂原生支持**——PiPER 是设计为双臂平台的：一只臂作为 Leader（主臂，用户拖动示教），一只臂作为 Follower（执行臂，执行策略）。两只臂通过各自的独立 CAN 口连接（主臂 `can0`，执行臂 `can1`），LeRobot 框架将 Leader 注册为 Teleoperator 类型，Follower 注册为 Robot 类型。这种架构与 LeRobot 的 Teleoperator → Robot 数据流水线完美契合。
-

本章小结

本章从源代码层面深入剖析了 LeRobot 框架的四层架构：从 CLI 入口点到 Robot/Teleoperator 抽象层，再到 MotorsBus 硬件抽象层，最终到达 `piper_sdk` 的硬件通信层。

核心设计理念可以概括为：

- **ChoiceRegistry** 的自动发现机制使得新增硬件类型只需一个装饰器声明，无需修改框架核心代码
- **MotorsBus** 的归一化/反归一化系统将所有不同量纲的原始编码器值统一映射到 `[-100, 100]` 或 `[0, 100]`，使得策略模型可以跨硬件迁移
- **Robot 与 Teleoperator** 的职责分离使数据采集流水线中的"谁产生动作"和"谁记录状态"两个角色清晰独立
- **MotorCalibration** 的 **per-joint** 标定确保每个关节的归一化精确反映其真实物理运动范围

理解这一层抽象对于后续理解 `lerobot-record` 的数据采集流程（第 5 章）和策略训练流程（第 6 章）至关重要，因为这些流程都建立在 Robot 和 Teleoperator 这两个抽象接口之上。

第五章 PiPER 电机总线适配层：PiperMotorsBus 完全解析

"适配层的本质不是翻译数据格式，而是翻译通信模型——把 CAN 总线的推模型翻译成 LeRobot 框架的拉模型。"

本章导览

本章是整个教程最重要的一章。前四章你已经理解了硬件基础（CAN 总线、电机型号）、SDK 原理（线程模型、CAN 编解码）和 LeRobot 框架（MotorsBus 基类）。现在所有知识点汇聚到同一个文件——`piper.py`。这个文件只有约 700 行，但它完成了一件极富挑战的工作：让一个为 Dynamixel 串口舵机设计的模仿学习框架，无缝驱动一台完全不同的 CAN 总线机械臂。

学完本章后，你应该能够：

- 完整解释 `get_action()` 和 `set_action()` 的每一步调用链
- 手写归一化/反归一化公式，并带入任意关节参数进行验算
- 理解为什么某些方法是空实现——不是因为"偷懒"，而是 CAN 总线和串口的通信模型根本不同
- 根据 PiPER 的校准表判断任意关节角度的有效范围
- 画出一帧完整数据帧（read-write cycle）的时序图

如果读完本章只能记住一句话，记住这句：

PiperMotorsBus 是一个**适配器**，它把 CAN 线上毫度为单位的电机编码器整数，翻译成 LeRobot 策略模型期望的 `[-100, 100]` 浮点数组，反之亦然。

5.1 "适配"到底是什么

5.1.1 问题的根源

LeRobot 是 HuggingFace 为 Dynamixel 和 Feetech 串口舵机设计的模仿学习框架。这类舵机的通信模型非常直接：



PiPER 机械臂则完全不同：



两者的通信模型差异可以归纳为下表：

维度	Dynamixel / Feetech	PiPER (CAN)
物理总线	UART 串口（TTL/RS-485）	CAN 总线（差分信号）
通信模型	拉模型：主机轮询每个舵机	推模型：机械臂主动上报
帧格式	Dynamixel Protocol 2.0（可变长包）	CAN 2.0B（8 字节固定数据场）
寻址方式	舵机 ID（0-252）	CAN ID（11 位或 29 位）
同步操作	<code>sync_read</code> / <code>sync_write</code> 广播指令	多帧发送，每帧对应不同 CAN ID
电机发现	<code>broadcast_ping()</code> 扫描所有 ID	不需要——只连一只臂
握手协议	Ping → 读型号 → 写配置	CAN 口 UP + 主动上报即建立通信

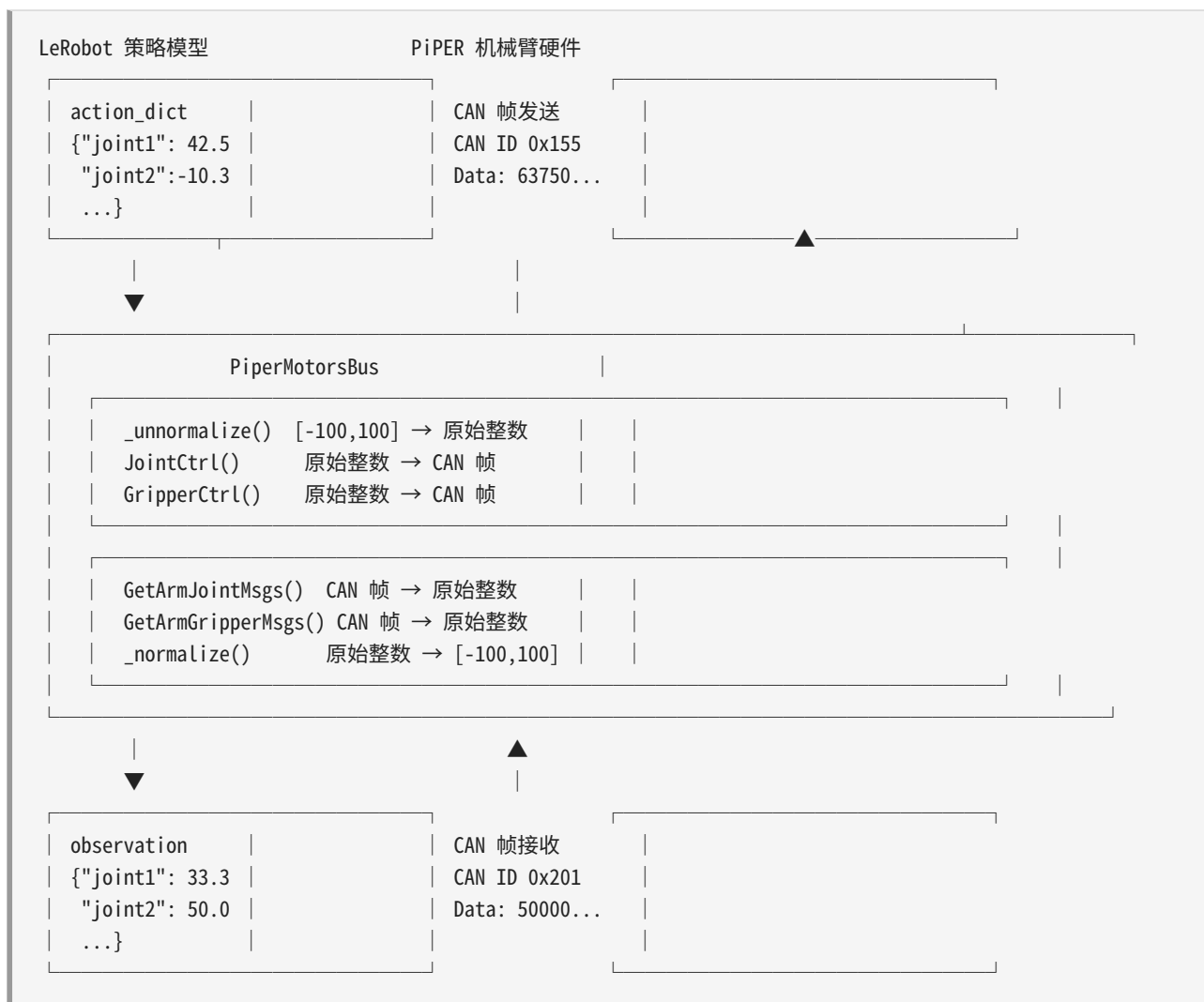
5.1.2 解决方案：适配器模式

软件工程有一个经典设计模式叫**适配器模式**（Adapter Pattern）：

将一个类的接口转换成客户期望的另一个接口。适配器让原本由于接口不兼容而不能一起工作的类可以合作。

在我们这个具体场景中：

- **客户期望的接口** = `MotorsBus` 基类（LeRobot 定义的统一接口）
- **被适配者** = `C_PiperInterface_V2`（SDK 的 CAN 通信接口）
- **适配器** = `PiperMotorsBus`



5.1.3 适配的三层含义

"适配"在这个项目中不是简单的数据格式转换。它发生在三个层面：

第一层：硬件通信层替代

Dynamixel 用串口协议包，PiPER 用 CAN 帧。在基类 `MotorsBus` 中，`_read()` / `_write()` / `_sync_read()` / `_sync_write()` 等方法底层都依赖 `PortHandler` + `PacketHandler` 收发串口数据包。PiPER 把这些方法全部替换为 SDK 的 `GetArmJointMsgs()` / `JointCtrl()` 等 CAN 原生调用。

第二层：数据格式转换

这是整个系统最核心的数学运算。SDK 读到的关节值是整型的毫度 (millidegrees) —— `joint1` 的范围是 `[-150000, 150000]`, `gripper` 的范围是 `[0, 68000]`。LeRobot 框架期望的是统一归一化到 `[-100, 100]` (关节) 或 `[0, 100]` (夹爪) 的浮点值。`_normalize()` 和 `_unnormalize()` 完成这层映射。

第三层：接口语义保留

尽管底层通信方式完全不同，但上层调用者（`PiperFollower`）看到的接口和 `Dynamixel` 版本一模一样——`get_action()` 返回 `dict[str, float]`，`set_action(action_dict)` 接收 `dict[str, float]`。这种语义保留使得策略模型和数据集格式无需任何修改就可以在不同硬件平台上切换。

5.1.4 比喻：电源适配器

用一个日常比喻来总结：

LeRobot 的 `MotorsBus` 接口就像 **USB-C 电源标准**（统一的电压/电流/协议）。PiPER 的 CAN 协议就像墙上 **220V 交流电插座**。`PiperMotorsBus` 就是那个 **充电头**：- 输入端（USB-C 口）接受标准的归一化值 `[-100, 100]` - 输出端（插头）输出 PiPER 特定的 CAN 协议（毫度整数 + CAN ID 帧）- 内部电路（`_normalize / _unnormalize`）完成交流→直流的数学转换

5.2 PiperMotorsBus 类结构

5.2.1 继承关系

```
abc.ABC
├── MotorsBus (motors_bus.py)    ← LeRobot 框架的电机总线抽象基类
└── PiperMotorsBus (piper.py)   ← PiPER CAN 总线实现
```

5.2.2 init 参数详解

```
def __init__(
    self,
    id: str,                    # 机器人标识
    port: str,                 # CAN 口名
    motors: dict[str, Motor],  # 电机定义字典
    calibration: dict[str, MotorCalibration] | None = None, # 标定信息字典
):
```

id: str — 机器人的唯一标识。在典型的 PC 中继遥操作（PC Relay Teleop）场景中，主臂 ID 为 `"pc_relay_leader"`，从臂 ID 为 `"pc_relay_follower"`。这个 ID 会被用于日志输出和校准文件路径规划。

port: str — Linux SocketCAN 接口名称，如 `"can0"` 或 `"can1"`。这个字符串直接传给 `C_PiperInterface_V2(port)` 构造函数，SDK 内部用它创建 `can.interface.Bus` 实例。

motors: dict[str, Motor] — 电机定义字典，键是关节名称（如 "joint1"），值是 **Motor** 数据类。每个 **Motor** 包含三个字段：

- **id: int** — 电机逻辑 ID（1~7）
- **model: str** — 电机型号字符串（"AGILEX-M" 或 "AGILEX-S"）
- **norm_mode: MotorNormMode** — 归一化模式，决定映射到 [-100, 100] 还是 [0, 100]

典型构造示例：

```
motors = {
    "joint1": Motor(1, "AGILEX-M", MotorNormMode.RANGE_M100_100),
    "joint2": Motor(2, "AGILEX-M", MotorNormMode.RANGE_M100_100),
    "joint3": Motor(3, "AGILEX-M", MotorNormMode.RANGE_M100_100),
    "joint4": Motor(4, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    "joint5": Motor(5, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    "joint6": Motor(6, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    "gripper": Motor(7, "AGILEX-S", MotorNormMode.RANGE_0_100),
}
```

calibration: dict[str, MotorCalibration] | None — 标定信息字典。每个 **MotorCalibration** 包含：

- **id: int** — 电机 ID
- **drive_mode: int** — 驱动模式（PiPER 不使用，始终为 0）
- **homing_offset: int** — 回零偏移（PiPER 不使用，始终为 0）
- **range_min: int** — 关节最小原始值（毫度）
- **range_max: int** — 关节最大原始值（毫度）

典型构造示例：

```
calibration = {
    "joint1": MotorCalibration(1, 0, 0, -150000, 150000),
    "joint2": MotorCalibration(2, 0, 0, 0, 180000),
    "joint3": MotorCalibration(3, 0, 0, -170000, 0),
    "joint4": MotorCalibration(4, 0, 0, -100000, 100000),
    "joint5": MotorCalibration(5, 0, 0, -65000, 65000),
    "joint6": MotorCalibration(6, 0, 0, -100000, 130000),
    "gripper": MotorCalibration(7, 0, 0, 0, 68000),
}
```

5.2.3 关键对象属性

构造函数初始化以下几个核心属性，它们是整个文件正常运转的基础：

self.piper: C_PiperInterface_V2 — 来自 AgileX 官方 SDK 的 CAN 接口对象。构造函数中通过 `C_PiperInterface_V2(port)` 创建，但不立即连接——连接操作推迟到 `connect()` 调用时。之后所有硬件操作都通过 `self.piper.XXX()` 调用。

self.port_handler: PortHandler — 来自 `wego_piper` 包的 CAN 口管理器。它只做三件事： 1. `setupPort(piper)` — 设置 CAN 口参数（波特率等） 2. `openPort()` — 打开 CAN 口 3. `closePort()` — 关闭 CAN 口

注意：`PortHandler` 来自第三方包 `wego_piper`，而 `C_PiperInterface_V2` 来自官方 `piper_sdk`。两者职责不同——`PortHandler` 只负责 CAN 口生命周期管理，`C_PiperInterface_V2` 负责 CAN 帧收发、编解码和控制命令。

self.id: str — 保存构造函数传入的机器人标识，用于日志和校准文件路径。

5.2.4 类属性（由父类方法引用）

`PiperMotorsBus` 定义了一些重要的类属性，它们会被父类 `MotorsBus` 的方法引用：

```
available_baudrates = [500_000, 1_000_000] # 支持的 CAN 波特率
default_timeout = 1000                      # 超时时间（毫秒）
apply_drive_mode = False                    # 不需要反转驱动模式
normalized_data = ["Present_Position", "Goal_Position"] # 需要归一化的数据字段
```

apply_drive_mode = False 是一个关键设计选择。在 Dynamixel 系统中，如果电机的 `drive_mode` 设为反转模式，`_normalize()` 会对结果取反（`-norm`），`_unnormalize()` 也会对输入取反。但 PiPER 不支持这种硬件级别的方向反转，也不需要通过软件来反转——因为校准表中的 `range_min` / `range_max` 已经隐式定义了方向（如 `joint3` 的 `min=-170000`, `max=0` 就是负向范围）。因此 `apply_drive_mode` 始终为 `False`，`drive_mode` 相关的取反逻辑永远不会触发。

5.3 覆写的方法（硬件层）

`PiperMotorsBus` 覆写了基类中所有需要直接与硬件交互的方法。这些方法构成了本类的"对外 API"，是策略模型、数据采集等上层代码实际调用的入口。

5.3.1 connect() — 建立 CAN 通信链路

```
def connect(self, handshake: bool = True) -> bool:
    self.port_handler.setupPort(self.piper) # ① 设置 CAN 口参数
    return self.port_handler.openPort()     # ② 打开 CAN 口
```

步骤 ① `setupPort(self.piper)`

调用 `wego_piper` 的 `PortHandler.setupPort()`。这个方法内部通过 `piper_sdk` 提供的接口设置 CAN 口波特率等底层参数。传入 `self.piper` 对象让 `PortHandler` 知道该在哪个 SDK 实例上操作。

步骤 ② `openPort()`

打开 CAN 口。返回 `True` 表示成功，返回 `False` 表示失败。成功打开后，机械臂的 CAN 上报帧（约 200Hz）开始流经 SocketCAN，SDK 内部的 `ReadCan` 线程开始接收和解析数据。

重要设计说明：`connect()` 只管通信链路，不管电机使能。

这是两层分离的架构设计：

- **Bus 层**（本层）：管通信——CAN 口是否打开，数据是否能收发
- **Robot 层**（`PiperFollower`）：管电机——何时使能，何时回零，何时初始化

电机使能由上层 `PiperFollower.connect()` 通过调用 `enable_torque()` 完成，不是 `PiperMotorsBus.connect()` 的职责。这种分离遵循了 LeRobot 的原始设计哲学，也让每个层次的责任更加清晰。

超时与重试

`connect()` 没有内置重试机制。如果 `openPort()` 返回 `False`，调用者（通常是 `PiperFollower.connect()`）会收到 `False` 并决定如何处理。常见的失败原因包括：

- CAN 口名称错误（如写成 `"can0"` 但实际是 `"can1"`）
- CAN 口未 `ip link set up`
- 权限不足（当前用户不在 `dialout` 或对应 group 中）
- USB-CAN 适配器未插入或被其他进程占用

5.3.2 `disconnect()` — 安全关闭连接

```
def disconnect(self, disable_torque: bool = False) -> None:
    if disable_torque:
        self.parking()           # ① 先回零位
        self.piper.DisablePiper() # ② 再失能电机
        self.port_handler.closePort() # ③ 最后关闭 CAN 口
```

步骤 ① `parking()` — 回到安全位置

调用 `parking()` 将所有关节移动到 `INITIALIZE_POSITION`（全零位姿）。这是安全措施——如果在机械臂处于极限位置时直接失能，手臂可能因重力猛然坠落，损坏硬件或伤及人员。

步骤② DisablePiper() — 释放扭矩

通过 CAN 发送失能命令，使电机进入自由状态。失能后，人可以轻松推动机械臂——这对于断电维护尤为重要。DisablePiper() 内部可能有重试逻辑（取决于 SDK 版本）。

步骤③ closePort() — 关闭 CAN 口

释放 SocketCAN 资源。关闭后，SDK 的 ReadCan 线程检测到 CAN 口关闭并退出。

参数说明：

- disable_torque=False（默认）：直接关闭 CAN 口，保持电机当前扭矩和位置。适用于程序临时重启（不改变机械臂姿态）。
- disable_torque=True：先回零再失能。适用于关机/维护场景。

5.3.3 get_action() — 读取当前关节状态（归一化）

这是整个系统调用最频繁的方法——每次数据采集（Record）、每次策略推理（Eval）、每次回放（Replay）都要调用。它完成从 CAN 帧到归一化字典的完整转换。

```
def get_action(self) -> dict[str, Any]:
    # ① 从 SDK 读取原始反馈值
    msg_joint = self.piper.GetArmJointMsgs()      # 读关节编码器
    msg_gripr = self.piper.GetArmGripperMsgs()    # 读夹爪编码器

    # ② 从消息对象中提取各字段
    rlt = {
        "joint1": float(msg_joint.joint_state.joint_1),
        "joint2": float(msg_joint.joint_state.joint_2),
        "joint3": float(msg_joint.joint_state.joint_3),
        "joint4": float(msg_joint.joint_state.joint_4),
        "joint5": float(msg_joint.joint_state.joint_5),
        "joint6": float(msg_joint.joint_state.joint_6),
        "gripper": float(msg_gripr.gripper_state.grippers_angle),
    }

    # ③ 归一化：原始整数 → [-100, 100] / [0, 100]
    rlt = self._normalize(rlt)
    return rlt
```

数据流全程追踪：

```
机械臂编码器
|
▼
[物理角度] joint1=45.0°, joint2=90.0°, ..., gripper=34.0°
|
▼ 驱动器 ADC 采样，转换为毫度整数
```

```

[原始值]    joint1=45000, joint2=90000, ..., gripper=34000
|
▼ 驱动器组装 CAN 帧 (CAN ID 0x201-0x20A)
[CAN 帧]    8 字节 data field, 200Hz 上报
|
▼ SocketCAN 接收 → ReadCan 线程回调 → Parser 解码
[SDK 消息]  msg_joint.joint_state.joint_1 = 45000
              msg_joint.joint_state.joint_2 = 90000
              ...
              msg_gripr.gripper_state.grippers_angle = 34000
|
▼ get_action() 提取各字段
[提取字典]  {"joint1": 45000, "joint2": 90000, ..., "gripper": 34000}
|
▼ _normalize() 应用归一化公式
[归一化]    {"joint1": 30.0, "joint2": 50.0, ..., "gripper": 50.0}

```

关键设计细节：

1. **SDK 消息对象不是实时读取 CAN 帧**，而是读取 `ReadCan` 线程缓存的最新值。因此 `GetArmJointMsgs()` 几乎没有延迟（ $\approx 0\text{ms}$ ），只是返回内存中已有的结构体。真正的延迟在 CAN 帧的采集到缓存更新之间。
2. **所有 6 个关节在一个消息对象中**，SDK 解析一帧 CAN 数据后同时更新全部 6 个关节的反馈值，这保证了关节状态的时间一致性。
3. **夹爪的读取是独立的**，因为它来自不同的 CAN ID（夹爪驱动器有自己的上报帧）。
4. **float() 转换**：SDK 返回的关节值是 `int` 类型，显式转为 `float` 是为了避免后续 `_normalize()` 中的整数除法问题（Python 中 `int / int` 会丢失精度）。

5.3.4 get_control() — 读取当前控制目标（未归一化）

```

def get_control(self) -> dict[str, Any]:
    msg_joint = self.piper.GetArmJointCtrl()    # 读取控制目标值
    msg_gripr = self.piper.GetArmGripperCtrl()  # 读取夹爪控制目标值
    rlt = {
        "joint1": msg_joint.joint_ctrl.joint_1,
        "joint2": msg_joint.joint_ctrl.joint_2,
        "joint3": msg_joint.joint_ctrl.joint_3,
        "joint4": msg_joint.joint_ctrl.joint_4,
        "joint5": msg_joint.joint_ctrl.joint_5,
        "joint6": msg_joint.joint_ctrl.joint_6,
        "gripper": msg_gripr.gripper_ctrl.grippers_angle,
    }
    return rlt

```

与 `get_action()` 的核心区别：

	get_action()	get_control()
数据来源	编码器反馈（实际位置）	控制命令值（目标位置）
SDK 方法	GetArmJointMsgs()	GetArmJointCtrl()
物理含义	"机械臂现在在哪"	"上次我让它去哪"
是否归一化	是（调用 <code>_normalize</code> ）	否（返回原始整数）
用途	训练/推理/数据集记录	确认命令已发出、调试
在 PC 中继中的特殊用途	—	读主臂当前位置（主臂处于示教模式时可能无数据） <code>GetArmJointMsgs()</code>

重要：为什么 PC 中继遥操作中读主臂要用 `GetArmJointCtrl()` ？

在 PC 中继模式中，主臂处于**示教模式**（Teaching Mode），电机释放扭矩，人可以随意拖动。在这种状态下，主臂驱动器的工作方式和正常伺服控制不同——它可能不产生常规的编码器反馈流（`GetArmJointMsgs()` 返回的可能不是最新的编码器值），但控制命令接口（`GetArmJointCtrl()`）仍然是活跃的，跟踪着主臂的当前位置。因此 `piper_pc_relay_teleop.py` 中读主臂用的是 `GetArmJointCtrl()`：

```
# piper_pc_relay_teleop.py 中的用法（示意）
leader_action = leader_bus.get_control() # 读主臂控制值
follower_bus.set_action(leader_action)   # 发给从臂执行
```

5.3.5 set_action() — 发送动作命令给机械臂

这是策略部署时最核心的方法——把策略模型输出的归一化动作翻译成 CAN 控制帧。

```
def set_action(self, action: dict[str, Any]) -> dict[str, Any]:
    # ① 反归一化: [-100,100]/[0,100] → SDK 原始整数
    action_denormalized = self._unnormalize(action)

    # ② 设置控制模式
    self.piper.ModeCtrl(0x01, 0x01, 30, 0x00)

    # ③ 发送关目标位置
    self.piper.JointCtrl(
        int(action_denormalized["joint1"]),
        int(action_denormalized["joint2"]),
        int(action_denormalized["joint3"]),
        int(action_denormalized["joint4"]),
        int(action_denormalized["joint5"]),
```

```

        int(action_denormalized["joint6"]),
    )

    # ④ 发送夹爪目标位置
    self.piper.GripperCtrl(
        abs(int(action_denormalized["gripper"])), 1000, 0x03, 0
    )

    # ⑤ 返回实际发出的命令值（用于日志/调试）
    return self.get_control()

```

参数：

- `action: dict[str, Any]` — 归一化值字典，形如 `{"joint1": 42.5, "joint2": -10.3, ..., "gripper": 75.0}`。键是关节名称，值是归一化浮点数。

返回值：

- `dict[str, Any]` — 通过 `get_control()` 返回当前控制目标值（原始整数）。返回值用于日志记录和调试，不是机械臂的实际到达位置。机械臂的运动是异步的——`set_action()` 返回后，电机还在执行运动中。

步骤 ① 反归一化 `_unnormalize()`

将 $[-100, 100]$ / $[0, 100]$ 的浮点动作值转换为 SDK 期望的毫度整数。例如 joint1 的 $42.5 \rightarrow 63750$ 毫度。详见 5.6 节的公式推导。

步骤 ② `ModeCtrl(0x01, 0x01, 30, 0x00)` — 设置控制模式

这是整条 CAN 控制链路的"开关"。参数含义：

参数位置	值	含义
第 1 参数	0x01	关节空间位置控制（Joint Position Control）
第 2 参数	0x01	在线控制模式
第 3 参数	30	速度百分比 30%（速度限制，安全考虑）
第 4 参数	0x00	不使用 MIT 模式（标准位置控制）

速度参数为什么是 30%？

这是安全设计。在 PC 中继遥操作场景（`piper_pc_relay_teleop.py`）中，速度通常设为 100%，因为主臂的动作由人实时控制，需要快速响应。而在策略部署场景（`lerobot-record` / `lerobot-eval`）中，策略模型的输出可能有抖动或不连续，30% 的速度限制可以减缓运动，降低碰撞风险。

模式控制帧的 CAN 通信过程：

`ModeCtrl(0x01, 0x01, 30, 0x00)` 内部会组装一条 CAN 帧并以 CAN ID `0x151` 发送到总线上。机械臂主控收到后，将控制模式切换到"关节位置控制"，并以 30% 的速度限制执行后续接收到的位置命令。

当前代码的行为：每次 `set_action()` 都发送 `ModeCtrl`

注意：在当前代码实现中，`ModeCtrl` 在每次 `set_action()` 调用中都发送一次。对于高速控制循环（如 30Hz 的策略推理），这意味着每秒发送 30 次相同的模式切换命令。虽然 CAN 总线设计上能够承受这种开销（每条 CAN 帧只有约 100 微秒的传输时间），但这仍然是不必要的冗余。

一个常见的优化是引入 `_mode_controlled` 标志位：

```
# 优化方案（当前代码未实现，仅供参考）
if not self._mode_controlled:
    self.piper.ModeCtrl(0x01, 0x01, 30, 0x00)
    self._mode_controlled = True
```

这样 `ModeCtrl` 只在第一帧发送，后续帧直接跳到 `JointCtrl`。需要注意：如果发生连接断开、错误恢复等异常，需要重置该标志位。

步骤 ③ `JointCtrl(j1, j2, j3, j4, j5, j6)` — 发送关节目标位置

一次性发送 6 个关节的目标位置（毫度整数）。SDK 内部将这 6 个值分别打包到对应的 CAN 帧中：

关节	CAN ID	负载
joint1, joint2	<code>0x155</code>	j1 (高字节 + 低字节), j2 (高字节 + 低字节)
joint3, joint4	<code>0x156</code>	j3, j4
joint5, joint6	<code>0x157</code>	j5, j6

每个关节值占 2 字节（`int16`）。三条 CAN 帧依次发送，总传输时间约 3ms（在 1Mbps 波特率下，每条 8 字节 CAN 帧约 100 微秒，加上帧间隔和可能的仲裁等）。

步骤 ④ `GripperCtrl(angle, speed, mode, block)` — 发送夹爪目标位置

参数	当前值	含义
<code>angle</code>	<code>abs(int(action_denormalized["gripper"]))</code>	目标位置（毫度），使用 <code>abs()</code> 确保非负
<code>speed</code>	<code>1000</code>	速度参数

参数	当前值	含义
mode	0x03	夹爪控制模式
block	0	非阻塞——不等待夹爪到达位置

为什么夹爪位置要用 `abs()` ？

`_unnormalize()` 对 `RANGE_0_100` 模式（夹爪）计算出的原始值：

```
raw = int((bounded_val / 100) * (max - min) + min)
```

其中 `min=0`, `max=68000`, `bounded_val` 被钳位在 `[0, 100]`。理论上讲结果一定非负。但为了防止浮点精度或异常输入导致微小负值（如 `-0.001` 经过 `int()` 变成 `-1`），加一层 `abs()` 保护是最稳妥的做法。

步骤 ⑤ 返回 `get_control()`

返回的不是机械臂的"实际位置"，而是刚发出的"目标命令值"。这提供了一个确认——命令确实已通过 CAN 发出。如果需要知道机械臂是否到达目标，应该在下一次循环中调用 `get_action()` 获取编码器反馈值，与期望值做比较。

5.3.6 Parking() — 回到初始安全位置

```
def parking(self):
    timeout = 100                                # 最多等 100 个周期
    self.set_action(INITIALIZE_POSITION)          # 发送初始位置作为目标
    time.sleep(0.1)
    status = self.piper.GetArmStatus()            # 读取运动状态

    while status.arm_status.motion_status and timeout:
        self.set_action(INITIALIZE_POSITION)      # 重发目标
        time.sleep(0.1)
        status = self.piper.GetArmStatus()
        timeout -= 1
```

`INITIALIZE_POSITION` 定义在 `tables.py` 中，所有 7 个关节（6 个关节 + 1 个夹爪）的目标值均为 0（归一化值）。这对应机械臂的"中立姿态"——所有关节回到原始位置，夹爪全开。

循环逻辑：

`GetArmStatus()` 返回的 `status.arm_status.motion_status` 是一个布尔值，表示机械臂当前是否正在运动中。`parking()` 的逻辑是：1. 先发送一次目标位置 2. 等待 100ms 3. 检查是否还在运动 4. 如果还在运动，重新发送目标位置（防止丢帧），再等 100ms 5. 重复，直到运动停止或超时

超时保护: `timeout = 100` 个周期, 每周期 100ms, 即最多等待 10 秒。如果 10 秒后机械臂仍未到达目标, 方法退出但不报错——这留给上层调用者处理。

安全提示:

`PiperFollower.connect()` 默认会调用 `parking()`。这意味着连接机械臂时它会自动运动到初始位姿。如果机械臂当前所在位置和初始位姿之间路径上有障碍物, 或者机械臂被卡住, 自动回零可能导致碰撞。此时应该使用 `connect(calibrate=False)` 跳过自动回零。

5.3.7 `clear_gripper()` — 张开夹爪

```
def clear_gripper(self):
    self.piper.GripperCtrl(0, 1000, 0x03, 0)
```

极其简单的方法: 发送目标位置 0 (毫度), 即夹爪完全张开。参数 1000 是速度, 0x03 是控制模式, 0 表示非阻塞。

注意: `clear_gripper()` 直接调用 SDK 方法, 跳过了 `set_action()` 的归一化流程——0 就是一个原始毫度值。这不会触发 `_mode_controlled` 标志位 (如果存在的话) 的检查。

5.3.8 `enable_torque()` — 使能电机

```
def enable_torque(self, motors=None, num_retry=0) -> bool:
    retry = 10
    while not self.piper.EnablePiper() and retry:
        retry -= 1
        time.sleep(0.1)
    logger.info(f"{self.piper.GetArmEnableStatus()}")
    if not retry:
        return False
    logger.info(f"{self.id} torque on.")
    return True
```

使能是通过 SDK 的 `EnablePiper()` 方法完成的, 它内部发送 CAN 命令到机械臂主控, 使所有电机进入伺服锁定状态。使能成功后, 电机保持当前角度, 抵抗外力——即俗称的"上电锁住"。

重试逻辑:

最多尝试 10 次, 每次间隔 100ms。所以总重试时间约为 1 秒。10 次全部失败返回 `False`。

参数说明:

- `motors` — 接受但不使用。接口保留是为了与基类 `MotorsBus` 的签名兼容 (Dynamixel 版本支持逐电机使能)。PiPER 不支持逐个电机单独使能——要么全部使能, 要么全不使能。

- `num_retry` — 同样接受但不使用。PiPER 版本使用自己内部的 `retry=10` 常量。

5.3.9 `disable_torque()` — 失能电机

```
def disable_torque(self, motors=None, num_retry=0) -> None:
    while self.piper.DisablePiper() and num_retry:
        num_retry -= 1
        time.sleep(0.01)
```

失能所有电机，释放扭矩。失能后，电机处于自由状态，手臂可以被人手轻易推动。

注意： 外层代码（如 `disconnect()`）通常在失能前会先调用 `parking()` 回到安全位置，防止自由落体。

5.3.10 主从模式配置

```
def set_slave(self):
    self.piper.MasterSlaveConfig(0xFC, 0, 0, 0)

def set_master(self):
    self.piper.MasterSlaveConfig(0xFA, 0, 0, 0)
```

这两个方法用于配置硬件级主从模式——两只 PiPER 机械臂连在同一 CAN 总线上，一只设为主臂（0xFA），一只设为从臂（0xFC），从臂自动跟随主臂运动。

当前项目中的应用：

在 PC 中继遥控方案中，主臂和从臂各连一条独立的 CAN 总线（`can0` 和 `can1`），电脑作为“中继站”从 `can0` 读取主臂状态，再通过 `can1` 发送命令给从臂。因此 `set_master()` / `set_slave()` 在当前实现中保留但未被调用。它们是为未来可能切换到纯硬件主从方案而预留的接口。

5.4 继承的方法（框架层）

以下方法和数据类直接继承自基类 `MotorsBus`，`PiperMotorsBus` 不做任何修改：

5.4.1 数据类

Motor — 描述一个电机的元数据：

```
@dataclass
class Motor:
    id: int          # 电机逻辑 ID
```

```
model: str          # 型号 ("AGILEX-M" 或 "AGILEX-S")
norm_mode: MotorNormMode # 归一化模式
```

MotorCalibration — 一个电机的标定信息：

```
@dataclass
class MotorCalibration:
    id: int          # 电机 ID
    drive_mode: int  # 驱动模式 (PiPER 不使用)
    homing_offset: int # 回零偏移 (PiPER 不使用)
    range_min: int   # 最小原始值 (毫度)
    range_max: int   # 最大原始值 (毫度)
```

MotorNormMode — 归一化模式枚举：

```
class MotorNormMode(str, Enum):
    RANGE_0_100 = "range_0_100"      # 映射到 [0, 100]
    RANGE_M100_100 = "range_m100_100" # 映射到 [-100, 100]
    DEGREES = "degrees"              # 映射到角度制 (PiPER 不使用)
```

5.4.2 继承的辅助方法

方法	功能	PiPER 是否需要
<code>_id_to_model(id_)</code>	电机 ID → 型号字符串	是——用于 <code>_normalize</code> 中的 DEGREES 模式
<code>_id_to_name(id_)</code>	电机 ID → 名称	是——用于日志和调试
<code>_get_motor_id(motor)</code>	统一获取电机 ID (名称或 ID 均可)	继承使用
<code>_get_motor_model(motor)</code>	统一获取电机型号	继承使用
<code>_get_motors_list(motors)</code>	参数归一化为电机名称列表	继承使用
<code>is_connected</code> (property)	检查 CAN 口是否已打开	继承使用 (依赖 <code>port_handler.is_open</code>)

5.4.3 `sync_read()` — 读取原始值

虽然 `get_action()` 是主要的读取方法，但 `sync_read()` 仍然通过继承保留，用于读取 `Present_Position` 等原始值（不做归一化）。在某些调试场景中很有用：

```
# 读取原始编码器值（不归一化）
raw_positions = bus.sync_read("Present_Position", normalize=False)
# 返回: {"joint1": 45000, "joint2": 90000, ...}
```

5.5 空实现的方法（Dynamixel 遗留）

以下是 PiperMotorsBus 中所有空实现/no-op 的方法，以及为什么它们是空的。理解这些"为什么"比"是什么"更重要——它们揭示了 CAN 总线和 Dynamixel 串口在通信模型上的根本差异。

5.5.1 方法清单

方法	返回值	为什么是空实现
<code>_assert_protocol_is_compatible()</code>	pass	Dynamixel 需要检查协议版本兼容性（1.0 vs 2.0），CAN 没有"协议版本"概念
<code>_handshake()</code>	pass	Dynamixel 需要通过 Ping 确认所有电机在线，CAN 的"握手"就是 <code>openPort()</code> 成功
<code>_find_single_motor()</code>	pass	Dynamixel 需要逐个扫描 ID 找到刚出厂的电机（丢失 ID），CAN 上只有一只臂
<code>broadcast_ping()</code>	pass	Dynamixel 通过广播地址扫描总线上所有舵机，CAN 没有广播寻址机制
<code>configure_motors()</code>	pass	Dynamixel 需要设置回包延迟、加速度限制等参数，PiPER 的电机参数由 SDK 管理
<code>read_calibration()</code>	pass	Dynamixel 的标定值存在电机 EEPROM 中需要读取，PiPER 的标定是 Python 代码中硬编码的
<code>write_calibration()</code>	pass	同上，PiPER 不需要向硬件写入标定值
<code>_get_half_turn_homings()</code>	pass	Dynamixel 的半圈回零概念基于 12-bit 编码器（一圈 4096），PiPER 使用毫度单位
<code>_encode_sign()</code>	返回原值	Dynamixel 需要对某些数据类型做符号编码（补码转换），PiPER 的值已是标准补码
<code>_decode_sign()</code>	返回原值	同上，PiPER 不需要额外的符号解码
<code>_split_into_byte_chunks()</code>	pass	Dynamixel 需要手动拆分整数为字节数组组装串口包，CAN 帧由 SDK 内部处理

5.5.2 深入理解：为什么 CAN 不需要这些

1. 不需要 Ping/握手/电机发现（`_handshake`，`broadcast_ping`，`_find_single_motor`）

在 Dynamixel 总线上，多个舵机以 Daisy Chain 方式串联。每个舵机有一个唯一的 ID (0-252)，但出厂时可能都是 ID=1。因此需要一套"电机发现"协议：先广播 Ping，收到响应的就确认存在；如果冲突（两个舵机都是 ID=1），需要先逐个断电、改 ID，再全部接回。

PiPER 的 CAN 总线上只有一只机械臂。你不需要"发现"它——`openPort()` 成功就意味着通信链路畅通。每个关节数据的 CAN ID 是固定的（由 PiPER 协议定义），不存在"舵机 ID 冲突"问题。

2. 不需要字节拆分 (`_split_into_byte_chunks`)

Dynamixel Protocol 2.0 的串口数据包需要手动组装：header (0xFF 0xFF 0xFD) + reserved (0x00) + ID + length + instruction + parameters + CRC。其中 parameters 需要按 little-endian 拆分成字节数组。

PiPER 使用的是 CAN 2.0B 协议。`JointCtrl()` 内部调用 `C_PiperParserV2` 自动将 6 个 `int16` 关节值打包到 3 条 8 字节 CAN 帧中。开发者不需要关心字节序或帧格式。

3. 不需要符号编解码 (`_encode_sign`, `_decode_sign`)

某些 Dynamixel 型号的 `Present_Position` 使用特殊编码（如补码格式），需要软件层面做符号转换。PiPER 的关节值就是标准的 `int16` 毫度值，无需额外编码。

4. 不需要写入硬件标定 (`write_calibration`, `read_calibration`)

Dynamixel 舵机有 EEPROM 存储区，可以持久化保存标定参数（最小/最大位置限制、回零偏移等）。因此标定流程是：先 `read_calibration()` 从硬件读取已有标定 → 修改 → 再 `write_calibration()` 写回硬件。

PiPER 的标定值是固定的——每个关节的范围由机械设计决定，不会改变。因此标定信息直接以 Python 字典形式硬编码在 `PiperFollower.__init__()` 中，不需要读写硬件。

5. 不需要角度制模式 (`DEGREES`)

Dynamixel 的编码器分辨率因型号而异（如 XM430 是 4096 脉冲/圈，XL330 是 4096），`DEGREES` 模式通过 `(val - mid) * 360 / max_res` 公式将编码器脉冲转换为角度制。

PiPER 直接使用毫度单位，所有值已经是角度单位（只是放大了 1000 倍）。`DEGREES` 模式不适用，因此虽然 `_normalize()` 和 `_unnormalize()` 保留了 `DEGREES` 分支，但在 PiPER 上永远不会进入。

5.5.3 设计哲学

这些空实现不是"尚未完成"的 TODO，而是一种有意的设计选择——**接口继承 + 空实现**比"创建全新的基类"更好，原因是：

1. **框架兼容性**：PiperFollower 等上层代码调用 `bus.broadcast_ping()` 时不会报 `AttributeError`，而是静默跳过。这让 PiPER 版本可以在不修改 LeRobot 框架代码的情况下运行。
 2. **遗留代码保护**：某些 LeRobot 工具脚本（如 `lerobot-find-port.py`、`calibrate.py`）可能会调用这些方法。空实现让这些脚本在不经过大改的情况下就能与 PiPER 共存。
 3. **未来扩展性**：如果将来 PiPER 支持类似功能（比如通过 CAN 命令读取电机型号），只需在这些空方法中添加实际代码，接口保持不变。
-

5.6 归一化数学详解

这是整个系统最核心的数学部分。如果你只能从本章带走一个知识点，就是这一节。

5.6.1 为什么需要归一化

原因一：统一接口

不同机械臂的编码器使用不同的单位。Dynamixel 舵机用"脉冲"（0-4095），PiPER 用"毫度"（-150000 ~ 150000），UR5 用"弧度"。如果没有归一化，策略模型的输入/输出格式与机械臂型号强绑定，切换硬件平台等于重写全部代码。

归一化后，**所有机械臂的策略输入都是 $[-100, 100]$** 。模型不需要知道底层用的是毫度还是脉冲还是弧度。

原因二：数值稳定性

神经网络训练对输入数值范围敏感。如果 joint1 的输入范围是 $[-150000, 150000]$ 而 joint3 的是 $[-170000, 0]$ ，量纲差异巨大。优化器（如 Adam）对不同量纲的梯度缩放不均匀，导致训练困难。归一化到固定范围消除这种不均衡。

原因三：迁移学习（理论）

如果两个机械臂的结构相似（如都是 6 轴串联），在机械臂 A 上训练的策略原则上可以迁移到机械臂 B 上推理。归一化提供了这种可能性——策略只学习"相对位置"而不是"绝对编码器值"。

5.6.2 RANGE_M100_100 归一化（关节，6 个）

公式：

$$\text{norm} = \frac{\text{raw} - \text{min}}{\text{max} - \text{min}} \times 200 - 100$$

分步解释：

```

步骤 1: (raw - min) / (max - min)    → 映射到 [0, 1]      (归一化到 0-1 区间)
步骤 2: × 200                        → 映射到 [0, 200]     (拉伸到 0-200)
步骤 3: - 100                       → 映射到 [-100, 100]  (平移到对称区间)

```

边界验证：

```

当 raw = min → norm = ((min - min) / (max - min)) × 200 - 100
               = 0 × 200 - 100
               = -100 ✓

当 raw = max → norm = ((max - min) / (max - min)) × 200 - 100
               = 1 × 200 - 100
               = 100  ✓

当 raw = (min + max) / 2 → norm = (0.5 × 200) - 100
                           = 100 - 100
                           = 0    ✓ 中位映射到 0

```

5.6.3 RANGE_M100_100 反归一化

公式：

$$\text{raw} = \left\lfloor \frac{\text{norm} + 100}{200} \times (\text{max} - \text{min}) + \text{min} \right\rceil$$

实际代码中 `round()` 由 `int()` 的截断实现（因为经过钳位后值为正且加上 `min` 后范围合理）：

```
raw = int(((bounded_val + 100) / 200) * (max - min) + min)
```

分步解释：

```

步骤 1: (norm + 100) / 200    → 从 [-100,100] 映射回 [0, 1]
步骤 2: × (max - min)         → 拉伸到原始范围宽度
步骤 3: + min                 → 平移到原始坐标系
步骤 4: int(...)              → 转为整数（毫度）

```

边界验证：

```

当 norm = -100 → raw = int((-100 + 100) / 200 × (max - min) + min)
                  = int(0 + min)
                  = min  ✓

```

```

当 norm = 100    → raw = int(((100 + 100) / 200) × (max - min) + min)
                  = int(1.0 × (max - min) + min)
                  = max  ✓

当 norm = 0      → raw = int((100 / 200) × (max - min) + min)
                  = int(0.5 × (max - min) + min)
                  = (min + max) / 2  ✓  0 映射到中点

```

5.6.4 RANGE_0_100 归一化（夹爪，1个）

归一化公式：

$$\text{norm} = \frac{\text{raw} - \text{min}}{\text{max} - \text{min}} \times 100$$

反归一化公式：

$$\text{raw} = \left\lfloor \frac{\text{norm}}{100} \times (\text{max} - \text{min}) + \text{min} \right\rceil$$

代码：

```

# 归一化
norm = ((bounded_val - min_) / (max_ - min_)) * 100

# 反归一化
raw = int((bounded_val / 100) * (max_ - min_) + min_)

```

与 RANGE_M100_100 的核心区别： 输出范围是 [0, 100] 而不是 [-100, 100]。这是因为夹爪只有"开"和"闭"两个方向（开口大小是单向的），不像关节有正负两个运动方向。

代码中的安全钳位：

在归一化（正向）中：

```
bounded_val = min(max_, max(min_, val)) # 将原始值钳位在 [min, max] 内
```

在反归一化（逆向）中：

```

# RANGE_M100_100:
bounded_val = min(100.0, max(-100.0, val)) # 钳位在 [-100, 100]

# RANGE_0_100:
bounded_val = min(100.0, max(0.0, val))    # 钳位在 [0, 100]

```

两层安全钳位保证了即使输入值异常（传感器跳变、模型输出越界），也不会发出超出机械限位的命令。

5.6.5 带具体数值的完整计算示例

以 **joint1** 为例，走一遍完整的归一化+反归一化流程。

给定参数：

- `calibration["joint1"]` : `range_min = -150000` , `range_max = 150000` (即 $\pm 150^\circ$)
- `motors["joint1"].norm_mode` : `RANGE_M100_100`
- 假设 SDK 读到的编码器反馈值: `raw = 45000` (即 45.0°)

归一化计算：

```
norm = ((45000 - (-150000)) / (150000 - (-150000))) × 200 - 100
      = (195000 / 300000) × 200 - 100
      = 0.65 × 200 - 100
      = 130 - 100
      = 30.0
```

验证物理意义： 45° 是 $\pm 150^\circ$ 范围的 65% 位置 (从 -150° 算起: $-150 + 300 \times 0.65 = 45 \checkmark$)。65% $\times 200 - 100 = 30 \checkmark$ 。

反归一化验证：

```
raw = int(((30.0 + 100) / 200) × (150000 - (-150000)) + (-150000))
     = int((130 / 200) × 300000 - 150000)
     = int(0.65 × 300000 - 150000)
     = int(195000 - 150000)
     = int(45000)
     = 45000 ✓
```

往返一致: $45000 \rightarrow 30.0 \rightarrow 45000$ 。

以 **joint3** 为例 (不对称、负向范围)：

- `range_min = -170000` , `range_max = 0` (范围为 $[-170^\circ, 0^\circ]$)
- 假设 `raw = -85000` (即 -85° , 范围中点)

```
norm = ((-85000 - (-170000)) / (0 - (-170000))) × 200 - 100
      = (85000 / 170000) × 200 - 100
      = 0.5 × 200 - 100
      = 0.0
```

中点映射到 0.0 $\checkmark$ 。虽然 **joint3** 的原始值全是负数 (-170000 到 0)，但归一化后中间位置仍然是 0。

以 gripper 为例（RANGE_0_100 模式）：

- `range_min = 0` , `range_max = 68000` （范围为 $[0^\circ, 68^\circ]$ ）
- 假设 `raw = 34000` （即 34° ，半开）

```
norm = ((34000 - 0) / (68000 - 0)) × 100
      = 0.5 × 100
      = 50.0
```

半开位置映射到 50.0 ✓。

5.6.6 归一化在数据 Pipeline 中的位置



关键洞察：**训练数据和推理数据使用完全相同的归一化公式**。如果训练时 `joint1` 的范围是 $[-150000, 150000]$ ，推理时也必须是这个范围。改变校准参数会导致“概念偏移”（concept drift）——模型学到的 30.0 对应 45° ，但你改变校准后 30.0 可能对应不同的角度，机械臂的行为就会出错。

5.7 校准范围表

以下是 PiPER 机械臂所有 7 个驱动电机的校准范围。这些值来自 PiPER 的机械结构设计——由关节的物理限位和驱动器软限位共同决定。

5.7.1 完整校准表

关节	电机型号	Motor ID	归一化模式	MIN (毫度)	MAX (毫度)	角度范围	范围宽度 (度)
joint1	AGILEX-M	1	RANGE_M100_100	-150000	150000	$\pm 150^\circ$	300°
joint2	AGILEX-M	2	RANGE_M100_100	0	180000	0 ~ 180°	180°
joint3	AGILEX-M	3	RANGE_M100_100	-170000	0	-170° ~ 0°	170°
joint4	AGILEX-S	4	RANGE_M100_100	-100000	100000	$\pm 100^\circ$	200°
joint5	AGILEX-S	5	RANGE_M100_100	-65000	65000	$\pm 65^\circ$	130°
joint6	AGILEX-S	6	RANGE_M100_100	-100000	130000	-100° ~ 130°	230°
gripper	AGILEX-S	7	RANGE_0_100	0	68000	0 ~ 68°	68°

5.7.2 电机型号分布

PiPER 机械臂使用两种型号的电机：

- **AGILEX-M** (3 个)：用于底座 3 个关节 (joint1, joint2, joint3) ——负载大，运动范围大，需要更大的扭矩。
- **AGILEX-S** (4 个)：用于腕部 3 个关节和夹爪 (joint4, joint5, joint6, gripper) ——负载小，对精度要求更高。

5.7.3 特殊范围解读

joint2 为什么 min=0?

joint2 是大臂的俯仰关节。它的机械结构不允许向后弯曲——大臂只能从垂直状态向前倾，不能向后仰（否则会撞到机械臂底座）。因此它的范围是单边正向 $[0^\circ, 180^\circ]$ ，min=0。在归一化时， 0° 对应 norm=-100， 180° 对应 norm=100。

joint3 为什么全是负数?

joint3 是小臂（肘关节）的俯仰关节。它的机械结构允许向下弯曲最多 170° ，但无法向上弯曲到超过大臂延长线（ 0° 被定义为与大臂共线）。因此它的范围是 $[-170^\circ, 0^\circ]$ 。虽然全是负数，归一化后中位（ -85° ）仍然映射到 norm=0。

joint4 为什么是 $\pm 100^\circ$?

joint4 是腕部旋转关节，负责手腕的 pronation/supination（旋前/旋后）。机械设计上可以有近 200° 的旋转范围。注意这个关节的旋转轴是沿小臂方向的，与其他俯仰关节不同。

joint5 为什么范围最小 ($\pm 65^\circ$) ?

joint5 是腕部俯仰关节。它的运动范围受限于腕部的机械结构和夹爪安装位置。过大的俯仰角度可能导致夹爪与机械臂本体碰撞。

joint6 为什么不对称 ($-100^\circ \sim 130^\circ$) ?

joint6 是腕部旋转关节（末端法兰旋转）。它的正负不对称可能是因为：- 正向 ($+130^\circ$) 旋转时不会拉扯内部线缆 - 负向 (-100°) 旋转时，内部线缆和气管的盘绕角度有限

gripper 为什么使用 RANGE_0_100?

夹爪只有"开"和"闭"两个方向，不像关节有正负两个运动方向。因此使用 RANGE_0_100: 0 表示全开，100 表示全闭。夹爪范围宽度 68° 意味着手指从完全打开到完全闭合运动 68° （毫度 68000）。

5.7.4 归一化中点对应的物理角度

关节	归一化值 0 对应的原始值	对应的物理角度
joint1	0	0° （中立，朝正前方）
joint2	90000	90° （水平前伸）
joint3	-85000	-85° （半弯）
joint4	0	0° （中立旋转位）
joint5	0	0° （中立腕部）
joint6	15000	15° （微偏）
gripper	34000	34° （半开）

5.8 tables.py 内容

tables.py 是 PiPER 电机的常量定义文件，虽然只有约 90 行，但包含了电机控制所必需的全部参数表。

5.8.1 MODEL_RESOLUTION_TABLE — 编码器分辨率

```
MODEL_RESOLUTION_TABLE = {  
    "AGILEX-M": 4096,  
    "AGILEX-S": 4096,  
}
```

两种型号的编码器分辨率都是 4096 脉冲/转。这个值只在 `DEGREES` 归一化模式下使用（`max_res = model_resolution - 1 = 4095`），PiPER 当前不使用该模式，因此这个表虽定义但实际未参与计算。

5.8.2 MODEL_NUMBER_TABLE — 型号编号

```
MODEL_NUMBER_TABLE = {  
    "AGILEX-M": 1190,  
    "AGILEX-S": 1191,  
}
```

每个电机型号在 Dynamixel 协议中对应一个唯一的型号编号。虽然 PiPER 不使用 Dynamixel 协议，但保留这些定义是为了框架兼容性（`MotorsBus.__init__` 中的 `_validate_motors()` 和 `_model_nb_to_model_dict` 会用到）。

5.8.3 INITIALIZE_POSITION — 初始化位置

```
INITIALIZE_POSITION = {  
    "joint1": 0,  
    "joint2": 0,  
    "joint3": 0,  
    "joint4": 0,  
    "joint5": 0,  
    "joint6": 0,  
    "gripper": 0,  
}
```

这是 `parking()` 方法使用的目标位置。所有值均为 0（归一化值），对应各关节的中立位置和夹爪的全开状态。注意：这里的 0 是归一化值，不是原始毫度值。在 `norm=0` 的情况下：

- joint1: 原始值 = 0 毫度 = 0°（中立）
- joint2: 原始值 = 90000 毫度 = 90°（前伸）
- joint3: 原始值 = -85000 毫度 = -85°（半弯）
- joint4: 原始值 = 0 毫度 = 0°
- joint5: 原始值 = 0 毫度 = 0°
- joint6: 原始值 = 15000 毫度 = 15°

- gripper: 原始值 = 0 毫度 = 0° (全开)

5.8.4 MODEL_BAUDRATE_TABLE — 波特率映射

```
MODEL_BAUDRATE_TABLE = {
    1_000_000: 3,
    2_000_000: 4,
    3_000_000: 5,
    4_000_000: 6,
}
```

将波特率 (bps) 映射到 Dynamixel 协议中的波特率编号。PiPER 使用 CAN 总线，波特率固定为 1Mbps 或 500kbps，这个表保留为框架兼容性。

5.8.5 其他表

```
AVAILABLE_BAUDRATES = [9600, 19200, ..., 4000000] # 支持的全部波特率列表
MODEL_ENCODING_TABLE = {"AGILEX-M": {}, "AGILEX-S": {}} # 编码参数 (空)
MODEL_CONTROL_TABLE = {"AGILEX-M": 1190, "AGILEX-S": 1191} # 控制表定义
MODEL_OPERATING_MODES = {
    "AGILEX-M": [0, 1, 3, 4, 5, 16],
    "AGILEX-S": [0, 1, 3, 4, 5, 16],
} # 支持的操作模式
```

这些表中大部分是 Dynamixel 协议的遗留定义，保留在 PiPER 版本中是为了保持框架兼容性。

`MODEL_CONTROL_TABLE` 实际上存储的是型号编号而非控制表定义，但变量名与 LeRobot 基类的 `model_ctrl_table` 保持一致。

5.9 get_action() vs set_action() 的完整时序

5.9.1 数据采集集中一个帧周期的时序图

在 `lerobot-record` (数据采集) 模式中，每一帧 (约 30 FPS，即 33ms 一个周期) 执行以下操作：





关键时间数据：

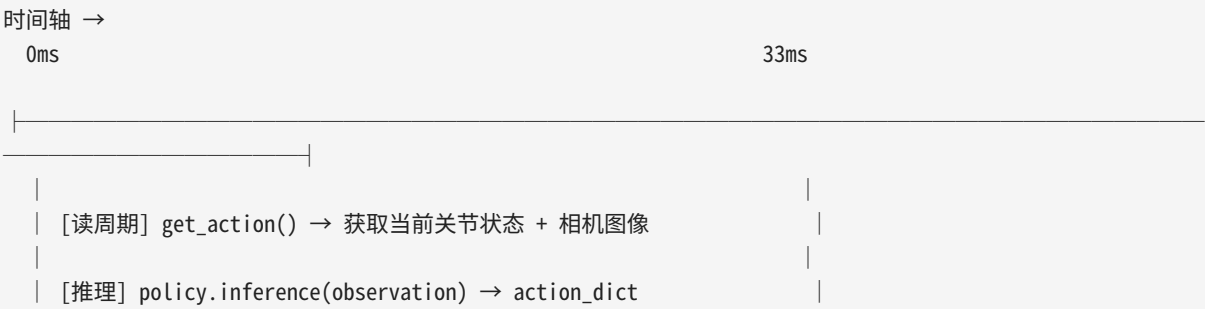
操作	耗时	占比
读取关节状态（缓存读）	< 0.1ms	< 0.3%
归一化 + 反归一化计算	< 0.02ms	< 0.1%
CAN 帧发送（4 条）	~0.5ms	~1.5%
相机帧捕获（异步）	~30ms	~91%
空闲等待	~3ms	~9%

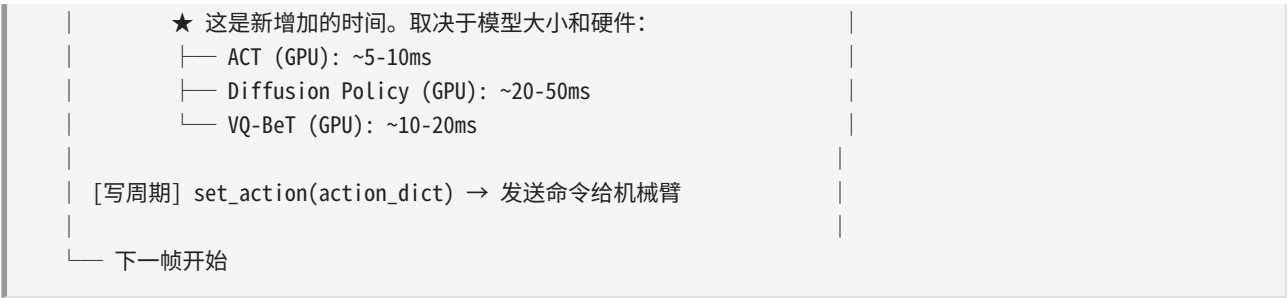
瓶颈分析：

显而易见，**相机采集是整个 Pipeline 的瓶颈**。CPU 计算（归一化/反归一化）和 CAN 通信的时间几乎可以忽略不计。这也是为什么 `get_action()` 中读取的是 `ReadCan` 线程的缓存值而不是实时触发一次 CAN 读取——在 33ms 的帧周期面前，缓存数据的新鲜度完全充足。

5.9.2 策略推理中一个帧周期的时序图

在 `lerobot-eval`（策略推理/部署）模式中，`timeline` 稍有不同：





与 Record 模式的核心区别： - Record 模式：写周期发送的是**来自主臂的动作**（通过 PC 中继转发），没有推理开销 - Eval 模式：写周期发送的是**策略模型的推理输出**，增加了推理时间

对于 Diffusion Policy 这种较大的模型（50ms 推理时间），33ms 帧周期可能不够。此时需要降帧率（如 15 FPS）或使用异步推理。这也是为什么 LeRobot 支持数据集的异步重放——某些策略模型达不到实时 30 FPS。

5.9.3 CAN 总线利用率分析

在 30 FPS 的帧率下，每秒发送的 CAN 帧数量：

CAN 帧	CAN ID	每帧调用数/秒	每秒总帧数	每帧比特数	每秒比特数
ModeCtrl	0x151	30	30	~100	3,000
JointCtrl	0x155	30	30	~100	3,000
JointCtrl	0x156	30	30	~100	3,000
JointCtrl	0x157	30	30	~100	3,000
GripperCtrl	0x159	30	30	~100	3,000
主动上报帧	0x201-0x20A	200	200×10	~100	200,000
合计			2,150		~215,000

CAN 1Mbps 的带宽利用率 = $215,000 / 1,000,000 \approx 21.5\%$ 。即使加上帧间隔（3 bit intermission）、填充位（bit stuffing）等开销，利用率也在 30% 以下，远未达到 CAN 总线的饱和点（通常 60-70% 为安全上限）。因此 CAN 带宽不是瓶颈。

5.10 完整代码走读

本节逐段走读 `piper.py` 的全部约 700 行代码，解释每一部分的用途、设计意图和与其他模块的交互。

5.10.1 文件头注释与版权 (第 1-32 行)

```
文件: PiperMotorsBus — LeRobot 与 PiPER 机械臂硬件之间的翻译器 + 搬运工
```

文件头的 ASCII 图示精确描述了本文件在整个系统中的位置——它是策略模型输出与真实机械臂运动之间的桥梁。依赖关系注释标注了四个关键外部模块：`piper_sdk`、`wego_piper`、`.tables` 和 `..motors_bus`。

5.10.2 导入部分 (第 34-81 行)

标准库导入： `time` (sleep 和超时控制)、`logging` (日志)、`typing.Any` (类型注解)。

LeRobot 框架导入：

```
from ..motors_bus import Motor, MotorCalibration, MotorsBus, MotorNormMode, NameOrID, Value, get_address
```

从上层 `motors_bus.py` 导入基类和核心数据类型。`Motor` 和 `MotorCalibration` 是数据类，`MotorsBus` 是抽象基类，`MotorNormMode` 是枚举。

PiPER SDK 导入：

```
from piper_sdk import *
```

导入所有 AgileX 官方 SDK 的公开符号。核心符号包括 `C_PiperInterface_V2` (主接口类)、`C_PiperInterface` (V1 版本) 和 `LogLevel` (日志级别枚举)。

第三方库导入：

```
from wego_piper.port_handler import PortHandler
```

`PortHandler` 来自 `wego_piper` 包，封装了 CAN 口的三个基本操作 (setup/open/close)。

常量表导入：

```
from .tables import (
    AVAILABLE_BAUDRATES, MODEL_BAUDRATE_TABLE, MODEL_CONTROL_TABLE,
    MODEL_ENCODING_TABLE, MODEL_NUMBER_TABLE, MODEL_RESOLUTION_TABLE,
    INITIALIZE_POSITION,
)
```

5.10.3 类定义与类属性 (第 89-120 行)

```
class PiperMotorsBus(MotorsBus):
```

类属性的作用域是类级别（所有实例共享）。关键类属性：

- `available_baudrates = [500_000, 1_000_000]`：覆盖基类的长列表，因为 CAN 只支持这两种波特率。
- `default_timeout = 1000`：超时 1 秒。
- `apply_drive_mode = False`：核心设计选择，让 `_normalize` / `_unnormalize` 中的 `drive_mode` 分支永不触发。
- `normalized_data = ["Present_Position", "Goal_Position"]`：标记哪些数据字段名需要经过归一化/反归一化处理。
- `model_baudrate_table` / `model_ctrl_table` / `model_encoding_table` / `model_number_table` / `model_resolution_table`：从 `tables.py` 导入的电机参数表，赋值为类属性供父类方法引用。

5.10.4 `init()` (第 125-170 行)

```
def __init__(self, id, port, motors, calibration=None):
    super().__init__(port, motors, calibration)
    self.port_handler = PortHandler()
    self.id = id
    self.piper = C_PiperInterface_V2(port)
```

调用顺序分析：

1. `super().__init__(port, motors, calibration)` — 先调用父类构造函数，它会：
2. 保存 `self.port`, `self.motors`, `self.calibration`
3. 构建 `_id_to_model_dict` 和 `_id_to_name_dict` 映射表
4. 调用 `_validate_motors()` 检查电机 ID 唯一性和控制表存在性
5. 创建 `PortHandler` 实例（暂不打开任何端口）
6. 保存 `self.id` 用于日志和校准文件路径
7. 创建 `C_PiperInterface_V2(port)` 实例（暂不连接 CAN 口）

注意： `C_PiperInterface_V2` 使用单例模式（按 `can_name`）。如果同一个 CAN 口已有其他代码创建了 `C_PiperInterface_V2` 实例，`C_PiperInterface_V2(port)` 会返回已缓存的实例。这种行为在单臂场景下没问题，但在双臂 PC 中继场景（`can0` 和 `can1` 各一个实例）中很重要——每个 `PiperMotorsBus` 持有对应 CAN 口的独立 SDK 实例。

5.10.5 连接与断开 (第 174-211 行)

```
def _assert_protocol_is_compatible(self, instruction_name):
    pass # PiPER 不需要协议兼容性检查

def _handshake(self):
    pass # PiPER 不需要握手协议

def _find_single_motor(self, motor, initial_baudrate):
    pass # PiPER 不需要逐电机扫描
```

三个空实现已在 5.5 节详细解释过。它们被保留是因为它们是 `MotorsBus` 的抽象方法，必须实现，但 PiPER 不需要这些功能。

```
def connect(self, handshake: bool = True) -> bool:
    self.port_handler.setupPort(self.piper)
    return self.port_handler.openPort()
```

极其简洁的 `connect()` ——只做 CAN 口的设置和打开。`handshake` 参数接受但不使用（保持接口兼容）。返回 `True`（成功）或 `False`（失败）。

```
def disconnect(self, disable_torque: bool = False) -> None:
    if disable_torque:
        self.parking()
        self.piper.DisablePiper()
    self.port_handler.closePort()
```

`disable_torque` 默认为 `False`，意味着默认断开连接时不失能电机——这适用于程序临时重启场景。

5.10.6 parking() 与 clear_gripper() (第 216-244 行)

```
def clear_gripper(self):
    self.piper.GripperCtrl(0, 1000, 0x03, 0)
```

跳过归一化流程，直接发送 SDK 原始命令。`0` 是目标毫度值（夹爪全开），`1000` 是速度参数，`0x03` 是控制模式。

```
def parking(self):
    timeout = 100
    self.set_action(INITIALIZE_POSITION)
    time.sleep(0.1)
    status = self.piper.GetArmStatus()
    while status.arm_status.motion_status and timeout:
        self.set_action(INITIALIZE_POSITION)
        time.sleep(0.1)
```

```
status = self.piper.GetArmStatus()
timeout -= 1
```

使用 `set_action()` 而非直接调 SDK，是因为 `INITIALIZE_POSITION` 是归一化值（全 0），需要通过 `_unnormalize()` 转换为原始值再发送。循环重发的目的已在 5.3.6 节详细讨论。

5.10.7 `_normalize()` (第 274-332 行)

```
def _normalize(self, ids_values: dict[int, int]) -> dict[int, float]:
```

这个方法的实现与父类 `MotorsBus._normalize()` 相同，但被覆写到了子类中。覆写的原因是两个细微但重要的差异：

1. **键的类型**：在父类中，`_normalize` 使用 `self._id_to_name(id_)` 将整数 ID 转为字符串名称来访问 `self.calibration`。在覆写版本中，直接使用整数 ID 访问 `self.calibration[motor]` —— 因为 PiPER 的 `calibration` 字典以整数 ID 为键。
2. **DEGREES 模式访问**：`self.model_resolution_table[self._id_to_model(id_)]` —— PiPER 版本使用整数 ID 而非字符串名称来查找模型。

方法内部逻辑（三个分支已在 5.6 节详细推导过）：
- `RANGE_M100_100`：`norm = ((bounded - min) / (max - min)) * 200 - 100`
- `RANGE_0_100`：`norm = ((bounded - min) / (max - min)) * 100`
- `DEGREES`：`norm = (val - mid) * 360 / max_res`（PiPER 不使用）

安全钳位 `bounded_val = min(max_, max(min_, val))` 在所有分支之前统一执行。

5.10.8 `_unnormalize()` (第 334-396 行)

```
def _unnormalize(self, ids_values: dict[int, float]) -> dict[int, int]:
```

与 `_normalize()` 的对称方法。同样有三个分支，每个分支中先钳位输入值再计算输出：

- `RANGE_M100_100`：`bounded = clamp(val, -100, 100)`, `raw = int(((bounded + 100) / 200) * (max - min) + min)`
- `RANGE_0_100`：`bounded = clamp(val, 0, 100)`, `raw = int((bounded / 100) * (max - min) + min)`
- `DEGREES`：`raw = int((val * max_res / 360) + mid)`

反归一化中 `int()` 的截断行为：对于正数结果，`int()` 等价于 `floor()`，会产生最多 1 毫度（0.001°）的截断误差。这在实践中完全可以忽略。

5.10.9 enable_torque() / disable_torque() (第 410-453 行)

```
def enable_torque(self, motors=None, num_retry=0) -> bool:
    retry = 10
    while not self.piper.EnablePiper() and retry:
        retry -= 1
        time.sleep(0.1)
    ...
    return True if retry else False
```

```
def disable_torque(self, motors=None, num_retry=0) -> None:
    while self.piper.DisablePiper() and num_retry:
        num_retry -= 1
        time.sleep(0.01)
```

`_disable_torque(self, motor, model, num_retry)` 调用 `self.piper.DisableArm(motor)` 尝试禁用单个电机——但 PiPER SDK 实际上不支持逐个电机禁用（要么全使能，要么全失能），所以这个方法主要是接口占位。

5.10.10 get_action() (第 459-497 行)

已在 5.3.3 节完整讲解。核心数据流：`GetArmJointMsgs()` / `GetArmGripperMsgs()` → 提取字段 → `_normalize()` → 返回。

5.10.11 get_control() (第 499-525 行)

已在 5.3.4 节完整讲解。与 `get_action()` 的区别在于数据来源（`GetArmJointCtrl()` vs `GetArmJointMsgs()`）和是否归一化。

5.10.12 set_action() (第 530-583 行)

已在 5.3.5 节完整讲解。核心数据流：`_unnormalize()` → `ModeCtrl()` → `JointCtrl()` → `GripperCtrl()` → `get_control()`。

5.10.13 主从模式方法 (第 603-609 行)

```
def set_slave(self):
    self.piper.MasterSlaveConfig(0xFC, 0, 0, 0)

def set_master(self):
    self.piper.MasterSlaveConfig(0xFA, 0, 0, 0)
```

已在 5.3.10 节讲解。`0xFC` 表示从臂模式，`0xFA` 表示主臂模式。

5.10.14 剩余空实现方法 (第 617-664 行)

```
def _get_half_turn_homings(self, positions): pass
def _encode_sign(self, data_name, ids_values): return ids_values
def _decode_sign(self, data_name, ids_values): return ids_values
def _split_into_byte_chunks(self, value, length): pass
def broadcast_ping(self, ...): pass
def configure_motors(self): pass
def read_calibration(self): pass

@property
def is_calibrated(self) -> bool:
    return True

def write_calibration(self, ...): pass
```

所有空实现已在 5.5 节解释过。特别值得再次强调 `is_calibrated` 始终返回 `True` ——PiPER 的标定是代码中硬编码的，不需要从硬件 EEPROM 读取，因此"是否已标定"这个问题永远回答"是"。

5.10.15 main 测试代码 (第 670-705 行)

```
if __name__ == "__main__":
    piper = C_PiperInterface(
        can_name="can0",
        judge_flag=False,
        can_auto_init=True,
        dh_is_offset=1,
        start_sdk_joint_limit=False,
        start_sdk_gripper_limit=False,
        logger_level=LogLevel.WARNING,
        log_to_file=False,
        log_file_path=None,
    )
    piper.ConnectPort()
    while True:
        print(piper.GetArmJointMsgs())
        time.sleep(0.005) # 5ms = 200Hz
```

这是一个独立的最小示例，不依赖 LeRobot 框架。直接运行本文件可以快速验证 `piper_sdk` 是否能正常连接和读取。注意这里使用的是 `C_PiperInterface (V1)`，不是 `PiperMotorsBus` 中使用的 `C_PiperInterface_V2` ——两个版本的 API 大体相似。

重要提示： 第一帧 `GetArmJointMsgs()` 数据是全 0（默认值），之后才是真实编码器反馈。这是因为 SDK 的 `ConnectPort()` 需要启动 `ReadCan` 线程并等待第一帧 CAN 数据到达，期间结构体保持默认初始化值。

5.10.16 方法与调用关系总结

外部调用 (PiperFollower)

└── connect()

└── port_handler.setupPort(piper)

设置 CAN 参数

└── port_handler.openPort()

打开 CAN 口

└── enable_torque()

└── piper.EnablePiper()

使能电机 (CAN 0x471)

└── get_action()

—— 每帧调用 ——

└── piper.GetArmJointMsgs()

读关节反馈

└── piper.GetArmGripperMsgs()

读夹爪反馈

└── _normalize(ids_values)

归一化

└── 对每个 motor 应用公式

└── set_action(action)

—— 每帧调用 ——

└── _unnormalize(action)

反归一化

└── 对每个 motor 应用公式

└── piper.ModeCtrl(...)

设置控制模式 (CAN 0x151)

└── piper.JointCtrl(j1..j6)

发送关节目标 (CAN 0x155-157)

└── piper.GripperCtrl(...)

发送夹爪目标 (CAN 0x159)

└── get_control()

返回控制值确认

└── parking()

└── set_action(INITIALIZE_POSITION)

发送初始位置

└── piper.GetArmStatus()

检查运动状态

└── 循环直到停止或超时

└── disconnect(disable_torque)

└── parking()

回零 (可选)

└── piper.DisablePiper()

失能 (可选)

└── port_handler.closePort()

关闭 CAN 口

└── clear_gripper()

└── piper.GripperCtrl(0, 1000, 0x03, 0) 直接打开夹爪

5.11 本章总结

5.11.1 核心概念回顾

1. **适配器模式**: `PiperMotorsBus` 是一个硬件适配器, 把 CAN 总线操作翻译成 LeRobot 期望的 Dynamixel 风格接口。适配发生在三个层面——硬件通信层 (CAN vs 串口)、数据格式层 (毫度整数 vs 归一化浮点)、接口语义层 (保持 get/set_action 的调用契约不变)。

2. **归一化是数学核心**： `_normalize()` 和 `_unnormalize()` 完成了 `[-100, 100]` / `[0, 100]` 与毫度整数之间的双向映射。6 个关节使用 `RANGE_M100_100`（对称区间），夹爪使用 `RANGE_0_100`（单向区间）。
3. **空实现不是偷懒**：近一半的方法是空实现，这是因为 CAN 总线的通信模型和 Dynamixel 串口完全不同——不需要 Ping、不需要握手、不需要字节拆分、不需要发现电机。
4. **两层分离架构**：Bus 层（`connect / disconnect`）只管 CAN 通信链路，Robot 层（`enable_torque / disable_torque / parking`）管电机使能和初始化。
5. **标定是硬编码的**：PiPER 的关节范围由机械结构决定，不会改变。标定字典在 Python 代码中定义，不需要像 Dynamixel 那样从硬件 EEPROM 读写。因此 `is_calibrated` 始终返回 `True`。

5.11.2 关键公式速查

模式	方向	公式
RANGE_M100_100	归一化	<code>norm = ((raw - min) / (max - min)) * 200 - 100</code>
RANGE_M100_100	反归一化	<code>raw = int(((norm + 100) / 200) * (max - min) + min)</code>
RANGE_0_100	归一化	<code>norm = ((raw - min) / (max - min)) * 100</code>
RANGE_0_100	反归一化	<code>raw = int((norm / 100) * (max - min) + min)</code>

5.11.3 下一步

本章完整讲解了 `PiperMotorsBus` ——整个项目中最重要的 700 行代码。你现在应该能够：

- 追踪从策略模型输出到电机物理运动的每一步
- 理解 CAN 总线和 Dynamixel 串口在通信模型上的根本差异
- 手写任意关节的归一化/反归一化计算
- 解释为什么某些方法是空实现

下一章将进入上层——`PiperFollower` 和 `PiperLeader` 类，它们基于 `PiperMotorsBus` 构建了完整的机器人控制逻辑，包括数据采集循环、策略推理循环和 PC 中继遥操作。

第六章 Robot 与 Teleoperator 封装详解

本章深入剖析 PiPER 机械臂在 LeRobot 框架中的两层封装——PiperFollower（从臂/Robot）和 PiperLeader（主臂/Teleoperator）。阅读本章后，你将理解 LeRobot 如何将异构硬件抽象为统一的 `get_observation()` / `send_action()` 接口，掌握电机配置、校准、特征定义、注册机制等核心概念，并获得直接使用这些 Python 类编程的能力。

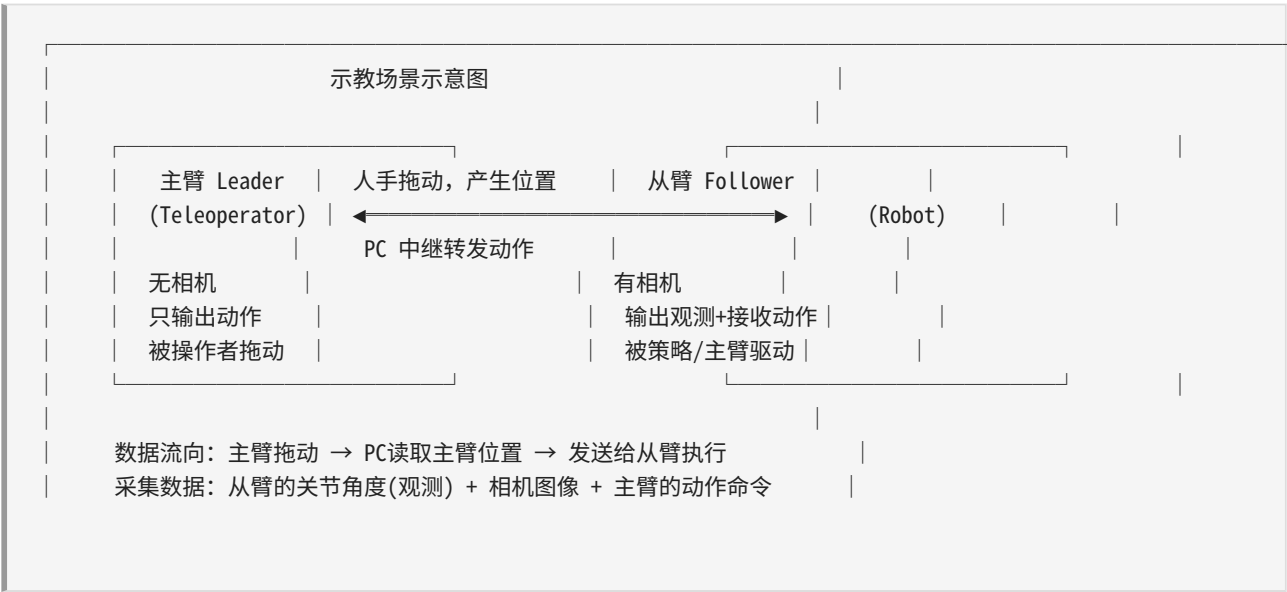
本章内容覆盖以下四个源文件：

文件	行数	角色
<code>lerobot/robots/piper_follower/piper_follower.py</code>	187	从臂驱动：采集观测、执行动作
<code>lerobot/robots/piper_follower/config_piper_follower.py</code>	43	从臂配置：CAN 口、相机、安全限幅
<code>lerobot/teleoperators/piper_leader/piper_leader.py</code>	111	主臂驱动：读取示教位置
<code>lerobot/teleoperators/piper_leader/config_piper_leader.py</code>	30	主臂配置：CAN 口、夹爪张开位置

6.1 Robot vs Teleoperator：概念辨析

6.1.1 物理角色的差异

在模仿学习场景中，一次完整的示教涉及两台机械臂，它们的物理角色截然不同：



- **Robot (从臂 / Follower)**：执行任务的机械臂。它拥有完整的感知和执行闭环：通过电机编码器感知自身关节位置（proprioception），通过相机感知环境（exteroception），接收动作命令后驱动电机到达目标位置。在数据采集时，它是"被记录者"——我们记录它的关节状态和相机画面作为训练数据。
- **Teleoperator (主臂 / Leader)**：被人操作的示教臂。它没有相机，唯一的"传感器"是人手的拖动力。在示教过程中，人被允许直接拖拽主臂的各关节，主臂的编码器实时返回被拖拽到的位置，这个位置经过 PC 中继方案转换为从臂的目标位置。

6.1.2 在 LeRobot 中的抽象基类

LeRobot 用两个相互独立但又结构对称的抽象基类来建模这两个角色：

Robot 基类（`lerobot/robots/robot.py`，186 行）：

方法/属性	方向	语义
<code>get_observation() → dict</code>	从硬件 向外 读	读取机械臂当前状态（关节角度 + 相机图像），返回一个扁平字典
<code>send_action(action) → dict</code>	从外部 向硬件 写	向机械臂发送目标关节位置命令，返回实际执行的动作
<code>observation_features</code>	元数据	描述 <code>get_observation()</code> 返回字典的 schema（键名和数据类型/形状）
<code>action_features</code>	元数据	描述 <code>send_action()</code> 期望接收字典的 schema
<code>connect()</code>	生命周期	建立硬件连接、校准、使能力矩
<code>disconnect()</code>	生命周期	断开连接、失能力矩、清理相机

Teleoperator 基类（`lerobot/teleoperators/teleoperator.py`，182 行）：

方法/属性	方向	语义
<code>get_action() → dict</code>	从硬件 向外 读	读取主臂被拖拽到的当前关节位置
<code>send_feedback(feedback) → None</code>	从外部 向硬件 写（可选）	向主臂发送力反馈（PiPER 主臂当前为空操作）
<code>action_features</code>	元数据	描述 <code>get_action()</code> 返回字典的 schema

方法/属性	方向	语义
<code>feedback_features</code>	元数据	描述 <code>send_feedback()</code> 期望接收字典的 schema
<code>connect()</code>	生命周期	建立 CAN 连接、使能力矩
<code>disconnect()</code>	生命周期	失能力矩、断开连接

6.1.3 关键差异总结

特性	Robot (PiperFollower)	Teleoperator (PiperLeader)
输入 (从外部接收)	<code>send_action(action)</code> — 目标关节位置	<code>send_feedback(feedback)</code> — 力反馈 (空实现)
输出 (向外部提供)	<code>get_observation()</code> — 关节状态 + 相机图像	<code>get_action()</code> — 被拖拽的关节位置
相机	有。通常配置 1-2 个 RealSense	无。主臂不采集视觉
校准	有。 <code>MotorCalibration</code> 定义每个关节的物理范围	无。不需要写入命令，不需要范围映射
连接后行为	连接 → 使能力矩 → parking (回初始位) → 连接相机	连接 → 使能力矩 (就这两步)
断开行为	断开所有相机 → 断开 CAN → 可选失能力矩	失能力矩 → 断开 CAN
电机型号	joint1-3: AGILEX-M, joint4-6+夹爪: AGILEX-S	全部 7 个电机: HTDW-5047
主从模式	<code>set_slave()</code>	<code>set_master()</code>

6.2 PiperFollower 完整解析

6.2.1 类定义和构造

```

class PiperFollower(Robot):
    # Set these in ALL subclasses
    config_class: PiperFollowerConfig
    name = "piper_follower"

    def __init__(self, config: PiperFollowerConfig):
        self.config = config

```

```

self.id = config.id
self.port = config.port
self.bus = PiperMotorsBus(
    id=config.id,
    port=config.port,
    motors={...},      # 电机型号配置
    calibration={...}  # 校准参数
)
self.cameras = make_cameras_from_configs(config.cameras)

```

两条类级别属性是强约束定：

- **config_class**：类型注解，指向 `PiperFollowerConfig`。LeRobot 框架通过 `draccus.ChoiceRegistry` 的多态机制，在运行时根据 `--robot.type=piper_follower` 自动选择对应的配置类并实例化。如果你忘写这个属性，框架将无法为你的 robot 类型找到正确的配置类。
- **name = "piper_follower"**：字符串标识，用于日志输出、注册表查找（`ROBOTS` 常量）、校准文件路径构造（`~/cache/huggingface/lerobot/calibration/robots/piper_follower/`）。

构造函数的职责非常清晰——它只在内存中创建对象的数据结构，**不建立任何硬件连接**：

1. 保存配置引用（`self.config`）。
2. 提取 `id` 和 `port` 作为便捷属性。
3. 创建 `PiperMotorsBus` 实例——这是整个机械臂操作的入口，封装了 CAN 通信、电机管理和校准逻辑。此处的 `motors` 参数定义电机型号和归一化模式，`calibration` 参数定义每个关节的物理角度范围。**这两个参数在构造时固化，之后不再改变。**
4. 调用 `make_cameras_from_configs(config.cameras)` 创建相机实例字典。这个工厂函数根据配置字典中的每个条目（指定了 `type`、`serial`、`width`、`height`、`fps`）实例化对应的相机驱动（如 `OpenCVCamera` 或 `RealSenseCamera`）。

6.2.2 电机配置（最关键的数据结构）

这是整个 `PiperFollower` 中最重要、也最容易被误解的代码段。让我们逐项拆解。

电机型号与归一化

```

self.bus = PiperMotorsBus(
    id=config.id,
    port=config.port,
    motors={
        "joint1": Motor(1, "AGILEX-M", MotorNormMode.RANGE_M100_100),
        "joint2": Motor(2, "AGILEX-M", MotorNormMode.RANGE_M100_100),
        "joint3": Motor(3, "AGILEX-M", MotorNormMode.RANGE_M100_100),
        "joint4": Motor(4, "AGILEX-S", MotorNormMode.RANGE_M100_100),
        "joint5": Motor(5, "AGILEX-S", MotorNormMode.RANGE_M100_100),
    }
)

```

```

        "joint6": Motor(6, "AGILEX-S", MotorNormMode.RANGE_M100_100),
        "gripper": Motor(7, "AGILEX-S", MotorNormMode.RANGE_0_100),
    },
    ...
)

```

每个 `Motor` 对象包含三个参数：

- **motor_id**（第 1 参数）：CAN 总线上的电机 ID，范围 1-7。这个 ID 是物理的——它对应电机驱动器上拨码开关设置的 CAN ID。joint1 = ID 1, joint2 = ID 2, ..., gripper = ID 7。注意：这里的 ID 与字典的 key 名是独立的两件事——key 名只是逻辑标识符，ID 才是 CAN 帧中实际使用的地址。
- **model**（第 2 参数）：电机型号字符串。控制 SDK 内部如何解析该电机的反馈数据（如编码器分辨率、电流/速度反馈格式）。不同型号对应不同的物理电机。
- **norm_mode**（第 3 参数）：归一化模式。定义该电机的归一化目标区间。
- **RANGE_M100_100**：归一化到 `[-100.0, 100.0]`。用于所有 6 个关节 (joint1-6)。对称双极性区间，0 代表物理中位。
- **RANGE_0_100**：归一化到 `[0.0, 100.0]`。仅用于夹爪 (gripper)。单极性区间，0 代表完全闭合，100 代表完全张开。

为什么需要归一化？

不同电机的物理角度范围差异很大。joint2 的运动范围是从 0 到约 180 度，joint6 是从约 -100 到 +130 度。如果神经网络直接使用物理角度值，它需要隐式地学习每个关节的不同尺度，这会使训练变得困难。归一化将所有关节统一映射到 `[-100, 100]`（或 `[0, 100]`）的公共数值空间，使得策略网络可以平等地对待所有关节，显著降低学习难度。

校准参数

```

calibration={
    "joint1": MotorCalibration(1, 0, 0, -150000, 150000),
    "joint2": MotorCalibration(2, 0, 0, 0, 180000),
    "joint3": MotorCalibration(3, 0, 0, -170000, 0),
    "joint4": MotorCalibration(4, 0, 0, -100000, 100000),
    "joint5": MotorCalibration(5, 0, 0, -65000, 65000),
    "joint6": MotorCalibration(6, 0, 0, -100000, 130000),
    "gripper": MotorCalibration(7, 0, 0, 0, 68000),
}

```

`MotorCalibration` 的构造函数签名为：

```
MotorCalibration(motor_id, offset_deg, direction, min, max)
```

参数	类型	含义
<code>motor_id</code>	int	CAN 总线上的电机 ID（必须与 <code>motors</code> 字典中对应电机的 ID 一致）
<code>offset_deg</code>	float	角度偏置（度）。PiPER 的出厂校准已完成，此处均为 0
<code>direction</code>	int	旋转方向。0 为默认方向（正向），1 为反向。此处均为 0（默认方向）
<code>min</code>	int	该关节的 最小物理限制 ，以 SDK 原始编码器单位表示
<code>max</code>	int	该关节的 最大物理限制 ，以 SDK 原始编码器单位表示

`min` 和 `max` 是校准参数中真正关键的值。它们定义了每个关节的**原始编码器值的合法范围**。归一化过程就是将原始编码器值按此范围线性映射到归一化区间：

$$\text{归一化值} = (\text{原始值} - \text{min}) / (\text{max} - \text{min}) * (\text{norm_max} - \text{norm_min}) + \text{norm_min}$$

例如，对于 `joint1`，原始值 0（物理中位）对应归一化值 0.0（`[-100, 100]` 的中点）。

完整参数对照表

关节	Motor ID	型号	校准范围 (min ~ max)	归一化区间	物理含义	编码器分辨率量级
<code>joint1</code>	1	AGILEX-M	-150000 ~ 150000	[-100, 100]	底座旋转 (J1)，范围约 $\pm 150^\circ$	~1000 单位/度
<code>joint2</code>	2	AGILEX-M	0 ~ 180000	[-100, 100]	大臂俯仰 (J2)，范围 $0^\circ \sim 180^\circ$	~1000 单位/度
<code>joint3</code>	3	AGILEX-M	-170000 ~ 0	[-100, 100]	肘关节俯仰 (J3)，范围约 $-170^\circ \sim 0^\circ$	~1000 单位/度
<code>joint4</code>	4	AGILEX-S	-100000 ~ 100000	[-100, 100]	腕部旋转 (J4)，范围约 $\pm 100^\circ$	~1000 单位/度
<code>joint5</code>	5	AGILEX-S	-65000 ~ 65000	[-100, 100]	腕部俯仰 (J5)，范围约 $\pm 65^\circ$	~1000 单位/度
<code>joint6</code>	6	AGILEX-S	-100000 ~ 130000	[-100, 100]	腕部末端旋转 (J6)，范围约 $-100^\circ \sim +130^\circ$	~1000 单位/度
<code>gripper</code>	7		0 ~ 68000	[0, 100]		

关节	Motor ID	型号	校准范围 (min ~ max)	归一化区间	物理含义	编码器分辨率量级
		AGILEX-S			夹爪开合, 0=闭合, 100=完全张开	~680 单位 (全程)

为什么 joint1-3 使用 AGILEX-M, 而 joint4-6 使用 AGILEX-S?

这是 PiPER 机械臂的硬件设计选择。**AGILEX-M** (M = Medium/Large) 是较大型的伺服电机, 具有更高的额定扭矩和更大的输出功率, 安装在底座、肩部和肘部 (joint1-3) ——这三个关节需要承受整个臂身的重量和末端负载。**AGILEX-S** (S = Small) 是小型伺服电机, 更紧凑、更轻量, 安装在腕部的三个关节 (joint4-6) ——这些关节靠近末端执行器, 需要低惯量以实现快速精确的腕部动作。夹爪也使用 AGILEX-S, 因为夹爪的开合力需求相对较低。

为什么 gripper 的归一化区间是 [0, 100] 而不是 [-100, 100]?

夹爪只有"开"和"合"两个方向, 不存在对称的双向运动。使用 [0, 100] 更符合直觉: 0 代表完全闭合 (夹紧物体), 100 代表完全张开 (释放物体)。如果使用 [-100, 100], 神经网络可能会学到负值对应半开状态, 反而引入不必要的复杂性。

6.2.3 observation_features 和 action_features

这两个属性定义了 LeRobot 数据集的 schema——它们告诉数据采集系统"这个 robot 会产生什么样的数据"。

_motors_ft —— 关节状态特征

```
@property
def _motors_ft(self) -> dict[str, type]:
    return {f"{motor}.pos": float for motor in self.bus.motors}
```

遍历 `self.bus.motors` 字典的所有键 (即 "joint1", "joint2", ..., "gripper"), 为每个电机构造一个以 `.pos` 为后缀的键名, 值类型为 `float`。生成的结果为:

```
{
    "joint1.pos": float,
    "joint2.pos": float,
    "joint3.pos": float,
    "joint4.pos": float,
    "joint5.pos": float,
    "joint6.pos": float,
```

```
"gripper.pos": float,
}
```

这里 `.pos` 后缀是 LeRobot 的数据集命名约定——它表示这是**位置 (position)** 特征。LeRobot 数据集（基于 HuggingFace Datasets）使用 `features` 字典来定义每帧数据的结构，`.pos` 后缀帮助下游的数据处理 pipeline 识别这是一个关节位置值。

`_cameras_ft` —— 相机图像特征

```
@property
def _cameras_ft(self) -> dict[str, tuple]:
    return {
        cam: (self.cameras[cam].height, self.cameras[cam].width, 3)
        for cam in self.cameras
    }
```

遍历相机字典，为每个相机生成一个键（通常为 `"wrist"` 和 `"global"`），值是一个三元组 `(height, width, 3)`，表示 RGB 图像的形状。例如，如果配置了腕部和全局两个 640x480 相机：

```
{
    "wrist": (480, 640, 3),
    "global": (480, 640, 3),
}
```

注意顺序是 `(H, W, C)` —— `height` 在前，`channels` 在后，这是一些深度学习框架（如 PyTorch）的图像数据约定。`3` 固定代表 RGB 三通道。

合并为完整 `features`

```
@cached_property
def observation_features(self) -> dict:
    return {**self._motors_ft, **self._cameras_ft}

@cached_property
def action_features(self) -> dict:
    return self._motors_ft
```

两个关键观察：

1. **`observation_features` = 关节特征 + 相机特征**。这意味着 `get_observation()` 返回的字典包含两类信息：proprioceptive（本体感知，即关节角度）和 exteroceptive（外部感知，即相机图像）。
2. **`action_features` = 关节特征（仅有）**。这意味着发送给机械臂的动作命令只包含目标关节位置，不涉及相机。动作是纯 proprioceptive 的——因为相机是传感器（输入），不是执行器（输

出)。神经网络策略根据观测中的图像和关节状态推理出目标关节位置，然后将这个目标关节位置发送给机械臂执行。

使用 `@cached_property`（`functools.cached_property`）而非 `@property` 是因为字典合并操作 `{**dict1, **dict2}` 每次调用都会创建新对象。缓存后，第一次访问计算一次，之后直接返回缓存值，避免了不必要的重复计算。

6.2.4 connect() 详细流程

```
def connect(self, calibrate: bool = True) -> bool:
    if not self.bus.connect():
        return False
    logger.info(f"{self} connected.")
    while not self.bus.enable_torque():
        logger.info(f"{self} retry torque on.")
    logger.info(f"{self} go to origin.")
    if calibrate:
        self.bus.parking()

    for cam in self.cameras.values():
        cam.connect()
    return True
```

连接流程按顺序分为 4 个阶段：

阶段 1：CAN 连接 — `self.bus.connect()`

这是最底层的连接操作。`PiperMotorsBus.connect()` 内部会调用 SDK 的 `C_PiperInterface_V2.ConnectPort()`，后者初始化 SocketCAN 接口、启动后台读取线程（ReadCan 线程，约 200Hz）、发送握手帧确认通信通路正常。

如果连接失败（例如 CAN 口不存在或机械臂未上电），返回 `False`，整个 `connect()` 返回 `False` 通知调用者。

阶段 2：使能力矩 — `self.bus.enable_torque()`（带重试循环）

使能力矩意味着向每个电机驱动器发送 `enable` 命令，让电机进入转矩控制模式。在失能状态下，电机处于自由旋转模式（可以被外力轻松转动）。在使能状态下，电机会抵抗外力、保持位置、响应位置命令。

`while not ... : retry` 循环的意义：在实际硬件中，CAN 总线通信有时会因为电气噪声或驱动器状态问题导致第一帧使能命令未成功送达。重试循环确保最终使能成功（如果持续失败则陷入无限循环——这是需要注意的风险点）。

阶段 3：parking（回初始位） — `self.bus.parking()`（当 `calibrate=True`）

`parking()` 方法将机械臂的所有关节缓慢移动到预设的"初始位置" (parking position)。这个初始位置是机械臂的一个安全姿态——所有关节都位于其运动范围中段附近，不会碰到极限位，也不会碰到桌面或机械臂自身。

`calibrate` 参数默认为 `True`。如果你希望在连接后保持机械臂当前位置不变（例如在实验间隙快速重连），可以传入 `calibrate=False`。

阶段 4：逐个连接相机 — `for cam in self.cameras.values(): cam.connect()`

遍历所有配置的相机，调用各自的 `connect()` 方法。对于 `RealSense` 相机，这会初始化 `librealsense2` 管道、启动彩色流、等待首帧到达。相机连接较慢（通常需要 1-3 秒），所以写在了 CAN 连接之后。

为什么相机连接放在最后？

如果先连相机再连 CAN，一旦 CAN 连接失败（例如机械臂未开机、CAN 线松动），相机已经建立了连接但无法使用。将相机连接放在最后，如果 CAN 连接或 `parking` 失败，调用者可以快速重试而不必重复等待相机初始化。

6.2.5 `get_observation()` 详细流程

```
def get_observation(self) -> dict[str, Any]:
    if not self.is_connected:
        raise DeviceNotConnectedError(f"{self} is not connected.")

    obs_dict = {}

    # Read arm position
    start = time.perf_counter()
    obs_dict = self.bus.get_action()
    obs_dict = {f"{motor}.pos": val for motor, val in obs_dict.items()}
    dt_ms = (time.perf_counter() - start) * 1e3
    logger.debug(f"{self} read state: {dt_ms:.1f}ms")

    # Capture images from cameras
    for cam_key, cam in self.cameras.items():
        start = time.perf_counter()
        obs_dict[cam_key] = cam.async_read()
        dt_ms = (time.perf_counter() - start) * 1e3
        logger.debug(f"{self} read {cam_key}: {dt_ms:.1f}ms")

    return obs_dict
```

这是一个典型的数据采集循环核心。每帧数据采集经过以下步骤：

步骤 1：连接检查

`is_connected` 属性返回 `self.bus.is_connected and all(cam.is_connected for cam in self.cameras.values())`——同时检查 CAN 总线和所有相机都处于连接状态。如果不满足，抛出 `DeviceNotConnectedError`。

步骤 2：读取关节位置

`self.bus.get_action()` 调用 SDK 的底层读取接口，从后台 `ReadCan` 线程的共享缓冲区中获取最新的电机位置值。返回的字典格式为：

```
{
    "joint1": -15.2,  # 归一化值，范围 [-100, 100]
    "joint2": 45.8,
    "joint3": -120.0,
    "joint4": 30.1,
    "joint5": 0.0,
    "joint6": -45.3,
    "gripper": 50.0,  # 归一化值，范围 [0, 100]
}
```

注意：这些值已经是归一化后的浮点数（经过 `calibration` 的 `min/max` 范围映射），直接可用于训练。

然后通过 `{f"{motor}.pos": val for motor, val in obs_dict.items()}` 为每个键添加 `.pos` 后缀，使其符合 LeRobot 数据集的特征命名约定：

```
{
    "joint1.pos": -15.2,
    "joint2.pos": 45.8,
    ...
    "gripper.pos": 50.0,
}
```

步骤 3：读取相机图像

使用 `cam.async_read()` 从每个相机获取最新的 RGB 帧。`async_read()` 是非阻塞调用——它读取相机后台线程的最新缓冲帧，不等待新帧到达。这确保了数据采集循环不会被相机帧率（通常 30 FPS）所阻塞：即使相机还没产生新帧，我们也使用上一帧的值，保证整体采集的实时性。

步骤 4：合并返回

将关节位置字典和相机图像字典合并为一个扁平字典返回。一个完整的返回示例如下：

```
{
    # 关节位置 (proprioception)
    "joint1.pos": -15.2,      # float: 归一化关节位置
    "joint2.pos": 45.8,
    "joint3.pos": -120.0,
```

```

    "joint4.pos": 30.1,
    "joint5.pos": 0.0,
    "joint6.pos": -45.3,
    "gripper.pos": 50.0,

    # 相机图像 (exteroception)
    "wrist": np.ndarray(shape=(480, 640, 3), dtype=uint8), # RGB 图像
    "global": np.ndarray(shape=(480, 640, 3), dtype=uint8), # RGB 图像
}

```

为什么方法名叫 `get_observation()` 但内部调用的是 `self.bus.get_action()` ?

这是一个命名上的潜在混淆点。 `PiperMotorsBus.get_action()` 的 "action" 是从电机驱动的角度而言的——电机把它当前的 "行为" (当前角度) 反馈出来。但从 `Robot/LeRobot` 的角度, 这属于 "观测" (observation), 因为它反映了机械臂当前的状态。 `get_action()` 这个命名源于 `PiperMotorsBus` 继承自更底层的 `motors bus` 接口, 其中 "action" 是通用术语。

6.2.6 `send_action()` 详细流程

```

def send_action(self, action: dict[str, Any]) -> dict[str, Any]:
    if not self.is_connected:
        raise DeviceNotConnectedError(f"{self} is not connected.")

    goal_pos = {key.removesuffix(".pos"): val
                 for key, val in action.items()
                 if key.endswith(".pos")}

    # Cap goal position when too far away from present position.
    if self.config.max_relative_target is not None:
        present_pos = self.bus.sync_read("Present_Position")
        goal_present_pos = {key: (g_pos, present_pos[key])
                           for key, g_pos in goal_pos.items()}
        goal_pos = ensure_safe_goal_position(
            goal_present_pos, self.config.max_relative_target
        )

    rlt = self.bus.set_action(goal_pos)

    return {f"{motor}.pos": val for motor, val in rlt.items()}

```

步骤 1: key 转换

策略网络输出的动作字典使用 `.pos` 后缀作为键名 (如 `"joint1.pos": 0.0`), 这是 `LeRobot` 数据集的命名约定。但在发送给 `PiperMotorsBus.set_action()` 之前, 需要去掉 `.pos` 后缀, 因为 `bus` 层使用裸的电机名作为键。

```
# 输入:
# {"joint1.pos": 10.0, "joint2.pos": 20.0, ...}
# 经过 key.removesuffix(".pos") 转换后:
# {"joint1": 10.0, "joint2": 20.0, ...}
```

注意 `if key.endswith(".pos")` 的过滤——如果动作字典中包含了非 `.pos` 后缀的键（例如某些外部添加的元数据），这些不会被传入 `bus`。

步骤 2：安全限幅（可选）

当 `config.max_relative_target` 不为 `None` 时，会启用相对位移安全检查：

```
if self.config.max_relative_target is not None:
    present_pos = self.bus.sync_read("Present_Position")
    goal_present_pos = {key: (g_pos, present_pos[key]) for key, g_pos in goal_pos.items()}
    goal_pos = ensure_safe_goal_position(goal_present_pos, self.config.max_relative_target)
```

这是防止机械臂发生剧烈运动的最后一道防线。其工作原理如下：

1. **读取当前位置：** `self.bus.sync_read("Present_Position")` 向 CAN 总线发送同步读取命令，获取所有电机的当前实际位置（归一化值）。与 `get_action()` 不同，`sync_read` 是阻塞的——它会等待硬件返回最新数据后才返回，保证数据的一致性。
2. **构造（目标，当前）对：** `goal_present_pos` 字典将目标位置和当前位置配对在一起：

```
python
{"joint1": (10.0, 5.0), # (目标, 当前) "joint2": (20.0, 25.0), ...}
```
3. **限幅检查：** `ensure_safe_goal_position()` 函数对每个关节检查 `|目标 - 当前|` 是否超过 `max_relative_target`。如果超过，将目标值 clamp 到 `当前位置 ± max_relative_target`。

`max_relative_target` 支持两种配置形式：

- **全局标量：** `max_relative_target = 5.0` — 所有关节的单步最大移动量统一为 5.0（在归一化空间 `[-100, 100]` 中，5.0 约对应几度的物理角度）。
- **per-joint 字典：** `max_relative_target = {"joint1": 3.0, "joint2": 2.0, ...}` — 为每个关节单独设定限制。这对于某些运动范围较小的关节（如 `joint5`，物理范围只有 $\pm 65^\circ$ ）特别有用。

性能注意： 注释中的警告 `/!\ Slower fps expected due to reading from the follower` 指出了这个安全检查的性能代价。每帧额外进行一次 `sync_read` 会增加约 2-5ms 的延迟（取决于 CAN 总线波特率和电机数量），从而降低数据采集或策略推理的帧率。在生产环境中，如果策略本身已经过充分训练且输出平滑，可以考虑将 `max_relative_target` 设为 `None` 以获取最高帧率。

步骤 3：发送动作

`self.bus.set_action(goal_pos)` 执行以下操作：

1. **反归一化**：将归一化的目标关节位置（`[-100, 100]` 范围）按 `calibration` 参数反向映射回原始编码器值。
2. **构造 CAN 帧**：为每个电机构造位置控制命令帧（CAN ID = `0x100 + motor_id`，8 字节数据负载包含目标位置编码）。
3. **发送到 CAN 总线**：通过 SocketCAN 的 `bus.send(msg)` 依次发送每个电机的命令帧。

由于 CAN 总线是广播式的，所有电机几乎同时收到命令帧，实现同步运动。

步骤 4：返回实际动作

`set_action()` 返回值是实际发送的归一化位置值（如果某些值被反归一化后超出物理范围会被 clamp）。函数再次添加 `.pos` 后缀后返回，保持与输入格式一致。

6.2.7 disconnect() —— 修复的 bug

这是一个值得单独讨论的修改。让我们对比原始版本和修复版本：

原始版本（有 bug，`lerobot_piper-piper-original`）：

```
def disconnect(self, disable_torque: bool = False) -> None:
    self.bus.disconnect(disable_torque)
```

修复版本（你们的版本）：

```
def disconnect(self, disable_torque: bool = False) -> None:
    for camera in self.cameras.values():
        if camera.is_connected:
            camera.disconnect()
    self.bus.disconnect(disable_torque)
```

Bug 分析：

原始版本在 `disconnect()` 时只断开了 CAN 总线，**没有断开相机**。这会导致以下问题：

1. **RealSense 相机资源泄漏**：`librealsense2` 的 pipeline 持有 USB 设备句柄和内核缓冲区。如果不调用 `pipeline.stop()` 而直接让 Python 进程退出，USB 设备可能保持在异常状态。
2. **Core dump 风险**：在某些 Linux 内核版本和 RealSense 固件组合下，未正确关闭的 RealSense 管道在进程退出时可能触发内核 USB 子系统的 core dump。
3. **下次连接失败**：如果相机驱动程序没有正确重置，下次调用 `connect()` 时可能无法正常打开设备（设备被标记为“忙”状态）。

修复的关键点：

- **先关相机，再关 CAN**：断开连接的顺序应当与连接顺序相反（LIFO 原则）。连接时是先 CAN 后相机，断开时先相机关 CAN。
- **检查 `is_connected`**：只断开已经连接的相机，避免对已断开的相机重复调用 `disconnect()`（虽然在大多数实现中是幂等的，但显式检查是更好的防御性编程）。
- **`disable_torque` 参数**：控制是否在断开前失能力矩。默认为 `False` 以保持机械臂的当前位置（避免断开后机械臂因重力下垂）。如果设为 `True`，电机会进入自由旋转模式。

6.3 PiperLeader 完整解析

6.3.1 与 PiperFollower 的关键差异

PiperLeader 继承自 `Teleoperator` 基类而非 `Robot` 基类，这带来了接口层面的根本性变化。同时，由于主臂的物理角色是"被人拖动"而非"自主运动"，其连接流程和电机配置也大不相同。

完整差异对照表：

特性	PiperFollower	PiperLeader
基类	<code>Robot</code>	<code>Teleoperator</code>
配置文件	<code>PiperFollowerConfig</code> (<code>RobotConfig</code> 子类)	<code>PiperLeaderConfig</code> (<code>TeleoperatorConfig</code> 子类)
注册装饰器	<code>@RobotConfig.register_subclass("piper_follower")</code>	<code>@TeleoperatorConfig.register_subclass("piper_l")</code>
电机型号	joint1-3: AGILEX-M, joint4-6+gripper: AGILEX-S	全部 7 个: HTDW-5047
校准 (calibration)	有 — <code>MotorCalibration</code> 定义每个关节的范围	无 — 构造函数未传入 <code>calibration</code> 参数
相机	有 — <code>self.cameras = make_cameras_from_configs(...)</code>	无 — 没有 <code>cameras</code> 属性
特征属性	<code>observation_features</code> + <code>action_features</code>	<code>action_features</code> + <code>feedback_features</code>
读方法	<code>get_observation()</code> — 读关节 + 相机	<code>get_action()</code> — 只读关节位置
写方法	<code>send_action()</code> — 发送目标位置	<code>send_feedback()</code> — 空实现 (pass)
<code>setup_motors()</code>	<code>set_slave()</code> — 从模式	<code>set_master()</code> — 主模式
连接后行为	CAN 连接 → 使能力矩 → parking → 连接相机	CAN 连接 → 使能力矩 (结束)
重试策略	<code>bus.connect()</code> 失败后立即返回 <code>False</code>	<code>bus.connect()</code> 失败后 <code>sleep(0.1)</code> 重试 (无限循环)

特性	PiperFollower	PiperLeader
is_calibrated	委托给 <code>bus.is_calibrated</code>	硬编码返回 <code>True</code> （主臂不需要校准）
disconnect	先断开所有相机 → 再断开 CAN	先失能力矩 → 再断开 CAN

6.3.2 电机配置

```
self.bus = PiperMotorsBus(
    id=config.id,
    port=config.port,
    motors={
        "joint1": Motor(1, "HTDW-5047", MotorNormMode.RANGE_M100_100),
        "joint2": Motor(2, "HTDW-5047", MotorNormMode.RANGE_M100_100),
        "joint3": Motor(3, "HTDW-5047", MotorNormMode.RANGE_M100_100),
        "joint4": Motor(4, "HTDW-5047", MotorNormMode.RANGE_M100_100),
        "joint5": Motor(5, "HTDW-5047", MotorNormMode.RANGE_M100_100),
        "joint6": Motor(6, "HTDW-5047", MotorNormMode.RANGE_M100_100),
        "gripper": Motor(7, "HTDW-5047", MotorNormMode.RANGE_0_100),
    }
)
```

与 PiperFollower 的 motors 配置对比：

对比维度	PiperFollower	PiperLeader
关节 1-3 型号	AGILEX-M	HTDW-5047
关节 4-6 型号	AGILEX-S	HTDW-5047
夹爪型号	AGILEX-S	HTDW-5047
归一化模式（关节）	RANGE_M100_100	RANGE_M100_100（相同）
归一化模式（夹爪）	RANGE_0_100	RANGE_0_100（相同）

HTDW-5047 是教学臂专用的伺服电机型号。它的特点是：

- **低齿槽转矩 (low cogging torque)**：电机内部设计的磁路经过优化，使得在失能状态下转子转动平滑、阻力小。这对主臂至关重要——人需要能够轻松拖拽主臂，如果电机阻转力矩太大，拖拽体验会很差。
- **高分辨率编码器**：可以提供与 AGILEX 系列相媲美的位置反馈精度，确保主臂读出的位置数据精度足够高。
- **不需要校准参数**：主臂不接收写入命令，因此不需要 `MotorCalibration` 来限定范围。归一化由 SDK 内部的默认范围（通常是电机的硬件限位）完成。

注意：PiperLeader 的构造函数中**没有传入 calibration 参数**。这意味着 PiperMotorsBus 的 calibration 参数被设为 None（或默认空字典）。当没有 calibration 时，set_action() 的反归一化步骤无法执行——但这对主臂来说没有影响，因为主臂根本不调用 set_action()。

6.3.3 为什么主臂不需要校准？

这个问题的答案涉及 LeRobot 架构中数据和命令流的根本不对称性：

从臂需要校准 因为它需要写入（set_action()）：

```
策略/主臂输出归一化值
↓
send_action() 接收归一化值 (如 joint1.pos = 10.0)
↓
bus.set_action() 需要将归一化值反归一化为原始编码器值
↓
MotorCalibration(min, max) 定义了反归一化的映射公式
↓
CAN 帧包含原始编码器值，发送给电机驱动器
```

主臂不需要校准 因为它只读取（get_action()）：

```
人拖拽主臂
↓
编码器产生原始值
↓
SDK 的 ReadCan 线程读取原始值
↓
bus.get_action() 使用默认范围归一化 (或 SDK 内部自带的范围)
↓
get_action() 返回归一化值
↓
(归一化值直接被中继程序转发给从臂)
```

主臂的归一化过程**不需要知道精确的物理角度范围**——只要归一化使用的映射在整条链路中保持一致即可。主臂读数归一化到 `[-100, 100]`，从臂也在 `[-100, 100]` 的归一化空间中接收命令，映射关系自动匹配。

6.3.4 setup_motors() 的特殊性

```
# PiperFollower
def setup_motors(self) -> None:
    self.bus.connect()
    self.bus.set_slave()

# PiperLeader
```

```
def setup_motors(self) -> None:
    self.bus.connect()
    self.bus.set_master()
```

`setup_motors()` 是 LeRobot 框架中的一个生命周期钩子，在 `connect()` 之后被调用（具体由 `lerobot-teleoperate` 等 CLI 工具协调）。它的作用是配置电机的主从角色：

- **`set_slave()`**：将从臂的电机配置为从模式（slave mode）。在从模式下，电机接收来自外部（PC 中继程序或策略推理）的位置命令并通过 CAN 总线执行。对应 CAN 协议中的 `MasterSlaveConfig` 值 `0xFC...`。
- **`set_master()`**：将主臂的电机配置为主模式（master mode）。在主模式下，电机的编码器数据通过 CAN 总线输出，但不响应外部位置命令。对应 CAN 协议中的 `MasterSlaveConfig` 值 `0xFA...`。

PC 中继方案不使用主从模式：在实际的 PC 中继遥操作方案中（`piper_pc_relay_teleop.py`），数据流是“主臂 SDK 读位置 → PC 转发 → 从臂 SDK 写位置”，即 PC 作为中继。此时两个臂之间的 CAN 总线是物理独立的（通常分属两个不同的 CAN 口），它们之间没有直接的 CAN 级主从同步。`set_master()` 和 `set_slave()` 仅在双臂直连的 CAN 主从同步方案中使用（该方案在本项目中不常用）。

6.3.5 connect() 对比

```
# PiperFollower.connect()
def connect(self, calibrate: bool = True) -> bool:
    if not self.bus.connect():
        return False
    # ... torque enable with retry ...
    if calibrate:
        self.bus.parking()
    for cam in self.cameras.values():
        cam.connect()
    return True

# PiperLeader.connect()
def connect(self, calibrate: bool = True) -> None:
    while not self.bus.connect():
        logger.info(f"{self} connection failed.")
        time.sleep(0.1)
    logger.info(f"{self} connected.")
    self.bus.enable_torque()
    logger.info(f"{self} torque on.")
```

关键差异：

1. **重试策略**：PiperFollower 的连接失败后直接返回 `False`，由调用者决定如何处理。PiperLeader 则采用无限重试循环——连接失败后 `sleep 0.1` 秒再重试。这种差异反映了两种场景的不同需求：从臂的连接通常由脚本控制，失败后可以由用户手动排查后重跑脚本；主臂的连接通常在人开始示教前进行，无限重试意味着"好了，你现在可以去给主臂上电了，程序会等着你"。
2. **无 parking**：主臂不需要 parking。主臂的初始位置就是它当前被搁置的位置——通常比 parking 位置更适合人开始拖拽（parking 位置是一个紧致的直立姿态，不太利于人操作）。
3. **无相机连接**：主臂没有相机，`connect()` 流程更简短。
4. **使能力矩无重试**：`self.bus.enable_torque()` 只调用一次（无 `while` 循环）。这是因为主臂的 HTDW-5047 电机使能响应更快，几乎不需要重试。

6.3.6 get_action() 流程

```
def get_action(self) -> dict[str, Any]:
    if not self.is_connected:
        raise DeviceNotConnectedError(f"{self} is not connected.")
    return self.bus.get_action()
```

这是整个 PiperLeader 中最简洁的方法——只有 2 行有效代码。`self.bus.get_action()` 返回的字典键名没有 `.pos` 后缀（直接是 `"joint1"`，`"joint2"` 等），这与 PiperFollower 的 `get_observation()` 不同。这是因为：

- PiperFollower 的观测数据需要写入 LeRobot 数据集，必须遵循数据集的 `.pos` 命名约定。
- PiperLeader 的动作数据由 PC 中继程序直接转发，不经过数据集序列化，因此可以使用更简短的键名。

6.3.7 send_feedback() —— 空实现

```
def send_feedback(self, feedback: dict[str, Any]) -> None:
    pass
```

这个方法是一个**存根 (stub)**——只有 `pass` 语句，什么都不做。在 LeRobot 的 Teleoperator 抽象中，`send_feedback()` 的目的是向主臂发送力反馈信号（例如在 VR 手柄或 haptic 设备中提供触觉反馈）。PiPER 的主臂是纯机械的被动示教臂，不具备力反馈执行器，因此该方法为空。

这个空实现的存在是为了满足 `Teleoperator` 抽象基类的接口约定——如果不在子类中实现 `send_feedback()`，Python 会在实例化时报 `TypeError: Can't instantiate abstract class`。

6.4 Config 类解析

6.4.1 PiperFollowerConfig

```
@RobotConfig.register_subclass("piper_follower")
@dataclass(kw_only=True)
class PiperFollowerConfig(RobotConfig):
    port: str
    disable_torque_on_disconnect: bool = True
    cameras: dict[str, CameraConfig] = field(default_factory=dict)
    max_relative_target: float | dict[str, float] | None = None
```

各字段说明：

字段	类型	默认值	说明
<code>port</code>	<code>str</code>	(必填)	CAN 口名称，如 <code>"can0"</code> 、 <code>"can1"</code> 。这是唯一没有默认值的字段，必须在构造时显式提供
<code>disable_torque_on_disconnect</code>	<code>bool</code>	<code>True</code>	断开连接时是否失能力矩。设为 <code>True</code> 时，断开后电机自由旋转，人可以手动移动机械臂；设为 <code>False</code> 时，断开后电机保持当前位置
<code>cameras</code>	<code>dict[str, CameraConfig]</code>	<code>{}</code> (空字典)	相机配置字典。key 为相机名（如 <code>"wrist"</code> ， <code>"global"</code> ），value 为 <code>CameraConfig</code> 对象。空字典意味着不使用任何相机
<code>max_relative_target</code>	<code>float</code> <code>dict[str, float]</code> <code>None</code>	<code>None</code>	单步最大相对位移限幅。 <code>None</code> 表示 unlimited； <code>float</code> 表示所有关节共用同一值；字典表示 per-joint 配置

从 `RobotConfig` 继承的字段（在 `RobotConfig.__init__` 中处理）：

字段	类型	默认值	说明
<code>id</code>	<code>str</code> \ \n <code>None</code>	<code>None</code>	机器人实例标识。同一台机器上连接多个同型号机械臂时用于区分
<code>calibration_dir</code>	<code>Path</code> \ \n <code>None</code>	<code>None</code>	校准文件存储目录。如果为 <code>None</code> ，自动使用 <code>~/.cache/huggingface/lerobot/calibration/robots/piper_follower/</code>

`@dataclass(kw_only=True)` 强制所有字段必须以关键字参数形式传递，避免位置参数导致的混淆（例如 `PiperFollowerConfig("can0", True)` 这种容易误读的写法被禁止，必须写作 `PiperFollowerConfig(port="can0", disable_torque_on_disconnect=True)`）。

`RobotConfig.__post_init__` 的相机验证（从 `RobotConfig` 继承）：

```
def __post_init__(self):
    if hasattr(self, "cameras") and self.cameras:
        for _, config in self.cameras.items():
            for attr in ["width", "height", "fps"]:
                if getattr(config, attr) is None:
                    raise ValueError(
                        f"Specifying '{attr}' is required for the camera to be used in a robot"
                    )
```

如果在 `cameras` 字典中配置了相机但没有指定 `width`、`height` 或 `fps`，会在实例化时直接抛出 `ValueError`。这确保了配置的完整性。

6.4.2 PiperLeaderConfig

```
@TeleoperatorConfig.register_subclass("piper_leader")
@dataclass
class PiperLeaderConfig(TeleoperatorConfig):
    port: str
    gripper_open_pos: float = 50.0
```

各字段说明：

字段	类型	默认值	说明
<code>port</code>	<code>str</code>	（必填）	CAN 口名称
<code>gripper_open_pos</code>	<code>float</code>	50.0	

字段	类型	默认值	说明
			夹爪张开位置（归一化值，范围 [0, 100]）。当主臂进入转矩模式时，夹爪电机收到该位置值。如果人捏合夹爪后释放，夹爪会自动弹回此位置

`gripper_open_pos = 50.0` 的含义：在归一化范围 [0, 100] 中，50.0 对应夹爪半开状态。这提供了一种“弹性夹爪”的体验——人可以用力捏合夹爪，松开后夹爪自动回到半开位置（而不是全开或全闭）。这个默认值可以按任务需要调整。

与 `PiperFollowerConfig` 不同，`PiperLeaderConfig` **没有相机配置**（因为主臂无相机），**没有安全限幅配置**（因为主臂只读不写），**没有 `disable_torque_on_disconnect`**（主臂的 `disconnect()` 总是会失能力矩）。

注意：`PiperLeaderConfig` 使用 `@dataclass` 而非 `@dataclass(kw_only=True)`。这意味着它接受位置参数——`PiperLeaderConfig("can0")` 是合法写法（但不推荐）。

6.5 注册机制深入

6.5.1 `register_subclass` 的工作原理

两个配置类顶部都有类似的装饰器：

```
@RobotConfig.register_subclass("piper_follower")    # PiperFollowerConfig
@TeleoperatorConfig.register_subclass("piper_leader") # PiperLeaderConfig
```

这个机制是 LeRobot 实现**多态配置**的核心。它的工作流程如下：

第 1 步：装饰器注册

`RobotConfig` 继承自 `draccus.ChoiceRegistry`（见 `RobotConfig` 的定义）：

```
@dataclass(kw_only=True)
class RobotConfig(draccus.ChoiceRegistry, abc.ABC):
    ...
```

`draccus.ChoiceRegistry` 维护一个全局的**注册表**（registry），将字符串名称映射到配置子类：

```
注册表（内部）：
{
    "piper_follower": PiperFollowerConfig,
    "so100":          So100Config,
```



```
"koch":      KochConfig,
...
}
```

当 Python 解释器执行到 `@RobotConfig.register_subclass("piper_follower")` 时（类定义时刻），装饰器将 `"piper_follower"` → `PiperFollowerConfig` 的映射写入注册表。这个过程在 `import` 时就已完成，不需要等到实例化。

第 2 步：CLI 参数解析

用户在命令行使用 `lerobot-record --robot.type=piper_follower --robot.port=can1` 时：

1. `draccus`（基于 `dataclasses` + `argparse / jsonargparse` 的配置解析库）解析 `--robot.type=piper_follower`。
2. `draccus` 查询 `RobotConfig` 的注册表，找到 `"piper_follower"` 对应的子类 `PiperFollowerConfig`。
3. `draccus` 使用 `PiperFollowerConfig` 的字段定义来解析剩余的 `--robot.*` 参数（如 `--robot.port=can1`）。
4. 如果用户传入 `--robot.cameras.wrist.type=realsense --robot.cameras.wrist.serial=...`，`draccus` 递归地使用 `CameraConfig` 的注册表（也是 `ChoiceRegistry` 的子类）来解析相机子配置。

第 3 步：类型属性验证

配置文件实例化后，`PiperFollower.__init__` 接收这个配置对象。由于类定义中有 `config_class: PiperFollowerConfig` 的类型注解，IDE 和类型检查器可以验证类型正确性。不过，在运行时 Python 并不会强制执行这个类型约束——如果错误地将 `PiperLeaderConfig` 传入 `PiperFollower()`，只有在访问不存在的属性时才会报 `AttributeError`。

6.5.2 type 属性

两个配置类都定义了 `type` 属性：

```
@property
def type(self) -> str:
    return self.get_choice_name(self.__class__)
```

`get_choice_name()` 是 `draccus.ChoiceRegistry` 提供的方法，它反向查找注册表——给定一个类，返回它被注册时的名称字符串。对于 `PiperFollowerConfig`，返回 `"piper_follower"`。

这个属性使得配置对象具有**自描述性**：给定任意一个配置对象，你可以通过 `config.type` 知道它属于哪种 robot/teleoperator。

6.6 完整使用示例

6.6.1 使用 PiperFollower 编程（不使用 CLI）

```
from lerobot.robots.piper_follower import PiperFollower, PiperFollowerConfig
from lerobot.cameras import OpenCVCameraConfig

# 第 1 步：构造配置
config = PiperFollowerConfig(
    port="can1",
    cameras={
        "wrist": OpenCVCameraConfig(
            camera_index=0,
            width=640,
            height=480,
            fps=30,
        ),
    },
    max_relative_target=5.0, # 安全限幅：单步不超过 5.0
)
print(f"Robot type: {config.type}") # 输出: piper_follower

# 第 2 步：创建 robot 对象（此时无硬件连接）
robot = PiperFollower(config)
print(f"Features: {robot.action_features}")
# 输出: {'joint1.pos': float, 'joint2.pos': float, ...,
#       'gripper.pos': float, 'wrist': (480, 640, 3)}

# 第 3 步：连接硬件
robot.connect(calibrate=True)
print(f"Connected: {robot.is_connected}") # 输出: True

# 第 4 步：读取观测
obs = robot.get_observation()
print(obs.keys())
# 输出: dict_keys(['joint1.pos', 'joint2.pos', 'joint3.pos',
#                  'joint4.pos', 'joint5.pos', 'joint6.pos',
#                  'gripper.pos', 'wrist'])
print(f"joint1.position = {obs['joint1.pos']}")
# 输出: joint1.position = -15.2 (示例值)

# 第 5 步：发送动作
action = {
    "joint1.pos": 10.0,
    "joint2.pos": 30.0,
    "joint3.pos": -50.0,
    "joint4.pos": 0.0,
    "joint5.pos": 20.0,
    "joint6.pos": -30.0,
    "gripper.pos": 80.0,
}
result = robot.send_action(action)
# 机械臂运动到目标位置，result 是实际执行的动作
```

```
# 第 6 步：断开连接
robot.disconnect(disable_torque=True)
print(f"Connected: {robot.is_connected}") # 输出: False
```

6.6.2 使用 PiperLeader 编程（不使用 CLI）

```
from lerobot.teleoperators.piper_leader import PiperLeader, PiperLeaderConfig

# 第 1 步：构造配置
config = PiperLeaderConfig(
    port="can0",
    gripper_open_pos=60.0, # 夹爪默认处于较开位置
)

# 第 2 步：创建 leader 对象
leader = PiperLeader(config)
print(f"Teleoperator type: {config.type}") # 输出: piper_leader

# 第 3 步：连接硬件
leader.connect()
print(f"Connected: {leader.is_connected}") # 输出: True

# 第 4 步：读取主臂位置（循环读取，模拟遥操作）
import time
for i in range(100):
    action = leader.get_action()
    print(f"Frame {i}: {action}")
    # 输出: {'joint1': -10.5, 'joint2': 45.2, ..., 'gripper': 55.0}
    time.sleep(0.033) # ~30 FPS

# 第 5 步：断开
leader.disconnect()
```

6.6.3 主从联动示例（PC 中继方案核心逻辑）

以下代码展示了 PC 中继遥操作中最核心的数据流——这实际上就是 `piper_pc_relay_teleop.py` 的简化版本：

```
from lerobot.robots.piper_follower import PiperFollower, PiperFollowerConfig
from lerobot.teleoperators.piper_leader import PiperLeader, PiperLeaderConfig

# 分别配置主臂和从臂（它们连接不同的 CAN 口）
leader_config = PiperLeaderConfig(port="can0")
follower_config = PiperFollowerConfig(port="can1")

leader = PiperLeader(leader_config)
follower = PiperFollower(follower_config)

# 连接
leader.connect()
```

```

follower.connect()

try:
    while True:
        # 第 1 步：读取主臂被拖动到的位置
        action = leader.get_action()
        # action = {"joint1": 10.5, "joint2": 45.2, ..., "gripper": 55.0}

        # 第 2 步：添加 .pos 后缀，转发给从臂
        action_with_suffix = {f"{k}.pos": v for k, v in action.items()}
        # action_with_suffix = {"joint1.pos": 10.5, ...}

        # 第 3 步：从臂执行动作
        follower.send_action(action_with_suffix)

finally:
    # 清理
    leader.disconnect()
    follower.disconnect()

```

这个循环的核心逻辑只有 3 行代码，体现了 LeRobot 封装带来的简洁性：

1. `leader.get_action()` — 读主臂位置
2. 添加 `.pos` 后缀 — 格式转换
3. `follower.send_action()` — 发送给从臂执行

6.7 小结

本章我们逐行剖析了 `PiperFollower` 和 `PiperLeader` 两个核心类的完整实现。以下是关键收获：

1. **Robot 和 Teleoperator 是两个独立的抽象层次。** Robot 管理感知（观测）和执行（动作）的完整闭环，Teleoperator 只负责输出人类示教的位置数据。
2. **电机配置是整个系统的数据根基。** `Motor` 的型号和归一化模式 + `MotorCalibration` 的物理范围，共同决定了归一化和反归一化的映射关系。AGILEX-M 用于大扭矩关节（1-3），AGILEX-S 用于小扭矩关节（4-6+夹爪），HTDW-5047 用于教学臂——这背后是硬件设计的工程权衡。
3. **校准只对写操作有意义。** `PiperFollower` 需要校准来确保发送的目标值在物理限制内，`PiperLeader` 只读不写，因此不需要校准。
4. **`max_relative_target` 是最后一道安全防线。** 它防止单步移动过大导致的机械臂抖动或碰撞，但会带来额外的 CAN 读取延迟。
5. **`disconnect` 的 bug 修复体现了防御性编程的重要性。** 相机和 CAN 总线都是需要显式清理的资源，断开顺序应当与连接顺序相反。

6. 注册机制使多态配置成为可能。 `@RobotConfig.register_subclass("piper_follower")` 通过 draccus 的 `ChoiceRegistry` 实现了"字符串 → 类"的自动路由, 使得 CLI 工具可以用 `--robot.type=piper_follower` 这样的简洁语法选择不同的硬件驱动。

在下一章, 我们将介绍数据采集的完整流程——包括 PC 中继遥操作脚本、LeRobot 的 `record` 命令、以及数据是如何被写入 HuggingFace Datasets 格式的。

第七章 PC 中继遥操作

本章是 PiPER + LeRobot 模仿学习流水线中**最核心的日常操作章节**。你将深入理解 PC 中继方案的设计动机、架构原理、代码实现和操作流程。阅读完本章后，你应当能够：

- 理解为什么需要 PC 中继，以及它与硬件主从模式和 LeRobot 内置 teleoperate 的区别
- 掌握 `piper_pc_relay_teleop.py` 的每一行代码逻辑
- 掌握 `record_with_pc_relay.sh` 的完整采集流程
- 独立调试中继遥操作中的常见问题
- 根据实际需求调整关节变换、速率限制等参数

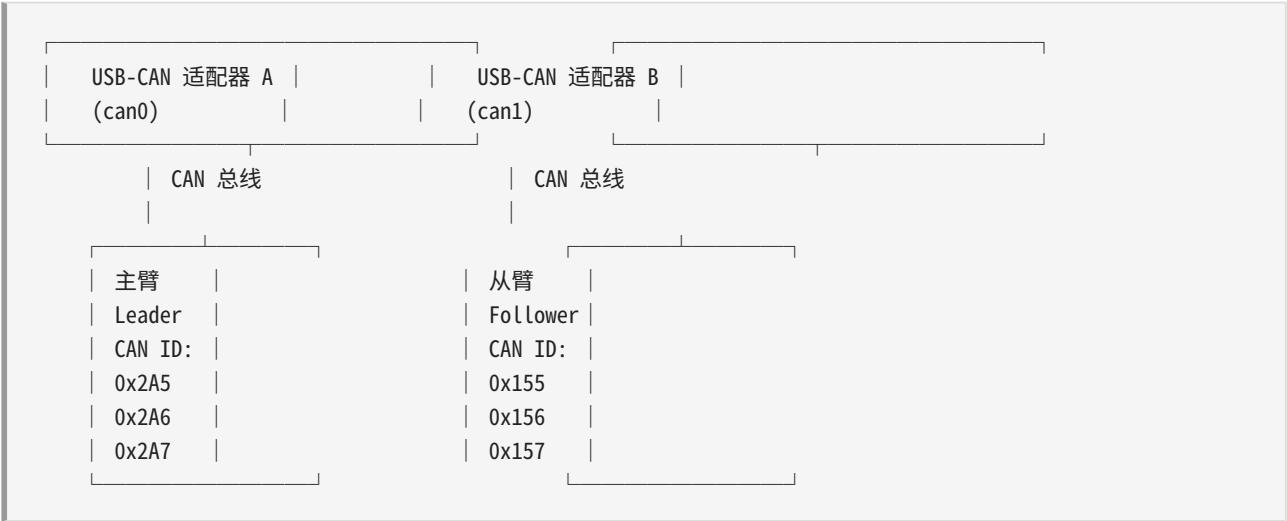
本章内容覆盖以下两个源文件：

文件	行数	角色
<code>scripts/piper_pc_relay_teleop.py</code>	309	PC 中继引擎：读主臂、变换、限速、写从臂
<code>record_with_pc_relay.sh</code>	228	采集编排器：启动中继、配置相机、调用 <code>lerobot-record</code>

7.1 为什么需要 PC 中继

7.1.1 问题场景：主臂和从臂在不同 CAN 总线上

在 PiPER 双臂协作机器人系统中，主臂（Leader）和从臂（Follower）各自连接一个独立的 USB-CAN 适配器，在操作系统中分别呈现为 `can0` 和 `can1` 两个网络接口：



主臂以 1kHz 频率通过 CAN ID 0x2A5 - 0x2A7 上报 6 个关节的实时位置。从臂通过 CAN ID 0x155 - 0x157 接收关节目标位置命令。这两个 CAN 总线在电气和协议层面完全隔离——主臂发出的 CAN 帧物理上无法到达从臂所在的 can1 总线，反之亦然。

这并非设计缺陷，而是一种**刻意为之的拓扑选择**： - 每个 USB-CAN 适配器有其带宽上限（通常 1 Mbps），将两个臂分到不同总线可避免带宽争用 - 调试时可以独立插拔某一个臂的 CAN 线，不影响另一个臂的通信 - 如果将来需要扩展更多外设（如力传感器、额外的编码器），独立的 CAN 总线提供了物理隔离的扩展空间

但这也意味着：如果不在 PC 上编写软件桥接程序，主臂和从臂之间无法直接通信。

7.1.2 硬件主从模式的局限

PiPER 的电机驱动器（型号 HTDW-5047 和 AGILEX-M/S）在固件层面支持一种"硬件主从模式"：将主臂的某个 CAN ID 配置为将关节状态直接转发到从臂对应的 CAN ID。这种模式在理论上可以实现"零延迟"的关节跟随。

然而，硬件主从模式有若干无法回避的局限：

1. 需要 CAN ID 偏移配置

硬件主从模式要求主臂和从臂的 CAN ID 之间有一个固定的偏移量。例如，主臂上报 0x2A5，从臂监听 0x2A5 - offset。这个偏移量需要写入电机固件的配置寄存器，操作繁琐且不可逆（需要通过专门的配置工具修改）。

在 PiPER 的默认固件中，主臂的上报 ID（0x2A5 - 0x2A7）和从臂的接收 ID（0x155 - 0x157）之间并无简单的偏移关系，这意味着如果使用硬件主从模式，需要**重新烧写至少一个臂的 CAN ID 配置**——这是一个高风险操作，稍有不慎会导致电机无法通信。

2. 无法做关节变换

硬件主从模式将主臂的编码器值**原封不动**地写入从臂。但实际使用中，以下场景需要关节变换：

- **方向翻转**：主臂和从臂可能采用镜像安装，关节 1 的正方向在主臂上是"顺时针"，在从臂上却是"逆时针"。如果直接转发编码器值，两个臂的运动方向会相反。
- **零点偏置**：主臂和从臂的机械零点可能不一致。主臂关节 2 在 0 度时，从臂关节 2 可能处于 +15 度的位置。需要加一个固定的偏置来对齐。

硬件主从模式只能做"1:1 直通"，无法插入任何数学变换。

3. 无法做速率限制

硬件主从模式将主臂的位置以 CAN 总线的原生速率（1 kHz）转发。当人快速拖拽主臂时，从臂会收到一个瞬时的大幅度位置跳变。虽然电机驱动器内部有自身的速度环和电流环保护，但**位置指令的突变**仍可能导致从臂产生不期望的抖动甚至过流保护。

PC 中继可以在软件层面做**增量限幅（per-step delta clamping）**，确保每一步的位置变化不超过安全阈值。

4. 无法与数据采集协调

硬件主从模式是一个纯硬件通路：主臂动 → 从臂动。它完全不知道"数据采集"这回事。如果 lerobot-record 在采集过程中需要暂停（例如进入 reset 阶段提示操作者复位环境），硬件主从模式下从臂会继续跟随主臂运动，导致复位过程中从臂处于不可控状态。

7.1.3 PC 中继的优势

PC 中继方案在 PC 上运行一个 Python 进程，通过 piper_sdk 同时连接 can0（读主臂）和 can1（写从臂），在软件层面构建一条可编程的数据通路。相比硬件主从模式，PC 中继有以下优势：

能力	硬件主从	PC 中继
关节方向翻转	不支持（需重新安装机械臂）	支持（ <code>--signs</code> 参数）
零点偏置对齐	不支持（需机械调整）	支持（ <code>--offset-deg</code> 参数）
速率限制	依赖驱动器自身保护	支持（ <code>--max-step-deg</code> 参数）
关节限位	依赖硬件限位开关	支持（软件层面 clamp 到关节允许范围）
与数据采集协调	不支持	支持（pause-file 机制）
安全开关	上电即跟随	<code>--execute</code> 显式启用，默认 dry-run
调试可见性	无	实时打印主臂/从臂/命令的角度值
退出保护	直接断电可能导致臂坠落	退出时自动发送 standby 模式命令

7.1.4 为什么不用 LeRobot 内置的 teleoperate 功能

LeRobot 框架提供了内置的遥操作机制（lerobot-record 的 `--teleoperator.type` 和 `--teleoperator.port` 参数）。它的设计假设是：

- 主臂（Teleoperator）和从臂（Robot）是**同一型号**的机械臂（如两个 SO-100 臂）
- 数据通路是 主臂.get_action() → lerobot-record → 从臂.send_action()，即 lerobot-record 进程同时连接主臂和从臂

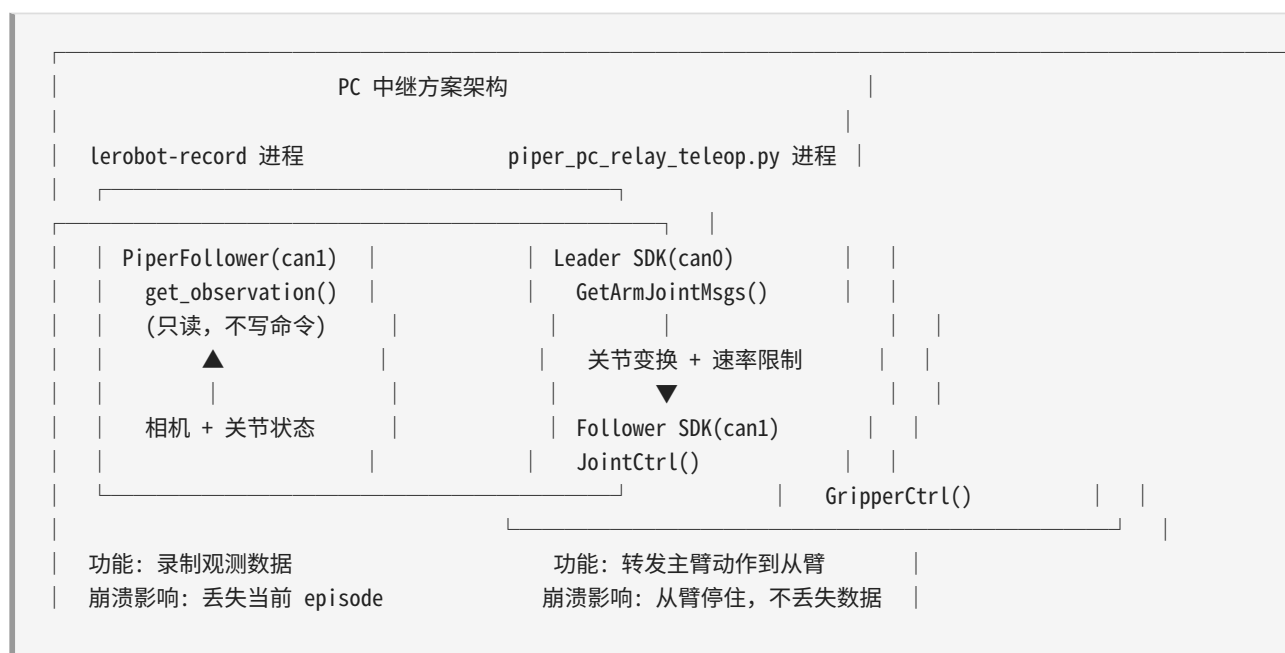
- 主臂和从臂可能共享同一个通信总线（如两个 SO-100 的 Dynamixel 舵机串联在同一条 RS-485 总线上）

这个假设在 PiPER 场景中不成立：

1. **PiPER 的 CAN 总线是独立的：** `can0` 和 `can1` 是两条物理上分离的 CAN 总线。lerobot-record 的架构期望一个 Robot 对象和一个 Teleoperator 对象，但 PiperFollower 和 PiperLeader 各自只能绑定一个 CAN 口。如果在同一个进程中创建两个 `C_PiperInterface_V2` 实例分别连 `can0` 和 `can1`，**piper_sdk 的单例模式会阻止第二个实例的创建**——因为 SDK 内部以 `can_name` 为键维护全局单例，但两个不同的 CAN 口需要两个独立的连接。
2. **需要一个独立于录制进程的中继：**如果中继逻辑嵌入在 lerobot-record 进程内，那么 lerobot-record 的崩溃（例如磁盘满导致写入失败）会导致中继停止，从臂失去控制。将中继放在独立进程中可以做到**故障隔离**——录制进程挂了，中继进程继续工作，从臂仍然跟随主臂。
3. **复位协调需要进程间通信：**lerobot-record 在 episode 之间需要暂停中继以执行复位操作。`pause-file` 是一种简单的进程间协调机制（文件系统作为信号量），两个独立进程可以通过文件的存在/不存在来同步状态。如果中继嵌入在 lerobot-record 内，这种协调会变得复杂（需要内部状态机）。

因此在 PiPER 的 PC 中继方案中：

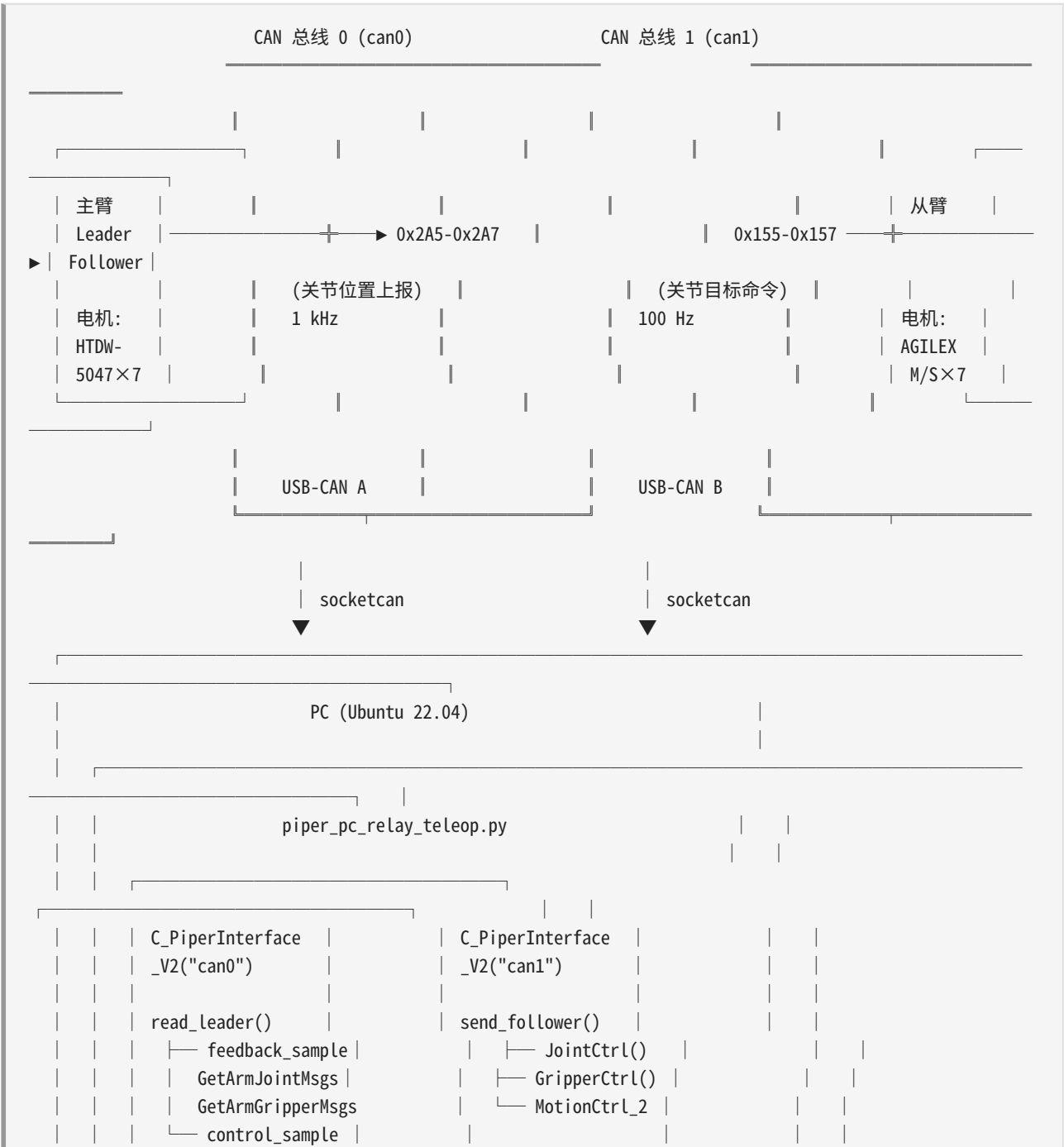
- **lerobot-record 只负责录制：**它连接从臂（`--robot.type=piper_follower --robot.port=can1`），读取从臂的关节状态和相机图像，但**不连接主臂也不发送动作命令**。注意 `record_with_pc_relay.sh` 中**没有** `--teleoperator.type` 参数。
- **piper_pc_relay_teleop.py 负责中继：**它独立运行，连接 `can0`（读）和 `can1`（写），负责从主臂到从臂的全量数据转发和变换。

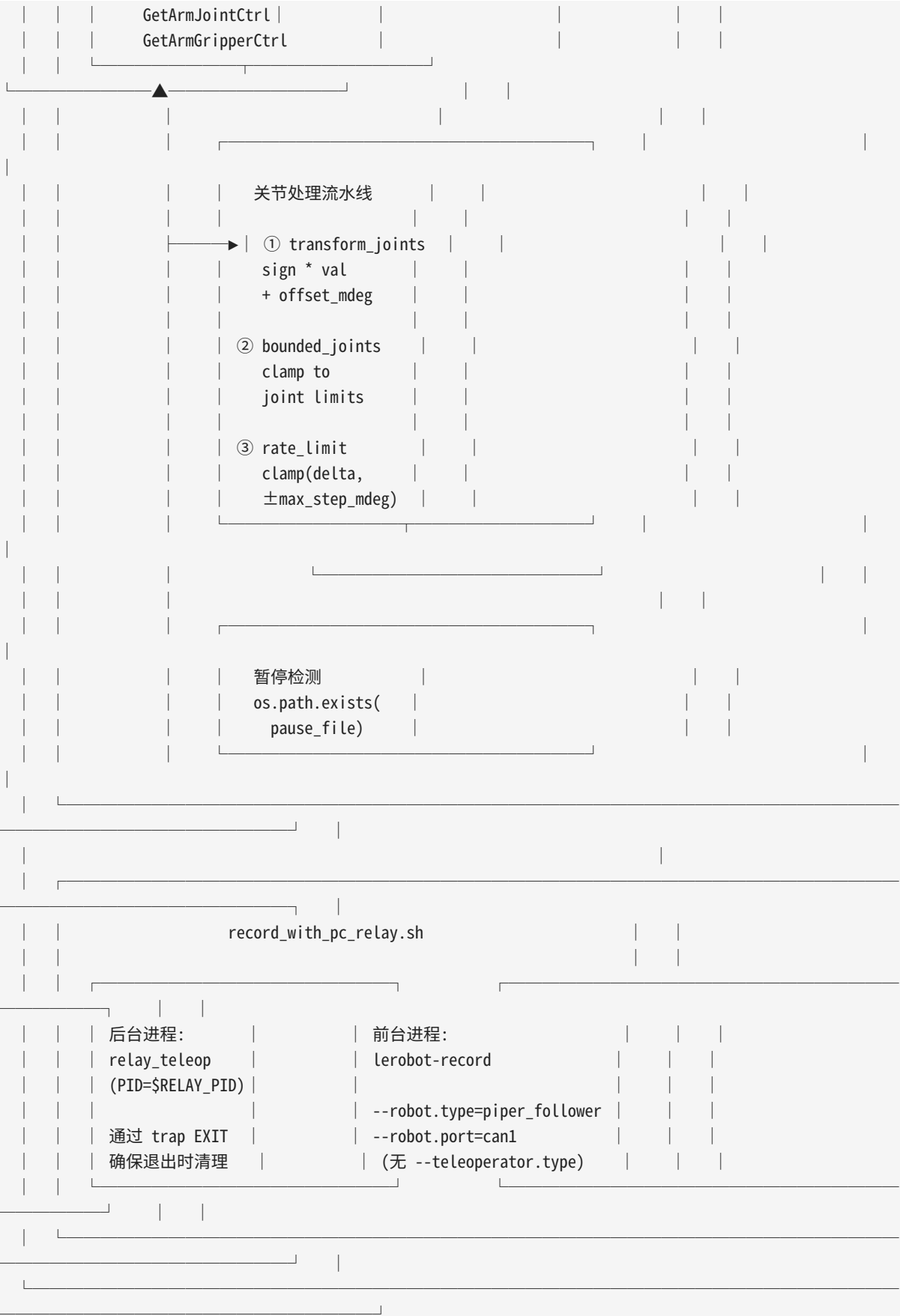


7.2 架构设计

7.2.1 整体数据流

以下 ASCII 图展示了 PC 中继方案的完整数据流。请仔细跟踪每一条箭头——它刻画了从"人的手拖动主臂"到"从臂执行动作"再到"数据写入硬盘"的全过程。





7.2.2 关键设计决策

决策 1：中继进程和录制进程分离

中继进程（Python 脚本）和录制进程（lerobot-record CLI）是两个独立的操作系统进程。它们之间唯一的通信渠道是文件系统（pause-file）。这样做的原因：

- **故障隔离**：如果 lerobot-record 因为磁盘满、相机断连等原因崩溃，中继进程不受影响，从臂仍然安全跟随主臂。如果中继进程崩溃，lerobot-record 会记录到"从臂不再移动"，但已写入硬盘的数据不会丢失。
- **独立生命周期**：中继进程需要在整个采集会话期间持续运行（可能数小时），而录制进程在每个 episode 之间会进入不同的状态（录制态 → 复位态 → 录制态）。独立进程使得各自的启动、停止、异常处理逻辑互不干扰。
- **可替换性**：你可以单独升级中继脚本而不影响 lerobot-record，反之亦然。你也可以在调试时手动启停中继进程，而不影响已录制的数据。

决策 2：默认 dry-run 模式（`--execute` 显式启用）

中继脚本的默认行为是**不发送任何命令给从臂**（dry-run 模式）。只有显式传入 `--execute` 参数后，才会真正通过 CAN 总线向从臂发送 JointCtrl / GripperCtrl / MotionCtrl_2 命令。

这是一个关键的安全设计：如果你忘记插从臂的 CAN 线，或者在错误的配置下运行脚本，dry-run 模式确保不会发生意外运动。你可以在 dry-run 模式下观察控制台打印的主臂位置、变换后位置和限速后位置，确认一切正确后再加 `--execute` 真正控制从臂。

决策 3：基于反馈位置（feedback position）的速率限制

速率限制算法使用**从臂当前的实际位置**（通过 `feedback_sample()` 读取）作为基线，而不是上一次发送的命令位置。这样做的原因是：从臂可能因为外部阻力、电机力矩不足、通信延迟等原因没有精确到达上一次命令指定的位置。如果基于"命令位置"做限速，累积误差会越来越大。基于"实际反馈位置"做限速，可以天然消除累积误差——每一步的增量都是相对于"从臂真正到达的位置"来计算的。

7.3 piper_pc_relay_teleop.py 逐段解析

本节按照脚本的代码结构，从上到下解析每一个模块和函数。建议在阅读时保持源码在另一个窗口打开，对照阅读。

7.3.1 导入和常量定义

```
from piper_sdk import C_PiperInterface_V2
```

脚本的唯一外部依赖是 `piper_sdk`，它提供了 CAN 通信的底层封装。`C_PiperInterface_V2` 是一个 C++ 扩展类（通过 `pybind11` 绑定），内部管理着一个后台线程持续收发 CAN 帧。注意 `v2` 后缀——这是 PiPER SDK 的第二个大版本，与第一版在 API 上有显著差异（例如 `ConnectPort` 的参数从位置参数变为关键字参数）。

```
JOINT_LIMITS_MDEG = [
    (-150_000, 150_000), # joint 1: ±150°
    (0, 180_000),        # joint 2: 0° ~ 180°
    (-170_000, 0),        # joint 3: -170° ~ 0°
    (-100_000, 100_000),  # joint 4: ±100°
    (-70_000, 70_000),    # joint 5: ±70°
    (-120_000, 120_000),  # joint 6: ±120°
]

GRIPPER_LIMIT_UM = (0, 68_000)
```

关节限位值以**毫度**（**mdeg**, **millidegree**）为单位。1 mdeg = 0.001 度，即 1000 mdeg = 1 度。使用整数的原因是 CAN 协议中角度值以编码器脉冲为单位传输，1 度对应 1000 个编码器单位。

为什么要存这些限位值？因为 PC 中继的 `bounded_joints()` 函数会在发送命令前将目标位置 clamp 到这些范围内。即使主臂的操作者将关节拖到了超出从臂物理极限的角度，从臂也不会尝试执行这个超出范围的目标——它会被 clamp 到最近的安全边界。这是一种纯软件层面的**二次安全保护**（一次保护是电机驱动器内部的限位）。

夹爪的限位单位是**微米**（**um**），范围 0 到 68,000 um = 68 mm。这对应了夹爪平行开闭的行程。

7.3.2 ArmSample 数据类

```
@dataclass
class ArmSample:
    joints: list[int]      # 6 个关节的编码器值 (mdeg)
    gripper: int           # 夹爪位置 (um)
    joint_hz: float        # 关节数据刷新率
    gripper_hz: float      # 夹爪数据刷新率
    joint_timestamp: float # 关节数据时间戳
    source: str            # 数据来源: "feedback" 或 "control"
```

`ArmSample` 是一个纯数据容器，封装了从机械臂读取的一组完整的关节和夹爪状态。`joint_hz` 和 `joint_timestamp` 是从 CAN 消息的元数据中提取的，用于判断数据流是否正常——如果 `joint_hz <= 0`

且 `joint_timestamp <= 0`，说明 CAN 总线上没有有效的关节数据（可能 CAN 口未 up、波特率不匹配、或机械臂未上电）。

7.3.3 读取函数：三种数据来源

脚本提供了三种读取主臂位置的方式，由 `--source` 参数控制：

```
JointSource = Literal["auto", "feedback", "control"]
```

方式 1: `feedback_sample` — 读取反馈位置

```
def feedback_sample(piper: C_PiperInterface_V2) -> ArmSample:
    joint_msg = piper.GetArmJointMsgs()
    gripper_msg = piper.GetArmGripperMsgs()
    joint = joint_msg.joint_state
    gripper = gripper_msg.gripper_state
    return ArmSample(
        joints=[int(joint.joint_1), ..., int(joint.joint_6)],
        gripper=int(gripper.grippers_angle),
        ...
        source="feedback",
    )
```

`GetArmJointMsgs()` 返回的是电机编码器**实际测量到的当前位置**（即反馈位置）。`GetArmGripperMsgs()` 同理。这是最“真实”的数据——它反映的是机械臂物理上所处的角度，而不是任何命令期望的角度。

使用场景：如果你希望从臂严格跟随主臂的**实际物理位置**（而不是主臂最后一次接收到的命令位置），使用 `feedback` 源。

方式 2: `control_sample` — 读取控制位置

```
def control_sample(piper: C_PiperInterface_V2) -> ArmSample:
    joint_msg = piper.GetArmJointCtrl()
    gripper_msg = piper.GetArmGripperCtrl()
    ...
    source="control",
```

`GetArmJointCtrl()` 返回的是**上一次发送给该臂的关节目标命令值**。在主臂（Leader）上，通常不会有外部进程向它发送命令——主臂的电机处于被拖动状态，电机驱动器持续采样编码器并更新此值。在某些固件版本中，`GetArmJointCtrl()` 返回的值可能比 `GetArmJointMsgs()` 更平滑（经过了驱动器内部的滤波）。

`GetArmGripperCtrl()` 同理，返回的是上一次发送给夹爪的目标位置。

使用场景：录制脚本 `record_with_pc_relay.sh` 默认使用 `RELAY_SOURCE=control`。

方式 3: auto — 自动选择

```
def read_leader(piper: C_PiperInterface_V2, source: JointSource) -> ArmSample:
    if source == "feedback":
        return feedback_sample(piper)
    if source == "control":
        return control_sample(piper)
    # auto
    ctrl = control_sample(piper)
    if ctrl.joint_hz > 0 or ctrl.joint_timestamp > 0:
        return ctrl
    return feedback_sample(piper)
```

`auto` 模式先尝试 `control_sample()`，如果控制数据流有效（`joint_hz > 0` 或 `joint_timestamp > 0`），则使用控制数据；否则回退到反馈数据。这是一种"优先使用更平滑的控制数据，但确保不会用无效数据"的策略。

三种方式的对比

来源	读取的 API	值的含义	平滑度	推荐场景
feedback	<code>GetArmJointMsgs()</code>	编码器实测角度	可能有微小的测量噪声	需要最真实的物理角度
control	<code>GetArmJointCtrl()</code>	上一次命令角度	经过驱动器滤波，更平滑	数据采集（默认）
auto	先 control 后 feedback	自动选择	取决于可用数据	不确定时使用

7.3.4 关节变换: `transform_joints`

```
def transform_joints(
    joints: list[int],
    signs: list[float],
    offsets_mdeg: list[float],
) -> list[int]:
    return [
        int(round(joint * sign + offset))
        for joint, sign, offset in zip(joints, signs, offsets_mdeg, strict=True)
    ]
```

变换公式：

```
cmd_val[i] = round(leader_val[i] × sign[i] + offset[i])
```

其中：- `sign[i]` 是关节 `i` 的方向符号。`1.0` 表示方向相同，`-1.0` 表示方向反转。- `offset[i]` 是以毫度为单位的偏置值。

为什么需要方向符号？

主臂和从臂的关节可能采用不同的安装方向。例如，如果主臂的关节 1 正方向是逆时针旋转（从顶部看），而从臂的关节 1 正方向是顺时针旋转。当人将主臂关节 1 逆时针旋转 30 度时，如果没有符号翻转，从臂也会逆时针旋转 30 度——但因为它在自己的坐标系中"逆时针"对应了相反的方向，实际效果是从臂向错误方向运动。

设置 `--signs "1,-1,1,1,1,1"` 即可只翻转关节 2 的方向。

为什么需要偏置？

主臂和从臂的机械零点可能不一致。例如，主臂关节 3 在编码器值为 0 时指向正前方，但从臂关节 3 在编码器值为 0 时指向下方 15 度。设置 `--offset-deg "0,0,-15,0,0,0"` 即可将关节 3 的零点偏移 -15 度，使两个臂在物理空间中对齐。

7.3.5 关节限位：bounded_joints

```
def bounded_joints(joints: list[int]) -> list[int]:
    return [
        clamp(joint, low, high)
        for joint, (low, high) in zip(joints, JOINT_LIMITS_MDEG, strict=True)
    ]
```

该函数将变换后的目标关节位置 clamp 到 `JOINT_LIMITS_MDEG` 定义的每个关节的物理允许范围内。这一步在速率限制之前执行——如果变换后的目标已经超出了关节限位，先把它拉回安全范围，再做速率限制，避免速率限制在非法值上运算。

7.3.6 速率限制：rate_limit

这是整个中继脚本中最核心的安全机制。

```
def rate_limit(
    current: Iterable[int],    # 从臂当前的反馈位置
    target: Iterable[int],    # 变换+限位后的目标位置
    max_step: int,            # 单步最大允许变化量 (mdeg)
) -> list[int]:
    limited = []
    for cur, tgt in zip(current, target, strict=True):
        delta = tgt - cur
        if delta > max_step:
            limited.append(cur + max_step)
        elif delta < -max_step:
            limited.append(cur - max_step)
```



```

else:
    limited.append(tgt)
return limited

```

算法逻辑（对每个关节独立执行）：

```

delta = target[i] - current[i]

if delta > +max_step:
    output[i] = current[i] + max_step # 向目标方向移动一步，但不超过 max_step
elif delta < -max_step:
    output[i] = current[i] - max_step # 向目标方向移动一步，但不超过 max_step
else:
    output[i] = target[i]             # 目标在安全范围内，直接采用

```

关键参数：max_step_mdeg

```
max_step_mdeg = int(round(args.max_step_deg * 1000))
```

`--max-step-deg` 的默认值是 1.0 度。在 `record_with_pc_relay.sh` 中，`RELAY_MAX_STEP_DEG` 默认设为 10 度，`RELAY_HZ` 默认设为 20 Hz。这意味着每步最多允许变化 10 度，即最大关节速度为 10 度/步 × 20 步/秒 = 200 度/秒。这个速度对于桌面级协作机械臂来说已经非常快，同时仍然在安全范围内。

如果你发现从臂在跟随主臂快速运动时有“追不上”的感觉（命令位置与实际位置之间有显著的持续偏差），可以适当增大 `--max-step-deg`。如果你发现从臂抖动明显（关节位置在目标附近来回震荡），可以适当减小 `--max-step-deg`。

为什么必须做速率限制？

1. **防止瞬时跳变：**如果主臂的 CAN 数据因为电磁干扰出现了一个异常值（例如关节 1 突然从 30 度跳到 300 度），没有速率限制的话，从臂会尝试以最大速度冲向 300 度，可能导致碰撞。速率限制将这一步的增量限制在 `max_step` 以内，从臂不会“相信”单步的突变。
2. **平滑 CAN 数据流：**主臂以 1 kHz 上报位置，但中继循环可能以 10-20 Hz 运行（受限于 USB-CAN 适配器的写入带宽和 Python GIL）。两个相邻的中继周期之间，主臂可能已经移动了相当大的角度。速率限制将这个大增量拆分为多个小步，使从臂的运动更平滑。
3. **防止操作者失误：**如果操作者不小心快速甩动主臂，速率限制确保从臂不会以同样危险的速度甩动。

7.3.7 从臂配置和命令发送

配置函数：configure_follower_for_joint_control

```
def configure_follower_for_joint_control(
    piper: C_PiperInterface_V2, speed: int, high_follow: bool
) -> None:
    mit_flag = 0xAD if high_follow else 0x00
    piper.MotionCtrl_2(0x01, 0x01, speed, mit_flag)
    time.sleep(0.05)
    piper.EnableArm()
    time.sleep(0.1)
```

`MotionCtrl_2` 的四个参数： - `0x01`：控制模式 = 关节位置控制 - `0x01`：使能状态 = 使能 - `speed`：速度百分比（0-100）。在 `record_with_pc_relay.sh` 中默认设为 100。 - `mit_flag`：MIT 模式标志。 `0xAD` 启用高跟随模式（higher torque bandwidth）， `0x00` 为普通模式。 `record_with_pc_relay.sh` 默认启用 `RELAY_HIGH_FOLLOW=1`。

然后调用 `EnableArm()` 使能电机力矩，等待 100ms 使能生效。如果不调用 `EnableArm()`，即使发送了 `JointCtrl` 命令，从臂的电机也不会输出力矩——关节处于自由状态。

如果你希望在脚本启动时**不自动使能从臂**（例如你想手动确认一切正常后再使能），可以使用 `--skip-follower-config` 参数。这在调试阶段非常有用。

命令发送函数：send_follower

```
def send_follower(
    piper: C_PiperInterface_V2,
    joints: list[int],          # 6 个关节的目标位置 (mdeg)
    gripper: int | None,        # 夹爪目标位置 (um), None 表示不发送夹爪命令
    speed: int,                 # 速度百分比
    high_follow: bool,          # MIT 模式
    send_mode: bool,            # 本周期是否同时发送 MotionCtrl_2
) -> None:
    if send_mode:
        mit_flag = 0xAD if high_follow else 0x00
        piper.MotionCtrl_2(0x01, 0x01, speed, mit_flag)
    piper.JointCtrl(*joints)
    if gripper is not None:
        piper.GripperCtrl(
            clamp(abs(gripper), *GRIPPER_LIMIT_UM), 1000, 0x03, 0
        )
```

三个 CAN 命令的发送顺序： 1. **MotionCtrl_2**（条件发送）：设置控制模式和使能状态。不每周期发送，由 `--mode-command-period` 控制发送间隔（例如每 1.0 秒发送一次），因为控制模式不需要每帧重复设置。 2. **JointCtrl**（每周期必发）：发送 6 个关节的目标位置。 `*joints` 将列表解包为 6 个位置参数。 3. **GripperCtrl**（条件发送）：发送夹爪的目标位置和速度/力矩参数。由 `--gripper-hz` 控制发送频率（默认 5 Hz），因为夹爪不需要像关节那样高的控制频率。

GripperCtrl 的参数： - clamp(abs(gripper), 0, 68000)：夹爪目标位置 clamp 到安全范围 - 1000：夹爪速度参数 - 0x03：夹爪控制模式 - 0：力矩参数

7.3.8 主循环逐行解析

让我们逐行分析 main() 函数中的主循环。这是整个 PC 中继的核心，它在一个无限循环中执行"读主臂 → 变换 → 限幅 → 写从臂"的流水线。

```
while not stop:
    now = time.monotonic()
```

time.monotonic() 返回单调递增的时钟（不受系统时间调整影响），用于计算每个循环迭代的耗时。

```
if args.duration > 0 and now - start_time >= args.duration:
    break
```

--duration 参数可以设置脚本自动退出（单位秒）。设为 0 表示无限运行直到 Ctrl-C。

```
if args.pause_file and os.path.exists(args.pause_file):
    time.sleep(period)
    continue
```

暂停机制的核心：如果 --pause-file 指定了一个文件路径，并且该文件在当前时刻存在，则跳过本周期——不读取主臂位置，不发送从臂命令。脚本 sleep period 秒后直接进入下一个循环。

注意 sleep(period) 的设计：它保证即使在暂停期间，循环频率也大致保持在 hz 的水平，不会因为 continue 而进入忙等待（busy loop）消耗 CPU。

暂停文件的**创建和删除**不由此脚本负责——它只是被动检查。文件的创建和删除由 lerobot-record 的 run_reset_command_if_configured() 函数管理（见 7.4.4 节）。

```
leader_sample = read_leader(leader, args.source)
follower_sample = feedback_sample(follower)
```

读取主臂和从臂的当前状态。注意：主臂的数据来源由 --source 决定，但从臂总是使用 feedback_sample()（读取实际位置）。这是因为速率限制需要基于从臂的**真实当前位置**，而不是它的控制目标位置（见 7.2.2 决策 3）。

```
if leader_sample.joint_hz <= 0 and leader_sample.joint_timestamp <= 0:
    ... # 打印警告, skip 本周期
    time.sleep(period)
    continue
```

如果主臂的 CAN 数据流无效（频率和时间戳均为 0），跳过发送。这处理了主臂未上电、CAN 线松动等异常情况。

```
raw_target = transform_joints(leader_sample.joints, signs, offsets_mdeg)
bounded_target = bounded_joints(raw_target)
limited_target = rate_limit(follower_sample.joints, bounded_target, max_step_mdeg)
```

三步流水线（顺序很重要）：

1. **transform_joints**: 应用方向符号和零点偏置
2. **bounded_joints**: clamp 到关节物理限位内
3. **rate_limit**: 基于从臂当前反馈位置，限制单步增量的幅度

顺序不可颠倒。如果先 `rate_limit` 再 `bounded_joints`，则 `rate_limit` 可能在越界值上计算（例如 `target` 为 200,000 mdeg，远超 `limit` 的 150,000 mdeg），导致 `rate_limit` 产生一个不合理的大步长。正确的顺序是先 clamp 到合法范围，再在合法范围内做增量限幅。

```
send_gripper = (
    args.include_gripper
    and (args.gripper_hz <= 0 or now - last_gripper_command >= 1.0 / args.gripper_hz)
)
gripper = leader_sample.gripper if send_gripper else None
send_mode = (
    args.mode_command_period > 0
    and now - last_mode_command >= args.mode_command_period
)
```

夹爪命令和模式命令的节流逻辑：

- 夹爪只有在 `--include-gripper` 启用且距上次发送超过 `1/gripper_hz` 秒时才发送。如果 `--gripper-hz` 设为 0，则每个周期都发。
- `MotionCtrl_2` 模式命令只有在距上次发送超过 `--mode-command-period` 秒时才附带发送。

```
if args.execute:
    send_follower(follower, limited_target, gripper, args.speed, args.high_follow, send_mode)
    if send_mode:
        last_mode_command = now
    if send_gripper:
        last_gripper_command = now
```

只有 `--execute` 启用时，才真正向从臂发送命令。如果没有 `--execute`，脚本仍然执行读取、变换、限速的全部计算逻辑，只是不发送——这是 `dry-run` 模式。

```
elapsed = time.monotonic() - now
time.sleep(max(0.0, period - elapsed))
```

频率控制：计算本周期已经消耗的时间，sleep 剩余时间以保持 `--hz` 的标称频率。`max(0.0, ...)` 防止在循环超时 (`elapsed > period`) 时 sleep 负数。

这个简单的前馈 sleep 方案（而非 PID 调节的变步长控制）在循环耗时稳定的情况下可以达到很好的频率精度。如果 `--hz` 设得太高（例如 100 Hz, `period = 0.01s`），但单次循环耗时超过 0.01s，则实际频率会低于标称值——`elapsed > period` 导致 sleep 0 秒，循环以其最快速度运行。

7.3.9 退出时的安全处理

```
finally:
    if args.execute:
        print("Stopping follower command stream; sending standby mode.")
        try:
            follower.MotionCtrl_2(0x00, 0x01, 0)
        except Exception as exc:
            print(f"Failed to send standby command: {exc}", file=sys.stderr)
    leader.DisconnectPort()
    follower.DisconnectPort()
```

`finally` 块保证无论脚本如何退出（Ctrl-C、异常、正常结束），以下清理动作都会执行：

1. **发送 standby 命令：** `MotionCtrl_2(0x00, 0x01, 0)` — 模式= `0x00` (standby/失能)，使能= `0x01` (使能)，速度= `0`。这会让从臂的电机退出关节位置控制模式，进入一种"无力矩但保持位置"的 standby 状态。如果不发送此命令，从臂可能在断开 CAN 连接后仍然保持最后一次接收到的力矩命令，导致关节在重力作用下缓慢下坠。
2. **断开 CAN 连接：** `DisconnectPort()` 释放 `piper_sdk` 的后台线程和 `socketcan` 资源。

7.3.10 命令行参数完整参考

```
parser.add_argument("--leader", default="can0",
    help="CAN interface connected to the leader arm")
parser.add_argument("--follower", default="can1",
    help="CAN interface connected to the follower arm")
parser.add_argument("--source", choices=["auto", "feedback", "control"], default="auto",
    help="Leader joint data source")
parser.add_argument("--hz", type=float, default=10.0,
    help="Relay loop frequency")
parser.add_argument("--speed", type=int, default=10,
    help="Follower speed percentage sent to MotionCtrl_2")
parser.add_argument("--max-step-deg", type=float, default=1.0,
    help="Max follower joint step per cycle (degrees)")
parser.add_argument("--mode-command-period", type=float, default=1.0,
    help="Seconds between repeated MotionCtrl_2 commands; 0 disables")
parser.add_argument("--gripper-hz", type=float, default=5.0,
    help="Max gripper command frequency")
parser.add_argument("--print-period", type=float, default=1.0,
```

```

    help="Seconds between status prints; 0 disables")
parser.add_argument("--pause-file", default="",
    help="If this file exists, pause follower command sending")
parser.add_argument("--duration", type=float, default=0.0,
    help="Seconds to run; 0 means until Ctrl-C")
parser.add_argument("--signs", default="1,1,1,1,1,1",
    help="Per-joint sign mapping, comma-separated")
parser.add_argument("--offset-deg", default="0,0,0,0,0,0",
    help="Per-joint offset in degrees, comma-separated")
parser.add_argument("--include-gripper", action="store_true",
    help="Relay gripper target too")
parser.add_argument("--high-follow", action="store_true",
    help="Use MotionCtrl_2 is_mit_mode=0xAD")
parser.add_argument("--skip-follower-config", action="store_true",
    help="Do not send initial mode/enable commands")
parser.add_argument("--execute", action="store_true",
    help="Actually send commands to the follower")

```

参数	类型	默认值	说明
<code>--leader</code>	str	can0	主臂连接的 CAN 接口名
<code>--follower</code>	str	can1	从臂连接的 CAN 接口名
<code>--source</code>	choice	auto	主臂数据来源: <code>auto</code> (优先 <code>control</code>) / <code>feedback</code> / <code>control</code>
<code>--hz</code>	float	10.0	中继循环频率。实际频率受限于 USB-CAN 带宽和 Python 耗时
<code>--speed</code>	int	10	从臂速度百分比 (0-100)。通过 MotionCtrl_2 发送
<code>--max-step-deg</code>	float	1.0	单步最大角度变化量 (度)。核心安全参数
<code>--mode-command-period</code>	float	1.0	MotionCtrl_2 命令的重复发送间隔 (秒)。0 表示只发送一次
<code>--gripper-hz</code>	float	5.0	夹爪命令的最大发送频率。0 表示每周期都发送
<code>--print-period</code>	float	1.0	控制台状态打印间隔 (秒)。0 表示不打印
<code>--pause-file</code>	str	""	暂停文件的路径。空字符串表示不使用暂停机制
<code>--duration</code>	float	0.0	运行时长 (秒)。0 表示无限运行直到 Ctrl-C
<code>--signs</code>	str	"1,1,1,1,1,1"	每个关节的方向符号, 逗号分隔。1 = 同向, -1 = 反向
<code>--offset-deg</code>	str	"0,0,0,0,0,0"	每个关节的角度偏置 (度), 逗号分隔
<code>--include-gripper</code>	flag	False	是否同时中继夹爪动作
<code>--high-follow</code>	flag	False	是否启用 MIT 高跟随模式 (0xAD)

参数	类型	默认值	说明
<code>--skip-follower-config</code>	flag	False	是否跳过从臂初始配置（不发送模式命令和使能）
<code>--execute</code>	flag	False	是否真正发送命令。不传则为 dry-run 模式

7.4 record_with_pc_relay.sh 逐段解析

`record_with_pc_relay.sh` 是 PC 中继方案的操作入口。它是一个 bash 脚本，负责：

1. 加载配置（相机序列号、CAN 口、采集参数）
2. 以后台进程启动中继脚本
3. 以前台进程启动 `lerobot-record`
4. 在录制结束时清理中继进程

7.4.1 目录和 Conda 环境配置

```
PIPER_ROOT="${PIPER_ROOT:-/home/liyuqi/Documents/Piper}"
LEROBOT_DIR="${LEROBOT_DIR:-${PIPER_ROOT}/lerobot_piper-piper}"
CONDA_ENV="${CONDA_ENV:-lerobot}"
```

所有路径都通过 bash 参数扩展提供默认值，可以使用环境变量覆盖：

```
PIPER_ROOT=/workspace CONDA_ENV=my_env ./record_with_pc_relay.sh
```

脚本使用 `${VAR:-default}` 语法（带冒号），这意味着只有当变量未设置或为空时才使用默认值。

7.4.2 相机配置加载

```
if [[ -f "${LEROBOT_DIR}/camera_config_realsense.env" ]]; then
    source "${LEROBOT_DIR}/camera_config_realsense.env"
fi
```

`camera_config_realsense.env` 文件的内容示例：

```
export WRIST_REALSENSE_SERIAL="238222076529"
export GLOBAL_REALSENSE_SERIAL="142122071524"
export ROBOT_CAMERAS='{
    wrist: {type: intelrealsense, serial_number_or_name: "WRIST_SERIAL", width: 640, height: 480, fps: 15},
```

```
global: {type: intelrealsense, serial_number_or_name: "GLOBAL_SERIAL", width: 640, height: 480, fps: 15}
}'
```

注意：ROBOT_CAMERAS 中使用了占位符 WRIST_SERIAL 和 GLOBAL_SERIAL。这是设计上的考量——相机配置模板保持通用性，实际序列号在脚本中通过字符串替换注入。

序列号替换逻辑：

```
if [[ -n "${WRIST_REALSENSE_SERIAL:-}" ]]; then
    ROBOT_CAMERAS="${ROBOT_CAMERAS//WRIST_SERIAL/${WRIST_REALSENSE_SERIAL}}"
fi
if [[ -n "${GLOBAL_REALSENSE_SERIAL:-}" ]]; then
    ROBOT_CAMERAS="${ROBOT_CAMERAS//GLOBAL_SERIAL/${GLOBAL_REALSENSE_SERIAL}}"
fi
```

`${ROBOT_CAMERAS//WRIST_SERIAL/...}` 是 bash 的全局替换语法。替换后，配置变为：

```
wrist: {type: intelrealsense, serial_number_or_name: "238222076529", width: 640, height: 480, fps: 15}
global: {type: intelrealsense, serial_number_or_name: "142122071524", width: 640, height: 480, fps: 15}
```

如果替换后仍然存在占位符（说明相机配置文件没有正确提供序列号），脚本会检测并报错退出：

```
if [[ "${ROBOT_CAMERAS}" == *"WRIST_SERIAL"* || "${ROBOT_CAMERAS}" == *"GLOBAL_SERIAL"* ]]; then
    echo "[record] ERROR: ROBOT_CAMERAS still contains placeholder serials:" >&2
    exit 1
fi
```

7.4.3 FPS 对齐

```
ROBOT_CAMERAS="${ROBOT_CAMERAS}" DATASET_FPS="${DATASET_FPS}" python - <<'PY'
import json, os, yaml

camera_cfg = yaml.safe_load(os.environ["ROBOT_CAMERAS"])
dataset_fps = int(os.environ["DATASET_FPS"])
if isinstance(camera_cfg, dict):
    for cfg in camera_cfg.values():
        if isinstance(cfg, dict) and "fps" in cfg:
            cfg["fps"] = dataset_fps
print(json.dumps(camera_cfg))
PY
)"
```

这段内联 Python 脚本将相机配置中的所有 fps 字段强制对齐到 DATASET_FPS（默认 30）。为什么要这样做？

在 LeRobot 的数据集中，所有相机必须以**相同的帧率**录制。如果腕部相机以 15 fps 运行而全局相机以 30 fps 运行，数据对齐会出现问题（某一帧的关节状态对应哪个相机图像？）。通过将所有相机 fps 统一到 DATASET_FPS，保证每个时间戳下所有数据（关节状态 + 所有相机帧）是一一对应的。

随后还有一个验证步骤：

```
mismatches = {
    name: cfg.get("fps")
    for name, cfg in camera_cfg.items()
    if isinstance(cfg, dict) and cfg.get("fps") is not None and int(cfg.get("fps")) != dataset_fps
}
if mismatches:
    raise SystemExit(f"ERROR: camera fps must match DATASET_FPS. mismatches={mismatches}")
```

这确保了（在替换之后）没有任何相机 fps 与目标 fps 不一致。

7.4.4 数据集路径处理

```
DATASET_BASE_DIR="${DATASET_BASE_DIR:-${DATASET_ROOT:-${PIPER_ROOT}/datasets}}"
DATASET_NAME="${DATASET_NAME:-piper_pc_relay_smoke}"
HF_USER="${HF_USER:-${HF_USER_DEFAULT:-local}}"
DATASET_REPO_ID="${DATASET_REPO_ID:-${HF_USER}/${DATASET_NAME}}"
DATASET_OUTPUT_ROOT="${DATASET_OUTPUT_ROOT:-${DATASET_BASE_DIR}/${DATASET_REPO_ID}}"

if [[ -e "${DATASET_OUTPUT_ROOT}" && "${AUTO_SUFFIX_DATASET_ROOT}" == "1" ]]; then
    DATASET_OUTPUT_ROOT="${DATASET_OUTPUT_ROOT}_${date +%Y%m%d_%H%M%S}"
fi
```

数据集默认路径为 `${PIPER_ROOT}/datasets/${HF_USER}/${DATASET_NAME}`。如果路径已存在且 `AUTO_SUFFIX_DATASET_ROOT=1`（默认启用），自动追加时间戳后缀（如 `_20260622_143025`），防止意外覆盖已有数据。

如果你要手动指定数据集目录，设置 `DATASET_OUTPUT_ROOT` 即可：

```
DATASET_OUTPUT_ROOT=/data/my_experiment ./record_with_pc_relay.sh
```

7.4.5 后台启动中继进程

```
RELAY_PAUSE_FILE="${RELAY_PAUSE_FILE:-/tmp/piper_pc_relay.pause}"

relay_cmd=(
    "${PIPER_ROOT}/piper_sdk/piper/bin/python"
    "${LEROBOT_DIR}/scripts/piper_pc_relay_teleop.py"
    --leader "${LEADER_CAN}"
    --follower "${FOLLOWER_CAN}"
    --source "${RELAY_SOURCE}"
```

```

--hz "${RELAY_HZ}"
--speed "${RELAY_SPEED}"
--max-step-deg "${RELAY_MAX_STEP_DEG}"
--mode-command-period "${RELAY_MODE_COMMAND_PERIOD}"
--gripper-hz "${RELAY_GRIPPER_HZ}"
--print-period "${RELAY_PRINT_PERIOD}"
--pause-file "${RELAY_PAUSE_FILE}"
--execute
)

```

关键默认值（在脚本顶部定义）：

变量	默认值	含义
RELAY_HZ	20	中继频率 20 Hz
RELAY_SPEED	100	从臂速度百分比 100%
RELAY_MAX_STEP_DEG	10	单步最大 10 度（相当于 200 度/秒）
RELAY_SOURCE	control	使用控制位置数据
RELAY_INCLUDE_GRIPPER	1	中继夹爪
RELAY_HIGH_FOLLOW	1	启用 MIT 高跟随模式
RELAY_MODE_COMMAND_PERIOD	1.0	每秒重发一次 MotionCtrl_2
RELAY_GRIPPER_HZ	5	夹爪命令 5 Hz
RELAY_PRINT_PERIOD	0	不打印状态（设为 1 或更大可以开启调试打印）

注意：`record_with_pc_relay.sh` 中使用的是项目自带的 **Python** 解释器（`${PIPER_ROOT}/piper_sdk/piper/bin/python`），而不是系统 Python 或 Conda 环境中的 Python。这是因为 `piper_sdk` 是一个编译好的 C++ 扩展包，对 Python 版本和 ABI 有严格依赖，使用自带的解释器可以避免版本冲突。

```

trap cleanup EXIT INT TERM
rm -f "${RELAY_PAUSE_FILE}" 2>/dev/null || true

echo "[relay] ${relay_cmd[*]}"
"${relay_cmd[@]}" &
RELAY_PID="$!"
sleep 2

```

`trap cleanup EXIT INT TERM` 注册了退出处理器，确保脚本被 Ctrl-C 或 kill 时，`cleanup` 函数一定会执行（删除暂停文件、kill 中继进程）。

sleep 2 给中继进程 2 秒的启动时间（piper_sdk 需要建立 CAN 连接和启动后台线程），然后才启动 lerobot-record。

7.4.6 复位协调机制

这是 PC 中继方案与 lerobot-record 深度集成的关键部分。

```
if [[ "${AUTO_RESET_ARMS}" == "1" || "${AUTO_RESET_ARMS}" == "true" ]]; then
    reset_ports=("${FOLLOWER_CAN}")
    master_home_args=()
    if [[ "${AUTO_RESET_LEADER}" == "1" || "${AUTO_RESET_LEADER}" == "true" ]]; then
        reset_ports=("${LEADER_CAN}" "${FOLLOWER_CAN}")
        master_home_args=(-master-home-ports "${LEADER_CAN}")
    fi
    export LEROBOT_RESET_COMMAND="${PIPER_ROOT}/piper_sdk/piper/bin/python ${LEROBOT_DIR}/scripts/
piper_reset_to_initial.py --ports ${reset_ports[*]} ${master_home_args[*]} --target-deg $
{RESET_TARGET_DEG} --speed ${RESET_SPEED} --hz ${RESET_HZ} --duration ${RESET_DURATION} --high-follow"
    export LEROBOT_RELAY_PAUSE_FILE="${RELAY_PAUSE_FILE}"
fi
```

两个关键的环境变量被导出： - LEROBOT_RESET_COMMAND：复位命令的完整 shell 命令字符串。在每个 episode 结束后，lerobot-record 会执行此命令。 - LEROBOT_RELAY_PAUSE_FILE：暂停文件的路径。lerobot-record 在执行复位命令前 touch 此文件，执行后删除。

lerobot-record 内部的处理逻辑（来自 src/lerobot/record.py）：

```
def run_reset_command_if_configured() -> None:
    command = os.environ.get("LEROBOT_RESET_COMMAND")
    if not command:
        return
    pause_file = os.environ.get("LEROBOT_RELAY_PAUSE_FILE")
    try:
        if pause_file:
            Path(pause_file).parent.mkdir(parents=True, exist_ok=True)
            Path(pause_file).touch() # ① 创建暂停文件
            subprocess.run(shlex.split(command), check=False) # ② 执行复位
    finally:
        if pause_file:
            Path(pause_file).unlink(missing_ok=True) # ③ 删除暂停文件
```

这个三段式协调保证： 1. **创建暂停文件** → 中继进程在下次循环中检测到文件存在 → 停止向从臂发送命令 2. **执行复位** → piper_reset_to_initial.py 将双臂移动到初始位置。此时从臂不受中继进程控制（因为暂停已生效），而是由复位脚本直接控制 3. **删除暂停文件** → 中继进程检测到文件消失 → 恢复向从臂发送命令

如果不使用 `pause-file` 机制，复位脚本和中继脚本会**同时**向从臂发送 CAN 命令——两个进程竞争同一个 CAN 总线，导致从臂的行为不可预测。

7.4.7 启动 `lerobot-record`

```
lerobot-record \
  --robot.type=piper_follower \
  --robot.port="${FOLLOWER_CAN}" \
  --robot.cameras="${ROBOT_CAMERAS}" \
  --robot.id=pc_relay_follower \
  --dataset.root="${DATASET_OUTPUT_ROOT}" \
  --dataset.repo_id="${DATASET_REPO_ID}" \
  --dataset.fps="${DATASET_FPS}" \
  --dataset.num_episodes="${NUM_EPISODES}" \
  --dataset.episode_time_s="${EPISODE_TIME_S}" \
  --dataset.reset_time_s="${RESET_TIME_S}" \
  --dataset.single_task="${TASK}" \
  --dataset.push_to_hub="${PUSH_TO_HUB}" \
  --display_data="${DISPLAY_DATA}" \
  --play_sounds="${PLAY_SOUNDS}" \
  --manual_step="${MANUAL_STEP}"
```

注意这里**没有** `--teleoperator.type` 参数。在标准的 `LeRobot` 录制备置中，通常会指定一个 `Teleoperator` 类型（如 `--teleoperator.type=piper_leader`），让 `lerobot-record` 同时管理主臂和从臂。但在 PC 中继方案中：

- 主臂的数据通路由独立的 `piper_pc_relay_teleop.py` 进程管理
- `lerobot-record` 只需要连接从臂（`--robot.type=piper_follower --robot.port=can1`）来采集观测数据
- 数据集中的 `action` 字段直接使用从臂的当前关节状态（因为从臂正在跟随主臂运动，它的关节状态反映了主臂的动作）

默认采集参数：

参数	默认值	含义
<code>NUM_EPISODES</code>	<code>5</code>	采集 5 个 episode
<code>EPISODE_TIME_S</code>	<code>20</code>	每个 episode 录制 20 秒
<code>RESET_TIME_S</code>	<code>10</code>	每个 episode 之间 10 秒复位时间
<code>DATASET_FPS</code>	<code>30</code>	数据集帧率 30 fps
<code>MANUAL_STEP</code>	<code>true</code>	手动步进模式（按回车进入下一 episode）

7.4.8 清理函数

```
cleanup() {
  rm -f "${RELAY_PAUSE_FILE}" 2>/dev/null || true
  if [[ -n "${RELAY_PID:-}" ]] && kill -0 "${RELAY_PID}" 2>/dev/null; then
    kill "${RELAY_PID}" 2>/dev/null || true
    wait "${RELAY_PID}" 2>/dev/null || true
  fi
}
```

清理函数被 `trap cleanup EXIT INT TERM` 绑定到脚本退出事件，确保无论脚本如何退出：1. 暂停文件被删除（防止下一次运行时中继误认为暂停）2. 中继进程被终止（`kill` + `wait` 确保进程完全退出）3. 中继进程的 `finally` 块会自动发送 `standby` 命令给从臂

7.5 PC 中继的完整工作流时序

以下时序图展示了一次完整的录制循环（从一个 episode 开始到下一个 episode 开始）中，人、主臂、中继进程、从臂和 `lerobot-record` 之间的交互。





关键时序说明

1. **录制阶段** (episode N)：人拖拽主臂执行任务，中继进程以 20 Hz 将主臂位置中继到从臂（经过变换和限速），从臂跟随主臂实时运动。lerobot-record 以 30 fps 记录从臂的关节状态和相机图像。此时 `pause_file` 不存在，中继进程正常运行。
2. **episode 结束时刻**：lerobot-record 的录制计时器到达 `episode_time_s`（默认 20 秒）。系统播放 "Reset the environment" 语音提示。`run_reset_command_if_configured()` 被调用。
3. **暂停中继**（关键步骤）：函数首先 `touch /tmp/piper_pc_relay.pause`。在中继进程的下一个循环周期内（最多 $1/20 = 0.05$ 秒），`os.path.exists(pause_file)` 返回 True，中继进程停止向从臂发送命令。**从臂停在最后接收到的位置，不再跟随主臂运动。**
4. **执行复位**：`piper_reset_to_initial.py` 脚本运行，连接 `can1`（从臂），将关节移动到目标零点位（默认 0,0,0,0,0,0 度）。如果 `AUTO_RESET_LEADER=1`，也同时复位主臂。
5. **恢复中继**：复位脚本执行完毕后，`finally` 块删除 `pause_file`。中继进程在下一个循环周期检测到文件消失，恢复向从臂发送命令。人重新开始拖拽主臂，进入下一 episode。

这个协调机制的优雅之处在于： - 不需要进程间通信（IPC），文件系统充当信号量 - 中继进程不需要知道"为什么"暂停——它只检查文件存在性 - 暂停和恢复的延迟上限为 `1/RELAY_HZ` 秒（默认 0.05 秒），足够快 - 如果复位脚本执行失败（返回非零退出码），`finally` 块仍然会删除 `pause_file`，中继不会永久卡在暂停状态

7.6 安全设计

PC 中继方案从多个层面确保了操作安全。理解这些安全机制不仅有助于避免事故，也有助于在遇到异常行为时快速定位问题。

7.6.1 速率限制（增量限幅）

这是最核心的安全层。`rate_limit()` 函数基于从臂**当前实际反馈位置**，对每一步的目标增量进行 clamp：

```
output[i] = clamp(target[i], current[i] - max_step, current[i] + max_step)
```

关键性质： - **增量限幅，不是绝对位置限幅**：速率限制不关心目标位置的绝对值是多少，只关心"这一步的变化量"。即使主臂报告了一个跳跃了 90 度的非法值，从臂也只会移动 `max_step_deg` 度（默认 1 度，在采集脚本中为 10 度）。 - **基于反馈位置，不自累积误差**：每一周期的基线是 `feedback_sample()` 读取的从臂实际位置。如果上一周期从臂因为某种原因没有到达目标（例如遇到障碍物），本周期会从"它实际在的位置"开始计算增量，不会在"未到达的旧目标"上叠加新的增量。 - **独立于硬件保护**：速率限制是纯软件层面的保护，与电机驱动器的速度环/电流环保护正交。两者同时生效，形成纵深防御。

7.6.2 关节限位 (bounded_joints)

在速率限制之前，`bounded_joints()` 将变换后的目标位置 clamp 到 `JOINT_LIMITS_MDEG` 定义的物理范围内。这确保了即使 `--signs` 和 `--offset-deg` 的配置错误导致变换后的目标超出了物理极限，从臂也不会收到越界命令。

7.6.3 暂停文件机制

暂停文件提供了一种**带外 (out-of-band)** 的紧急停止能力。即使中继进程正常运行，只要在文件系统中创建 `pause_file` (`touch /tmp/piper_pc_relay.pause`)，中继就会在 0.05 秒内停止发送命令。你可以通过以下方式手动触发暂停：

```
# 手动暂停中继
touch /tmp/piper_pc_relay.pause

# 手动恢复中继
rm /tmp/piper_pc_relay.pause
```

这在以下场景中非常有用： - 从臂出现异常抖动，需要立即停止 - 工作区有障碍物，需要先移开再继续操作 - 调试关节变换参数，需要观察从臂停在某位置的行为

7.6.4 进程隔离

中继进程和录制进程是独立的操作系统进程。这种隔离带来了以下安全属性：

故障场景	影响	安全性
lerobot-record 崩溃（磁盘满）	当前 episode 数据丢失，但中继继续工作	从臂仍然受控
中继进程崩溃（Python 异常）	从臂停在最后位置，lerobot-record 继续运行（见不到新数据）	从臂不会意外运动
两个进程同时崩溃	从臂停在最后位置（USB-CAN 保持最后输出）	从臂静止

故障场景	影响	安全性
操作者按 Ctrl-C	bash trap 触发 → kill 中继 → 中继 finally 发送 standby → 从臂进入无力矩保持	安全停止

7.6.5 Dry-run 模式

中继脚本的默认行为是**不发送任何命令**。`--execute` 必须显式传入。这意味着：

- 第一次启动时，你可以在 `dry-run` 模式下观察控制台输出，确认关节变换和速率限制的逻辑正确
- 如果你忘记了正确的参数组合，`dry-run` 模式给了你试错的空间
- 在演示或培训场景中，可以先 `dry-run` 一遍，确认所有参数无误后再加 `--execute`

7.6.6 退出时的 Standby 命令

中继进程退出时（无论是正常退出还是被信号杀死），`finally` 块会向从臂发送 `MotionCtrl_2(0x00, 0x01, 0)` ——进入 `standby` 模式但不失能。这比直接断开 CAN 连接更安全：

- 如果直接断开 CAN，从臂可能保持上一次的力矩命令，在重力作用下关节缓慢移动
- `Standby` 模式让驱动器切换到位置保持状态（无力矩输出但保持当前位置），关节不会下坠
- 如果你需要完全失能力矩（例如检修），可以在 `standby` 之后再手动发送失能命令

7.7 常见问题和调试

7.7.1 中继不工作：从臂完全不动

可能原因及排查步骤：

1. 检查 CAN 口是否 UP

```
ip link show can0
ip link show can1
```

输出中应当看到 `state UP`。如果状态是 `DOWN`：

```
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up
sudo ip link set can1 type can bitrate 1000000
sudo ip link set can1 up
```


PiPER 的 CAN 波特率是 1 Mbps (1,000,000 bps) 。注意不是 500k 或 125k。

2. 检查是否传了 `--execute`

在没有 `--execute` 时，脚本是 `dry-run` 模式。检查控制台输出：第一行会打印 `Mode: DRY-RUN: no` `follower commands will be sent` 或 `Mode: EXECUTE: follower will move`。

3. 检查主臂是否上电且 CAN 线连接

在 `dry-run` 模式下观察控制台输出。如果看到 `No leader ... joint stream on can0`，说明主臂的 CAN 数据没有到达 PC。检查主臂电源和 USB-CAN 适配器的连接。

4. 检查 USB-CAN 适配器的指示灯

- 绿灯常亮：适配器已上电，`socketcan` 已绑定
- 绿灯闪烁：CAN 总线上有数据帧传输
- 红灯：总线错误（可能是终端电阻缺失或波特率不匹配）

若红灯常亮，请检查 CAN 总线的 120 欧终端电阻是否正确安装。

7.7.2 从臂抖动

可能原因和解决方案：

1. `--max-step-deg` 过大

如果 `max-step-deg` 设置过高（例如 50 度/步），从臂会在目标位置附近来回超调。降低此值：

```
# 在 record_with_pc_relay.sh 调用时覆盖
RELAY_MAX_STEP_DEG=3 ./record_with_pc_relay.sh
```

推荐从 3 度/步开始测试，逐步增大直到找到既不抖动又能跟上的最佳值。

2. `--hz` 过高

如果中继频率设置过高（例如 100 Hz），USB-CAN 适配器可能无法以这个频率稳定发送命令。尝试降低：

```
RELAY_HZ=10 ./record_with_pc_relay.sh
```

3. 主臂传感器噪声

尝试切换 `--source`：

```
# 如果当前用的是 feedback, 尝试 control
RELAY_SOURCE=control ./record_with_pc_relay.sh

# 如果当前用的是 control, 尝试 feedback
RELAY_SOURCE=feedback ./record_with_pc_relay.sh
```

通常 `control` 源的数据比 `feedback` 源更平滑（经过了驱动器内部滤波）。

4. CAN 总线电磁干扰

检查 CAN 线是否靠近大功率设备（电机驱动器、开关电源）。尝试重新走线，使 CAN 线远离干扰源。确保 CAN 线使用双绞线，且屏蔽层正确接地。

7.7.3 主臂和从臂运动方向不一致

问题：人把主臂往某个方向拖动，但从臂往相反方向运动。

原因：主臂和从臂的某个（或某几个）关节安装方向相反。

解决方案：修改 `--signs` 参数。例如关节 2 方向反转：

```
# 直接在命令行测试 (dry-run 模式)
python scripts/piper_pc_relay_teleop.py --signs "1,-1,1,1,1,1" --print-period 1

# 确认正确后, 在 record_with_pc_relay.sh 中设置环境变量 (当前脚本未暴露此参数,
# 需要手动修改 relay_cmd 数组或在脚本中添加环境变量支持)
```

如何确定哪个关节需要反转：1. 将主臂的**单个关节**从零位向正方向缓慢移动（例如只动关节 1，其他关节保持不动）2. 观察从臂对应关节的运动方向 3. 如果方向相反，将该关节的 `sign` 改为 `-1` 4. 重复以上步骤，逐个关节验证

7.7.4 从臂零点与主臂不对齐

问题：主臂和从臂在相同姿态下，从臂的关节角度有明显偏置。

解决方案：使用 `--offset-deg` 参数：

```
# 例如关节 3 需要偏移 -15 度
python scripts/piper_pc_relay_teleop.py --offset-deg "0,0,-15,0,0,0" --print-period 1
```

如何确定偏置值：

1. 将主臂手动拖到零位姿态（或某个已知姿态）
2. 读取从臂的当前关节角度（dry-run 模式的控制台会打印 `follower deg`）

3. 计算差值: `offset = master_pos - follower_pos`

4. 将差值作为 `--offset-deg` 的值

7.7.5 频率跟不上（实际频率低于设定频率）

现象：控制台打印的 hz 信息显示实际中继频率显著低于 `--hz` 的设定值。

原因： - USB-CAN 适配器的写入带宽不足（廉价适配器可能无法做高频写入） - Python GIL 导致 piper_sdk 的后台线程得不到足够的 CPU 时间 - 系统负载过高（其他进程占用 CPU）

解决方案：

1. **降低中继频率：** `bash RELAY_HZ=10 ./record_with_pc_relay.sh` 10-20 Hz 对大多数操作任务来说已经足够。人类操作者的手部动作频率通常在 2-5 Hz 范围内。
2. **关闭不必要的后台进程：** 特别是浏览器、IDE 等占用 CPU 和内存的应用。
3. **检查 USB-CAN 适配器：** 使用 `candump` 检查总线负载: `bash candump can0 #` 观察主臂的上报频率 应当看到约 1000 帧/秒 ($1\text{ kHz} \times 3$ 个 CAN ID)。如果实际频率远低于此，可能是 USB-CAN 适配器硬件性能不足。

7.7.6 录制数据中从臂动作不连续

现象：录制的数据集中，从臂的关节轨迹出现跳变或不连续。

可能原因：

1. **复位阶段数据未过滤：** 在 reset 阶段，从臂被 `piper_reset_to_initial.py` 驱动回零位，这个运动会被 lerobot-record 记录（如果 reset 阶段也在录制）。这是预期行为——reset 阶段的数据在训练时通常会被跳过。
2. **中继频率与录制频率不一致：** 中继以 20 Hz 发送命令，录制以 30 fps 采样。在两次中继命令之间，从臂可能已经到达了目标位置，录制的相邻两帧关节值可能相同。这也是预期行为——训练时模型学到的是"动作可以重复"。
3. **CAN 通信丢帧：** 使用 `candump` 检查从臂 CAN 总线是否有错误帧: `bash cat /sys/class/net/can1/statistics/rx_errors cat /sys/class/net/can1/statistics/tx_errors` 如果错误计数器持续增长，说明 CAN 物理层有问题（终端电阻、线缆质量、接地等）。

7.7.7 夹爪不跟随主臂

问题：主臂夹爪的开合不影响从臂夹爪。

可能原因： 1. 中继脚本没有加 `--include-gripper` 参数。`record_with_pc_relay.sh` 默认设置了 `RELAY_INCLUDE_GRIPPER=1`，如果手动运行中继脚本需要显式添加。 2. 从臂夹爪的通信协议与主臂夹爪不同（主臂用 HTDW-5047 的夹爪通道，从臂用 AGILEX-S 的夹爪通道）。这是硬件层面的已知差异——PC 中继方案暂不支持主臂夹爪到从臂夹爪的转发。如需夹爪控制，请使用 `lerobot-record` 中配置的夹爪动作直接控制从臂夹爪。

7.8 操作检查清单

在每次开始数据采集前，建议按以下清单逐项确认：

- ☐ 1. 硬件检查
 - ☐ 主臂和从臂的电源线已连接并上电
 - ☐ USB-CAN A 连接 can0（主臂），USB-CAN B 连接 can1（从臂）
 - ☐ CAN 终端电阻已安装（120 欧）
 - ☐ 相机 USB 线已连接（腕部和全局）
 - ☐ 机械臂工作区内无障碍物
- ☐ 2. CAN 总线检查
 - ☐ `ip link show can0 → state UP`
 - ☐ `ip link show can1 → state UP`
 - ☐ can0 波特率 = 1,000,000
 - ☐ can1 波特率 = 1,000,000
- ☐ 3. 相机检查
 - ☐ `lerobot-find-cameras realsense` 返回两个相机
 - ☐ `camera_config_realsense.env` 中的序列号与实际相机匹配
- ☐ 4. 中继脚本 dry-run 测试
 - ☐ `python scripts/piper_pc_relay_teleop.py --print-period 1`
 - ☐ 控制台打印 `leader deg` 数值随拖拽主臂实时变化
 - ☐ 控制台打印 `follower deg` 数值与从臂实际姿态一致
 - ☐ 主臂和从臂运动方向一致（如不一致，调整 `--signs`）
 - ☐ 从臂零点与主臂对齐（如有偏置，调整 `--offset-deg`）
- ☐ 5. 采集配置确认
 - ☐ `DATASET_NAME` 已设置为有意义的数据集名称
 - ☐ `NUM_EPISODES` 和 `EPISODE_TIME_S` 满足需求
 - ☐ `TASK` 描述准确描述了当前采集的任务
- ☐ 6. 开始采集
 - ☐ `./record_with_pc_relay.sh`
 - ☐ 等待 "Press Enter to continue..." 提示，按 Enter 开始第一个 episode
 - ☐ 操作过程中如遇异常，Ctrl-C 安全退出

7.9 本章小结

PC 中继是 PiPER 双臂系统中连接"人的操作"和"机器的执行"的核心桥梁。本章涵盖了以下关键知识点：

1. **为什么需要 PC 中继**：主臂和从臂在不同 CAN 总线上，硬件主从模式无法满足关节变换、速率限制和采集协调的需求。LeRobot 内置的 teleoperate 机制假设双臂同型号、同总线，不适用于 PiPER。
2. **架构设计**：中继进程和录制进程分离，通过文件系统（pause-file）协调。进程隔离提供了故障隔离，dry-run 模式提供了安全试错空间。
3. **数据处理流水线**：读取主臂 → 关节变换 ($\text{sign} \times \text{val} + \text{offset}$) → 限位 (clamp to limits) → 速率限制 (clamp delta) → 发送从臂。三步顺序不可颠倒。
4. **录制编排**：record_with_pc_relay.sh 负责加载相机配置、启动后台中继、配置复位命令、启动前台录制、退出时清理。
5. **安全机制**：速率限制（增量限幅）、关节限位、暂停文件、进程隔离、dry-run 默认模式、退出 standby 命令——六层安全保护形成纵深防御。
6. **调试方法**：通过 dry-run 模式的 console 输出来验证关节变换和速率限制的正确性，通过 candump 和 ip link show 来诊断 CAN 通信问题。

掌握了 PC 中继方案后，你就掌握了 PiPER 双臂系统日常数据采集的完整工作流。下一章将进入**策略训练**——如何使用采集到的数据训练行为克隆模型。

源码参考： - scripts/piper_pc_relay_teleop.py (309 行) — PC 中继引擎 - record_with_pc_relay.sh (228 行) — 采集编排脚本 - src/lerobot/record.py — run_reset_command_if_configured() 函数（第 146-159 行） - scripts/piper_reset_to_initial.py — 双臂复位脚本，由 LEROBOT_RESET_COMMAND 调用

第八章 数据录制：从示教到数据集

本章全面讲解 PiPER 机械臂在 LeRobot 框架中的数据录制管线。从 `lerobot-record` CLI 入口点出发，逐层深入录制循环、时间控制、双相机配置、语音提示系统和数据集输出结构，最后覆盖数据验证（Replay）和常见故障排查。阅读本章后，你将能够独立完成从硬件接线到产出可训练数据集的全流程操作。

本章内容覆盖以下源文件：

文件	行数	角色
<code>src/lerobot/record.py</code>	596	录制主逻辑：CLI 入口、录制循环、手动步进、复位命令
<code>src/lerobot/utils/utils.py</code>	503	语音提示系统：TTS 引擎链、提示音生成、预生成播放
<code>src/lerobot/cameras/realsense/camera_realsense.py</code>	556	RealSense 相机驱动：同步/异步读取、超时控制
<code>src/lerobot/replay.py</code>	134	数据回放：逐帧重放已录制的动作
<code>camera_config_realsense.env</code>	16	双相机环境变量配置模板

8.1 lerobot-record 完整流程

8.1.1 入口点与命令结构

`lerobot-record` 是一个通过 `pyproject.toml` 注册的 CLI 命令，其执行入口为 `src/lerobot/record.py:main()`。`main()` 函数极为简洁——它只做一件事：调用 `record()`。

```
# src/lerobot/record.py 第 591-592 行
def main():
    record()
```

`record()` 函数被 `@parser.wrap()` 装饰器包裹，这个装饰器负责三件事：

- 解析命令行参数：**将 `--robot.type=piper_follower` 这类 `key=value` 参数解析为嵌套的 `dataclass` 实例。

2. 自动生成帮助文档：基于 dataclass 的字段类型和默认值自动构建 `--help` 输出。
3. 类型校验与转换：将字符串参数自动转换为对应的 Python 类型（如 `"30"` $\rightarrow$ `30`）。

8.1.2 命令行参数结构：RecordConfig $\rightarrow$ DatasetRecordConfig

录制命令的参数体系由三层 dataclass 嵌套组成：

```
RecordConfig
├── robot: RobotConfig          # 从臂配置
│   ├── type: str = "piper_follower" # 机器人类型
│   ├── port: str = "can0"         # CAN 接口
│   ├── cameras: dict             # 相机配置
│   └── ...
├── dataset: DatasetRecordConfig # 数据集配置
│   ├── repo_id: str              # 数据集标识符（如 "local/my_task"）
│   ├── single_task: str          # 任务描述
│   ├── root: str | Path | None   # 数据集存储根目录
│   ├── fps: int = 30             # 录制帧率
│   ├── episode_time_s: int | float = 60 # 每个 episode 的录制时长（秒）
│   ├── reset_time_s: int | float = 60 # 每个 episode 之间的复位等待时间（秒）
│   ├── num_episodes: int = 50    # 录制 episode 数量
│   ├── video: bool = True        # 是否将图像编码为视频
│   ├── push_to_hub: bool = True  # 录制完成后是否上传到 Hugging Face Hub
│   ├── private: bool = False     # 上传时是否设为私有仓库
│   ├── tags: list[str] | None    # Hub 标签
│   ├── num_image_writer_processes: int = 0 # 图像写入子进程数
│   ├── num_image_writer_threads_per_camera: int = 4 # 每相机的图像写入线程数
│   └── video_encoding_batch_size: int = 1 # 视频批量编码大小
├── teleop: TeleoperatorConfig | None # 主臂配置（可选）
├── policy: PreTrainedConfig | None  # 策略配置（可选，替代遥操作）
├── display_data: bool = False       # 是否使用 rerun 实时可视化
├── play_sounds: bool = True         # 是否启用语音/提示音播报
├── manual_step: bool = False        # 是否启用手动步进模式
└── resume: bool = False             # 是否在已有数据集上续录
```

关键参数详解：

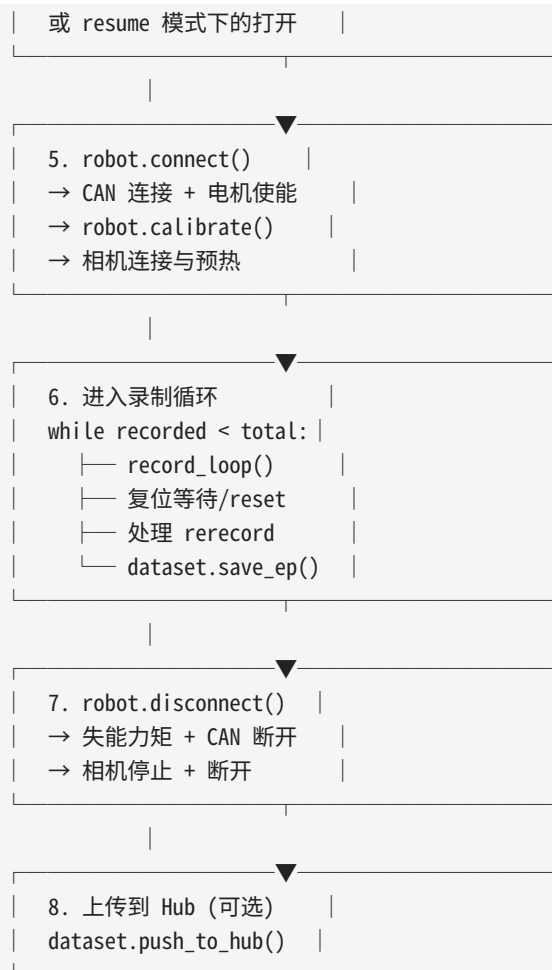
参数路径	类型	默认值	说明
<code>--robot.type</code>	str	—	从臂类型，PiPER 项目始终用 <code>piper_follower</code>
<code>--robot.port</code>	str	—	从臂 CAN 接口，如 <code>can0</code> 或 <code>can1</code>
<code>--robot.cameras</code>	dict	—	相机配置字典，支持 <code>opencv</code> 和 <code>intelrealsense</code> 两种类型
<code>--robot.id</code>	str	—	从臂标识名（如 <code>black</code> ），用于区分多台机械臂
<code>--teleoperator.type</code>	str	—	主臂类型，PiPER 项目用 <code>piper_leader</code>

参数路径	类型	默认值	说明
<code>--teleoperator.port</code>	str	—	主臂 CAN 接口
<code>--dataset.repo_id</code>	str	—	数据集路径标识, 如 <code>local/piper_pick_cube</code>
<code>--dataset.num_episodes</code>	int	50	要录制的 episode 总数
<code>--dataset.fps</code>	int	30	目标录制帧率 (帧/秒)
<code>--dataset.episode_time_s</code>	int	60	每个 episode 的录制时长
<code>--dataset.single_task</code>	str	—	任务文本描述 (必填)
<code>--control.time_s</code>	—	—	注意: 实际参数名为 <code>--dataset.episode_time_s</code> , 旧版文档可能引用 <code>control.time_s</code>
<code>--display_data</code>	bool	false	启用 rerun 可视化面板
<code>--manual_step</code>	bool	false	启用手动步进: 每 episode 前后等待 Enter 确认
<code>--resume</code>	bool	false	断点续录模式

8.1.3 完整执行流程

`record()` 函数的执行可划分为 8 个阶段:





阶段 2 — 创建 Robot 实例:

```
# record.py 第 443 行
robot = make_robot_from_config(cfg.robot)
```

`make_robot_from_config` 根据 `cfg.robot.type` 的值 (此处为 `"piper_follower"`) 从注册表中查找对应的 `dataclass` 并实例化。`PiperFollowerConfig` 在此过程中被构造为包含 CAN 端口、相机列表、关节限位等信息。

阶段 3 — 创建 Teleoperator: 在本项目的 PC 中继方案中, 主臂 (PiperLeader) 通过 CAN 直接向从臂 (PiperFollower) 发送目标位置, PC 端不需要再创建 `teleoperator` 对象来转发动作, 因此此处 `teleop = None`。不过代码中保留了 `teleoperator` 的完整加载路径, 以便其他机器人类型使用。

阶段 4 — 创建数据集:

```
# record.py 第 480-490 行
dataset = LeRobotDataset.create(
```

```

        cfg.dataset.repo_id,
        cfg.dataset.fps,
        root=cfg.dataset.root,
        robot_type=robot.name,
        features=dataset_features,
        use_videos=cfg.dataset.video,
        ...
    )

```

`LeRobotDataset.create()` 在磁盘上创建以下结构： - `meta/` 目录，包含 `info.json`（元数据）和 `stats.json`（在后续训练步骤中填入） - `data/` 目录，包含分块（chunk）的 `parquet` 文件 - `videos/` 目录，按相机名称分为子目录

数据集的 `features`（特征 schema）由两部分合并而来： 1. **动作特征**（`robot.action_features`）：从臂关节的维度信息 2. **观测特征**（`robot.observation_features`）：关节角度 + 相机图像信息

阶段 5 — `robot.connect()`：这一步骤执行以下子操作： 1. 打开 CAN 接口，与从臂的 7 个电机建立通信 2. 切换电机到使能（enable）状态，施加力矩 3. 执行 `calibrate()`：将电机驱动到 parking 位置（初始位姿）。此过程约需 2-3 秒 4. 连接所有配置的相机（如两个 RealSense），启动预热读取

预热期间，相机被连续读取若干帧以确保自动曝光、白平衡等参数稳定。具体代码：

```

# camera_realsense.py 第 184-191 行
if warmup:
    time.sleep(1) # RealSense 硬件需要短时间稳定
    start_time = time.time()
    while time.time() - start_time < self.warmup_s:
        self.read()
        time.sleep(0.1)

```

8.2 录制循环详解

8.2.1 `record_loop()` 函数完整流程

`record_loop()` 是数据录制的核心函数，它被调用两次： - **录制阶段**：以 `episode_time_s` 为时长，同时采集数据并写入数据集 - **复位阶段**：以 `reset_time_s` 为时长，只采集观测并发送动作（不写入数据集），给操作者时间复位环境

函数签名：

```

def record_loop(
    robot: Robot,
    events: dict,

```

```

    fps: int,
    teleop_action_processor: RobotProcessorPipeline,
    robot_action_processor: RobotProcessorPipeline,
    robot_observation_processor: RobotProcessorPipeline,
    dataset: LeRobotDataset | None = None,
    teleop: Teleoperator | None = None,
    policy: PreTrainedPolicy | None = None,
    preprocessor: PolicyProcessorPipeline | None = None,
    postprocessor: PolicyProcessorPipeline | None = None,
    control_time_s: int | None = None,
    single_task: str | None = None,
    display_data: bool = False,
):

```

每帧的执行步骤如下：

```

for timestamp in range(0, control_time_s, 1/fps): # 实际上用 while + time.perf_counter()

```

```

    | Step 1: 检查事件 |
    | - events["exit_early"] → 立即终止当前 episode |
    | - events["stop_recording"] → 由外层循环处理 |
    | - events["rerecord_episode"] → 由外层循环处理 |

```

```

    | Step 2: robot.get_observation() |
    | → 读取从臂 7 个电机当前角度 + 2 个相机图像 |
    | → 返回扁平 dict, 键如 "observation/arm/joint_1", |
    | "observation/images/wrist", ... |

```

```

    | Step 3: robot_observation_processor(obs) |
    | → 默认是 IdentityProcessor (不做处理) |
    | → 可配置为数据增强、归一化等 |

```

```

    | Step 4: 获取动作 |
    | 分支 A (有 policy): predict_action(observation, policy) |
    | 分支 B (有 teleop): teleop.get_action() |
    | 分支 C (PiPER 中继): 见下方详解 |

```

```

    | Step 5: 动作处理 |
    | teleop_action_processor((action, obs)) |

```

```

    | Step 6: 构建 frame dict → dataset.add_frame() |

```

```
| frame = {
|     **observation_frame, # 观测数据
|     **action_frame,     # 动作数据
|     "task": single_task, # 任务文本
| }
```

```
| Step 7: display_data (可选)
| → log_rerun_data(obs_processed, sent_action)
| → rerun 面板实时显示相机画面和关节状态
```

```
| Step 8: 帧率控制
| dt_s = time.perf_counter() - start_loop_t
| busy_wait(1/fps - dt_s)
| → 如果 dt_s > 1/fps, busy_wait 为负数, 立即进入下一帧
```

8.2.2 PiPER 的动作获取特殊路径

由于 PiPER 采用 PC 中继方案（主臂 CAN 直接中继到从臂 CAN），`teleop` 在本项目中为 `None`，动作来源不是遥操作臂也不是策略，而是直接从观测值中提取。对应代码：

```
# record.py 第 389-391 行 (PiPER 专用分支)
act_processed_teleop = teleop_action_processor((obs, obs))
action_values = act_processed_teleop
```

这里将 `obs`（从臂的当前关节角度）直接作为动作值记录。背后的原理是：在 PC 中继方案中，主臂发出的目标位置命令被 CAN 中继软件（如 `cangw`）直接转发给从臂，从臂的反馈控制会将其实际位置驱动到目标位置。当 PC 通过 `get_observation()` 读取从臂关节角度时，这个角度近似等于主臂被拖拽到的位置（忽略电机的跟踪延迟）。因此，**记录"从臂的实际关节角度"等价于记录"主臂的命令位置"**。

这一设计使得记录命令极简：不需要 `--teleoperator` 参数，不需要 `teleop.connect()`，整个录制流程只需要一台从臂的 CAN 连接。

8.2.3 时间控制机制

录制循环的时间控制是理解 FPS 精度的关键。代码中使用 `time.perf_counter()` 获取高精度时间戳（单位：秒），并用两种时间尺度管理流程：

```
| episode 级别的时间线 (以 episode_time_s=30, fps=30 为例)
```

8.3.1 manual_step 模式

问题背景：原始录制流程中，每个 episode 之间自动等待固定的 `reset_time_s` 秒后立即开始下一个 episode。但在实际使用中，操作者需要摆放操作物品、清理桌面、调整相机角度——这些操作的耗时不确定。固定的等待时间要么太短（操作者还没准备好），要么太长（浪费时间）。

解决方案：新增 `manual_step` 参数。

```
# RecordConfig 第 218 行
manual_step: bool = False
```

当 `manual_step=True` 时，录制流程的行为变化如下：



代码实现分为两处：

(1) Episode 开始前等待：

```
# record.py 第 519-520 行
if cfg.manual_step:
    input(f"\n[record] Press Enter to start episode {dataset.num_episodes}...")
else:
    log_say(f"Recording episode {dataset.num_episodes}", cfg.play_sounds, blocking=True)
```

(2) Episode 结束后等待：

```
# record.py 第 543-551 行
should_reset = not events["stop_recording"] and (
    (recorded_episodes < cfg.dataset.num_episodes - 1) or events["rerecord_episode"]
)
if should_reset and cfg.manual_step:
    run_reset_command_if_configured()
    input("[record] Episode finished. Reset complete or manually adjusted. Press Enter to continue...")
```

```

elif should_reset:
    log_say("Reset the environment", cfg.play_sounds, blocking=True)
    run_reset_command_if_configured()
    # 自动模式下使用固定 reset_time_s 等待
    record_loop(robot=robot, ..., control_time_s=cfg.dataset.reset_time_s, ...)

```

注意：在 `manual_step` 模式下，复位阶段不再调用 `record_loop` 来占用固定时间，而是直接等待用户输入。这大大提高了工作效率——操作者只需花费实际复位所需的时间。

8.3.2 run_reset_command_if_configured()

问题背景：许多操作环境在每轮录制后需要自动化复位——例如通过另一个脚本驱动传送带将新工件送到工位，或者控制气动执行器将物品推回初始位置。这些操作可能是系统级别的（需要 root 权限或调用其他硬件驱动），不便写入 LeRobot Python 代码中。

解决方案：通过环境变量注入任意复位命令，在录制循环的复位阶段自动执行。

```

# record.py 第 146-159 行
def run_reset_command_if_configured() -> None:
    command = os.environ.get("LEROBOT_RESET_COMMAND")
    if not command:
        return
    pause_file = os.environ.get("LEROBOT_RELAY_PAUSE_FILE")
    print(f"[record] Running reset command: {command}")
    try:
        if pause_file:
            Path(pause_file).parent.mkdir(parents=True, exist_ok=True)
            Path(pause_file).touch() # 创建暂停文件 → relay 暂停
            subprocess.run(shlex.split(command), check=False) # 执行复位脚本
    finally:
        if pause_file:
            Path(pause_file).unlink(missing_ok=True) # 删除暂停文件 → relay 恢复

```

设计要点分析：

- 环境变量驱动，零代码侵入：**只需在启动录制前 `export LEROBOT_RESET_COMMAND="..."` 即可激活，不需要修改 Python 代码。适合快速切换不同工位的复位逻辑。
- 暂停文件协调机制：**`LEROBOT_RELAY_PAUSE_FILE` 是一个可选的路径。当指定时，执行复位命令前该文件被 `touch` 创建，命令执行完后被 `unlink` 删除。PC 中继程序（`cangw` 或等价组件）可以监控这个文件的存在性——文件存在时暂停 CAN 中继，防止复位命令引起的从臂运动被误认为是主臂的命令输入。
- try/finally 确保清理：**无论复位命令成功、失败还是被 Ctrl+C 中断，`finally` 块都会执行，确保暂停文件一定被删除。这是关键的可靠性保证——如果暂停文件残留在磁盘上，`relay` 将被永久暂停。

4. **非零返回码不中断流程**: `check=False` 意味着即使复位脚本返回错误，录制流程也会继续。原因是在录制的容错性优于单次复位的成功——操作者可以在终端看到错误信息并手动干预，而不是整个录制会话崩溃。

使用示例:

```
# 场景: 录制 pick-and-place, 每轮结束后传送带送新物品
export LEROBOT_RESET_COMMAND="python3 /home/user/scripts/advance_conveyor.py --steps 1"
export LEROBOT_RELAY_PAUSE_FILE="/tmp/lerobot_relay_pause"

lerobot-record \
  --robot.type=piper_follower --robot.port=can0 \
  --dataset.repo_id=local/pick_place \
  --dataset.num_episodes=10 \
  --dataset.episode_time_s=30 \
  --dataset.single_task="Pick the object and place it in the bin"
```

8.3.3 有效 FPS 统计

问题背景: `--dataset.fps=30` 表示目标帧率，但实际录制帧率取决于硬件性能。如果相机读取、数据处理、磁盘写入的总耗时超过 33ms (1/30秒)，实际帧率就会低于 30。用户需要知道实际帧率来判断录制质量。

解决方案: 在每个 episode 结束时打印有效 FPS 统计。

```
# record.py 第 424-432 行
elapsed_s = time.perf_counter() - start_episode_t
if dataset is not None and elapsed_s > 0:
    logging.info(
        "Recorded %d frames in %.2fs, effective fps %.2f / requested fps %s",
        num_frames,
        elapsed_s,
        num_frames / elapsed_s,
        fps,
    )
```

输出示例:

```
INFO Recorded 897 frames in 30.12s, effective fps 29.78 / requested fps 30
```

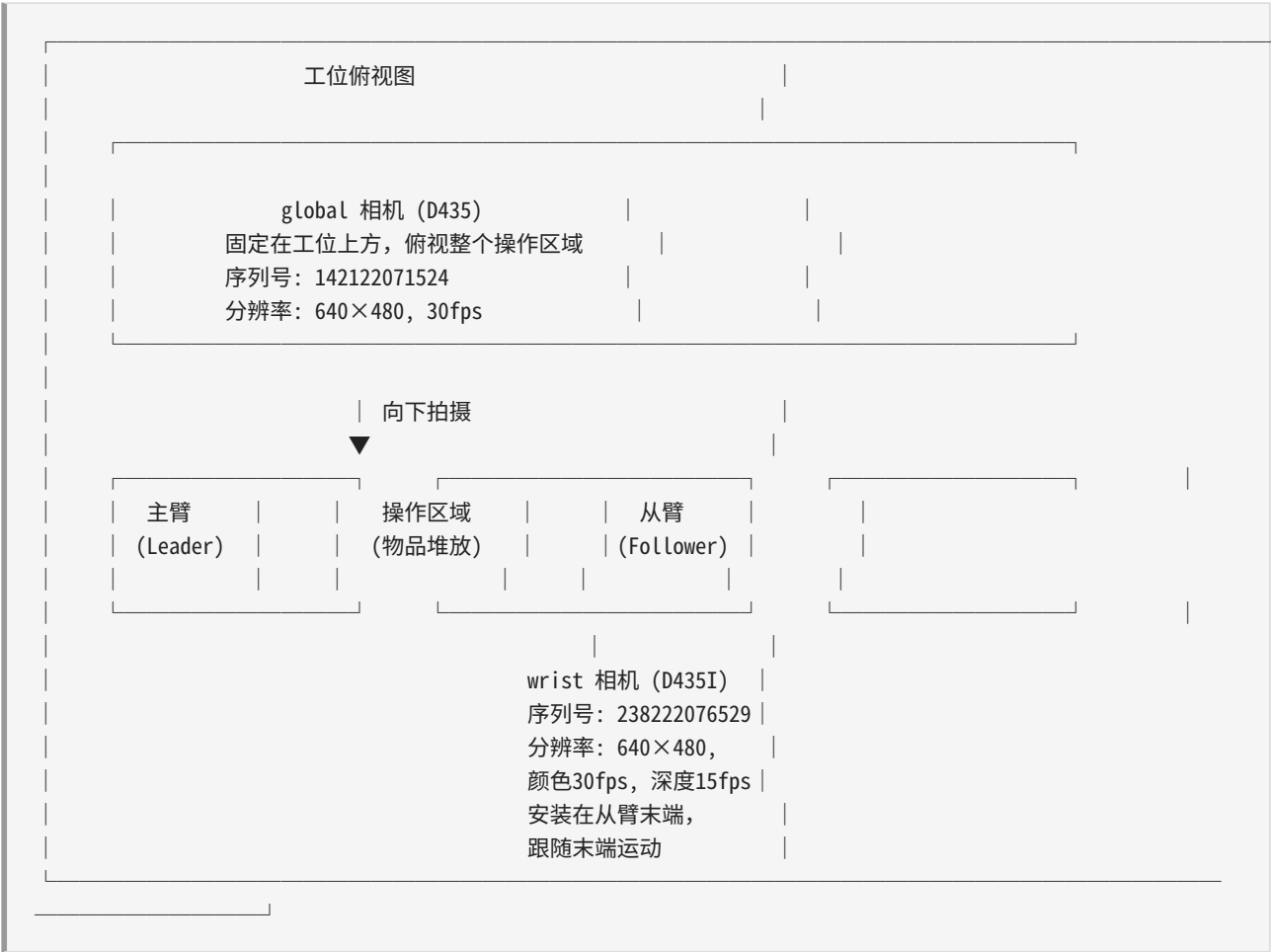
如果有效 FPS 显著低于目标 FPS (例如 22.5 / requested fps 30)，说明存在性能瓶颈。常见的瓶颈排查方法:

瓶颈来源	症状	排查手段
相机读取慢	单帧耗时 > 40ms	降低相机分辨率、减少 fps 设置
USB 带宽不足	双相机时明显	将两个相机插到不同的 USB 控制器上
磁盘 I/O 慢	写入峰值时丢帧	使用 SSD、减少 <code>num_image_writer_threads</code>
CPU 忙于图像编码	CPU 使用率 > 90%	增大 <code>video_encoding_batch_size</code> 推迟编码

8.4 双 RealSense 配置

8.4.1 物理布局

本项目使用两台 Intel RealSense 相机从不同视角采集操作场景：



- **wrist（腕部相机）**：Intel RealSense D435I，安装在从臂末端执行器附近，随机械臂运动。提供近距离的操作对象精细视角，适合精确抓取任务。

- **global（全局相机）**：Intel RealSense D435，固定在工位上方约 1.2m 处，视角不变。提供整个操作空间的全景视图，适合定位物体和判断大范围运动。

8.4.2 camera_config_realsense.env 配置详解

```
# 相机序列号——每台 RealSense 出厂时固化，USB 口换了也不影响识别
export WRIST_REALSENSE_SERIAL="238222076529"
export GLOBAL_REALSENSE_SERIAL="142122071524"

# LeRobot 框架使用的 Robot Cameras 配置
export ROBOT_CAMERAS='{
  wrist: {
    type: intelrealsense,
    serial_number_or_name: "238222076529",
    width: 640, height: 480, fps: 15,
  },
  global: {
    type: intelrealsense,
    serial_number_or_name: "142122071524",
    width: 640, height: 480, fps: 15,
  }
}'
```

序列号区分的好处： - RealSense 相机在 Linux 下的 `/dev/video*` 编号可能因 USB 口插拔而变化 - 序列号是出厂烧录的，永久不变 - 使用 `serial_number_or_name` 参数可确保无论相机插在哪个 USB 口，都能正确识别

获取相机序列号：

```
# 使用 LeRobot 内置工具
lerobot-find-cameras realsense

# 或直接使用 realsense-viewer
realsense-viewer

# GUI 中会显示每个相机的序列号
```

fps 设置说明：虽然相机硬件支持 30fps，但此处配置为 15fps。这是因为：1. 两台相机共享同一个 USB 控制器的带宽，总带宽有限 2. 模仿学习任务通常不需要 30fps 的视频数据（每帧之间的变化很小） 3. 降低 fps 可减少数据集的存储体积，加速训练

8.4.3 async_read 超时修改

原始代码：

```
# camera_realsense.py 第 489 行（修改前）
def async_read(self, timeout_ms: float = 200) -> np.ndarray:
```

修改后：

```
# camera_realsense.py 第 489 行 (修改后)
def async_read(self, timeout_ms: float = 1000) -> np.ndarray:
```

修改原因分析：

`async_read` 的工作机制如下： 1. 后台线程 `_read_loop` 持续从 `RealSense` 读取帧，存入 `latest_frame`，并通过 `new_frame_event` 事件通知 2. `async_read` 被主线程调用时，等待 `new_frame_event` 被设置（最多等 `timeout_ms` 毫秒） 3. 如果超时仍未等到新帧，抛出 `TimeoutError`

原始值 `200ms` 的矛盾： - `RealSense` 相机以 `30fps` 运行，帧间隔约 `33ms` - 但进程调度、图像传输、后台线程切换都会引入额外延迟 - 当**两台**相机同时运行时，主线程需要在两个相机之间切换读取，可能出现“刚读了一个相机，另一个相机的后台线程还在等待新帧”的情况 - `200ms` 对于单相机足够，但双相机时偶尔会因为 CPU 调度而超时

改为 `1000ms` 后： - 给予充足的等待时间（`30fps` 下约可完成 30 帧的读取周期） - 几乎消除了因调度延迟导致的误报超时 - 代价极低：正常情况下几毫秒就能等到新帧，超时只是一个安全上限

8.5 语音提示系统

语音提示系统帮助操作者在不需要看屏幕的情况下感知录制状态。这在操作环境中非常重要——操作者的注意力应该集中在机械臂和操作物品上，而不是终端输出。

8.5.1 TTS 引擎降级链

```
# utils.py 第 202-232 行
def say(text: str, blocking: bool = False):
    system = platform.system()

    if system == "Darwin":
        # macOS: 系统内置 say 命令
        cmd = ["say", text]

    elif system == "Linux":
        if shutil.which("espeak-ng"):
            # 首选: eSpeak NG (优秀的中文支持)
            cmd = ["espeak-ng", text]
        elif shutil.which("espeak"):
            # 备选: eSpeak (旧版)
            cmd = ["espeak", text]
        else:
            # 兜底: speech-dispatcher
            cmd = ["spd-say", text]
            if blocking:
                cmd.append("--wait")

    elif system == "Windows":
```

```
cmd = ["PowerShell", "-Command",
      "Add-Type -AssemblyName System.Speech; "
      f"(New-Object System.Speech.Synthesis.SpeechSynthesizer).Speak('{text}')]"]
```

引擎对比:

引擎	中文发音质量	安装方式	说明
espeak-ng	良好（支持拼音和声调）	<code>sudo apt install espeak-ng</code>	首选推荐。支持多语言，中文合成效果明显优于 spd-say
espeak	一般	<code>sudo apt install espeak</code>	espeak-ng 的前身，中文支持较弱
spd-say	差	Ubuntu 自带的 speech-dispatcher	英文尚可，中文发音非常机械，不建议用于中文提示

为什么需要更换：原版 LeRobot 只使用 `spd-say` 作为 Linux TTS 方案。在英文环境下尚可接受，但在中文操作提示场景下（如"开始录制"、"请复位环境"），`spd-say` 的中文输出难以辨认。升级到 `espeak-ng` 显著改善了中文语音的可懂度。

安装依赖:

```
sudo apt update
sudo apt install espeak-ng
```

8.5.2 play_cached_voice_prompt() — 预生成语音播放

问题背景：TTS 实时合成存在两个问题：1. **延迟大：**`espeak-ng` 合成一段短文本约需 200~500ms，这个延迟会在录制循环中累积 2. **质量不稳定：**同一段文本每次合成的韵律可能略有不同

解决方案：预先用高质量的云 TTS（如 Azure Neural TTS）生成语音文件，录制时直接播放这些文件。

语音文件存放在 `audio_prompts/` 目录下，按语言和语音名称组织：

```
audio_prompts/
├── en-US-AriaNeural/          # 英文女声 (Azure Neural)
│   ├── recording_episode_0.wav # "Recording episode 0"
│   ├── recording_episode_1.wav # "Recording episode 1"
│   ├── ...
│   ├── recording_episode_19.wav # "Recording episode 19"
│   ├── reset_the_environment.wav # "Reset the environment"
│   ├── re_record_episode.wav    # "Re-record episode"
│   ├── stop_recording.wav       # "Stop recording"
│   └── exiting.wav              # "Exiting"
└──
```

```

└── zh-CN-XiaoxiaoNeural/      # 中文女声 (Azure Neural, 晓晓)
    ├── recording_episode_0.mp3 # "正在录制第0个episode"
    ├── recording_episode_1.mp3
    ├── ...
    ├── recording_episode_50.mp3
    ├── reset_the_environment.mp3 # "请复位操作环境"
    ├── re_record_episode.mp3    # "重新录制这个episode"
    └── stop_recording.mp3       # "停止录制"

```

语音目录选择：通过环境变量 `LEROBOT_VOICE_PROMPT_DIR` 指定。例如：

```

# 使用中文语音
export LEROBOT_VOICE_PROMPT_DIR="audio_prompts/zh-CN-XiaoxiaoNeural"

# 使用英文语音
export LEROBOT_VOICE_PROMPT_DIR="audio_prompts/en-US-AriaNeural"

```

播放机制：

```

# utils.py 第 253-283 行
def play_cached_voice_prompt(text: str) -> bool:
    prompt_dir = os.environ.get("LEROBOT_VOICE_PROMPT_DIR")
    if not prompt_dir:
        return False                                # 未配置目录, 回退到 TTS

    key = _voice_prompt_key(text)
    if key is None:
        return False                                # 文本无法映射到预生成文件

    wav_path = Path(prompt_dir) / f"{key}.wav"
    if wav_path.exists() and shutil.which("aplay"):
        subprocess.run(["aplay", "-q", str(wav_path)], check=False)
        return True                                  # WAV 用 aplay 播放 (低延迟 ~5ms)

    path = Path(prompt_dir) / f"{key}.mp3"
    if not path.exists():
        return False                                  # 文件不存在, 回退到 TTS

    try:
        import pygame
        if not pygame.mixer.get_init():
            pygame.mixer.init()
        pygame.mixer.music.load(str(path))
        pygame.mixer.music.play()
        while pygame.mixer.music.get_busy():        # 阻塞等待播放完成
            time.sleep(0.02)
        return True
    except Exception as exc:
        logging.warning("Failed to play cached voice prompt %s: %s", path, exc)
        return False                                  # 播放失败, 回退到 TTS

```

文件格式选择： - `.wav` 格式优先：Linux 下 `aplay` 直接播放，延迟约 5ms，无额外依赖 - `.mp3` 格式备选：需要 `pygame` 库，播放延迟约 10ms，但文件体积更小

文本到文件名的映射（`_voice_prompt_key`）：

```
# utils.py 第 239-250 行
def _voice_prompt_key(text: str) -> str | None:
    # "Recording episode 3" → "recording_episode_3"
    match = re.fullmatch(r"Recording episode (\d+)", text)
    if match:
        return f"recording_episode_{match.group(1)}"

    # 固定文本映射
    mapping = {
        "Reset the environment": "reset_the_environment",
        "Re-record episode": "re_record_episode",
        "Stop recording": "stop_recording",
        "Exiting": "exiting",
    }
    return mapping.get(text)
```

8.5.3 `play_phase_beep()` — 不同阶段的提示音

语音播放适合告知“正在发生什么”，但有时候操作者只需要一个简短的声音信号来判断状态变化。提示音系统为不同事件分配不同频率的蜂鸣音：

事件	频率	模式	含义
开始录制 episode	880 Hz	单声（0.16s）	高音短促——“准备好了”
复位环境	440 Hz	双声（0.12s × 2）	中音双声——“请复位”
重新录制 episode	660 Hz	三声（0.1s × 3）	中高音三连——“重来”
停止/退出	220 Hz	单长声（0.3s）	低音长——“结束”

```
# utils.py 第 308-327 行
def play_phase_beep(text: str):
    if not _truthy_env("LEROBOT_PHASE_BEEP"):
        return # 环境变量未启用，跳过

    lower = text.lower()
    if "recording episode" in lower:
        pattern = [(880, 0.16)] # 高音单声
    elif "reset" in lower:
        pattern = [(440, 0.12), (440, 0.12)] # 中音双声
    elif "re-record" in lower:
        pattern = [(660, 0.1), (660, 0.1), (660, 0.1)] # 中高音三声
    elif "stop" in lower or "exiting" in lower:
```

```

        pattern = [(220, 0.3)]                # 低音长声
    else:
        pattern = [(660, 0.1)]                # 默认中高音

    for index, (freq, duration) in enumerate(pattern):
        if index > 0:
            time.sleep(0.08)                  # 多声之间 80ms 间隔
            _play_tone(freq, duration)

```

计算机声学设计原理： - **高音 (880Hz)** 对应开始：高频声音具有"唤醒"和"注意"的心理暗示，类似比赛开始时的哨声 - **中音 (440Hz/660Hz)** 对应中间状态：不突兀，提示但不打断注意力 - **低音 (220Hz)** 对应结束：低频声音具有"终止"和"确认"的感觉，类似心跳声的结束 - 频率选择避开了 8 度重复（如 440Hz 与 880Hz 相差一个八度），确保操作者能清晰分辨不同阶段的提示

提示音生成实现（`_play_tone`）：

```

# utils.py 第 286-305 行
def _play_tone(freq_hz: int, duration_s: float, volume: float = 0.35):
    if not shutil.which("aplay"):
        return                                # aplay 不可用则跳过

    sample_rate = 16_000                      # 16kHz 采样率（语音品质）
    samples = int(sample_rate * duration_s)
    amplitude = int(32767 * volume)           # 35% 音量，避免刺耳

    path = Path("/tmp/lerobot_phase_beep.wav")

    with wave.open(str(path), "wb") as wav:
        wav.setnchannels(1)                   # 单声道
        wav.setsampwidth(2)                   # 16-bit 采样
        wav.setframerate(sample_rate)

        frames = bytearray()
        for i in range(samples):
            # 生成正弦波:  $y(t) = A * \sin(2\pi * f * t)$ 
            sample = int(amplitude * math.sin(2 * math.pi * freq_hz * i / sample_rate))
            frames.extend(sample.to_bytes(2, byteorder="little", signed=True))
        wav.writeframes(frames)

    subprocess.run(["aplay", "-q", str(path)], check=False)

```

关键技术选择： - **16kHz 采样率：**奈奎斯特频率为 8kHz，880Hz 的最高频率远在其下，16kHz 足够且文件极小 - **直接生成 WAV 而非调用外部合成器：**避免额外依赖，生成耗时约 2ms - **/tmp/lerobot_phase_beep.wav**：固定临时路径，每次覆写，不积累文件垃圾

启用提示音：

```
export LEROBOT_PHASE_BEEP=1
```

8.5.4 log_say() — 统一的语音播报接口

```
# utils.py 第 330-337 行
def log_say(text: str, play_sounds: bool = True, blocking: bool = False):
    logging.info(text)                # 始终写日志

    if play_sounds:
        if not play_cached_voice_prompt(text): # 优先级 1: 预生成文件
            say(text, blocking)              # 优先级 2: 实时 TTS
    if blocking:
        play_phase_beep(text)              # 关键节点播提示音
```

调用场景分析：

`log_say` 在录制流程中被调用 8 次，每次的语义和策略不同：

调用位置	text 内容	blocking	说明
录制 episode 前	"Recording episode N"	True	阻断：确保操作者听到提示后再开始录制
自动复位开始	"Reset the environment"	True	阻断：提示操作者开始复位动作
重新录制	"Re-record episode"	True	阻断：操作者需要知道发生了重录
停止录制	"Stop recording"	True	阻断：最终确认，避免操作者误以为还在录制
退出程序	"Exiting"	False	非阻断：退出时不需要等待

`blocking=True` 的含义是：语音播报会阻塞主线程直到播放完成（通常 1~3 秒），确保操作者在录制开始前已收到完整提示。`blocking=False` 则异步播放，不阻塞录制流程。

8.6 录制启动脚本

8.6.1 直连 CAN 方案（简单版）

当主臂和从臂通过 CAN 硬件中继直接通信（不需要 PC 中继软件）时，录制命令最简洁：

```
#!/bin/bash
# 4__record.sh — 直连 CAN 录制脚本

source ~/miniconda3/etc/profile.d/conda.sh
conda activate lerobot

HF_USER=$(hf auth whoami | head -n 1)
echo "HuggingFace user: $HF_USER"
```



```

lerobot-record \
  --robot.type=piper_follower \
  --robot.port=can0 \
  --robot.cameras='{
    top: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30},
    left: {type: opencv, index_or_path: 4, width: 640, height: 480, fps: 30}
  }' \
  --robot.id=black \
  --dataset.num_episodes=50 \
  --dataset.single_task="Grab the yellow car and put in the box" \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/piper_pick_yellow_car

```

关键点：

- `robot.port=can0`：从臂连接到 CAN0 接口 - 不需要 `--teleoperator` 参数：主臂的命令由 CAN 硬件中继直接转发
- 相机使用 `opencv` 类型（USB 摄像头）或 `intelrealsense` 类型（RealSense）
- `display_data=true`：打开 `rerun` 可视化面板，实时查看相机和关节状态
- `dataset.repo_id` 使用 HuggingFace 用户名作为前缀

8.6.2 PC 中继方案（record_with_pc_relay.sh）

当使用 PC 中继方案时，录制前需要：1. 加载相机环境变量配置 2. 启动 PC 中继程序（另开终端）

```

# 终端 1: 启动 PC 中继
./cangw --leader can0 --follower can1

# 终端 2: 加载相机配置并开始录制
source ./camera_config_realsense.env
source ~/miniconda3/etc/profile.d/conda.sh
conda activate lerobot

# 启用语音提示（中文）和提示音
export LEROBOT_VOICE_PROMPT_DIR="audio_prompts/zh-CN-XiaoxiaoNeural"
export LEROBOT_PHASE_BEEP=1

# 可选：配置自动复位命令
export LEROBOT_RESET_COMMAND="python3 /home/user/scripts/reset_env.py"
export LEROBOT_RELAY_PAUSE_FILE="/tmp/lerobot_relay_pause"

lerobot-record \
  --robot.type=piper_follower \
  --robot.port=can1 \
  --robot.cameras="${ROBOT_CAMERAS}" \
  --robot.id=black \
  --dataset.repo_id=local/piper_bimanual_task \
  --dataset.num_episodes=30 \
  --dataset.episode_time_s=60 \
  --dataset.fps=30 \
  --dataset.single_task="Pick the red block and place it on the blue platform" \
  --display_data=true \

```

```
--manual_step=true \  
--play_sounds=true
```

注意: `--robot.cameras="${ROBOT_CAMERAS}"` 使用了从 `camera_config_realsense.env` 中加载的环境变量, 这样 camera 配置就可以独立于启动脚本管理。

8.7 数据集输出结构

8.7.1 完整目录树

录制完成后, 数据集在磁盘上的完整结构如下:

```
datasets/local/piper_task/          # <root>/<repo_id>  
|  
├── meta/  
|   ├── info.json                  # 数据集元信息  
|   |   ├── "fps": 30              # 录制帧率  
|   |   ├── "total_episodes": 10   # episode 总数  
|   |   ├── "total_frames": 8990   # 总帧数  
|   |   ├── "total_tasks": 1       # 任务数  
|   |   ├── "features": {...}      # 特征 schema  
|   |   └── "robot_type": "piper_follower" # 机器人类型  
|   └── stats.json                 # 归一化统计量  
|       ├── "observation.state": {  # 关节角度  
|       |   ├── "mean": [...], "std": [...], #  
|       |   ├── "min": [...], "max": [...]   #  
|       |   └── }                         #  
|       └── "action": {             # 动作命令  
|           ├── "mean": [...], "std": [...], #  
|           ├── "min": [...], "max": [...]   #  
|           └── }                         #  
└── tasks.jsonl                    # 任务描述 (每行一个 episode)  
    ├── {"task_index": 0, "task": "Pick the red block..."}  
    ├── {"task_index": 1, "task": "Pick the red block..."}  
    └── ...  
  
├── data/  
|   ├── chunk-000/                 # 第一个数据块  
|   |   ├── episode_000000.parquet  # Episode 0 所有帧的非图像数据  
|   |   ├── episode_000001.parquet  # Episode 1  
|   |   ├── ...  
|   |   └── episode_000009.parquet  # 每个 chunk 可包含多个 episode  
|   └── ...  
└── videos/  
    ├── observation.images.wrist/  # 腕部相机视频  
    |   ├── episode_000000.mp4     # Episode 0 的视频  
    |   └── episode_000001.mp4
```

```

|   |   ...
|   |   |
|   |   |— observation.images.global/      # 全局相机视频
|   |   |   |— episode_000000.mp4
|   |   |   |— episode_000001.mp4
|   |   |   |
|   |   |   ...

```

8.7.2 关键文件说明

info.json — 数据集元信息:

```

{
  "fps": 30,
  "total_episodes": 10,
  "total_frames": 8990,
  "total_tasks": 1,
  "total_chunks": 1,
  "chunks_size": 1000,
  "features": {
    "observation.state": {
      "dtype": "float32",
      "shape": [7],
      "names": ["joint_1", "joint_2", "joint_3", "joint_4", "joint_5", "joint_6", "gripper"]
    },
    "observation.images.wrist": {
      "dtype": "video",
      "shape": [3, 480, 640],
      "names": ["channel", "height", "width"]
    },
    "action": {
      "dtype": "float32",
      "shape": [7],
      "names": ["joint_1", "joint_2", "joint_3", "joint_4", "joint_5", "joint_6", "gripper"]
    },
    "task": {
      "dtype": "string",
      "shape": [1],
      "names": null
    }
  },
  "robot_type": "piper_follower"
}

```

stats.json: 此文件在录制阶段仅包含占位值，真正的统计量在训练脚本的 `compute_stats()` 步骤中计算。它包含每个浮点特征维度的均值、标准差、最小值和最大值，用于训练时的数据归一化。

tasks.jsonl: 每行一个 JSON 对象，记录该 episode 的任务文本。如果所有 episode 使用相同任务，每行内容相同。

parquet 文件：每个 episode 的表格数据以 Apache Parquet 格式存储。每行对应一帧，列包括：

- `observation.state` (`float32[7]`): 从臂 7 个关节的实际角度
- `action` (`float32[7]`): 记录的动作（在 PiPER 方案中等于从臂角度）
- `task` (`string`): 任务描述文本
- `episode_index` (`int64`): episode 编号
- `frame_index` (`int64`): 帧在 episode 内的编号
- `timestamp` (`float32`): 相对于 episode 开始的时间戳（秒）
- `index` (`int64`): 全局帧编号

视频文件：每个 camera key 对应一个视频子目录。视频以 MP4 (H.264) 格式编码，帧率与数据集 fps 一致。视频帧索引与 parquet 文件的 `frame_index` 列对齐。

8.7.3 数据读取示例

录制完成后，可以用以下 Python 代码快速检查数据质量：

```
from lerobot.datasets.lerobot_dataset import LeRobotDataset

# 加载本地数据集
dataset = LeRobotDataset("local/piper_task")

# 查看元信息
print(f"FPS: {dataset.fps}")
print(f"Total episodes: {dataset.num_episodes}")
print(f"Total frames: {dataset.num_frames}")
print(f"Features: {dataset.features}")

# 读取某个 episode 的所有帧
episode_0 = dataset.hf_dataset.filter(
    lambda x: x["episode_index"] == 0
)
print(f"Episode 0 has {len(episode_0)} frames")

# 查看第一帧的数据
first_frame = dataset[0]
print(f"Joint angles: {first_frame['observation.state']}")
print(f"Task: {first_frame['task']}")
```

8.8 Replay 功能 — 数据验证

8.8.1 功能概述

`lerobot-replay` 是一个独立的数据验证工具，功能是读取已录制的数据集，将保存的动作命令逐帧发送给从臂，让从臂"重放"示教过程。它的核心应用场景：

1. **验证数据质量：**重放能展示数据中是否存在不合理的大幅度跳跃、抖动或噪声。如果从臂在重放时剧烈摇摆，说明原始数据有质量问题。

2. **验证动作可行性**: 不是所有从数据中学到的行为都能在物理机器人上执行。Replay 在真实硬件上验证记录的关节轨迹是否平滑、可达。
3. **调试硬件问题**: 如果 replay 中动作表现与录制时不一致（例如末端位置偏移），可以定位是电机校准、CAN 延迟还是关节零位漂移问题。
4. **演示和展示**: 无需操作者再次示教，机器人可以自动重复之前的操作过程。

8.8.2 命令用法

```
lerobot-replay \  
  --robot.type=piper_follower \  
  --robot.port=can0 \  
  --robot.id=black \  
  --dataset.repo_id=local/piper_pick_yellow_car \  
  --dataset.episode=0
```

关键参数: - `--robot.type/port/id`: 与录制时相同的从臂配置 - `--dataset.repo_id`: 已录制数据集的路径标识 - `--dataset.episode`: 要回放的具体 episode 编号

8.8.3 回放循环

```
# replay.py 第 107-125 行  
log_say("Replaying episode", cfg.play_sounds, blocking=True)  
  
for idx in range(len(episode_frames)):  
    start_episode_t = time.perf_counter()  
  
    # 从数据集中取出第 idx 帧的动作  
    action_array = actions[idx]["action"]  
    action = {}  
    for i, name in enumerate(dataset.features["action"]["names"]):  
        action[name] = action_array[i]  
  
    # 读取当前观测（用于 action processor 的处理上下文）  
    robot_obs = robot.get_observation()  
  
    # 处理动作  
    processed_action = robot_action_processor((action, robot_obs))  
  
    # 发送动作到从臂  
    _ = robot.send_action(processed_action)  
  
    # 帧率控制  
    dt_s = time.perf_counter() - start_episode_t  
    busy_wait(1 / dataset.fps - dt_s)  
  
robot.disconnect()
```

与录制循环的核心区别： - **不需要相机**：Replay 只发送关节命令，不需要读取相机图像 - **动作来自数据集而非主臂或策略**：逐帧读取 parquet 文件中的 `action` 列 - **不需要写数据集**：纯回放，无数据写入 - **同样的帧率控制**：保持与录制时相同的帧率，确保回放速度一致

8.9 常见问题与排查

8.9.1 FPS 不稳定 / 有效 FPS 远低于目标值

现象：终端输出 `effective fps 18.34 / requested fps 30`，录制的视频明显卡顿。

原因与解决方案：

可能原因	排查方法	解决方案
相机 USB 带宽不足	查看 <code>dmesg grep -i usb</code> 是否有带宽错误	将两个相机插到不同的 USB 控制器（查看 <code>lsusb -t</code> 确认总线拓扑）
相机分辨率/帧率过高	计算带宽： <code>640×480×3×30×2</code> 相机 ≈ 165 MB/s	降低到 <code>640×480, fps=15</code> 或 <code>320×240, fps=30</code>
磁盘写入慢	<code>iotop -o</code> 观察写入速率	使用 SSD；增大 <code>video_encoding_batch_size</code> 推迟视频编码
图像写入线程过多	线程竞争导致主线程阻塞	减少 <code>num_image_writer_threads_per_camera</code> 到 2
CPU 负载高	<code>htop</code> 观察 CPU 利用率	增加 <code>num_image_writer_processes</code> 利用多核并行

系统性优化命令（使用前先备份数据）：

```
lerobot-record \
  --robot.type=piper_follower --robot.port=can0 \
  --robot.cameras='{wrist: {type: intelrealsense, serial_number_or_name: "238222076529", width: 320, height: 240, fps: 15}}' \
  --dataset.repo_id=local/test_optimized \
  --dataset.fps=15 \
  --dataset.num_image_writer_threads_per_camera=2 \
  --dataset.video_encoding_batch_size=10 \
  ...
```

8.9.2 录制中断 / 电机失能

现象：录制过程中从臂突然失能，终端输出 CAN 通信错误。

排查清单：

1. **CAN 物理连接：** `bash # 检查 CAN 接口是否 up ip link show can0 # 应显示 "state UP", 如果显示 "state DOWN": sudo ip link set can0 up type can bitrate 1000000`
2. **CAN 线缆：** 检查 DB9 或 M12 接头是否松动。双臂系统通常有 4 个 CAN 节点（主臂 × 2 口、从臂 × 2 口），每个节点都需要可靠连接。
3. **终端电阻：** CAN 总线两端必须有 120Ω 终端电阻。如果使用 USB-CAN 适配器，确认适配器是否已内置终端电阻。
4. **电机温度保护：** 电机长时间工作可能过热触发保护。如果从臂突然失能，等待 5 分钟冷却后再试。
5. **电源不足：** 7 个 AGILEX 电机（尤其是 joint1-3 的大扭矩 AGILEX-M）对电源要求高。确认 24V 电源额定电流 $\geq 15A$ 。

8.9.3 相机不工作

现象： `lerobot-find-cameras realsense` 找不到相机，或连接时超时。

排查步骤：

```
# 1. 确认 USB 设备被系统识别
lsusb | grep -i intel
# 应输出类似：Bus 003 Device 005: ID 8086:0b07 Intel Corp. Intel(R) RealSense(TM) Depth Camera 435

# 2. 确认序列号
lerobot-find-cameras realsense
# 对比输出的序列号与 camera_config_realsense.env 中的序列号是否一致

# 3. 检查 USB 线缆
# RealSense 对 USB 线缆质量敏感。建议使用：
# - 原装线缆 或 高质量的 USB 3.0 线缆
# - 长度不超过 2m（过长会导致信号衰减）
# - 避免使用 USB Hub（直接插主机 USB 口）

# 4. 重置 USB 设备
sudo usbreset <bus>/<device> # 例如 sudo usbreset 003/005
```

8.9.4 语音不播放 / 中文发音差

排查步骤：

```
# 1. 确认 espeak-ng 已安装
which espeak-ng # 应输出 /usr/bin/espeak-ng
espeak-ng "测试中文" # 听听是否正常
```

```
# 2. 安装 espeak-ng (如果未安装)
sudo apt update && sudo apt install espeak-ng

# 3. 确认语音提示目录正确
ls audio_prompts/zh-CN-XiaoxiaoNeural/
# 应列出 .mp3 文件

# 4. 确认环境变量已设置
echo $LEROBOT_VOICE_PROMPT_DIR
# 应输出 audio_prompts/zh-CN-XiaoxiaoNeural (或完整路径)

# 5. 确认 pygame 已安装 (用于播放 mp3)
python3 -c "import pygame; print('ok')"
```

8.9.5 磁盘空间不足

现象：录制到一半报错 `No space left on device`。

空间估算：以 640×480 , 30fps, 60s episode 为例：

数据类型	单帧大小	一个 episode (1800 帧)	50 个 episode
Parquet (关节数据)	~200 bytes	~0.36 MB	~18 MB
视频 (H.264, 1 相机)	~15 KB	~27 MB	~1.35 GB
视频 (H.264, 2 相机)	~30 KB	~54 MB	~2.7 GB
原始 png (录制过程中)	~150 KB	~270 MB	— (编码后转为视频)
总计 (2相机, 50ep)	—	—	约 3-5 GB

预防措施：

```
# 录制前检查磁盘空间
df -h datasets/

# 建议至少保留录制预估空间的 3 倍余量 (考虑临时 png 文件)
```

8.9.6 录制但未生成视频

现象：`videos/` 目录为空，只有 png 文件残留。

原因：视频编码在 episode 保存后异步执行。如果录制提前终止或编码进程被杀，视频可能未完成编码。

解决方案：


```
# 确认 video encoding batch size 设置
# 如果设得过大（如 50），所有 episode 录制完后才编码
# 建议设为 1（每个 episode 录完立即编码）
# 或设为合理值如 5（每录完 5 个 episode 批量编码一次）

lerobot-record ... --dataset.video_encoding_batch_size=1
```

8.10 章节总结

本章从 CLI 入口点 `lerobot-record` 出发，完整剖析了 PiPER 机械臂的数据录制管线。核心要点回顾：

1. **管线的入口和参数体系**：`record.py:main()` → `record()`，通过嵌套的 `RecordConfig` dataclass 管理所有参数，`@parser.wrap()` 装饰器自动完成解析和校验。
2. **录制循环的逐帧时序**：`record_loop()` 每帧依次执行事件检查 → `get_observation()` → 处理管线 → 动作获取 → `dataset.add_frame()` → 帧率控制。PiPER 的特殊之处在于动作直接从从臂观测中提取（因为主臂命令通过 CAN 中继已在从臂上体现为实际位置）。
3. **手动步进模式**（`manual_step=True`）：将固定时间的复位等待改为等待操作者按 Enter，更适合需要手动摆放物品的场景。
4. **可配置复位命令**（`LEROBOT_RESET_COMMAND`）：通过环境变量注入外部脚本，配合暂停文件机制协调 PC 中继。
5. **双 RealSense 配置**：利用序列号区分相机，wrist（腕部）和 global（全局）两个视角互补。`async_read` 超时从 200ms 增加到 1000ms 解决了双相机读取的调度问题。
6. **语音提示系统**：包括 TTS 引擎降级链（`espeak-ng` → `espeak` → `spd-say`）、预生成高质量语音文件播放、以及不同频率的蜂鸣提示音。整体设计让操作者不需要看屏幕即可知晓录制状态。
7. **数据集输出结构**：`meta/`（`info.json` + `stats.json` + `tasks.jsonl`）、`data/`（parquet 文件）、`videos/`（mp4 文件）三层结构，支持 HuggingFace datasets 库直接加载。
8. **Replay 回放验证**：`lerobot-replay` 逐帧读取数据集中的动作并重放到从臂上，用于验证数据质量和动作可行性。
9. **常见问题排查**：覆盖了 FPS 不稳定、录制中断、相机不工作、语音播放失败、磁盘空间不足等实际使用中的高频问题。

第九章 模仿学习与 Action Chunking Transformer (ACT)

学习目标：理解模仿学习的基本原理，掌握 Action Chunking Transformer (ACT) 的架构设计、训练流程与部署方法，能够将 ACT 策略应用于双臂机器人操作任务。

9.1 模仿学习基础

什么是模仿学习

模仿学习 (Imitation Learning) 是机器人学习中最直观的范式之一。其核心思想可以概括为一句话：**让人来演示任务，让机器人学习模仿人的行为**。从机器学习的角度看，模仿学习本质上是一种监督学习——只是输入和输出都活在物理世界中。

具体而言，模仿学习将机器人控制问题转化为一个映射问题：

- **输入 (observation)**：机器人当前感知到的所有信息，包括相机拍摄的 RGB 图像、机械臂各关节的当前位置、夹爪的开合状态等。
- **输出 (action)**：机器人下一步应该执行的动作，通常表示为各关节的目标角度位置和夹爪的目标开度。
- **训练数据**：由人类操作者通过遥操作采集的 (observation, action) 对，按时间顺序排列形成若干段演示 (demonstration episodes)。

与强化学习 (Reinforcement Learning) 相比，模仿学习最大的区别在于：**模仿学习不需要设计 reward 函数**。reward 函数的设计是强化学习中最困难的部分之一——对于一个复杂的双臂操作任务，如何用数学定义“做得好”往往比直接演示几次要困难得多。模仿学习通过人类演示绕过了这个问题，让数据本身定义什么是好的行为。

行为克隆 (Behavioral Cloning)

行为克隆是模仿学习中最简单、最直接的方法。它的数学形式非常朴素：

$$\min_{\pi} \mathbb{E}_{(o_t, a_t) \sim \mathcal{D}} \left[\|\pi(o_t) - a_t\|^2 \right]$$

其中 π 是我们学习的策略网络， $\mathcal{D}$ 是人类的演示数据集， s_t 是 t 时刻的观测， a_t 是 t 时刻人类执行的动作。换句话说，行为克隆的目标就是让策略网络对每一个观测下的预测动作，尽可能接近人类在该观测下执行的动作。

然而，行为克隆存在两个根本性的挑战：

问题一：复合误差 (Compounding Errors)。假设策略每次预测的误差是 ϵ 。在开环执行中，第一步会产生一个小偏差，这导致机器人到达的状态略微偏离训练分布；第二步的策略在这个“没见过的状态”上进行预测，可能产生更大的偏差；如此往复，误差随时间呈指数级累积，最终机器人可能完全偏离演示过的轨迹。这是模仿学习最核心的问题——策略的误差分布与训练时的误差分布不同（分布偏移，distribution shift）。

问题二：多模态 (Multi-modality)。对于同一个初始状态，可能存在多种同样正确的操作方式。例如，把桌上的杯子拿起来，可以从左边接近也可以从右边接近。如果用一个简单的回归模型（输出单一的动作向量），它会学习输出所有可能动作的平均值——而这个平均值恰恰可能是一个无效动作（比如夹爪从中间穿过了杯子）。标准的 L2 回归无法处理这种一个输入对应多个合理输出的情况。

本章介绍的 ACT 算法通过预测动作序列 (action chunking) 和 Transformer 架构，有效缓解了上述两个问题。

LeRobot 数据集格式

在深入算法细节之前，我们首先需要理解训练数据的组织方式。LeRobot 框架使用了一种统一的数据格式来存储演示数据。一个数据集包含多个 episode（每次人类遥操作录制一段称为一个 episode），每个 episode 是一个时间序列：

```
Episode k:
  t=0: observation_0 → action_0
  t=1: observation_1 → action_1
  ...
  t=T: observation_T → action_T
```

每个时间步的 observation 和 action 的具体结构如下：

```
observation = {
  "observation.state": tensor([joint1.pos, joint2.pos, ..., joint6.pos, gripper.pos]), # (7,)
  "observation.images.wrist": tensor(480, 640, 3), # RGB 图像, 腕部相机
  "observation.images.global": tensor(480, 640, 3), # RGB 图像, 全局相机
}

action = tensor([joint1.pos, joint2.pos, ..., joint6.pos, gripper.pos]) # (7,)
```

注意 `action` 和 `observation.state` 共享相同的维度结构（都是 7 维：6 个关节角度 + 1 个夹爪开度），但含义不同：`observation.state` 是当前关节的实际位置，而 `action` 是人类演示的目标位置。训练时，模型学习从 `observation.state` 映射到 `action` 的偏移关系。

在代码中，`action` 的键名列表定义在 `piper_act_safe_rollout.py` 中：

```
ACTION_KEYS = [
    "joint1.pos",
    "joint2.pos",
    "joint3.pos",
    "joint4.pos",
    "joint5.pos",
    "joint6.pos",
    "gripper.pos",
]
```

为什么 ACT 选择关节空间而不是像素空间

在第一章中，我们讨论了关节空间控制和笛卡尔空间控制的分野。现在我们从 ACT 算法的具体实现角度，审视这个设计选择如何影响学习问题的难度和策略的性能。

ACT 的输入和输出

回顾 ACT 的实际数据流：

输入：

- 腕部相机图像 ($640 \times 480 \times 3 = 921,600$ 维) $\longrightarrow$ ResNet-18 $\longrightarrow$ 特征向量 (512 维)
- 全局相机图像 ($640 \times 480 \times 3 = 921,600$ 维) $\longrightarrow$ ResNet-18 $\longrightarrow$ 特征向量 (512 维)
- 关节当前角度 + 夹爪开度 (7 维) $\longrightarrow$ 直接拼接

输出：

- 未来 30 步的关节目标位置： $30 \times 7 = 210$ 维
(每步： `joint1.pos`, `joint2.pos`, ..., `joint6.pos`, `gripper.pos`)

ACT 输出的不是像素空间的动作（比如“在图像坐标 (320, 240) 处抓取”），不是末端笛卡尔坐标（xyz + 姿态），而是**关节角度**。这个选择与 Behavior Transformer (BeT) 和 Diffusion Policy 等主流模仿学习算法完全一致。

为什么：动作空间维度的指数级差异

这可能是模仿学习中最重要“免费午餐”——动作空间的维度差异：

动作空间类型	维度	学习难度
像素空间动作	921,600	极高（语义鸿沟）

(预测下一帧像素)

笛卡尔轨迹 6 + IK 中等 (需要 IK 求解)
(x, y, z, roll, pitch, yaw)

关节空间动作 7 低 (直接拟合)
(joint1..6, gripper) ← ACT 的选择

从 921,600 维到 7 维，维度降低了超过 10 万倍。在机器学习中，**学习一个 7→7 的映射远易于学习一个 921,600→921,600 的映射**。这不仅体现在需要的数据量上，也体现在训练的稳定性和预测的精确度上。

更根本的原因：避免"中间表示"的累积误差

如果策略输出的是笛卡尔坐标（末端 xyz + 姿态），轨迹需要经过 IK 求解器才能转化为电机命令：

```
策略 → [x, y, z, roll, pitch, yaw] → IK 求解器 → [θ1, ..., θ6] → 电机
                                   ↑
                               多解性、奇异性、
                               数值迭代误差
```

IK 求解器在以下情况下可能失效： - 目标位姿在奇异点附近（Jacobian 病态，关节速度趋于无穷）
- 目标位姿有多个解，IK 选了"错误"的那个（比如导致关节 3 撞到桌面） - 数值 IK 不收敛（目标位姿稍微超出工作空间边界）

当 IK 求解器出错时，策略不知道自己犯了什么错——它只能看到"机械臂没有到达我指定的位置"，但无法知道是 IK 的问题还是自己的问题。这个反馈回路断裂使得 end-to-end 学习的梯度无法从动作效果回传到策略参数。

在关节空间中，这个问题完全不存在：

```
策略 → [θ1, ..., θ6, gripper] → 直接发给电机
```

策略的输出和电机的输入是同一种数据类型，不需要任何中间变换。这意味着： - 策略可以精确控制每个关节的运动量 - 训练信号（L2 loss）直接反映策略参数的优化方向 - 没有奇异性、多解性、收敛性等"黑箱"问题

为什么这不是一个"偷懒"的选择

有人可能会问：笛卡尔空间控制更"通用"——策略与机械臂解耦，同一个策略可以用于不同的机械臂。为什么不做笛卡尔？

答案是：在当前模仿学习的发展阶段，让一个机械臂可靠地完成 3-5 个桌面任务，比让一个策略跨机械臂泛化更紧迫。先让一台机械臂"好用"，再考虑跨硬件的泛化。跨硬件泛化是 VLA 路线的目标——用海量多机器人数据训练一个通用策略——而不是 ACT 要解决的问题。

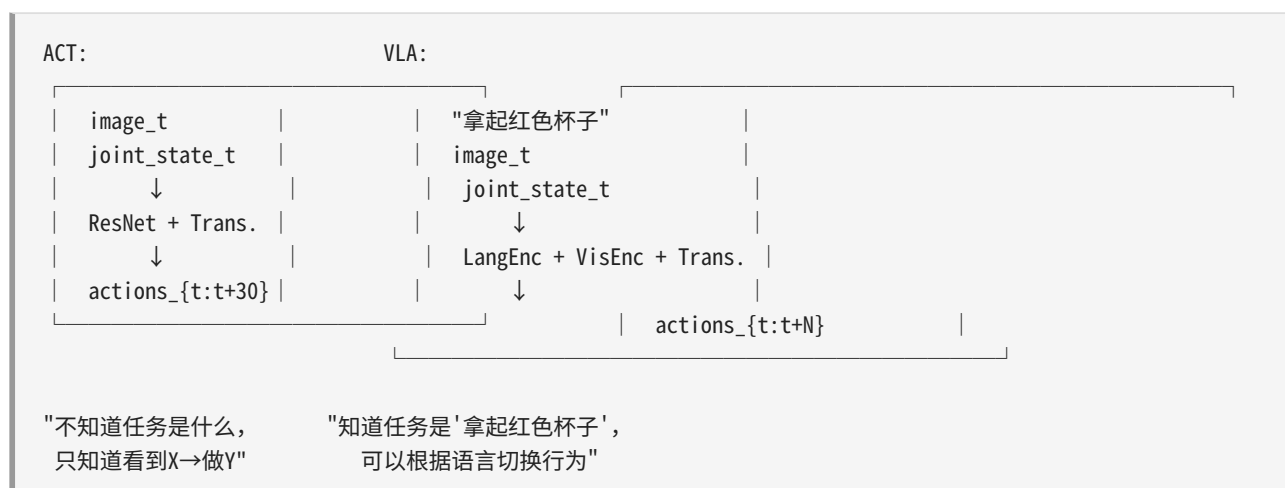
ACT 的务实哲学：在低维关节空间中，用 50-200 episodes 的数据，让一个真实的机械臂学会做一件事。这个目标在当前阶段是合理且可实现的。

ACT 与 VLA 的关系

ACT 是视觉模仿学习, VLA 是其多模态扩展

ACT (Action Chunking Transformer) 是一种**视觉模仿学习 (Visual Imitation Learning)** 方法。它的核心能力是从图像和关节状态直接预测动作序列，但策略不知道"任务是什么"——它只是复现训练数据中的行为模式。

从 ACT 升级到 VLA 的核心变化是引入语言条件:



如何将 ACT 改造为 VLA

从架构角度看，将 ACT 改造为 VLA 需要做以下扩展：

```
# 当前 ACT: 仅视觉输入
class ACTPolicy(nn.Module):
    def forward(self, image, joint_state):
        visual_features = self.vision_encoder(image)           # ResNet-18
        state_features = self.state_encoder(joint_state)       # MLP
        fused = torch.cat([visual_features, state_features], dim=-1)
        actions = self.transformer_decoder(fused)              # 输出 30×7
        return actions

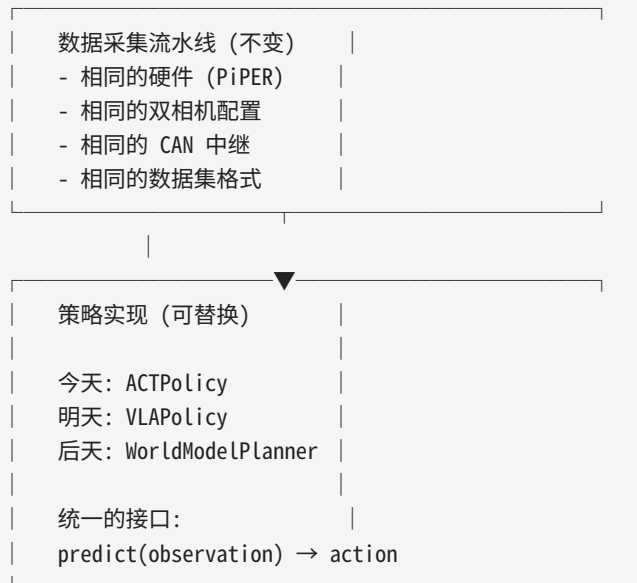
# 改造后的 VLA:
class VLAPolicy(nn.Module):
```

```
def forward(self, image, joint_state, language_instruction):
    visual_features = self.vision_encoder(image)          # SigLIP/ViT (更大的视觉 backbone)
    text_features = self.language_encoder(language)       # T5/Gemma (新增语言编码器)
    state_features = self.state_encoder(joint_state)     # MLP
    # 通过交叉注意力融合三种模态
    fused = self.multimodal_transformer(
        visual_features, text_features, state_features
    )
    actions = self.action_decoder(fused)
    return actions
```

这就是 RT-1 (Google Robotics)、RT-2 (Google DeepMind)、Octo (Stanford) 等模型的核心架构思路——在模仿学习的视觉-动作流水线上，增加一个语言编码分支，通过交叉注意力机制让语言语义影响动作生成。

LeRobot 的模块化设计使 ACT → VLA 的迁移变得简单

本项目的核心设计优势在于：**数据采集流水线与策略实现是解耦的。**



你今天在 PiPER 上采集的每一条数据（双相机 RGB 图像 + 关节状态 + 动作），只要再配上语言标签（如 "拿起红色方块"），就是一条标准的 VLA 训练数据。不需要换硬件，不需要改相机布局，不需要重新设计数据格式。

语言标签可以来自： - 录制时的语音输入（你已经集成了语音提示功能） - 手动标注（在 dataset metadata 中添加 "task": "pick up red block"） - 自动标注（通过 VLM 在回放视频时自动生成描述）

现在采集的数据就是"VLA-ready"

这是理解本项目价值的一个关键视角：你每天的 ACT 实验数据并不仅仅服务于当前的 ACT 训练。只要 episode 的数据结构是完整的（observation + action + metadata），这些数据在未来可以直接合并到更大的 VLA 训练集中。这就是为什么 LeRobot 的标准数据格式把 metadata 设为可选但强烈推荐——它为未来扩展预留了空间。

ACT 与世界模型的关系

ACT 是 Model-Free，世界模型是 Model-Based

这是强化学习和机器人学习中两个根本不同的范式：

Model-Free (ACT 所属的范式)	
策略 = $f(\text{observation}) \rightarrow \text{action}$	
直接学习"看到什么 $\rightarrow$ 做什么"的映射。 不构建环境模型，不做预测，不"思考"未来。	
代表方法：ACT, Diffusion Policy, Behavior Transformer, SAC, PPO, DrQ-v2	
Model-Based (世界模型所在的范式)	
世界模型 = $f(s_t, a_t) \rightarrow s_{t+1}$	
策略 = $\text{argmax}_a [\text{奖励}(s_t, a) + V(\text{世界模型}(s_t, a))]$	
先学一个内部的世界模型（环境的预测器）， 然后在世界模型内部做规划，选出最优动作。	
代表方法：Dreamer (v1/v2/v3), TD-MPC (v1/v2), MuZero, PlaNet	

为什么 Model-Free 目前赢了（在真实机器人上）

截至 2025-2026 年，真实机器人操作任务的 SOTA 方法仍然主要是 model-free 的（ACT, Diffusion Policy 等）。原因在于：

世界模型在视觉空间中积累预测误差。 每一步的微小预测误差会复合增长：


```
初始状态 s_0
→ 预测 s_1 (误差  $\epsilon$ )
  → 以 s_1 (有误差) 为起点预测 s_2 (误差  $2\epsilon$ )
    → 以 s_2 (有误差) 为起点预测 s_3 (误差  $\approx 4\epsilon$ )
      → ...
        → 第 30 步的预测已经与实际无关
```

对于 ACT 的 30 步 (`chunk_size = 30`) 这样的时间跨度，显式世界模型的视觉预测通常已经崩溃。没有可靠的未来预测，基于世界模型的规划就失去了基础。

世界模型需要比策略更多的训练数据。 一个行为克隆策略只需要学习 `observation` $\rightarrow$ `action` 的映射（监督信号是动作的 `L2 loss`）。一个世界模型需要学习 `(observation, action)  $\rightarrow$  next_observation` 的映射，输出空间是完整的观测（包括高维图像），这本质上是一个更难的问题。

ACT 的 Action Chunking 作为隐式世界模型

ACT 的一个精妙之处在于：它的 **action chunking** 在功能上等价于一种有限的、隐式的世界模型。

```
显式世界模型的预测过程：
s_0, a_0  $\rightarrow$  world_model  $\rightarrow$  s_1_pred
s_1_pred, a_1  $\rightarrow$  world_model  $\rightarrow$  s_2_pred
...

ACT 的动作分块预测：
observation_t  $\rightarrow$  ACT  $\rightarrow$  [a_t, a_{t+1}, a_{t+2}, ..., a_{t+29}]

这些动作序列本身编码了"接下来会发生什么"的信息——
不需要预测像素，但序列结构反映了任务的时间逻辑：

a_t     $\approx$  接近目标物体
a_{t+5}  $\approx$  到达抓取位置
a_{t+8}  $\approx$  闭合夹爪
a_{t+15}  $\approx$  抬起物体
a_{t+25}  $\approx$  移动到目标位置
a_{t+29}  $\approx$  释放物体
```

ACT 不做视觉预测，不做显式的状态转移，但它的 30 步动作序列**隐式地将时间结构融入了策略输出**。这既得到了时间抽象的好处（平滑、减少累积误差、抑制高频抖动），又避免了显式世界模型的视觉预测困难。

这是一种务实的折中——"我们不预测世界会变成什么样，但我们预测机械臂应该怎么动"。

但 ACT 不能做什么：长期规划

隐式世界模型（action chunking）的局限在于：它只能预测"按计划进行时"的动作，无法推理"如果出错了怎么办"。

考虑以下场景：

任务：抓取杯子并放到左边

情况 A（训练数据中的）：

接近杯子 → 合拢夹爪 → 杯子在夹爪中 → 移动到左边 → 释放
ACT 可以完美处理（这是训练分布内的）

情况 B（分布外）：

接近杯子 → 合拢夹爪 → 杯子滑落了！（夹爪中空无一物）
ACT 会继续执行后续动作："移动到左边 → 释放"
它"不知道"杯子已经掉了——因为它没有世界模型来检查
"夹爪中还有杯子吗？"

一个完整的世界模型 + 规划系统可以处理情况 B：在每一步根据预测的未来状态进行重新规划（re-planning）。如果世界模型预测"再这样下去杯子会掉"，规划器可以调整抓取力度或改变接近角度。

这是一种不同的能力层级： - **ACT (model-free)**：适用于"如果一切按预期进行"的 tasks，对分布外情况无抵抗力 - **世界模型 (model-based)**：理论上可以推理"如果...会怎样"，能够响应意外情况 - **中间地带**：ACT + 高频重规划（在 ACT 输出的 30 步中，每执行 5 步就用新观测重新调用 ACT 预测下一个 30 步）——这是一种折中，利用高频的 model-free 推理来近似 re-planning

为什么理解这层关系很重要

理解 ACT 和世界模型的关系，有助于理解本项目架构的演进空间：

今天：

PiPER 双臂 + LeRobot + ACT = 可靠的桌面级任务模仿学习

明天：

同样的硬件 + 同样的数据 + WorldModelPolicy =
能做在线规划的"更聪明"的机器人

后天：

同样的硬件 + 更多用户的共享数据 + VLA + World Model =
通用语言条件、能做长期规划、自适应意外情况的机器人

LeRobot 的设计保证了你不需要为这个演进重写任何数据采集代码——只需要替换 **Policy** 的实现。这是框架层面"为未来设计"的范例。

9.2 Action Chunking Transformer (ACT) 原理

核心思想

ACT 的核心创新在于：不是预测下一个时刻的单个动作，而是预测未来一段时间的完整动作轨迹（chunk）。

具体来说，传统的行为克隆策略在时刻 t 接收观测 o_t ，输出单个动作 a_t 。而 ACT 在时刻 t 接收观测 o_t ，一次性输出 K 个未来动作：

$$\pi(o_t) = [\hat{a}_t, \hat{a}_{t+1}, \hat{a}_{t+2}, \dots, \hat{a}_{t+K-1}]$$

其中 K 称为 `chunk_size`，典型值为 30（在 30 fps 下对应 1 秒的动作序列）。

为什么要预测多步动作？

- 减少高频抖动：**单步预测容易受到传感器噪声和模型微小扰动的影响，导致输出在相邻帧之间发生剧烈变化。多步预测天然地编码了动作的时间平滑性——chunk 内的动作序列是模型在同一次推理中生成的，彼此之间是协调的。
- 利用时间一致性：**机器人操作中的连续动作高度相关。例如执行一个抓取动作，整个过程中各关节的运动轨迹通常是一条平滑的曲线。一次性预测整条曲线比逐帧预测每个点更能保证轨迹的连贯性。
- 降低推理频率需求：**如果 `n_action_steps` 设置为 30，意味着策略推理一次后，可以将输出的 30 步动作逐帧发送给机器人执行整整 1 秒，而无需每帧都进行推理。这大大降低了对推理速度的要求。不过在实践中，为了获得更好的控制效果，通常 `n_action_steps` 会结合 `temporal ensemble` 使用。

网络架构

ACT 的网络结构可以拆解为三个主要模块：

输入模块：

- 腕部图像 (wrist) → ResNet-18 backbone → 图像特征 (512维)
- 全局图像 (global) → ResNet-18 backbone → 图像特征 (512维)
- 关节位置 (obs.state) → MLP → 关节特征 (512维)

处理模块 (Transformer Encoder + Decoder)：

- 图像特征 + 关节特征 + 位置编码 → Transformer Encoder
- K 个可学习 query tokens → Transformer Decoder (cross-attend to encoder output)

输出模块：

- K 个动作向量，每个 7 维 (joint1..6 + gripper)

整个数据流可以描述为以下步骤：

步骤一：视觉特征提取。两个相机各自拍摄的 $480 \times 640 \times 3$ 图像分别送入一个 ResNet-18 网络。每个 ResNet 的输入经过 conv1 到 layer4 的逐层处理后，得到一张空间尺寸缩小的特征图，再经过全局平均池化 (Global Average Pooling) 压缩为一个 512 维的向量。两个相机的特征向量拼接 (concatenate) 后得到 1024 维的视觉特征。

步骤二：本体感知特征提取。当前的 7 维关节位置通过一个线性层投影到 512 维的空间中，与视觉特征的维度对齐。

步骤三：位置编码。在进入 Transformer Encoder 之前，每个 token 都需要加上位置编码以保留空间或序列信息：

- 对于一维的 token（如 latent、robot state），使用可学习的 Embedding 作为位置编码。
- 对于二维的图像特征图，使用正弦位置编码 (2D Sinusoidal Position Embedding)，对每个像素位置 (x, y) 生成唯一的编码向量。这种编码方式借鉴了 Transformer 论文中的思路：对偶数维度使用正弦函数，对奇数维度使用余弦函数，不同维度具有不同的频率。

步骤四：Transformer Encoder。Encoder 由 4 层 Transformer Encoder Layer 堆叠而成。每层包含：
- Multi-head Self-Attention (8 个头，每个头 64 维)
- Feed-forward Network (将 512 维扩展到 3200 维，再压缩回 512 维)
- Layer Normalization + Residual Connection

Encoder 对所有输入 token (latent + state + 各像素的图像特征) 进行 self-attention，让模型学习不同模态之间的交互关系——例如图像中的"杯子"应该和关节状态中的"夹爪靠近杯子"产生关联。

步骤五：Transformer Decoder。Decoder 的输入是 K 个可学习的 query token (decoder_pos_embed)，每个 query 对应未来一个时间步的动作预测。Decoder 通过 cross-attention 机制从 Encoder 的输出中"查询"相关信息。具体来说：

- Self-Attention: query 之间互相通信，确保预测的动作序列内部保持一致。
- Cross-Attention: query 去 attend Encoder 的输出，从中提取与当前时间步最相关的视觉和状态信息。

Decoder 只有 1 层（这是一个历史遗留问题——原始 ACT 实现宣称有 7 层但因代码 bug 实际上只用到了第 1 层，LeRobot 的实现保留了这个设定）。

步骤六：动作回归头。Decoder 的每个输出 token（维度 512）通过一个线性层 (action_head) 投影到动作空间（7 维），得到最终的 $K \times 7$ 的动作 chunk。

ResNet-18 Backbone

ACT 选择 ResNet-18 作为视觉 backbone，这并非因为 ResNet-18 是"最好"的视觉模型，而是出于精确的工程考虑：

- **轻量**：ResNet-18 只有约 11M 参数，远小于 ResNet-50 (25M) 或 ViT (86M+)。在实际部署中，推理速度直接影响控制回路的延迟。
- **ImageNet 预训练**：通过加载 `ResNet18_Weights.IMAGENET1K_V1` 预训练权重，ResNet-18 已经具备了通用的视觉特征提取能力——它在 ImageNet 上学会了识别边缘、纹理、形状等基础视觉元素，这些能力可以直接迁移到机器人操作场景中。
- **冻结 BatchNorm**：在 LeRobot 的实现中，ResNet 的 BatchNorm 层被替换为 `FrozenBatchNorm2d` (`norm_layer=FrozenBatchNorm2d`)，意味着在训练过程中 BatchNorm 的统计量始终使用 ImageNet 预训练的值，不会根据机器人数据更新。这有助于在小数据集上稳定训练。
- **可选的膨胀卷积**：`replace_final_stride_with_dilation` 参数允许将 ResNet 最后一层的 2x2 stride 替换为膨胀卷积 (dilated convolution)，以保留更高的空间分辨率。这在对位置敏感的精细操作任务中可能会有帮助，但默认保持关闭以降低计算量。

Transformer 详解

Transformer 是 ACT 的核心处理模块。理解它对于理解 ACT 为什么有效至关重要。

Self-Attention 的本质。Self-Attention 允许输入序列中的每个元素 (token) 与序列中的所有其他元素进行交互。在 ACT 中：

- Encoder 的 Self-Attention 让不同来源的信息——来自腕部相机的像素特征、来自全局相机的像素特征、当前关节状态——彼此"沟通"。例如，腕部相机中看到"夹爪旁边有一个红色积木"，这个视觉 token 可以和"关节 4 的角度是 45 度"这个状态 token 通过 attention 建立关联。
- Decoder 的 Self-Attention 让不同时间步的 query 之间相互协调。例如，第 5 步的 query 知道第 4 步预测了什么动作，从而保证动作序列的时序一致性。

Cross-Attention。Decoder 中的 Cross-Attention 是实现"条件生成"的关键。Decoder 的 query (代表"我想知道第 t 步该做什么") 去 attend Encoder 的所有输出 token，从中提取相关的信息。注意力权重 $\alpha_{i,j}$ 表示第 i 个 query 对第 j 个 encoder token 的关注程度：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 Q 来自 Decoder query， K 和 V 来自 Encoder 输出。 $\sqrt{d_k}$ 是缩放因子，防止点积过大导致 softmax 梯度消失。

VAE (Variational Auto-Encoder)。ACT 在训练时使用了一个额外的 Transformer（称为 VAE Encoder，不要和主 Transformer Encoder 混淆）来引入变分目标。VAE Encoder 的作用是：

1. 将未来的真实动作序列编码为一个低维的 latent 向量（32 维）。
2. 训练时，主 Transformer 的解码器基于这个 latent 向量来重建动作序列。
3. 同时，最小化 latent 分布与标准正态分布之间的 KL 散度，使 latent 空间更加规整。

这样做的目的是让模型不仅仅学习“给定观测，输出动作”的确定性映射，还能学习到动作序列的概率分布——这有助于处理多模态问题：同一个观测下可能有多种合理动作，VAE 的随机采样可以覆盖多种模式。

然而，VAE 相关代码在推理时并不使用（推理时 latent 设置为全零向量），因此 VAE 更像是一个训练正则化手段而非推理时的必要组件。`use_vae` 参数可以关闭 VAE。

损失函数

ACT 的损失函数由两部分组成：

L1 损失 (Smooth L1 / Huber)。与标准的行为克隆使用 MSE (L2 Loss) 不同，ACT 使用 L1 Loss：

$$\mathcal{L}_{L1} = \frac{1}{K \cdot d} \sum_{k=0}^{K-1} \sum_{j=0}^{d-1} |\hat{a}_{t+k}^{(j)} - a_{t+k}^{(j)}|$$

其中 K 是 `chunk_size`， $d=7$ 是动作维度。使用 L1 Loss 而非 L2 Loss 的原因是：L1 对异常值 (outlier) 不敏感。在人类演示数据中，偶尔会有突兀的大幅动作（如突然松开夹爪），L2 Loss 会将这些异常值的影响放大（平方效应），而 L1 Loss 的梯度是常数，不会被个别异常值主导。

在代码中，L1 损失的计算还会考虑 padding 掩码：数据集中不同 episode 的长度不同，短的 episode 会被 padding 到统一长度，这些 padding 位置不参与损失计算：

```
l1_loss = (
    F.l1_loss(batch[ACTION], actions_hat, reduction="none") * ~batch["action_is_pad"].unsqueeze(-1)
).mean()
```

KL 散度损失 (仅在 `use_vae=True` 时生效)：

$$\mathcal{L}_{KL} = -\frac{1}{2} \sum_{j=1}^L \left(1 + \log \sigma_j^2 - \mu_j^2 - \frac{\mu_j^2}{\sigma_j^2} \right)$$

其中 $L=32$ 是 latent 维度, μ 和 $\log \sigma^2$ 是 VAE Encoder 输出的分布参数。KL 散度衡量 latent 分布与标准正态分布 $\mathcal{N}(0, I)$ 的差异。总损失为:

$$\mathcal{L} = \mathcal{L}_{L1} + \beta \cdot \mathcal{L}_{KL}$$

其中 β 是 `kl_weight`, 默认值为 10.0。

关键超参数

以下是在 PiPER 项目中使用的典型 ACT 超参数配置:

参数	典型值	含义
<code>chunk_size</code>	30	每次推理预测多少步未来动作
<code>n_action_steps</code>	30	每次推理后实际执行多少步 ($\leq$ <code>chunk_size</code>)
<code>learning_rate</code>	1e-5	AdamW 优化器的学习率
<code>optimizer_weight_decay</code>	1e-4	权重衰减系数
<code>batch_size</code>	8	每次训练的 batch 大小, 取决于 GPU 显存
<code>steps</code>	20000	总训练步数
<code>n_encoder_layers</code>	4	Transformer Encoder 的层数
<code>n_decoder_layers</code>	1	Transformer Decoder 的层数
<code>dim_model</code>	512	Transformer 的隐藏维度
<code>n_heads</code>	8	Multi-head Attention 的头数
<code>dim_feedforward</code>	3200	Feed-forward 层的扩展维度
<code>latent_dim</code>	32	VAE 的 latent 空间维度
<code>dropout</code>	0.1	Transformer 中的 Dropout 率
<code>kl_weight</code>	10.0	KL 散度在总损失中的权重
<code>n_obs_steps</code>	1	每次推理使用多少个历史观测帧
<code>temporal_ensemble_coeff</code>	0.01	时间集成的指数衰减系数

9.3 时间集成 (Temporal Ensemble)

问题：Chunk 边界的不连续性

虽然 ACT 一次性预测一个完整的动作 chunk (30 步)，但在实际部署中，我们可能希望在 chunk 之间重新推理以获得更好的控制效果。假设 `n_action_steps=1` (每帧都推理)：

- 在 $t=0$ 时刻，策略输出 chunk $\mathbf{A} = [a_0, a_1, \dots, a_{29}]$ ，执行 a_0 。
- 在 $t=1$ 时刻，策略输入新的观测，输出新 chunk $\mathbf{B} = [b_0, b_1, \dots, b_{29}]$ 。

问题在于： a_1 (来自第一次推理，预测的是时刻 1 的动作) 和 b_0 (来自第二次推理，预测的也是时刻 1 的动作) 可能不相等。由于两次推理的输入观测略有不同，或者模型本身的不确定性，这两个预测值之间会有差异。如果我们直接在当前帧无缝切换到新 chunk 的第一个动作，就可能产生动作跳变 (jump)。

解决：加权平均重叠时间步

时间集成 (Temporal Ensemble) 的核心思想是：对同一时刻来自不同推理的多组预测进行指数加权平均。这正是 ACT 原论文中 Algorithm 2 描述的方法。

具体而言，对于时刻 t 的实际执行动作，它是所有“覆盖了时刻 t 的预测 chunk”中对应位置动作的加权平均：

$$\text{executed_action}[t] = \frac{\sum_i w_i \cdot \text{predicted_action}_i[t]}{\sum_i w_i}$$

其中 i 是推理的索引 ($i=0$ 表示最新的一次推理)，权重 w_i 按指数衰减：

$$w_i = \exp(-\lambda \cdot i)$$

λ 是 `temporal_ensemble_coeff` (默认 0.01)。注意这个系数的含义是： i 越大代表推理越旧，`temporal_ensemble_coeff` 为正时，旧推理的权重更高 ($\exp(-0.01 \cdot i)$ 接近 1 时权重更大)。这看似反直觉——为什么旧推理权重要更高？原论文的实验表明，过高的新推理权重会削弱 action chunking 的时间平滑优势，让动作序列变得过于 responsive 而失去连贯性。

在线计算算法

在代码实现中 (`ACTTemporalEnsembler` 类)，时间集成并非在每次推理时重新计算整个加权平均 (那样需要缓存所有历史预测 chunk，内存开销太大)，而是使用在线递推方式：

```
class ACTTemporalEnsembler:
    def __init__(self, temporal_ensemble_coeff, chunk_size):
```



```

self.chunk_size = chunk_size
# 预计算所有权重和累积权重
self.ensemble_weights = torch.exp(-temporal_ensemble_coeff * torch.arange(chunk_size))
self.ensemble_weights_cumsum = torch.cumsum(self.ensemble_weights, dim=0)

def update(self, actions):
    if self.embled_actions is None:
        # 第一次推理：直接存储整个 chunk
        self.embled_actions = actions.clone()
        self.embled_actions_count = torch.ones((chunk_size, 1))
    else:
        # 在线更新：将现有 ensemble 乘以旧权重，加入新预测乘以新权重
        # （实际实现有更精细的处理，包括处理 chunk 的截断和拼接）
        ...

    # 弹出并返回第一个动作
    action = self.embled_actions[:, 0]
    self.embled_actions = self.embled_actions[:, 1:]
    return action

```

使用条件

要启用时间集成，需要同时满足：

- `temporal_ensemble_coeff` 不为 `None`（典型值 0.01）。
- `n_action_steps` 必须等于 1——因为时间集成需要每帧都进行推理以形成 ensemble。当 `n_action_steps > 1` 时策略内置的 action queue 接管（参见下文“动作队列”）。

动作队列 (Action Queue)

当不使用时间集成时（`temporal_ensemble_coeff` 为 `None`），ACT 使用更简单的动作队列机制：推理一次得到 K 步动作后，全部放入一个队列，然后逐帧弹出执行，直到队列耗尽再触发下一次推理。这适用于对推理延迟敏感的场景——推理一次管 30 步，大大降低了 GPU 占用。

9.4 训练流程

server_train_act_from_dataset.sh 详解

PiPER 项目提供了一个完整的训练脚本 `server_train_act_from_dataset.sh`，用于在训练服务器上启动 ACT 训练。下面逐段解析该脚本。

1. 环境配置

```

PIPER_ROOT="${PIPER_ROOT:-/root/autodl-tmp/Piper}"
LEROBOT_DIR="${LEROBOT_DIR:-${PIPER_ROOT}/lerobot_piper-piper}"
DATASET_DIR="${DATASET_DIR:-}"
OUTPUT_BASE="${OUTPUT_BASE:-${PIPER_ROOT}/outputs/train}"

```

```
JOB_NAME="${JOB_NAME:-piper_act}"
OUTPUT_DIR="${OUTPUT_DIR:-${OUTPUT_BASE}/${JOB_NAME}_${date +%Y%m%d_%H%M%S}}"
```

脚本使用 Bash 的 `${var:-default}` 语法，为每个关键路径提供默认值。`OUTPUT_DIR` 自动加入时间戳以避免多次训练相互覆盖。核心可调参数通过环境变量传入：

```
POLICY_DEVICE="${POLICY_DEVICE:-cuda}"
BATCH_SIZE="${BATCH_SIZE:-8}"
STEPS="${STEPS:-20000}"
CHUNK_SIZE="${CHUNK_SIZE:-30}"
N_ACTION_STEPS="${N_ACTION_STEPS:-30}"
VIDEO_BACKEND="${VIDEO_BACKEND:-pyav}"
PRETRAINED_BACKBONE_WEIGHTS="${PRETRAINED_BACKBONE_WEIGHTS:-ResNet18_Weights.IMAGENET1K_V1}"
```

2. 数据集验证

训练开始前，脚本通过一段内联 Python 代码验证数据集是否可以正确加载：

```
from lerobot.datasets.lerobot_dataset import LeRobotDataset

ds = LeRobotDataset(
    repo_id="${DATASET_REPO_ID}",
    root="${DATASET_DIR}",
    video_backend="${VIDEO_BACKEND}",
)
print("num_frames:", ds.num_frames)
print("num_episodes:", ds.num_episodes)
print("fps:", ds.fps)
sample = ds[0]
print("action:", tuple(sample["action"].shape))
print("state:", tuple(sample["observation.state"].shape))
for key in sorted(k for k in sample if k.startswith("observation.images.")):
    print(key + ":", tuple(sample[key].shape))
```

这段代码输出数据集的帧数、episode 数、帧率，以及第一个样本中各模态的形状。如果数据集的 `meta/info.json` 缺失或格式不符合预期，这一步会报错并中止训练——避免了训练到一半才发现数据问题的情况。

3. 训练命令

```
lerobot-train \
  --policy.type=act \
  --policy.device="${POLICY_DEVICE}" \
  --policy.push_to_hub=false \
  --policy.chunk_size="${CHUNK_SIZE}" \
  --policy.n_action_steps="${N_ACTION_STEPS}" \
  --dataset.repo_id="${DATASET_REPO_ID}" \
  --dataset.root="${DATASET_DIR}" \
```

```
--dataset.video_backend="${VIDEO_BACKEND}" \
--batch_size="${BATCH_SIZE}" \
--steps="${STEPS}" \
--log_freq="${LOG_FREQ}" \
--save_freq="${SAVE_FREQ}" \
--eval_freq="${EVAL_FREQ}" \
--num_workers="${NUM_WORKERS}" \
--output_dir="${OUTPUT_DIR}" \
--job_name="${JOB_NAME}" \
--wandb.enable=false
```

各参数的含义：

- `--policy.type=act`：指定使用 ACT 策略。
- `--policy.device=cuda`：在 GPU 上训练。
- `--policy.push_to_hub=false`：不自动推送到 HuggingFace Hub（本地训练场景）。
- `--dataset.repo_id` 和 `--dataset.root`：分别指定数据集在 HuggingFace Hub 上的标识符和本地存储路径。
- `--wandb.enable=false`：默认关闭 Weights & Biases 日志（可以按需开启）。

ResNet 预训练权重的处理有一条特殊逻辑：

```
if [[ "${PRETRAINED_BACKBONE_WEIGHTS}" == "null" || ... ]]; then
  ARGS+=(--policy.pretrained_backbone_weights=null)
else
  ARGS+=(--policy.pretrained_backbone_weights="${PRETRAINED_BACKBONE_WEIGHTS}")
fi
```

这允许用户选择从头训练 ResNet（传入 `null`）或使用 ImageNet 预训练权重（默认）。

训练监控

训练过程中，以下参数控制日志和模型保存的频率：

- `--log_freq=100`：每 100 步输出一次训练指标（loss、学习率等）。
- `--save_freq=5000`：每 5000 步保存一个 checkpoint。
- `--eval_freq=0`：默认为 0 表示不在训练过程中进行评估（评估通常离线进行）。
- 输出目录结构：

```
outputs/train/piper_act_20250101_120000/ |—— checkpoints/ # 模型权重和配置文件 |
|—— 0005000/ | |—— 0010000/ | |—— ... |—— logs/ # 训练日志
```

每个 checkpoint 目录包含：

- `config.json`：策略的完整配置
- `model.safetensors`：模型权重
- `preprocessor.json`：预处理 pipeline 配置（归一化参数等）
- `postprocessor.json`：后处理 pipeline 配置（反归一化参数等）

训练配置的底层机制

在 LeRobot 框架中，`TrainPipelineConfig`（定义在 `src/lerobot/configs/train.py`）是整个训练流程的总配置类。它使用 `draccus` 库将命令行参数与 `dataclass` 字段绑定，实现了类型安全的参数解析。训练 pipeline 配置的核心字段包括：

```
@dataclass
class TrainPipelineConfig(HubMixin):
    dataset: DatasetConfig
    policy: PreTrainedConfig | None = None
    output_dir: Path | None = None
    job_name: str | None = None
    resume: bool = False
    seed: int | None = 1000
    num_workers: int = 4
    batch_size: int = 8
    steps: int = 100_000
    eval_freq: int = 20_000
    log_freq: int = 200
    save_freq: int = 20_000
```

当 `use_policy_training_preset=True` 时，优化器和学习率调度器的配置直接从策略类中获取（`self.policy.get_optimizer_preset()` 和 `self.policy.get_scheduler_preset()`），确保训练超参数与策略设计者的建议保持一致。对于 ACT，`get_optimizer_preset` 返回 `AdamWConfig(lr=1e-5, weight_decay=1e-4)`，`get_scheduler_preset` 返回 `None`（不使用学习率调度器）。

9.5 策略部署

`piper_act_safe_rollout.py` 详解

训练完成后，需要将策略部署到真实机器人上执行推理。PiPER 项目提供了 `piper_act_safe_rollout.py` 脚本，其名称中的 "safe" 暗示了其核心设计理念——**安全第一，先验证后执行**。

整体架构

脚本的执行流程可以概括为以下几个阶段：

初始化阶段：

1. 解析命令行参数 (`parse_args`)
2. 验证 checkpoint 完整性 (`require_checkpoint`)
3. 加载 ACT 策略和预处理/后处理 pipeline (`load_policy`)
4. 创建 `PiperFollower` 实例并连接从臂 (`make_robot + robot.connect`)
5. 映射 robot 的键名到 LeRobot 标准名称 (`map_robot_keys_to_lerobot_features`)

推理循环（每步）：

1. robot.get_observation() → 原始观测（图像 + 关节位置）
2. 将原始观测转换为 LeRobot 格式 (raw_observation_to_observation)
3. 预处理 (preprocessor): 归一化 + 加 batch 维度 + 移入设备
4. policy.select_action(obs) → K 步动作 chunk
5. 后处理 (postprocessor.process_action): 反归一化 + 移回 CPU
6. limit_action(): 增量限幅安全检查
7. smooth_action(): EMA 平滑
8. 如果是执行模式, robot.send_action(smoothed) → 发送到从臂

干运行模式 (--dry-run)

脚本的默认行为是干运行 (dry run)——不添加 `--execute` 参数时，策略会完整地执行推理、限幅、平滑等所有计算，但**不会**将动作发送给机器人。终端输出如下格式：

```
[000] current=[ 30.00, 45.00, ..., 50.00] pred=[ 32.15, 46.20, ..., 51.30] limited=[ 32.15, 46.20, ..., 51.30] cmd=[ 32.15, 46.20, ..., 51.30]
```

每列的含义： - `current`：机械臂当前的实际关节位置 - `pred`：策略的原始预测值 - `limited`：经过增量限幅 (limit_action) 后的值 - `cmd`：经过 EMA 平滑 (smooth_action) 后的最终命令值

通过对比 `pred` 和 `cmd`，可以直观地评估限幅和平滑是否对策略输出产生了显著影响。如果两者差异很大，说明策略输出的幅度超出了安全阈值。

limit_action() — 增量限幅

增量限幅是保障安全的第一道防线。它限制每一步的动作变化量不超过预设阈值：

```
def limit_action(predicted, current, max_delta, max_gripper_delta, disable_gripper):
    limited = {}
    for key in ACTION_KEYS:
        delta_limit = max_gripper_delta if key == "gripper.pos" else max_delta
        abs_low, abs_high = (0.0, 100.0) if key == "gripper.pos" else (-100.0, 100.0)
        if key == "gripper.pos" and disable_gripper:
            limited[key] = float(np.clip(current[key], abs_low, abs_high))
            continue
        delta = np.clip(predicted[key] - current[key], -delta_limit, delta_limit)
        limited[key] = float(np.clip(current[key] + float(delta), abs_low, abs_high))
    return limited
```

工作原理： 1. 计算预测动作与当前位置的差值 $\Delta = a_{\text{pred}} - a_{\text{current}}$ 。 2. 将 Δ clamp 到 $[-\text{max_delta}, +\text{max_delta}]$ 范围内。 3. 最终动作 $a_{\text{cmd}} = a_{\text{current}} + \text{clamp}(\Delta)$ 。 4. 再对绝对值进行一次 clamp（关节角度的合法范围是 $[-100, 100]$ 度，夹爪是 $[0, 100]$ ）。

`--max-delta` 的默认值为 2.0（关节角度单位），这意味着每步（0.2 秒，如果 `control_fps=5`）关节最多移动 2 个单位。这样即使策略输出了一个极端的预测值（例如由于 chunk 切换时的 jump 或模型对异常观测的不合理响应），机器人也不会突然大幅度运动。

夹爪有独立的 `--max-gripper-delta` 参数（默认 3.0），因为夹爪的开合动作通常比关节转动更剧烈。此外，`--disable-gripper` 开关允许在测试手臂关节时固定夹爪位置，避免夹爪意外开合导致抓取的物体掉落。

smooth_action() — EMA 平滑

EMA（指数移动平均）是额外的平滑层，用于降低动作序列中的高频抖动：

```
def smooth_action(action, previous, alpha):
    if previous is None or alpha >= 1.0:
        return dict(action)
    if alpha <= 0.0:
        return dict(previous)
    return {key: alpha * action[key] + (1.0 - alpha) * previous[key] for key in ACTION_KEYS}
```

EMA 的递推公式为：

$$a_{\text{smooth}}^{(t)} = \alpha \cdot a_{\text{new}}^{(t)} + (1 - \alpha) \cdot a_{\text{smooth}}^{(t-1)}$$

其中 α 是 `--ema-alpha` 参数（默认 1.0 表示不启用平滑）。当 $\alpha < 1$ 时，当前执行的命令不仅是当前策略预测的结果，还“记忆”了上一步的命令—— α 越小，对历史命令的依赖越强，输出越平滑，但响应也越慢。

EMA 与 Temporal Ensemble 的区别：

- Temporal Ensemble 由策略内部实现（`ACTTemporalEnsembler`），在动作空间中对多个推理 chunk 的重叠部分进行加权平均——解决的是跨 chunk 的一致性問題。
- EMA 是在策略输出已经确定之后，在命令发送给机器人之前额外施加的时间平滑——解决的是帧间高频抖动问题。

两者可以同时使用，形成双层平滑。在 chunk 边界（即从队列中弹出下一帧动作时），EMA 能够显著减少相邻两帧之间的不连续性。

完整推理循环

推理循环的核心代码结构如下（简化版）：

```
previous_limited = None
period = 1.0 / args.control_fps # 控制周期，如 0.2 秒

for step in range(args.steps):
```

```

loop_start = time.perf_counter()

# 1. 获取观测
raw_obs = robot.get_observation()
obs = raw_observation_to_observation(raw_obs, lerobot_features,
                                     cfg.image_features, args.device)
obs = preprocessor(obs)

# 2. 策略推理
with torch.inference_mode():
    raw_action = policy.select_action(obs)
    action = postprocessor.process_action(raw_action)

# 3. 转换格式
predicted = action_tensor_to_dict(action)
current = current_state_from_obs(raw_obs)

# 4. 安全检查
limited = limit_action(predicted, current,
                      args.max_delta, args.max_gripper_delta,
                      args.disable_gripper)
smoothed = smooth_action(limited, previous_limited, args.ema_alpha)
previous_limited = smoothed

# 5. 执行（仅在 --execute 模式下）
if args.execute:
    robot.send_action(smoothed)

# 6. 帧率控制
elapsed = time.perf_counter() - loop_start
time.sleep(max(0.0, period - elapsed))

```

关键设计要点： - `torch.inference_mode()` 上下文管理器禁用梯度计算和 `autograd` 跟踪，减少推理时的内存和计算开销。 - `time.perf_counter()` 精确测量每步耗时，通过 `time.sleep` 补足到目标控制周期，保持稳定的控制帧率。 - `robot.disconnect(disable_torque=False)` 在脚本退出时（包括 `Ctrl+C` 中断）确保从臂不断电（保持当前姿态），防止因突然断电导致机械臂坠落。

9.6 为什么 ACT 适合双臂操作

双臂操作 (Bimanual Manipulation) 是机器人学习中最具挑战性的场景之一。ACT 的设计恰好针对了双臂操作的几个关键需求：

协调性： 双臂操作要求左右臂的动作严格协调——例如双手配合拧开瓶盖，左臂固定瓶身、右臂旋开瓶盖，两个动作在时序上有精确的依赖关系。如果只预测单步动作，模型需要隐式地学习这种协调性；而 ACT 预测完整的动作序列，可以在 `chunk` 内部通过 Decoder 的 Self-Attention 显式地协调不同时间步的动作。

时间尺度匹配：典型的操作子任务——如抓起物体、移动到目标位置、放下——通常持续 1 到数秒。`chunk_size=30`（在 30 fps 下为 1 秒）恰好覆盖了这些短时操作子任务的时间跨度。如果只需要 0.5 秒的子任务，30 步 chunk 提供了足够的余量；如果需要 2 秒的复杂操作，可以在 chunk 之间重新推理以更新策略。

推理频率降低：双臂操作的物理约束（惯性、摩擦、电机转速）意味着不需要每毫秒都调整控制指令。`n_action_steps=30` 配置下，模型推理一次可以管 1 秒的动作，远比双臂物理系统的时间常数长。这降低了对 GPU 推理延迟的要求，也减少了计算开销。

对演示噪声的鲁棒性：人类演示者在遥操作时不可避免地会产生手抖、犹豫、纠正等噪声。ACT 的 L1 Loss 和 action chunking 机制对这类噪声具有天然的鲁棒性——L1 不会被个别的突兀动作主导损失，而 chunking 让模型关注的是整段轨迹的"形状"而非每个点的精确值。

9.7 ACT 的局限性

尽管 ACT 在双臂操作任务中表现出色，但它并非没有局限：

数据需求大：ACT 通常需要 50-200 个 episode 的高质量演示才能学到可靠的行为。数据采集是遥操作机器人项目中最耗时的环节——每个 episode 可能需要数分钟的人工操作，而且需要覆盖不同的初始条件、物体位置和光照变化。数据不足时，ACT 学到的策略容易过拟合训练场景中的特定视觉模式。

对分布外 (Out-of-Distribution) 状态的泛化弱：ACT 是一个确定性策略（忽略 VAE 的随机采样），当机器人偏离训练数据中的状态分布时（例如物体被移动到了训练时从未见过的位置），策略的输出可能变得不可靠。这是所有模仿学习方法的通病——当策略逐步偏离演示轨迹时，它面临的是训练数据中从未出现过的观测。

不建模物理约束：ACT 的神经网络完全不理解物理世界的规律——它不知道机械臂有最大扭矩限制，不知道夹爪不能穿透物体，不知道桌面是一个刚性平面。这些约束全部依赖人类演示数据中的"常识"来隐式编码。当策略输出一个违反物理规律的动作时，唯一的安全保障是部署脚本中的 `limit_action()` 和 `smooth_action()` 等启发式规则。

只能复现见过的行为模式：如果训练数据中只有"从左向右推"的演示，ACT 无法自主学会"从右向左拉"。它不具备组合泛化或推理能力——不会有类似"既然学会了推，倒过来应该也能拉"的 generalize。这使得 ACT 更像是一个动作回放引擎（有条件的回放）而非一个真正"理解"任务的智能体。

长序列的 VAE 编码效率：32 维的 latent 向量需要编码 30 步 $\times$ 7 维 = 210 个浮点数的动作序列信息，压缩比接近 7:1。对于复杂的、变化丰富的动作序列，这个压缩可能导致信息丢失。

9.8 其他 LeRobot 支持的策略

LeRobot 框架不仅支持 ACT，还集成了多种策略算法，适用于不同的机器人和任务场景。以下是简要对比：

策略	类型	核心特点	适用场景
ACT	模仿学习	Action Chunking + Transformer + VAE	双臂操作、精细任务
Diffusion Policy	模仿学习	扩散模型生成动作分布，天然处理多模态	高精度位置控制、需要多模态输出的任务
TDMPC	模型预测控制	基于模型的规划，在隐空间中搜索最优动作序列	需要长期规划的任务、接触丰富的操作
VQ-BET	模仿学习	使用 Vector Quantization 对动作离散化，显式建模多模态	行为模式多样的任务（如多种抓取策略）
SmolVLA	视觉语言动作	跨任务泛化，支持语言指令	需要跨任务泛化、多任务学习的场景
SAC	强化学习	Actor-Critic 架构，最大化累积 reward	仿真环境中的训练（需要 reward 函数）

选择策略时的主要考量：

- 如果有高质量的演示数据且任务相对单一，**ACT** 是一个稳定和可靠的首选。
- 如果同一状态存在多种同样正确的动作（例如抓取不同部位），**Diffusion Policy** 或 **VQ-BET** 对多模态的处理更好。
- 如果任务涉及接触物理、需要精确的力控或轨迹规划，**TDMPC** 的模型预测能力可能更合适。
- 如果希望一个策略处理多种不同的任务，**SmolVLA** 的多任务能力值得尝试。

9.9 常见问题

策略输出抖动

现象： 机器人动作不流畅，关节在相邻帧之间来回振荡，尤其是在 chunk 边界处。

原因： 连续两次推理产生的 action chunk 在重叠位置不完全一致，或者模型本身对观测的微小变化过于敏感。

解决方案： 1. 降低 `--max-delta`（如从 2.0 降到 1.0），增加增量限幅的强度。2. 降低 `--ema-alpha`（如从 1.0 降到 0.5），启用更激进的 EMA 平滑。3. 在策略配置中启用 temporal ensemble（设置 `temporal_ensemble_coeff=0.01` 且 `n_action_steps=1`）。

策略不跟随

现象： 机械臂几乎不动，或者动作与人类期望完全不符。

可能原因： 1. 加载了错误的 checkpoint（例如不同任务或不同机器人的模型）。2. 数据集统计信息（归一化参数）与 checkpoint 不匹配。3. 相机没有正确初始化或图像格式不匹配。

排查步骤： 1. 检查 `checkpoint` 目录中是否包含 `config.json`、`model.safetensors`、`preprocessor.json`、`postprocessor.json` 四个文件——脚本中 `require_checkpoint()` 会验证这一点。2. 启用干运行模式（默认行为），观察 `pred` 列的值是否有意义——如果所有预测值都接近零或某个常数，可能是归一化/反归一化出了问题。3. 确认数据集使用的 `DATASET_REPO_ID` 与训练时一致。4. 确认相机分辨率（`--width`，`--height`）与训练时的分辨率一致。

Out of Memory (OOM)

现象： 训练时报错 `CUDA out of memory`。

原因： `batch_size` 或 `chunk_size` 超出了 GPU 显存容量。ACT 的内存占用主要来自： - 图像 batch ($\$B \times 2 \times 3 \times H \times W$) - Transformer 的激活值（与 $\$K$ 和序列长度成正比）

解决方案： 1. 减小 `--batch_size`（如从 8 降到 4 或 2）。2. 减小 `--chunk_size`（如从 30 降到 15）——但这会改变策略的语义，需要重新训练。3. 降低图像分辨率（如从 640x480 降到 320x240）。4. 如果 GPU 显存实在不足，可以使用 `--policy.device=cpu` 进行纯 CPU 训练（非常慢，仅用于验证流程）。

推理太慢

现象： 每步推理耗时过长，导致控制帧率无法达到设定值（如 5 fps）。

原因： ResNet-18 虽然是轻量 backbone，但在 CPU 上推理 640x480 的双相机图像仍然较慢；或者 GPU 推理时显存不足导致内存交换。

解决方案： 1. 使用 GPU 推理（`--device cuda`）而非 CPU。2. 降低相机分辨率（`--width 320 --height 240`），注意需要与训练时的分辨率一致，或重新训练。3. 增大 `n_action_steps`（如设为 30），让策略推理一次后使用较长时间的动作队列，减少推理频率。4. 如果图像预处理 pipeline 耗时长，可以将其与推理解耦到单独的线程中。

机械臂在退出时坠落

现象： 停止脚本后机械臂失去力矩 (torque) 而自然下落。

原因： `robot.disconnect()` 被调用时传入了 `disable_torque=True`，或者在异常退出时没有正确调用 `disconnect()`。

解决方案： - 脚本的 `finally` 块中使用了 `robot.disconnect(disable_torque=False)`，确保从臂在断开连接时保持力矩。不要修改这个参数。 - 如果机械臂仍然坠落，检查从臂的物理连接（CAN 总线线缆是否松动）和电源状态。

小结

本章从模仿学习的基本概念出发，深入讲解了 Action Chunking Transformer (ACT) 的原理、架构、训练流程和部署方法。ACT 通过 action chunking 机制解决了传统行为克隆中的复合误差问题，通过 Transformer 架构实现了多模态信息的融合和时序动作的协调生成，通过时间集成 (temporal ensemble) 保证了推理时的动作平滑性。

在实际应用中，ACT 需要对硬件安全有充分的考虑——本章介绍的 `piper_act_safe_rollout.py` 脚本通过干运行模式、增量限幅和 EMA 平滑三道防线，确保策略的输出在发送给物理机器人之前经过了充分的约束和过滤。

至此，本教程的理论部分完结。后续章节将聚焦于具体实验、性能分析和进阶优化。

附录 A：脚本参考手册

本附录为 PiPER 双臂机器人 + LeRobot 模仿学习流水线中所有核心脚本提供完整的参考文档。每个脚本均涵盖用途说明、参数列表、使用示例和关键注意事项。

A.1 scripts/piper_pc_relay_teleop.py — PC 中继遥操作脚本

用途

将主臂（leader）的关节指令通过 PC 实时转发给从臂（follower），实现主从遥操作。典型拓扑如下：

```
主臂  -> USB-CAN A -> can0 -> PC
从臂  -> USB-CAN B -> can1 -> PC
```

脚本从 `can0` 读取主臂关节目标，经过符号映射、偏移补偿、速率限制和安全限位后，将指令发送到 `can1` 上的从臂。**默认为干运行（dry-run）模式，不会驱动物理臂**，需要显式指定 `--execute` 才会实际发送控制指令。

环境要求

- 两条 CAN 总线均已激活（`ip link set can0 up / can1 up`），波特率 1 Mbps
- 主臂与从臂均已上电，且分别连接至对应的 USB-CAN 适配器
- `piper_sdk` 已安装，CAN 接口权限正常（通常需要 `sudo` 或 `udev` 规则）
- Python 解释器：`piper_sdk/piper/bin/python`（SDK 自带虚拟环境）

完整参数列表

参数	类型	默认值	说明
<code>--leader</code>	<code>str</code>	<code>"can0"</code>	连接主臂的 CAN 接口名称
<code>--follower</code>	<code>str</code>	<code>"can1"</code>	连接从臂的 CAN 接口名称
<code>--source</code>	<code>str</code>	<code>"auto"</code>	读取主臂关节数据的数据源。可选值： <code>auto</code> （优先 <code>control</code> 流，其次 <code>feedback</code> 流）、 <code>feedback</code> （仅读反馈帧）、 <code>control</code> （仅读控制帧）

参数	类型	默认值	说明
<code>--hz</code>	float	10.0	中继循环频率（Hz）。必须为正数
<code>--speed</code>	int	10	从臂运动速度百分比，写入 <code>MotionCtrl_2</code> 命令。取值范围 [0, 100]
<code>--max-step-deg</code>	float	1.0	单个控制周期内从臂每个关节的最大步进量（度）。用于速率限制，避免从臂突跳
<code>--mode-command-period</code>	float	1.0	重复发送 <code>MotionCtrl_2</code> 模式命令的间隔（秒）。设为 0 则禁用重复模式命令
<code>--gripper-hz</code>	float	5.0	夹爪指令的最大发送频率（Hz）。设为 0 表示不限频
<code>--print-period</code>	float	1.0	状态打印间隔（秒）。设为 0 禁用周期性打印
<code>--pause-file</code>	str	""	暂停文件路径。当该文件存在时暂停向从臂发送指令；删除后恢复发送
<code>--duration</code>	float	0.0	运行时长（秒）。设为 0 表示持续运行直到 Ctrl-C
<code>--signs</code>	str	"1,1,1,1,1,1"	每关节符号映射（6 个逗号分隔的浮点数）。例如 <code>-1</code> 表示反转该关节方向
<code>--offset-deg</code>	str	"0,0,0,0,0,0"	每关节偏移量（度），6 个逗号分隔的浮点数
<code>--include-gripper</code>	flag	False	是否同时中继夹爪目标
<code>--high-follow</code>	flag	False	启用高跟随模式（ <code>MotionCtrl_2</code> 的 <code>is_mit_mode=0xAD</code> ）
<code>--skip-follower-config</code>	flag	False	跳过初始模式/使能配置，不发送初始 <code>MotionCtrl_2</code> 和 <code>EnableArm</code> 命令
<code>--execute</code>	flag	False	实际发送指令 。不指定则脚本仅打印状态，不驱动物理臂（干运行模式）

使用示例

```
# 干运行（安全预览，不驱动物理臂）
python scripts/piper_pc_relay_teleop.py

# 实际执行遥操作，包含夹爪，高跟随模式
python scripts/piper_pc_relay_teleop.py \
    --leader can0 --follower can1 \
```

```
--hz 20 --speed 80 --max-step-deg 5.0 \  
--include-gripper --high-follow \  
--source control \  
--execute  
  
# 干运行, 反转关节 2 和关节 4 方向  
python scripts/piper_pc_relay_teleop.py --signs "1,-1,1,-1,1,1"
```

关键说明

- **安全机制**: 默认不执行 (dry-run) ; 关节限位由 `JOINT_LIMITS_MDEG` 和 `GRIPPER_LIMIT_UM` 定义; 速率限制通过 `--max-step-deg` 控制单步增量; `--pause-file` 提供外部暂停机制
- **数据源选择**: `auto` 模式优先读取控制帧 (`GetArmJointCtrl`) , 当控制帧无数据时回退到反馈帧 (`GetArmJointMsgs`) ; `control` 模式在主臂实际发送控制指令时更实时
- **退出行为**: 收到 `SIGINT / SIGTERM` 后, 如果处于执行模式, 脚本会向从臂发送待机命令 (`MotionCtrl_2(0x00, 0x01, 0)`) , 然后断开连接

A.2 scripts/piper_act_safe_rollout.py — ACT 策略安全部署脚本

用途

加载训练好的 ACT (Action Chunking Transformer) 策略, 读取机器人状态和相机图像, 预测动作, 并对从臂执行安全限幅后的动作命令。**默认模式为干运行 (dry-run)** : 加载策略、读取传感器、推理动作、打印结果, 但不向机械臂发送任何控制指令。

干运行模式说明

干运行 (`--execute` 未指定) 是本脚本的核心安全设计:

1. 加载完整的策略检查点 (配置文件 + 模型权重 + 预处理器/后处理器)
2. 连接从臂和 RealSense 相机
3. 每步读取当前观测 (关节位置 + 图像) , 运行策略推理
4. 计算预测动作, 应用 delta 限幅和 EMA 平滑
5. 打印当前状态、预测值和限幅后的指令, **不驱动物理臂**
6. 上下文内容

此模式用于: 验证检查点完整性、检查推理速度、评估动作范围合理性、在真实部署前做最后的 dry-run 验证。

完整参数列表

必需参数

参数	类型	默认值	说明
<code>--checkpoint</code>	str	<code>"../checkpoints/piper_act_smoke_10ep_001000"</code>	预训练模型目录路径，需包含 <code>config.json</code> 、 <code>model.safetensors</code> 、 <code>preprocessor.json</code> 、 <code>postprocessor.json</code>

硬件/相机参数

参数	类型	默认值	说明
<code>--port</code>	str	<code>"can1"</code>	从臂（执行策略输出的臂）的 CAN 接口
<code>--wrist-serial</code>	str	环境变量 <code>WRIST_REALSENSE_SERIAL</code> 或 <code>"238222076529"</code>	腕部 RealSense 相机序列号
<code>--global-serial</code>	str	环境变量 <code>GLOBAL_REALSENSE_SERIAL</code> 或 <code>"142122071524"</code>	全局 RealSense 相机序列号
<code>--width</code>	int	640	相机采集宽度
<code>--height</code>	int	480	相机采集高度
<code>--fps</code>	int	15	相机采集帧率

推理参数

参数	类型	默认值	说明
<code>--device</code>	str	<code>"cpu"</code>	策略推理设备。可选： <code>cpu</code> 、 <code>cuda</code>
<code>--steps</code>	int	20	控制迭代步数
<code>--control-fps</code>	float	5.0	执行循环频率（Hz）

安全限幅参数

参数	类型	默认值	说明
<code>--max-delta</code>	float	2.0	每步关节 1-6 最大归一化动作变化量

参数	类型	默认值	说明
<code>--max-gripper-delta</code>	float	3.0	每步夹爪最大归一化动作变化量
<code>--disable-gripper</code>	flag	False	保持夹爪当前位置不变（仅测试臂关节时使用）
<code>--ema-alpha</code>	float	1.0	动作 EMA（指数移动平均）平滑系数。1.0 禁用平滑；0.3-0.6 可减少 ACT chunk 边界抖动。取值范围 (0, 1]

执行控制

参数	类型	默认值	说明
<code>--execute</code>	flag	False	实际向从臂发送限幅动作。不指定则为干运行
<code>--calibrate</code>	flag	False	允许 <code>PiperFollower.connect()</code> 执行 <code>parking()</code> 校准。首次测试建议关闭

使用示例

```
# 干运行验证（推荐首次使用）
python scripts/piper_act_safe_rollout.py \
  --checkpoint /path/to/checkpoints/piper_act_10ep_001000 \
  --device cuda --steps 30

# 实际执行，带 EMA 平滑和安全限幅
python scripts/piper_act_safe_rollout.py \
  --checkpoint /path/to/checkpoints/piper_act_10ep_001000 \
  --device cuda --steps 50 --execute \
  --max-delta 1.5 --ema-alpha 0.5 \
  --port can1

# 仅测试臂关节，禁用夹爪
python scripts/piper_act_safe_rollout.py \
  --checkpoint /path/to/checkpoints --execute \
  --disable-gripper --max-delta 1.0
```

安全特性

- **默认干运行**：不指定 `--execute` 则绝不驱动物理臂
- **Delta 限幅**：`--max-delta` 限制单步关节目标变化，`--max-gripper-delta` 限制夹爪变化。动作被 clamp 到归一化范围 [-100, 100]（关节）和 [0, 100]（夹爪）
- **EMA 平滑**：`--ema-alpha` 通过指数移动平均消除 ACT chunk 边界的动作抖动

- **检查点完整性校验**：启动时检查 `config.json`、`model.safetensors`、`preprocessor.json`、`postprocessor.json` 四个文件是否齐全
- **异常退出保护**：Ctrl-C 后保持 torque 不禁用（`disable_torque=False`），防止突然掉力
- **范围校验**：`--ema-alpha` 必须在 (0, 1] 范围内，超出则报错退出

A.3 `scripts/piper_reset_to_initial.py` — 机械臂复位到初始位置

用途

将一个或多个 PiPER 机械臂移动到统一的初始关节姿态，用于任务间复位。默认目标姿态为零位姿（6 个关节均为 0 度）。

完整参数列表

参数	类型	默认值	说明
<code>--ports</code>	<code>nargs="+"</code>	<code>["can0", "can1"]</code>	需要复位的 CAN 端口列表
<code>--target-deg</code>	<code>str</code>	<code>"0.0,0.0,0.0,0.0,0.0,0.0"</code>	目标关节角度（度），6 个逗号分隔的浮点数
<code>--gripper-um</code>	<code>int</code>	<code>0</code>	目标夹爪位置（微米）。0 表示闭合
<code>--speed</code>	<code>int</code>	<code>50</code>	运动速度百分比
<code>--hz</code>	<code>float</code>	<code>10.0</code>	控制发送频率
<code>--duration</code>	<code>float</code>	<code>5.0</code>	命令发送持续时间（秒）
<code>--high-follow</code>	<code>flag</code>	<code>False</code>	启用高跟随模式（ <code>mit_flag=0xAD</code> ）
<code>--master-home-ports</code>	<code>nargs="*"</code>	<code>["can0"]</code>	需要额外发送 <code>ReqMasterArmMoveToHome(1)</code> 主臂归位命令的端口列表。设为空值可禁用

使用示例

```
# 双臂同时复位到零位
python scripts/piper_reset_to_initial.py --ports can0 can1

# 仅复位从臂，使用自定义目标位置
python scripts/piper_reset_to_initial.py \
    --ports can1 \
    --target-deg "10.0,-20.0,45.0,0.0,0.0,0.0" \
    --speed 30 --duration 3.0
```

```
# 主臂发出归位命令
python scripts/piper_reset_to_initial.py \
  --ports can0 can1 \
  --master-home-ports can0
```

关键说明

- 脚本在发送目标指令前会先发送 `ModeCtrl` 和 `MotionCtrl_2` 配置命令，然后尝试最多 20 次使能（`EnableArm`），直到所有臂均已使能
- `--master-home-ports` 中的端口会额外接收 `ReqMasterArmMoveToHome(1)`，这是 PiPER SDK 中主臂的专用归位协议
- 需确保 CAN 接口状态为 UP，否则脚本会抛出错误并给出激活命令
- 复位期间持续指定时长内按指定频率发送关节和夹爪目标指令

A.4 scripts/realsense_live_view.py — 双 RealSense 实时预览

用途

实时显示一个或两个 Intel RealSense 彩色相机流，支持 Web 浏览器查看和 OpenCV 窗口查看两种模式。可用于验证相机安装位置、对焦和帧率。

完整参数列表

参数	类型	默认值	说明
<code>--wrist-serial</code>	str	"238222076529"	腕部相机序列号
<code>--global-serial</code>	str	"142122071524"	全局相机序列号
<code>--only</code>	str	"both"	显示哪些相机。可选： <code>both</code> 、 <code>wrist</code> 、 <code>global</code>
<code>--width</code>	int	640	彩色流宽度
<code>--height</code>	int	480	彩色流高度
<code>--fps</code>	int	30	彩色流帧率
<code>--timeout-ms</code>	int	1000	取帧超时（毫秒）
<code>--layout</code>	str	"horizontal"	双画面布局。可选： <code>horizontal</code> （水平并排）、 <code>vertical</code> （垂直堆叠）

参数	类型	默认值	说明
<code>--viewer</code>	str	"web"	查看模式。可选： <code>web</code> （浏览器 MJPEG 流）、 <code>opencv</code> （本地 OpenCV 窗口）
<code>--host</code>	str	"127.0.0.1"	Web 模式下的 HTTP 绑定地址
<code>--port</code>	int	8765	Web 模式下的 HTTP 端口
<code>--snapshot-dir</code>	str	"../camera_check/ live_view"	截图保存目录

截图功能

- **Web 模式：**访问 `http://<host>:<port>/snapshot` 可触发截图，保存为 `realsense_live_<timestamp>.jpg`
- **OpenCV 模式：**按 `s` 键保存截图，文件名格式同上，保存至 `--snapshot-dir` 指定目录
- 截图输出目录会自动创建（不存在时）

使用示例

```
# 浏览器预览（默认），打开 http://127.0.0.1:8765
python scripts/realsense_live_view.py

# OpenCV 窗口预览，仅腕部相机
python scripts/realsense_live_view.py --viewer opencv --only wrist

# 自定义分辨率和帧率
python scripts/realsense_live_view.py --width 1280 --height 720 --fps 15

# 垂直布局双画面
python scripts/realsense_live_view.py --layout vertical
```

控制快捷键（OpenCV 模式）

按键	功能
<code>q / Esc</code>	退出
<code>s</code>	保存截图

关键说明

- Web 模式启动一个轻量 HTTP 服务器，页面通过 MJPEG 流推送实时画面，局域网内其他设备也可访问（将 `--host` 设为 `0.0.0.0`）

- 画面左上角显示相机名称和实时帧率（FPS）
- 如果相机暂时无数据，会显示 `waiting for <name>` 提示，并显示上一帧或黑画面

A.5 `scripts/compare_piper_settings.py` — 双臂参数对比

用途

只读方式读取并对比两个 CAN 端口上 PiPER 臂的关键设置和状态参数。不使能电机，不发送运动指令。

完整参数列表

参数	类型	默认值	说明
<code>--ports</code>	<code>nargs=2</code>	<code>["can0", "can1"]</code>	需要对比的两个 CAN 端口（必须恰好 2 个）
<code>--settle</code>	<code>float</code>	<code>0.5</code>	连接后等待 SDK 数据流稳定的时间（秒）

对比内容

脚本读取并逐行对比以下参数：

参数	来源	说明
<code>status_hz</code>	<code>GetArmStatus</code>	状态反馈频率
<code>joint_hz</code>	<code>GetArmJointMsgs</code>	关节反馈流频率
<code>ctrl_hz</code>	<code>GetArmJointCtrl</code>	控制帧频率
<code>control_mode</code>	<code>arm_status</code>	控制模式
<code>arm_status</code>	<code>arm_status</code>	臂状态
<code>mode_feed</code>	<code>arm_status</code>	模式反馈
<code>teach_status</code>	<code>arm_status</code>	示教状态
<code>motion_status</code>	<code>arm_status</code>	运动状态
<code>error_code</code>	<code>arm_status</code>	错误码
<code>enable_status</code>	<code>GetArmEnableStatus</code>	使能状态（6 个 bool 值列表）

此外还单独打印每个臂的详细信息：关节反馈位置（mdeg）、关节控制目标位置（mdeg）、ModeCtrl 设置、CtrlCode151、末端速度加速度参数、碰撞保护等级反馈、夹爪示教器参数反馈。

输出格式

```
===== can0 =====
status_hz=100.0 joint_hz=100.0 ctrl_hz=100.0
control_mode=...
...
[gripper_teaching]
...

===== can1 =====
...

===== SUMMARY =====
OK status_hz: can0=100.0 | can1=100.0
DIFF joint_hz: can0=100.0 | can1=0.0
OK control_mode: ...
...
WARN can1 has no joint feedback stream.
NOTE neither arm currently exposes a control-frame stream.
```

- **OK**：两侧参数一致
- **DIFF**：两侧参数不一致，需关注
- **WARN**：某个臂无关节反馈流（可能未上电、CAN 未激活或连接问题）
- **NOTE**：两个臂均无控制帧暴露（正常，取决于 SDK 当前控制模式）

使用示例

```
# 对比默认的 can0 和 can1
python scripts/compare_piper_settings.py

# 指定自定义端口
python scripts/compare_piper_settings.py --ports can0 can2

# 增加等待稳定时间
python scripts/compare_piper_settings.py --settle 1.0
```

关键说明

- 完全只读：通过 ArmParamEnquiryAndConfig 发送参数查询请求（query=0x01, 0x02, 0x04），不改变任何设置
- 适用于故障排查：快速判断两个臂的控制模式、使能状态、反馈流是否一致

- 典型使用场景：遥操作前验证主臂和从臂处于匹配状态；记录故障前的参数快照

A.6 scripts/identify_piper_can_roles.py — CAN 端口角色识别

用途

只读方式识别每个 CAN 端口上连接的 PiPER 臂是主臂 (leader/master) 还是从臂 (follower/slave)。不发送任何运动指令。

工作原理

根据 PiPER SDK 通信协议：

- **主臂 (leader/master)**：主要发送控制帧，可通过 `GetArmJointCtrl()` 和 `GetArmGripperCtrl()` 看到活跃的控制消息
- **从臂 (follower/slave)**：主要发送反馈帧，可通过 `GetArmJointMsgs()` 和 `GetArmGripperMsgs()` 看到活跃的反馈消息

脚本连接每个 CAN 端口，等待 SDK 数据流积累一段时间，然后分别打印反馈消息和控制消息的完整内容。根据哪个端口有活跃的控制消息即可判断主臂所在。

完整参数列表

参数	类型	默认值	说明
<code>--ports</code>	<code>nargs="+"</code>	<code>["can0", "can1"]</code>	需要识别的 CAN 端口列表
<code>--seconds</code>	<code>float</code>	<code>2.0</code>	每个端口连接后等待数据积累的时间（秒）

使用示例

```
# 识别默认的 can0 和 can1 上的角色
python scripts/identify_piper_can_roles.py

# 更长的等待时间以获得更准确的结果
python scripts/identify_piper_can_roles.py --ports can0 can1 --seconds 5.0
```

输出格式

```
===== can0 =====
[feedback] joint:
<JointMsg detail>
```

```
[feedback] gripper:
<GripperMsg detail>
[control] joint:
<JointCtrl detail>
[control] gripper:
<GripperCtrl detail>
[status]:
<Status detail>
```

```
===== can1 =====
```

```
[feedback] joint:
...
[control] joint:
...
```

Interpretation hint: if one port has active control messages and the other has active feedback messages, the control-heavy port is likely the leader/master and the feedback-heavy port is likely the follower/slave.

关键说明

- 完全只读：仅连接和读取，不使能电机，不发送运动命令
- 输出包含底层 SDK 消息对象的 `__repr__` 完整打印，包括消息计数器等详细信息
- 如果某个端口连接失败（CAN 未激活、设备未连接），会打印异常信息但不中断其余端口检测

A.7 scripts/server_train_act_from_dataset.sh — 服务端 ACT 训练脚本

用途

在训练服务器上，从本地 LeRobot 数据集目录启动 ACT 策略训练。需在 `lerobot` conda 环境下运行，通过环境变量配置训练参数。

所有环境变量

路径类

环境变量	默认值	说明
<code>PIPER_ROOT</code>	<code>/root/autodl-tmp/Piper</code>	项目根目录
<code>LEROBOT_DIR</code>	<code>\${PIPER_ROOT}/lerobot_piper-piper</code>	LeRobot 代码目录
<code>DATASET_DIR</code>	(必填，无默认值)	数据集目录，需包含 <code>data/</code> 、 <code>meta/</code> 、 <code>videos/</code>

环境变量	默认值	说明
DATASET_NAME	DATASET_DIR 的 basename	数据集名称
DATASET_REPO_ID	local/\${DATASET_NAME}	数据集仓库 ID
OUTPUT_BASE	\${PIPER_ROOT}/outputs/train	训练输出根目录
OUTPUT_DIR	\${OUTPUT_BASE}/\${JOB_NAME}_<timestamp>	具体输出目录（自动加时间戳）
JOB_NAME	pipec_act	训练任务名称

训练超参数

环境变量	默认值	说明
POLICY_DEVICE	cuda	策略训练/推理设备
BATCH_SIZE	8	批次大小
STEPS	20000	总训练步数
LOG_FREQ	100	日志打印频率（步）
SAVE_FREQ	5000	检查点保存频率（步）
EVAL_FREQ	0	评估频率（步），0 表示不评估
NUM_WORKERS	4	数据加载工作进程数
CHUNK_SIZE	30	ACT 的 chunk 大小
N_ACTION_STEPS	30	每次执行的动作步数
VIDEO_BACKEND	pyav	视频解码后端
PRETRAINED_BACKBONE_WEIGHTS	ResNet18_Weights.IMAGENET1K_V1	预训练骨干网络权重。设为 <code>null / None / none</code> 则随机初始化

训练前检查

脚本在启动训练前执行以下检查：

1. DATASET_DIR 是否为空（必填）
2. LEROBOT_DIR 目录是否存在
3. 数据集目录下 meta/info.json 是否存在
4. lerobot-train 命令是否可用（验证 conda 环境）

5. 通过 Python 脚本验证数据集可加载，并打印 `num_frames`、`num_episodes`、`fps` 及首帧 `action`、`state`、图像特征 `shape`

使用示例

```
# 在服务器上运行
DATASET_DIR=/path/to/dataset/piper_task_v1 \
STEPS=50000 \
BATCH_SIZE=16 \
bash scripts/server_train_act_from_dataset.sh
```

关键说明

- 使用 `set -euo pipefail`，任何命令失败都会中止脚本
- 训练参数通过 `lerobot-train` 命令行传递，`wandb` 默认关闭（`--wandb.enable=false`）
- 输出目录自动加时间戳以避免覆盖
- 训练完成后打印最新检查点文件列表

A.8 `scripts/package_lerobot_dataset.sh` — 数据集打包脚本

用途

将本地 LeRobot 数据集目录打包为 `.tar.gz` 压缩包并计算 SHA-256 校验和，方便传输到训练服务器。

环境变量

环境变量	默认值	说明
<code>DATASET_DIR</code>	（必填，无默认值）	待打包的数据集目录，需包含 <code>data/</code> 、 <code>meta/</code> 子目录和 <code>meta/info.json</code>
<code>OUT_DIR</code>	<code>/home/liyuqi/Documents/Piper/exported_datasets</code>	压缩包输出目录

输入

- `DATASET_DIR` 下的完整数据集目录（包含 `data/`、`meta/` 及可选的 `videos/`）

输出

- `<OUT_DIR>/<数据集名>.tar.gz` — 压缩包
- `<OUT_DIR>/<数据集名>.tar.gz.sha256` — SHA-256 校验文件

使用示例

```
DATASET_DIR=/home/liyuqi/Documents/Piper/datasets/local/piper_task_v1 \
bash scripts/package_lerobot_dataset.sh

# 自定义输出目录
DATASET_DIR=/path/to/dataset OUT_DIR=/mnt/share/datasets \
bash scripts/package_lerobot_dataset.sh
```

关键说明

- 使用 `set -euo pipefail` 严格错误处理
- 打包前检查 `meta/info.json`、`data/`、`meta/` 是否齐全
- 使用 `readlink -f` 解析绝对路径，避免相对路径问题
- 打包后打印文件大小和 SHA-256 校验值

A.9 `record_with_pc_relay.sh` — 主数据采集编排器

用途

这是整个数据采集流程的核心编排脚本。它同时启动 PC 中继遥操作进程（`piper_pc_relay_teleop.py`）和 LeRobot 录制进程（`lerobot-record`），实现"主臂遥操作 + 从臂状态记录"的完整数据采集流水线。

系统拓扑

```
主臂  -> USB-CAN A -> can0 -> [PC 中继] -> can1 -> 从臂
      ↑
      lerobot-record 读取从臂观测 + 相机图像
      → 存储为 LeRobot 数据集（从臂状态=action）
```

中继进程将主臂指令发送到从臂驱动运动；`lerobot-record` 仅读取从臂的观测和相机数据，并将从臂状态作为动作（action）存储。

所有环境变量

路径类

环境变量	默认值	说明
PIPER_ROOT	/home/liyuqi/Documents/Piper	项目根目录
LEROBOT_DIR	\${PIPER_ROOT}/lerobot_piper-piper	LeRobot 代码目录
CONDA_ENV	lerobot	Conda 环境名称
VOICE_PROMPT_DIR	\${LEROBOT_DIR}/audio_prompts/en-US-AriaNeural	语音提示音频目录

相机配置

环境变量	说明
ROBOT_CAMERAS	相机配置 YAML 字符串。为空时自动尝试从 camera_config_realsense.env 读取
WRIST_REALSENSE_SERIAL	腕部 RealSense 序列号（由 camera_config_realsense.env 提供）
GLOBAL_REALSENSE_SERIAL	全局 RealSense 序列号（由 camera_config_realsense.env 提供）

中继参数

环境变量	默认值	说明
LEADER_CAN	can0	主臂 CAN 接口
FOLLOWER_CAN	can1	从臂 CAN 接口
RELAY_HZ	20	中继循环频率
RELAY_SPEED	100	从臂运动速度百分比
RELAY_MAX_STEP_DEG	10	最大关节步进量（度）
RELAY_SOURCE	control	主臂数据源（control / feedback / auto）
RELAY_INCLUDE_GRIPPER	1	是否中继夹爪（1=是）
RELAY_HIGH_FOLLOW	1	高跟随模式（1=启用）
RELAY_MODE_COMMAND_PERIOD	1.0	模式命令重复间隔（秒）
RELAY_GRIPPER_HZ	5	夹爪指令频率
RELAY_PRINT_PERIOD	0	状态打印间隔（0=禁用）

环境变量	默认值	说明
RELAY_PAUSE_FILE	/tmp/piper_pc_relay.pause	暂停文件路径

数据集录制参数

环境变量	默认值	说明
DATASET_BASE_DIR / DATASET_ROOT	\${PIPER_ROOT}/datasets	数据集存储根目录
DATASET_NAME	piper_pc_relay_smoke	数据集名称
DATASET_FPS	30	录制帧率
NUM_EPISODES	5	录制轮次（episode）数
EPISODE_TIME_S	20	每轮时长（秒）
RESET_TIME_S	10	轮间复位时间（秒）
TASK	"Teleoperate the follower PiPER with the leader PiPER"	任务描述文本
DISPLAY_DATA	false	是否显示录制数据
PUSH_TO_HUB	false	是否推送到 Hugging Face Hub
PLAY_SOUNDS	false	是否播放提示音
HF_USER	hf auth whoami 结果或 local	Hugging Face 用户名
DATASET_REPO_ID	\${HF_USER}/\${DATASET_NAME}	数据集仓库 ID
DATASET_OUTPUT_ROOT	\${DATASET_BASE_DIR}/\${DATASET_REPO_ID}	实际输出目录
AUTO_SUFFIX_DATASET_ROOT	1	自动在已存在的输出目录名后加时间戳

行为控制

环境变量	默认值	说明
PHASE_BEEP	true	是否播放阶段提示音
MANUAL_STEP	true	手动步进模式（需按键确认每轮开始）

环境变量	默认值	说明
AUTO_RESET_ARMS	true	每轮间自动复位从臂
AUTO_RESET_LEADER	false	每轮间是否也复位主臂

复位参数

环境变量	默认值	说明
RESET_SPEED	50	复位运动速度
RESET_DURATION	5	复位命令持续时间（秒）
RESET_HZ	10	复位控制频率
RESET_TARGET_DEG	0,0,0,0,0,0	复位目标关节角度

执行流程

1. 加载 conda 环境（lerobot）
2. 读取 camera_config_realsense.env（如果存在）解析相机序列号
3. 验证相机配置中无占位符（WRIST_SERIAL / GLOBAL_SERIAL）
4. 将相机帧率统一为 DATASET_FPS
5. 设置 trap 清理函数（退出时自动杀 relay 进程、删 pause 文件）
6. 启动 piper_pc_relay_teleop.py 后台进程（带 --execute）
7. 等待 2 秒让中继稳定
8. 配置 LEROBOT_PHASE_BEEP（阶段提示音）
9. 配置 LEROBOT_RESET_COMMAND（自动复位命令，可选）
10. 启动 lerobot-record 录制进程
11. 录制完毕后进入 cleanup 清理

使用示例

```
# 无相机冒烟测试
cd /home/liyuqi/Documents/Piper/lerobot_piper-piper
bash record_with_pc_relay.sh

# 带双 RealSense 相机的真实数据采集
ROBOT_CAMERAS='{ wrist: {type: realsense, serial: WRIST_SERIAL, width: 640, height: 480, fps: 30}, global:
{type: realsense, serial: GLOBAL_SERIAL, width: 640, height: 480, fps: 30} }' \
NUM_EPISODES=10 EPISODE_TIME_S=60 \
RELAY_SPEED=80 RELAY_HZ=30 \
    bash record_with_pc_relay.sh

# 自定义数据集名称和低位关注参数
DATASET_NAME=piper_pour_water_v2 \
NUM_EPISODES=30 EPISODE_TIME_S=30 \
```

```
RELAY_MAX_STEP_DEG=5.0 \  
bash record_with_pc_relay.sh
```

关键说明

- **安全机制**：退出时自动执行 `cleanup`，发送 `SIGTERM` 给中继进程（中继进程收到后发送待机命令并断开连接），删除 `pause` 文件
- **复位机制**：通过设置 `LEROBOT_RESET_COMMAND` 和 `LEROBOT_RELAY_PAUSE_FILE` 环境变量，让 `lerobot-record` 在轮间自动暂停中继、复位机械臂、再恢复中继
- **相机占位符检查**：如果 `ROBOT_CAMERAS` 中仍包含 `WRIST_SERIAL` 或 `GLOBAL_SERIAL` 占位符字面量，脚本会中止并提示先配置真实序列号
- **键盘控制**（在 `lerobot-record` 录制过程中）：右键 = 完成当前 episode 进入复位阶段；左键 = 重新录制当前 episode；Esc = 停止录制

A.10 根目录编号 Shell 脚本（`1__init_can.sh` ~ `8__run_client.sh`）

这些脚本是流水线的快速入口，按编号顺序执行可实现从 CAN 初始化到策略部署的完整流程。每个脚本均为单行或数行命令的快捷封装。

`1__init_can.sh` — CAN 总线初始化

角色：激活 CAN0 总线，波特率 1 Mbps。

```
bash ~/piper_sdk/piper_sdk/can_activate.sh can0 1000000
```

使用时机：每次系统启动后、连接机械臂之前执行。如果使用双臂（`can0` + `can1`），需额外执行 `can1` 的初始化。

`2__find_camera.sh` — 查找可用相机

角色：列出系统中所有可用的相机设备，并打开文件管理器查看之前捕获的图像。

```
source ~/miniconda3/etc/profile.d/conda.sh && conda activate lerobot  
python ~/lerobot/src/lerobot/find_cameras.py  
nautilus ~/lerobot/outputs/captured_images
```

使用时机：连接新相机后验证系统是否识别；确认相机索引号。

3__set_camera.sh — 设置相机属性

角色：调整相机曝光、增益等属性参数。

```
python ./src/lerobot/camera_prop.py
```

使用时机：调整相机参数以获得最佳图像质量。通常在 2__find_camera.sh 确认相机可用之后执行。

4__record.sh — 录制数据集（独立采集）

角色：使用 lerobot-record 录制数据集，采用**独立采集**模式（无 PC 中继遥操作）。机械臂由人手动操作（直接示教），lerobot 读取从臂状态和相机图像作为数据。

关键参数示例（脚本中的硬编码值）： - 机器人类型： piper_follower，端口 can0 - 2 个 OpenCV 相机（top 索引 0， left 索引 4），640x480@30fps - 50 个 episode

```
source ~/miniconda3/etc/profile.d/conda.sh && conda activate lerobot
HF_USER=$(hf auth whoami | head -n 1)
lerobot-record \
  --robot.type=piper_follower --robot.port=can0 \
  --robot.cameras="{ ... }" --robot.id=black \
  --dataset.num_episodes=50 \
  --dataset.single_task="Grab the yellow car and put in the box" \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/piper_pick_yellow_car
```

使用时机：手动示教数据采集模式，适用于不需要主从遥操作的简单任务。

5__replay.sh — 数据集重放

角色：从已有数据集中回放录制的动作到机械臂，用于验证数据集质量。

```
source ~/miniconda3/etc/profile.d/conda.sh && conda activate lerobot
HF_USER=$(hf auth whoami | head -n 1)
lerobot-replay \
  --robot.type=piper_follower --robot.port=can0 \
  --robot.id=black \
  --dataset.repo_id=${HF_USER}/piper_pick_yellow_car \
  --dataset.episode=0
```

使用时机：数据采集后验证动作序列是否合理；演示训练数据中的标准轨迹。

6__train.sh — 训练策略 (SmolVLA)

角色：在服务器上使用 `lerobot-train` 训练 SmolVLA 策略。

关键参数（脚本中硬编码）：- 策略类型： `smolvla` - 设备： `cuda`，批次大小 64 - 训练步数 20000，评估频率每 5000 步

```
lerobot-train \
  --policy.device=cuda --policy.type=smolvla \
  --dataset.repo_id=wego-hansu/piper_pick_yellow_car \
  --dataset.video_backend=pyav --batch_size=64 \
  --steps=20000 --eval_freq=5000 \
  --output_dir=outputs/train/piper_smolvla_pick_yellow_cars_new \
  --job_name=piper_smolvla_yellow_car --wandb.enable=false
```

使用时机：数据集准备好后，在 GPU 服务器上启动训练。注意这是 SmolVLA 训练模板，需替换 `dataset.repo_id` 和 `output_dir` 为实际值。

7__run_server.sh — 启动策略服务器

角色：启动 LeRobot 的 `policy_server.py`，在本地端口提供策略推理服务。

```
python src/lerobot/scripts/server/policy_server.py --host=127.0.0.1 --port=8080
```

使用时机：训练完成后，在部署机械臂的机器上启动策略服务器，等待 `robot_client` 连接。

8__run_client.sh — 启动机器人客户端

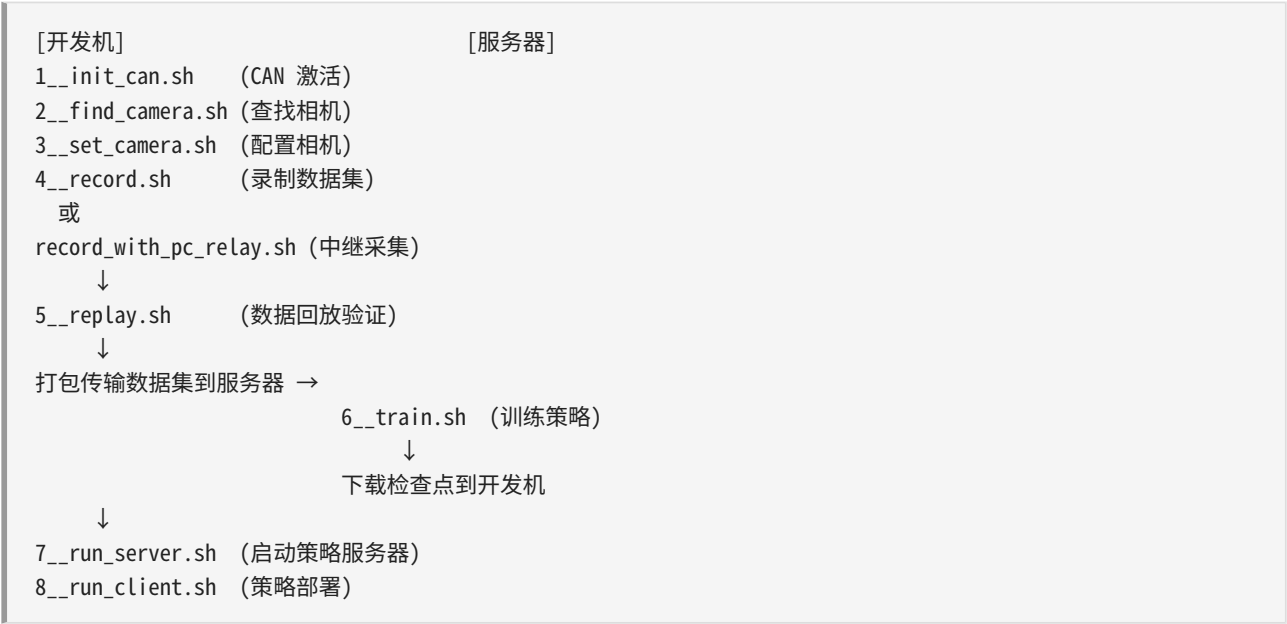
角色：启动 LeRobot 的 `robot_client.py`，连接策略服务器并驱动机器人执行策略推理后的动作。

关键参数示例：- 策略服务器地址： `127.0.0.1:8080` - 机器人类型： `piper_follower`，端口 `can0` - 2 个 OpenCV 相机，任务描述，策略类型 `smolvla` - 策略路径：本地训练的检查点目录

```
python src/lerobot/scripts/server/robot_client.py \
  --server_address=127.0.0.1:8080 \
  --robot.type=piper_follower --robot.port=can0 \
  --robot.id=black --robot.cameras="{ ... }" \
  --task="Grasp the object and put it in the bin" \
  --policy_type=smolvla \
  --pretrained_name_or_path=outputs/train/.../checkpoints/020000/pretrained_model \
  --policy_device=cuda --actions_per_chunk=50 \
  --chunk_size_threshold=0.5 \
  --aggregate_fn_name=weighted_average \
  --debug_visualize_queue_size=True
```


使用时机：策略服务器运行后，在连接机械臂的机器上启动客户端执行策略部署。注意需替换 `pretrained_name_or_path` 为实际检查点路径。

编号脚本使用流程概览



文档版本：附录 A v1.0 最后更新：2026-06-22 对应代码库： `lerobot_piper-piper`

附录 B CAN ID 快速参考

B.1 CAN ID 总表

PiPER 机械臂的 CAN 总线通信使用标准帧格式（11-bit 标识符），波特率为 1 Mbps。下表列出了所有 CAN ID，按功能分为反馈指令、控制指令、配置指令和驱动器信息四大类。

B.1.1 主动反馈指令（机械臂 → 上位机）

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x2A1	反馈	ARM_STATUS_FEEDBACK	Byte 0-7: 机械臂状态信息	机械臂状态反馈：电机使能状态、错误码、当前控制模式等
0x2A2	反馈	ARM_END_POSE_FEEDBACK_1	Byte 0-7: 末端位姿 X, Y (双 int32)	机械臂末端位姿反馈：X (4B, int32, 单位 0.001mm), Y (4B)
0x2A3	反馈	ARM_END_POSE_FEEDBACK_2	Byte 0-7: 末端位姿 Z, RX (双 int32)	末端位姿反馈：Z (4B, int32, 单位 0.001mm), RX (4B, 单位 0.001°)
0x2A4	反馈	ARM_END_POSE_FEEDBACK_3	Byte 0-7: 末端位姿 RY, RZ (双 int32)	末端位姿反馈：RY (4B, 单位 0.001°), RZ (4B, 单位 0.001°)
0x2A5	反馈	ARM_JOINT_FEEDBACK_12	Byte 0-3: J1 (int32), Byte 4-7: J2 (int32)	关节 1、2 反馈角度，单位 0.001°
0x2A6	反馈	ARM_JOINT_FEEDBACK_34	Byte 0-3: J3 (int32), Byte 4-7: J4 (int32)	关节 3、4 反馈角度，单位 0.001°
0x2A7	反馈	ARM_JOINT_FEEDBACK_56	Byte 0-3: J5 (int32), Byte 4-7: J6 (int32)	关节 5、6 反馈角度，单位 0.001°
0x2A8	反馈	ARM_GRIPPER_FEEDBACK	Byte 0-3: 夹爪角度 (int32, 单位 0.001°), Byte 4-5: 扭矩	夹爪状态反馈

CAN ID (Hex)	方向	消息名称	数据格式	说明
			(uint16, 单位 0.001N·m), Byte 6: 状态码, Byte 7: 保留	

偏移说明：反馈指令 ID 可通过 0x470 指令整体偏移。基 ID 0x2Ax 可偏移为 0x2Bx（偏移值 0x10）或 0x2Cx（偏移值 0x20），用于主从模式下区分双臂的反馈数据。

B.1.2 运动控制指令（上位机 → 机械臂）

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x150	控制	ARM_MOTION_CTRL_1	Byte 0: 急停 (uint8), Byte 1: 轨迹控制 (uint8), Byte 2: 拖动示教 (uint8), Byte 3: 轨迹索引 (uint8), Byte 4-5: NameIndex (uint16), Byte 6-7: CRC16 (uint16)	运动控制指令 1: 急停/轨迹/示教控制
0x151	控制	ARM_MOTION_CTRL_2	Byte 0: 控制模式 (uint8), Byte 1: MOVE 模式 (uint8), Byte 2: 速度百分比 0~100 (uint8), Byte 3: MIT 模式 (uint8), Byte 4: 离线轨迹停留时间 (uint8, 单位 s), Byte 5: 安装位置 (uint8)	运动控制指令 2: 模式选择与参数设置
0x152	控制	ARM_MOTION_CTRL_CARTESIAN_1		笛卡尔空

CAN ID (Hex)	方向	消息名称	数据格式	说明
			Byte 0-3: X (int32, 单位 0.001mm), Byte 4-7: Y (int32)	间坐标: X, Y 指令值
0x153	控制	ARM_MOTION_CTRL_CARTESIAN_2	Byte 0-3: Z (int32, 单位 0.001mm), Byte 4-7: RX (int32, 单位 0.001°)	笛卡尔空间坐标: Z, RX 指令值
0x154	控制	ARM_MOTION_CTRL_CARTESIAN_3	Byte 0-3: RY (int32, 单位 0.001°), Byte 4-7: RZ (int32, 单位 0.001°)	笛卡尔空间坐标: RY, RZ 指令值
0x155	控制	ARM_JOINT_CTRL_12	Byte 0-3: J1 (int32, 单位 0.001°), Byte 4-7: J2 (int32)	关节 1、2 目标角度
0x156	控制	ARM_JOINT_CTRL_34	Byte 0-3: J3 (int32, 单位 0.001°), Byte 4-7: J4 (int32)	关节 3、4 目标角度
0x157	控制	ARM_JOINT_CTRL_56	Byte 0-3: J5 (int32, 单位 0.001°), Byte 4-7: J6 (int32)	关节 5、6 目标角度
0x158	控制	ARM_CIRCULAR_PATTERN_COORD_NUM_UPDATE_CTRL	Byte 0: 指令序号 (uint8, 0x01=起点, 0x02=中点, 0x03=	圆弧模式坐标

CAN ID (Hex)	方向	消息名称	数据格式	说明
			终点), Byte 1-7: 圆弧坐标数据	序号更新指令
0x159	控制	ARM_GRIPPER_CTRL	Byte 0-3: 角度 (int32, 单位 0.001°), Byte 4-5: 扭矩 (uint16, 单位 0.001N·m, 范围 0~5000), Byte 6: 状态码 (uint8), Byte 7: 设零标志 (uint8)	夹爪控制指令

偏移说明：控制指令 ID 可通过 0x470 指令整体偏移。基 ID 0x15x 可偏移为 0x16x（偏移值 0x10）或 0x17x（偏移值 0x20），用于主从模式下区分双臂的控制数据。

B.1.3 MIT 关节控制指令（基于 V1.5-2 版本）

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x15A	控制	ARM_JOINT_MIT_CTRL_1	J1 MIT 控制参数	关节 1 MIT 模式控制指令
0x15B	控制	ARM_JOINT_MIT_CTRL_2	J2 MIT 控制参数	关节 2 MIT 模式控制指令
0x15C	控制	ARM_JOINT_MIT_CTRL_3	J3 MIT 控制参数	关节 3 MIT 模式控制指令
0x15D	控制	ARM_JOINT_MIT_CTRL_4	J4 MIT 控制参数	关节 4 MIT 模式控制指令
0x15E	控制	ARM_JOINT_MIT_CTRL_5	J5 MIT 控制参数	关节 5 MIT 模式控制指令
0x15F	控制	ARM_JOINT_MIT_CTRL_6	J6 MIT 控制参数	关节 6 MIT 模式控制指令

B.1.4 参数配置与查询指令

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x470		ARM_MASTER_SLAVE_MODE_CONFIG		

CAN ID (Hex)	方向	消息名称	数据格式	说明
	控制		Byte 0: 联动配置 (uint8), Byte 1: 反馈偏移 (uint8), Byte 2: 控制偏移 (uint8), Byte 3: 联动偏移 (uint8)	主从模式设置: 联动配置、ID 偏移
0x471	控制	ARM_MOTOR_ENABLE_DISABLE_CONFIG	Byte 0: 电机序号 (uint8, 1~7 或 0xFF), Byte 1: 使能标志 (uint8, 0x01=失能, 0x02=使能)	电机使能/失能指令
0x472	控制	ARM_SEARCH_MOTOR_MAX_SPD_ACC_LIMIT	Byte 0: 电机序号 (uint8, 1~7), Byte 1: 查询内容 (uint8, 0x01=角度限制/最大速度, 0x02=最大加速度)	查询电机角度限制、最大速度、加速度限制
0x473	反馈	ARM_FEEDBACK_CURRENT_MOTOR_ANGLE_LIMIT_MAX_SPD	Byte 0-1: 最小角度 (int16, 单位 0.1°), Byte 2-3: 最大角度 (int16, 单位 0.1°), Byte 4-5: 最大速度 (int16,	反馈电机角度限制与最大速度

CAN ID (Hex)	方向	消息名称	数据格式	说明
			单位 0.001rad/s), Byte 6-7: 保留	
0x474	控制	ARM_MOTOR_ANGLE_LIMIT_MAX_SPD_SET	Byte 0: 电机序号 (uint8, 1~7), Byte 1-2: 最小角度 (int16, 单位 0.1°), Byte 3-4: 最大角度 (int16), Byte 5-6: 最大速度 (int16, 单位 0.001rad/s), Byte 7: 保留	设置电机角度限制与最大速度
0x475	控制	ARM_JOINT_CONFIG	Byte 0: 电机序号 (uint8, 1~7), Byte 1: 设零点 (uint8, 0xAE=生效), Byte 2: 加速度生效标志 (uint8, 0xAE), Byte 3-4: 最大加速度 (uint16, 单位 0.01rad/s ² , 范围 0~500), Byte 5: 清除错误 (uint8, 0xAE), Byte 6-7: 保留	关节设置指令: 零点、加速度、错误清除
0x476	反馈	ARM_INSTRUCTION_RESPONSE_CONFIG	Byte 0: 应答码 (uint8,	

CAN ID (Hex)	方向	消息名称	数据格式	说明
			0x50=ACK), Byte 1: 指令类型, Byte 2: 序号/数据, Byte 3-7: 附加数据	指令 应答 确认
0x477	控制	ARM_PARAM_ENQUIRY_AND_CONFIG	Byte 0: 参数查询 (uint8), Byte 1: 参数设置 (uint8), Byte 2: 0x48X 反馈设置 (uint8, 0x01=关, 0x02=开), Byte 3: 负载生效标志 (uint8, 0xAE), Byte 4: 末端负载设置 (uint8)	参数 查询 与设置: 末端 速度/ 加速度、 关节 限位、 碰撞 防护
0x478	反馈	ARM_FEEDBACK_CURRENT_END_VEL_ACC_PARAM	Byte 0-3: 末端速度限制 (uint32), Byte 4-7: 末端加速度限制 (uint32)	反馈 当前 末端 速度/ 加速度 参数
0x479	控制	ARM_END_VEL_ACC_PARAM_CONFIG	Byte 0-3: 末端速度 (uint32), Byte 4-7: 末端加速度 (uint32)	设置 末端 速度/ 加速度 参数

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x47A	控制	ARM_CRASH_PROTECTION_RATING_CONFIG	Byte 0: 碰撞保护等级 (uint8, 0x00~0x03)	碰撞防护等级设置
0x47B	反馈	ARM_CRASH_PROTECTION_RATING_FEEDBACK	Byte 0: 当前碰撞保护等级 (uint8)	碰撞防护等级反馈
0x47C	反馈	ARM_FEEDBACK_CURRENT_MOTOR_MAX_ACC_LIMIT	Byte 0-1: 最大加速度 (int16, 单位 0.01rad/s ²)	反馈当前电机最大加速度限制

B.1.5 示教器/夹爪参数（基于 V1.5-2 版本）

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x47D	控制	ARM_GRIPPER_TEACHING_PENDANT_PARAM_CONFIG	示教器参数配置数据	夹爪/示教器参数设置
0x47E	反馈	ARM_GRIPPER_TEACHING_PENDANT_PARAM_FEEDBACK	示教器参数反馈数据	夹爪/示教器参数反馈

B.1.6 关节末端速度/加速度反馈

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x481	反馈	ARM_FEEDBACK_JOINT_VEL_ACC_1	关节 1 速度/加速度数据	反馈关节 1 当前末端速度/加速度

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x482	反馈	ARM_FEEDBACK_JOINT_VEL_ACC_2	关节 2 速度/加速度数据	反馈关节 2 当前末端速度/加速度
0x483	反馈	ARM_FEEDBACK_JOINT_VEL_ACC_3	关节 3 速度/加速度数据	反馈关节 3 当前末端速度/加速度
0x484	反馈	ARM_FEEDBACK_JOINT_VEL_ACC_4	关节 4 速度/加速度数据	反馈关节 4 当前末端速度/加速度
0x485	反馈	ARM_FEEDBACK_JOINT_VEL_ACC_5	关节 5 速度/加速度数据	反馈关节 5 当前末端速度/加速度
0x486	反馈	ARM_FEEDBACK_JOINT_VEL_ACC_6	关节 6 速度/加速度数据	反馈关节 6 当前末端速度/加速度

B.1.7 驱动器信息反馈

高速反馈（电机驱动器 → 主控，每 2ms）

CAN ID (Hex)	方向	消息名称	说明
0x251	反馈	ARM_INFO_HIGH_SPD_FEEDBACK_1	驱动器 1（关节 1）高速信息
0x252	反馈	ARM_INFO_HIGH_SPD_FEEDBACK_2	驱动器 2（关节 2）高速信息
0x253	反馈	ARM_INFO_HIGH_SPD_FEEDBACK_3	驱动器 3（关节 3）高速信息
0x254	反馈	ARM_INFO_HIGH_SPD_FEEDBACK_4	驱动器 4（关节 4）高速信息
0x255	反馈	ARM_INFO_HIGH_SPD_FEEDBACK_5	驱动器 5（关节 5）高速信息
0x256	反馈	ARM_INFO_HIGH_SPD_FEEDBACK_6	驱动器 6（关节 6）高速信息

低速反馈（电机驱动器 → 主控，每 10ms）

CAN ID (Hex)	方向	消息名称	说明
0x261	反馈	ARM_INFO_LOW_SPD_FEEDBACK_1	驱动器 1（关节 1）低速信息
0x262	反馈	ARM_INFO_LOW_SPD_FEEDBACK_2	驱动器 2（关节 2）低速信息
0x263	反馈	ARM_INFO_LOW_SPD_FEEDBACK_3	驱动器 3（关节 3）低速信息
0x264	反馈	ARM_INFO_LOW_SPD_FEEDBACK_4	驱动器 4（关节 4）低速信息

CAN ID (Hex)	方向	消息名称	说明
0x265	反馈	ARM_INFO_LOW_SPD_FEEDBACK_5	驱动器 5（关节 5）低速信息
0x266	反馈	ARM_INFO_LOW_SPD_FEEDBACK_6	驱动器 6（关节 6）低速信息

B.1.8 其他指令

CAN ID (Hex)	方向	消息名称	数据格式	说明
0x121	控制	ARM_LIGHT_CTRL	Byte 0: 灯光控制数据 (uint8)	灯光控制指令 (节点 ID 0x1)
0x422	控制	ARM_CAN_UPDATE_SILENT_MODE_CONFIG	Byte 0: 静默模式标志 (uint8)	CAN 升级总线静默模式设定
0x4AF	控制	ARM_FIRMWARE_READ	固件读取指令数据	固件版本读取指令

B.2 编码规则与单位换算

B.2.1 角度编码

- 关节角度控制/反馈：单位 0.001° （千分之一度），以 int32 有符号整数编码
- 例：90000 编码值 $\rightarrow 90000 * 0.001 = 90.000^{\circ}$
- 例：-45000 编码值 $\rightarrow -45000 * 0.001 = -45.000^{\circ}$
- 末端姿态角度 (RX/RY/RZ)：单位 0.001° ，以 int32 有符号整数编码
- 同关节角度换算方式
- 夹爪角度：单位 0.001° ，以 int32 有符号整数编码
- 角度限制参数 (0x473/0x474)：单位 0.1° ，以 int16 有符号整数编码
- 例：-1350 编码值 $\rightarrow -1350 * 0.1 = -135.0^{\circ}$
- 例：1350 编码值 $\rightarrow 1350 * 0.1 = 135.0^{\circ}$

B.2.2 位置编码

- 末端位置 (X/Y/Z)：单位 0.001 mm（千分之一毫米），以 int32 有符号整数编码

· 例：500000 编码值 → 500.000 mm = 0.5 m

B.2.3 速度/加速度编码

- 关节最大速度 (0x473/0x474)：单位 0.001 rad/s，以 int16 编码
- 最大关节加速度 (0x475)：单位 0.01 rad/s²，范围 [0, 500] 对应 [0, 5.0] rad/s²
- 末端速度：单位 0.001 mm/s，以 uint32 编码
- 末端加速度：单位 0.001 mm/s²，以 uint32 编码

B.2.4 扭矩编码

- 夹爪扭矩 (0x159)：单位 0.001 N·m，以 uint16 编码，范围 0~5000 对应 0~5.0 N·m

B.3 关节限位参考表

以下为 PiPER 机械臂各关节的典型限位值（实际值需通过 0x472 指令查询获取）。

关节	最小角度 (编码值)	最大角度 (编码值)	最小角度 (度)	最大角度 (度)	运动范围 (度)	最大速度 (rad/s)	最大加速度 (rad/s ²)
J1 (底座)	~-1700	~+1700	~-170.0°	~+170.0°	~340.0°	~3.14	~5.0
J2 (肩部)	~-1350	~+1350	~-135.0°	~+135.0°	~270.0°	~3.14	~5.0
J3 (肘部)	~-1500	~+1500	~-150.0°	~+150.0°	~300.0°	~3.14	~5.0
J4 (腕部俯仰)	~-1800	~+1800	~-180.0°	~+180.0°	~360.0°	~3.14	~5.0
J5 (腕部偏航)	~-1800	~+1800	~-180.0°	~+180.0°	~360.0°	~3.14	~5.0
J6 (腕部旋转)	~-1800	~+1800	~-180.0°	~+180.0°	~360.0°	~6.28	~10.0

注意：以上限位值为设计参考值。实际限位由电机驱动器固件存储，部署时必须以 0x472 指令查询的实际值为准。使用 0x477 Byte 1 = 0x02 可将所有关节限位、最大速度、加速度恢复为默认出厂值。

编码值 ↔ 度数换算公式

- 关节角度 (控制/反馈): $\text{度数} = \text{编码值} / 1000$
- 角度限制: $\text{度数} = \text{编码值} * 0.1$

B.4 控制模式代码参考

B.4.1 控制模式 (ctrl_mode) — Byte 0 of 0x151

代码	名称	说明
0x00	待机模式 (IDLE)	停止/空闲, 电机保持当前位置但不可控
0x01	CAN 指令控制	通过 CAN 总线控制, 标准操作模式
0x03	以太网控制	通过以太网控制
0x04	WiFi 控制	通过 WiFi 控制
0x07	离线轨迹模式	执行预录的离线轨迹

B.4.2 运动模式 (move_mode) — Byte 1 of 0x151

代码	名称	说明
0x00	MOVE P	位置模式 (Position) - 点到点运动
0x01	MOVE J	关节空间 (Joint) - 关节插补运动
0x02	MOVE L	笛卡尔直线 (Linear) - 直线路径
0x03	MOVE C	笛卡尔圆弧 (Circular) - 圆弧路径
0x04	MOVE M	MIT 模式 - 基于 V1.5-2 版本

B.4.3 MIT 模式 (mit_mode) — Byte 3 of 0x151

代码	名称	说明
0x00	位置速度模式	标准位置/速度控制模式
0xAD	MIT 模式	MIT 阻抗控制模式
0xFF	无效	无 MIT 模式

B.4.4 安装位置 (installation_pos) — Byte 5 of 0x151

代码	名称	说明
0x00	无效值	安装位置未设定
0x01	水平正装	桌面水平安装（默认接线朝后）
0x02	侧装左	左侧壁挂安装
0x03	侧装右	右侧壁挂安装

B.5 夹爪控制代码参考

B.5.1 夹爪状态码 (status_code) — Byte 6 of 0x159

代码	名称	功能
0x00	失能	夹爪电机断电（无力输出）
0x01	使能	夹爪电机上电（进入力矩/位置控制）
0x02	失能清除错误	失能并清除错误状态
0x03	使能清除错误	使能并清除错误状态

B.5.2 夹爪设零标志 (set_zero) — Byte 7 of 0x159

代码	名称	说明
0x00	无效	不执行设零操作
0xAE	设零点	将当前位置设定为零点

B.5.3 夹爪角度与力矩

- **角度范围**: 0 ~ 90000 编码值（约 0 ~ 90° 行程）
- **扭矩范围**: 0 ~ 5000 编码值（0 ~ 5.0 N·m）
- **典型用法**: 使能后 (status_code = 0x01)，设定目标角度及扭矩上限，夹爪自行完成位置闭环

B.6 电机使能控制代码 (0x471)

B.6.1 电机序号 (motor_num) — Byte 0

代码	说明
1 ~ 6	对应关节 1~6 的电机驱动器
7	夹爪电机
0xFF	全部关节电机 (含夹爪)

B.6.2 使能标志 (enable_flag) — Byte 1

代码	名称	说明
0x01	失能 (Disable)	电机断电, 自由转动
0x02	使能 (Enable)	电机上电, 进入力矩/位置控制

B.7 主从模式配置代码 (0x470)

B.7.1 联动配置 (linkage_config) — Byte 0

代码	名称	说明
0x00	无效	不改变当前配置
0xFA	设为示教输入臂	将当前机械臂配置为主从跟随中的输入 (Leader Arm)
0xFC	设为运动输出臂	将当前机械臂配置为主从跟随中的输出 (Follower Arm)

典型流程: 1. Leader 臂发送 0x470: [0xFA, 0x00, 0x00, 0x00] (设为示教输入臂) 2. Follower 臂发送 0x470: [0xFC, 0x00, 0x00, 0x00] (设为运动输出臂) 3. Leader 臂主动发送关节控制指令到 Follower 臂

B.7.2 ID 偏移配置

Byte	名称	可选值	说明
Byte 1	反馈指令偏移	0x00 (不偏移), 0x10 (→0x2Bx), 0x20 (→0x2Cx)	区分双臂的反馈数据

Byte	名称	可选值	说明
Byte 2	控制指令偏移	0x00 (不偏移), 0x10 (→0x16x), 0x20 (→0x17x)	区分双臂的控制数据
Byte 3	联动目标地址偏移	0x00 (不偏移), 0x10 (→0x16x), 0x20 (→0x17x)	设置联动控制目标的 ID 偏移

B.7.3 主从模式双机械臂 ID 分配示例

角色	反馈 ID (偏移=0x10)	控制 ID (偏移=0x10)	备注
Leader (输入臂)	0x2B1 ~ 0x2B8	0x161 ~ 0x169	Leader 的反馈和控制使用偏移后 ID
Follower (输出臂)	0x2A1 ~ 0x2A8 (默认)	0x151 ~ 0x159 (默认)	Follower 使用默认 ID

原理：设置偏移后，Leader 机械臂自动将其反馈报文和控制响应发送到偏移后的 ID 上。上位机通过监听两组不同的 ID 同时获取双臂数据，实现 1 台 PC 控制 2 台机械臂。

B.8 参数查询代码参考 (0x477)

B.8.1 参数查询 (Byte 0)

代码	名称	说明
0x00	空闲	不查询
0x01	末端 V/acc	查询末端速度/加速度参数 → 应答于 0x478
0x02	碰撞防护	查询碰撞防护等级 → 应答于 0x47B
0x03	轨迹索引	查询当前轨迹包名称索引
0x04	夹爪/示教器参数	查询夹爪或示教器参数 → 应答于 0x47E (V1.5-2)

B.8.2 参数设置 (Byte 1)

代码	名称	说明
0x00	空闲	不设置
0x01	末端 V/acc 默认值	将末端速度/加速度恢复为默认初始值

代码	名称	说明
0x02	关节限位默认值	将所有关节限位、关节最大速度、关节加速度恢复为默认出厂值

B.8.3 0x48X 报文反馈设置 (Byte 2)

代码	名称	说明
0x00	无效	不改变设置
0x01	关闭周期反馈	关闭 0x481~0x486 的周期上报
0x02	开启周期反馈	开启 0x481~0x486 1~6 号关节末端速度/加速度周期上报

B.9 常用指令速查表

操作	CAN ID	数据字节	说明
使能全部电机	0x471	[0xFF, 0x02, 0, 0, 0, 0, 0, 0]	motor_num=0xFF, enable=0x02
失能全部电机	0x471	[0xFF, 0x01, 0, 0, 0, 0, 0, 0]	motor_num=0xFF, enable=0x01
设置为 CAN 控制+关节模式	0x151	[0x01, 0x01, 50, 0x00, 0, 0, 0, 0]	ctrl_mode=0x01, move_mode=0x01, speed=50%
急停	0x150	[0x01, 0, 0, 0, 0, 0, 0, 0]	emergency_stop=0x01
恢复急停	0x150	[0x02, 0, 0, 0, 0, 0, 0, 0]	emergency_stop=0x02
查询全部电机限位	0x472	[7, 0x01, 0, 0, 0, 0, 0, 0]	motor_num=7, search_content=0x01
查询全部电机加速度	0x472	[7, 0x02, 0, 0, 0, 0, 0, 0]	motor_num=7, search_content=0x02
设为示教输入臂	0x470	[0xFA, 0, 0, 0, 0, 0, 0, 0]	linkage_config=0xFA
设为运动输出臂	0x470	[0xFC, 0, 0, 0, 0, 0, 0, 0]	linkage_config=0xFC

操作	CAN ID	数据字节	说明
恢复默认限位	0x477	[0, 0x02, 0, 0, 0, 0, 0, 0]	param_setting=0x02

B.10 多字节字段的字节序

PiPER CAN 协议中所有多字节整数字段均采用**小端序 (Little-Endian)**，即低字节在前、高字节在后。

例如，0x2A5 消息中 J1 = 90000（十进制）的编码：

```
Decoded: 90000 = 0x00015F90
On CAN bus: Byte 0=0x90, Byte 1=0x5F, Byte 2=0x01, Byte 3=0x00
```

B.11 CAN 总线硬件参数

参数	值
波特率	1 Mbps
帧格式	标准帧（11-bit ID），CAN 2.0A
上层协议	CANopen-like（自定义，非标准 CANopen）
物理接口	DB9 或 M12 航空插头（视机械臂型号而定）
终端电阻	120 Ω（总线两端各一个）
推荐线缆	双绞屏蔽线，特性阻抗 120 Ω
最大总线长度	40 m @ 1 Mbps

附录 C 故障排除指南

本附录汇总了 PiPER + LeRobot 教程完整流程中可能遇到的常见问题，按故障类别分组，每组提供症状识别、诊断命令和解决方案。

C.1 CAN 总线问题

C.1.1 CAN 端口未出现

症状：执行 `ifconfig -a` 或 `ip link show` 看不到任何 `can0` 接口。

诊断命令：

```
# 检查 USB 设备是否被识别
lsusb | grep -i "can\|gs_usb\|peak\|kvaser\|socketcan"

# 检查 dmesg 中的 USB 设备日志
dmesg | tail -30 | grep -i "usb\|can"

# 检查是否有待加载的内核模块
lsmod | grep can

# 查看所有网络接口
ip link show
```

解决方案： 1. **USB CAN 适配器未识别：**重新拔插 USB 连接线，尝试不同 USB 端口（优先使用 USB 2.0 端口，部分 USB 3.0 控制器存在兼容性问题）。 2. **驱动未加载：**对于 `gs_usb` 适配器（如 CANable），执行：`bash sudo modprobe gs_usb` 对于 MCP251x SPI 转 CAN：执行 `sudo modprobe mcp251x`。 3. **固件问题：**部分 CANable 适配器需要刷入 `CandleLight` 固件才能被 `gs_usb` 驱动识别。参考 `CandleLight_fw`。 4. **内核版本过低：**确保 Linux 内核版本 ≥ 5.4 （`uname -r` 检查），旧内核可能缺少 SocketCAN 支持。

C.1.2 CAN 端口无法启动 (bring up)

症状：执行 `sudo ip link set can0 up type can bitrate 1000000` 时报错 `Cannot find device "can0"` 或 `RTNETLINK answers: Operation not permitted`。

诊断命令：

```
# 检查 CAN 设备是否在 /sys 中注册
ls /sys/class/net/ | grep can

# 查看内核 CAN 日志
dmesg | grep -i can | tail -20

# 检查当前 CAN 状态
ip -details link show can0 2>&1
```

解决方案： 1. **设备名不对：**检查实际的接口名称（可能是 `can1`、`slcan0` 等）。执行 `ip link show | grep -E "[0-9]+:"` 查看全部网络接口。 2. **设备未初始化：**
`bash # 先设置类型和比特率再启动 sudo ip link set can0 type can bitrate 1000000 sudo ip link set can0 up`
3. **终端电阻未接：**确认 CAN 总线两端各有一个 120 Ω 终端电阻。缺少终端电阻会导致无法正常通信甚至控制器报错。 4. **权限不足：**确保使用 `sudo`，或将用户加入 `dialout` 组：`sudo usermod -aG dialout $USER`。

C.1.3 CAN 端口已启动但无数据

症状： `candump can0` 没有任何输出，或只偶尔出现零星报文。

诊断命令：

```
# 实时监测 CAN 数据
candump can0

# 查看接口统计信息（错误计数器）
ip -details -statistics link show can0

# 发送测试帧检查 CAN 回路是否正常
cansend can0 123#DEADBEEF

# 检查总线负载
canbusload can0@1000000
```

解决方案： 1. **波特率不匹配：**PiPER 机械臂 CAN 总线默认为 **1 Mbps (1000000)**。确认与机械臂固件波特率一致。 2. **机械臂未上电：**PiPER 机械臂需外部供电（通常 24V DC），仅 USB 连接不足以驱动电机控制器。 3. **终端电阻问题：**缺少终端电阻会导致信号反射，测量总线 CAN_H 和 CAN_L 间静态电阻应约为 60 Ω （两个 120 Ω 并联）。 4. **CAN 控制器 bus-off：**如果 `ip link show can0` 显示 `ERROR-PASSIVE` 或 `BUS-OFF`（错误计数器 `txerr/rxerr` $\geq$ 96/128），需要重启：
`bash sudo ip link set can0 down sudo ip link set can0 up type can bitrate 1000000` 5. **电机未使能：**机械臂上电后不会主动发送数据，需要先通过 0x471 指令使能电机，才会收到 0x2A5~0x2A8 的反馈数据。

C.2 机械臂连接问题

C.2.1 SDK 无法连接机械臂

症状：运行 `detect_arm.py` 或任何 SDK 代码时报错：`ConnectionError`、`TimeoutError`、`Cannot connect to arm` 或程序卡死无响应。

诊断命令：

```
# 确认 CAN 端口工作正常
candump can0 -n 5

# 检查 Python 环境和 SDK 安装
python3 -c "import piper_sdk; print(piper_sdk.__version__)"

# 运行 SDK 自带的探测脚本
python3 -m piper_sdk.demo.detect_arm

# 检查是否有进程占用 CAN 接口
sudo lsof /dev/ttyUSB* 2>/dev/null
```

解决方案： 1. **CAN 端口未启动：**按 C.1 节检查 CAN 连接。 2. **机械臂供电不足：**PiPER 机械臂建议使用 24V / 10A 以上的直流电源，电压不足时控制器无法正常启动。 3. **SDK 版本不匹配：**`bash pip show piper_sdk` # 确认版本号与机械臂固件版本兼容 4. **CAN 接口被其他程序占用：**确保同一时间只有一个进程使用 CAN 接口（SocketCAN 支持多进程同时读写，但某些 SDK 实现可能要求独占）。 5. **连接超时：**检查 SDK 初始化代码中的 `can_interface` 参数是否正确：`python piper = Piper(interface="can0", bitrate=1000000)`

C.2.2 扭矩无法使能

症状：调用 `piper.EnableArm(7)`（或等效的使能函数）后，电机无响应，关节仍处于自由状态。

诊断命令：

```
# 直接发送使能 CAN 指令测试
cansend can0 471#FF02000000000000

# 查看反馈数据确认使能状态
candump can0,2A1:7FF # 监听状态反馈

# 检查是否有错误/警告码
python3 -c "
import piper_sdk
p = piper_sdk.Piper()
print(p.GetArmStatus())
"
```

解决方案： 1. **使能顺序：** PiPER 要求先设置控制模式 (0x151)，再使能电机 (0x471)：

```
python # 正确顺序 piper.MotionCtrl_2(ctrl_mode=0x01, move_mode=0x01, move_spd_rate_ctrl=50)
piper.EnableArm(7) # 使能全部电机 2. 急停状态未清除： 如果之前触发过急停，需要先解除： python
piper.MotionCtrl_1(emergency_stop=0x02) # 恢复急停 3. 软限位触发： 如果关节当前位置超出软限位，部
分电机拒绝使能。可以先关闭软限位测试： python piper.init_soft_joint_limit_on() # 开启软限位保护
piper.init_soft_joint_limit_off() # 关闭软限位保护（调试用） 4. 硬件错误锁存： 通过查询清除错误：
python # 使用 0x475 清除关节错误 piper.JointConfig(joint_motor_num=7, clear_joint_err=0xAE)
```

C.2.3 机械臂不响应控制指令

症状： 使能成功、发送关节控制指令后，机械臂不运动。

诊断命令：

```
# 检查实际发送的控制指令
candump can0 | grep "155\|156\|157"

# 检查反馈角度是否在变化
python3 -c "
import piper_sdk
p = piper_sdk.Piper()
states = p.GetArmJointMsgs()
print(states) # 查看反馈角度
"
```

解决方案： 1. **运动速度设置过低：** 0x151 Byte 2 速度百分比过低会导致运动极慢。设为 50~100。
2. **控制模式未正确设置：** 确认 `ctrl_mode = 0x01` (CAN 指令控制模式)。 3. **目标角度超出限位：** 发送的目标角度超出软限位范围时，机械臂拒绝执行。可以查询当前限位： `python`
`piper.SearchAllMotorAngleLimitMaxSpd()` `limits = piper.GetAllMotorAngleLimitMaxSpd()` # 比较目标角度是否在 `min_angle_limit` 和 `max_angle_limit` 之间 4. **指令编码错误：** 角度以 0.001° 为单位编码为 int32。0.151 弧度 $\approx 9^\circ$ ，编码值约为 9000。如果编码值小于 10（即目标角度小于 0.01° ），可能无可见运动。
5. **电机处于失能状态：** 调用 `EnableArm(7, 0x02)` 重新使能。

C.3 摄像头问题

C.3.1 摄像头未检测到

症状： LeRobot 报告 No camera found、Cannot open video device。

诊断命令：

```
# 列出所有视频设备
ls /dev/video*

# 检查 USB 摄像头
lsusb | grep -i "camera\|webcam\|imaging"

# 使用 OpenCV 测试打开摄像头
python3 -c "
import cv2
cap = cv2.VideoCapture(0)
print('Opened:', cap.isOpened())
cap.release()
"

# 使用 v4l2 查看设备
v4l2-ctl --list-devices

# 使用 lerobot 内置工具
lerobot-find-cameras
```

解决方案： 1. **设备索引不对：** `/dev/video0` 不一定是实际的摄像头，可能被内置摄像头或虚拟设备占用。用 `v4l2-ctl --list-devices` 确认正确索引。 2. **权限不足：** 将用户加入 `video` 组：`bash sudo usermod -aG video $USER` # 重新登录或执行 `newgrp video` 3. **USB 供电不足：** 摄像头通过无源 USB Hub 连接可能供电不足。直接连接到主机 USB 端口。 4. **UVC 驱动问题：** `bash sudo modprobe uvcvideo dmesg | grep uvcvideo` 5. **LeRobot 配置：** 确认录制命令中的 `--robot.cameras` 参数正确：`bash lerobot-record --robot.type piper_follower --robot.cameras '0' '1'`

C.3.2 摄像头超时

症状： 录制时频繁出现 `Camera timeout` 警告，或帧获取延迟很大。

诊断命令：

```
# 测试摄像头 FPS
python3 -c "
import cv2, time
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FPS, 30)
t0 = time.time()
for i in range(100):
    ret, frame = cap.read()
print(f'Actual FPS: {100 / (time.time() - t0):.1f}')
cap.release()
"
```

解决方案： 1. **USB 带宽饱和：** 多个摄像头共用同一 USB 控制器的带宽。将摄像头分散到不同的 USB 端口（物理上位于不同 USB Host Controller 上）。查看 USB 树状拓扑：`bash lsusb -t` 2. **降**

低分辨率：在 LeRobot 配置中降低摄像头分辨率（如 640x480）：`python # 在 robot config 中设置 CameraConfig(width=640, height=480, fps=30)`

3. 关闭自动曝光/自动白平衡：`python cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 1) # 手动曝光 cap.set(cv2.CAP_PROP_EXPOSURE, -6) # 设定曝光值`

C.3.3 帧率过低 (Low FPS)

症状：录制显示实际 FPS 远低于目标 FPS（如目标 30 FPS，实际仅 5-10 FPS）。

诊断命令：

```
# 使用 v4l2 查看摄像头支持的格式和帧率
v4l2-ctl -d /dev/video0 --list-formats-ext

# LeRobot 录制时观察 FPS 日志
lerobot-record --robot.type piper_follower --robot.cameras '0' 2>&1 | grep -i fps
```

解决方案：

- 1. MJPEG vs YUYV：**MJPEG 格式通常可获得更高 FPS。在配置中指定：`python CameraConfig(fps=30, color_mode='rgb', pixel_format='MJPEG')`
- 2. 系统负载过高：**录制过程中避免运行其他计算密集型任务。
- 3. 硬件瓶颈：**USB 2.0 摄像头理论最大带宽 480 Mbps，单个 1080p@30FPS YUYV 流约占 140 MB/s=1120 Mbps，超出 USB 2.0 带宽。此时必须降低分辨率或使用 MJPEG 压缩格式。

C.4 录制问题

C.4.1 帧率下降

症状：录制开始时 FPS 正常，几分钟后出现明显下降。

诊断命令：

```
# 监控磁盘 I/O
iostat -x 1

# 检查磁盘剩余空间
df -h

# 监控 CPU 使用率
htop

# 检查录制文件大小增长速率
watch -n 1 'du -sh ~/.cache/lerobot/'
```


解决方案： 1. **磁盘 I/O 瓶颈：** 录制数据写入速度不足。将数据目录迁移到 SSD（固态硬盘）：`bash export LEROBOT_HOME=/path/to/ssd/lerobot_data` 2. **内存缓冲区不足：** 录制时 Python 进程内存使用过高触发 GC，导致主循环阻塞。增加系统可用内存，或减少同时录制的摄像头数量。 3. **编码/压缩开销：** 如果录像编码格式设为原始 YUYV，磁盘写入量大，考虑使用 MJPEG 或降低分辨率。 4. **CPU 过热降频：** 检查 CPU 温度 (`sensors`)，如果过热触发了降频保护，改善散热条件。

C.4.2 数据帧丢失/帧不完整

症状： 录制完成后检查 HDF5 文件，时间戳不均匀或关节角度数据出现跳变/缺失。

诊断命令：

```
# 检查 HDF5 数据集完整性
python3 -c "
import h5py, numpy as np
f = h5py.File('path/to/episode_0.h5', 'r')
for key in f['data'].keys():
    arr = f['data'][key][:]
    print(f'{key}: shape={arr.shape}, NaN={np.isnan(arr).sum()}, Zero={np.sum(arr==0)}')
f.close()
"

# 检查时间戳间隔
python3 -c "
import h5py, numpy as np
f = h5py.File('path/to/episode_0.h5', 'r')
ts = f['observation/timestamp'][:]
diffs = np.diff(ts.squeeze())
print(f'Mean dt={diffs.mean():.3f}s, Std={diffs.std():.3f}s, Max gap={diffs.max():.3f}s')
f.close()
"
```

解决方案： 1. **CAN 数据丢包：** 如果 CAN 总线上同时有大量报文，可能丢包。检查 `ip -details -statistics link show can0` 中的 `dropped` 计数。如持续增长，说明内核 SocketCAN 缓冲区不足：`bash sudo ip link set can0 txqueuelen 1000 #` 或增大内核接收缓冲区（需要重新加载 `can` 模块） 2. **ROS/中间件竞争：** 如果同时运行 ROS 节点与 LeRobot，确保它们使用不同的 CAN 接口或同一接口的 SocketCAN 多进程模式。 3. **线程同步问题：** LeRobot 数据采集使用多线程（图像采集线程 + 主控制线程）。如果关节读取在图像线程之后但时间戳取自不同的时钟，会导致时间戳不一致。可考虑使用统一的 `monotonic clock`。

C.4.3 磁盘空间不足

症状： 录制中途报错 `No space left on device` 或 `OSError: [Errno 28]`。

诊断命令：

```
# 查看磁盘使用情况
df -h

# 查看 LeRobot 数据目录大小
du -sh ~/.cache/lerobot/

# 预估一次录制的磁盘占用
# 2 cameras × 640×480×3 bytes × 30 FPS × 60s ≈ 3.3 GB (原始) / 0.5 GB (MJPEG)
```

解决方案： 1. **清理旧数据：** `bash rm -rf ~/.cache/lerobot/old_episodes/` 2. **更改数据存储路径到更大磁盘：** `bash export LEROBOT_HOME=/mnt/large_disk/lerobot_data mkdir -p $LEROBOT_HOME` 3. **减小录制参数：** - 降低摄像头分辨率 - 降低目标帧率 - 减少录制时长 4. **使用视频压缩格式：** MJPEG 比原始 YUYV 省磁盘空间约 5~10 倍。

C.5 主从跟随（Relay）问题

C.5.1 从机械臂不跟随运动

症状： Leader 臂可自由拖动，但 Follower 臂保持静止。

诊断命令：

```
# 检查 Leader 臂是否在发送数据
candump can0 # 检查 0x2A1~0x2A8 是否有持续反馈

# 检查 Follower 臂是否收到控制指令
candump can1 | grep "155\|156\|157"

# 确认 Follower 臂使能状态
python3 -c "
import piper_sdk
p = piper_sdk.Piper(interface='can1')
p.SearchAllMotorMaxAngleSpdAccLimit()
status = p.GetArmStatus()
print(status)
"
```

解决方案： 1. **Follower 臂未使能：** 确保 Follower 臂的 EnableArm 已调用且成功。 2. **CAN 接口混淆：** Leader 和 Follower 必须连接到不同的 CAN 接口（如 can0 和 can1）。确认物理连接和 SDK 初始化参数一致。 3. **主从模式未设置：** 在 PC Relay 模式下需要正确配置主从关系。确保两支机械臂分别发送了正确的主从配置指令： - Leader: 0x470 [0xFA]（示教输入臂） - Follower: 0x470 [0xFC]（运动输出臂） 4. **ID 偏移配置错误：** 如果两支臂共用一个 CAN 总线（不推荐），必须设置 ID 偏移以区分数据。参见附录 B.7.3。 5. **关节映射错误：** 确认 Leader 的 J1~J6 关节角度被

正确映射到 Follower 的对应关节。某些安装方式下需要做角度取反。6. **软件逻辑**：检查 PC Relay 脚本的循环是否在正常运行（打印调试信息/日志）。

C.5.2 Follower 臂运动抖动严重

症状：Follower 臂在跟随 Leader 时出现明显的抖动/震颤/高频振动。

诊断命令：

```
# 观察反馈角度的时间变化
candump can1 | grep "2A5\|2A6\|2A7" | head -50

# 检查控制帧的发送频率
timeout 5 candump can1 | grep "155\|156\|157" | wc -l
```

解决方案：

1. **控制频率不稳定**：控制循环的频率不均导致抖动。使用固定频率的控制循环（如 `time.perf_counter() + sleep` 或 RT 线程）：

```
python import time period = 1.0 / 100 # 100 Hz while True:
t_start = time.perf_counter() # 发送控制指令 elapsed = time.perf_counter() - t_start time.sleep(max(0,
period - elapsed))
```
2. **角度噪声放大**：Leader 臂反馈的角度本身有少量噪声（ $\sim 0.1^\circ$ ），直接映射会放大为抖动。在控制回路中加入低通滤波：

```
python alpha = 0.2 # 滤波系数 (0~1) filtered_angle = alpha * raw_angle + (1 - alpha) * filtered_angle
```

3. **PID 参数过激**：如果使用额外 PID 控制，P 增益过大会导致超调和振荡。降低 Kp 值。
4. **加速度限制过低**：机械臂默认的加速度限制可能过高，导致速度突变。通过 0x475 适度降低最大加速度。

C.5.3 Follower 臂运动方向与预期相反

症状：Leader 臂向上抬时 Follower 向下压，或者旋转方向相反。

诊断命令：

```
# 对比 Leader 和 Follower 的同关节角度
python3 -c "
import piper_sdk
leader = piper_sdk.Piper(interface='can0')
follower = piper_sdk.Piper(interface='can1')
l = leader.GetArmJointMsgs()
f = follower.GetArmJointMsgs()
for i in range(6):
    print(f'J{i+1}: Leader={getattr(l.joint_state, f"joint_{i+1}")*0.001:.1f}°, '
          f'Follower={getattr(f.joint_state, f"joint_{i+1}")*0.001:.1f}°')
"
```

解决方案：

1. **安装姿态差异**：两支臂的物理安装方向不同（如一支水平正装，一支倒装）。检查 `installation_pos` (0x151 Byte 5) 设置，并在软件中对需要翻转的关节做取反处理。
2. **关节编号不一**

致：确认两支臂的 J1~J6 关节物理定义一致。必要时在代码中建立关节映射表。3. **解决方案代码示**

例： `python # 关节方向取反示例 joint_offsets = [+1, +1, -1, +1, +1, +1] # 1 表示同向, -1 表示反向 for i, direction in enumerate(joint_offsets): follower_cmd[i] = leader_pos[i] * direction`

C.6 训练问题

C.6.1 显存不足 (OOM)

症状：训练启动后报错 `RuntimeError: CUDA out of memory`。

诊断命令：

```
# 查看 GPU 显存使用
nvidia-smi

# 训练启动前确认显存余量
python3 -c "import torch; print(f'Total: {torch.cuda.get_device_properties(0).total_mem/1e9:.1f} GB, '
          f'Free: {torch.cuda.memory_reserved(0)/1e9:.1f} GB')"
```

解决方案： 1. **减小 batch size：**在训练配置中将 `batch_size` 从默认值减半或更小。 2. **减少图像分辨率：**训练所用图像分辨率直接决定显存占用量。将分辨率从 `640x480` 降至 `320x240` 或 `224x224`。 3. **减少上下文长度：**如果使用 ACT 或类似序列模型，减小 `chunk_size` 或 `horizon` 参数。 4. **使用混合精度训练：**启用 `torch.amp` (自动混合精度)： `python # 在配置中添加 use_amp = true` 5. **梯度累积：**增大 `gradient_accumulation_steps` 以在较小 batch size 下模拟较大 batch size。 6. **减小模型容量：**减少 transformer 层数、注意力头数或隐藏维度。

C.6.2 损失不下降

症状：训练了多个 epoch，损失曲线基本持平或只有微小下降。

诊断命令：

```
# 查看训练曲线
tensorboard --logdir outputs/train/

# 检查数据分布
python3 -c "
import h5py, numpy as np
f = h5py.File('episode_0.h5', 'r')
actions = f['action'][:]
print(f'Action mean: {actions.mean(axis=0)}')
print(f'Action std: {actions.std(axis=0)}')
print(f'Action range: {actions.min(axis=0)} to {actions.max(axis=0)}')
```

```
f.close()
"
```

解决方案： 1. **数据量不足：** 模仿学习需要大量示教数据。单任务至少 50 条 episode，复杂任务可能需要 100 条以上。 2. **数据质量差：** 示教动作不一致、抖动、停顿过长。重新录制更流畅的示教数据。 3. **学习率设置不当：** 尝试减小或增大学习率一个数量级（默认 $1e-4$ ，尝试 $5e-5$ 或 $5e-4$ ）。 4. **归一化问题：** 确认动作和观测的归一化统计量正确（mean/std）。LeRobot 会自动计算并使用数据集统计量。 5. **任务难度与模型容量不匹配：** 简单的策略用大模型会过拟合，复杂任务用小模型则欠拟合。调整模型大小。 6. **检查数据加载：** 确认摄像头图像和关节角度的时间对齐正确，没有偏移。

C.6.3 检查点无法加载

症状： 训练中断后恢复时报错 `KeyError: 'model_state_dict'` 或 `Checkpoint corrupted`。

诊断命令：

```
# 列出 checkpoint 文件
ls -lh outputs/train/*/checkpoints/

# 检查 checkpoint 文件完整性
python3 -c "
import torch
ckpt = torch.load('outputs/train/.../checkpoints/ckpt_1000.pth', map_location='cpu')
print('Keys:', list(ckpt.keys()))
print('Epoch:', ckpt.get('epoch', 'N/A'))
print('Loss:', ckpt.get('loss', 'N/A'))
"
```

解决方案： 1. **版本不兼容：** checkpoint 可能由不同版本的 LeRobot 或 PyTorch 生成。尝试安装与训练时相同的依赖版本。 2. **模型结构变化：** 如果修改了模型代码，旧的 checkpoint 结构不再匹配。从头训练，或用兼容的模型结构加载后迁移。 3. **文件损坏：** 如果训练被强制终止（kill -9），checkpoint 文件可能写入不完整。使用上一个完整的 checkpoint 恢复： `bash # 检查文件大小异常的 checkpoint ls -lhS outputs/train/*/checkpoints/` 4. **恢复训练的命令：** `bash lerobot-train \ --config.output_dir outputs/train/my_experiment \ --resume 1`

C.7 部署问题

C.7.1 策略部署后机械臂不运动

症状： 训练完成的策略成功加载，但机械臂不动。

诊断命令:

```
# 确认策略输出值范围
python3 -c "
import torch
model = torch.load('policy.pth')
# 打印模型输出归一化参数
print('Action mean:', model.get('action_mean', 'N/A'))
print('Action std:', model.get('action_std', 'N/A'))
"

# 观察实际控制指令流
candump can0 | grep "155\\|156\\|157" | head -20
```

解决方案: 1. **动作反归一化:** 策略输出的动作值是归一化后的 (通常均值 0, 标准差 1), 需要反向映射到实际的关节角度编码值。确认反归一化步骤正确: `python action_raw = model_output # 范围 ~ [-1, 1] action_denorm = action_raw * action_std + action_mean # 反归一化 joint_cmd = (action_denorm * 1000).astype(int) # 转为 0.001° 编码` 2. **动作空间维度:** 确认策略输出的维度和机械臂可接受的关节数量一致 (6 个关节 + 1 个夹爪 = 7 维)。 3. **机械臂未使能:** 检查 Follower 臂是否已处于使能状态和控制模式下。 4. **控制模式:** 确认控制模式为 CAN 指令控制 (`ctrl_mode = 0x01`)。 5. **安全限位阻止:** 策略输出的目标角度可能超出机械臂当前设置的软限位。

C.7.2 运动抖动/不平滑

症状: 策略推理产生的运动有明显的抖动、卡顿或步进感。

诊断命令:

```
# 检查推理频率
python3 -c "
import time
times = []
for _ in range(100):
    t0 = time.perf_counter()
    # 模拟一次推理
    time.sleep(0.01)
    times.append(time.perf_counter() - t0)
print(f'Mean inference time: {np.mean(times)*1000:.1f}ms, Max: {np.max(times)*1000:.1f}ms')
"
```

解决方案: 1. **推理频率不足:** 如果模型一次推理耗时超过控制周期, 会导致帧间间隙过大。优化模型推理: - 使用 ONNX Runtime 或 TensorRT 加速推理 - 减小模型输入分辨率 - 使用更轻量级的模型架构 2. **动作平滑 (Action Chunking):** 如果使用 ACT 模型, 它本身会输出一个动作块 (chunk), 可以减少帧间跳变。确认 `chunk_size` 设置合理 (通常 10~100)。 3. **时间平滑 (Temporal Smoothing):** 在后处理中加入指数移动平均平滑输出: `python current_action = 0.8`

* raw_output + 0.2 * previous_action 4. **观测跳变**: 摄像头图像不稳定或有噪点导致策略输出抖动。确保摄像头固定、光照条件稳定。

C.7.3 部署时程序崩溃/异常退出

症状: 部署脚本运行一段时间后崩溃, 或按 Ctrl+C 后机械臂不释放/锁死。

诊断命令:

```
# 查看崩溃前的 Python 异常
python3 deploy.py 2>&1 | tee deploy.log

# 检查核心转储
coredumpctl list

# 检查系统日志
journalctl -xe | tail -30
```

解决方案:

- 异常处理不完整**: 部署脚本缺少 `try-finally` 保证安全退出的逻辑。必须包装:
`python try: # 部署循环 while running: action = policy.predict(obs) piper.set_joints(action) except KeyboardInterrupt: print("Interrupted, cleaning up...") finally: piper.disable_all() # 失能所有电机 piper.close() # 关闭 CAN 连接`
- CAN 通信超时**: 长时间运行时 CAN 通信偶尔超时。添加重试逻辑:
`python import time def send_with_retry(piper, cmd, max_retries=3): for attempt in range(max_retries): try: piper.send(cmd) return True except TimeoutError: time.sleep(0.01) return False`
- 内存泄漏**: 长时间运行的推理可能导致 GPU 内存缓慢增长。在循环中定期清空 CUDA 缓存: `python if step % 1000 == 0: torch.cuda.empty_cache()`
- 紧急停止按钮失效**: 部署环境必须确保物理急停按钮可用。测试急停响应是否正常。
- 程序退出后机械臂未失能**: 如果异常处理未能执行, 可以手动发送急停指令或切断机械臂电源。也可以在启动脚本中加入 trap: `bash #!/bin/bash trap "cansend can0 150#0100000000000000" EXIT # 退出时发送急停 python3 deploy.py`

C.8 环境与依赖问题速查

症状	可能原因	解决方向
<code>ImportError: No module named 'piper_sdk'</code>	SDK 未安装或不在 PYTHONPATH	<code>pip install -e /path/to/piper_sdk</code>
<code>ModuleNotFoundError: No module named 'lerobot'</code>	LeRobot 未安装	<code>pip install -e "[piper]"</code>
<code>ImportError: libpython3.x.so</code>	Python 动态库缺失	<code>sudo apt install libpython3.x-dev</code>

症状	可能原因	解决方向
can0: unknown interface	CAN 驱动未加载	<code>sudo modprobe can; sudo modprobe can_raw; sudo modprobe gs_usb</code>
RTNETLINK answers: Operation not supported	CAN 配置参数错误	检查 bitrate 参数拼写: bitrate 不是 bit_rate
socket.error: [Errno 1] Operation not permitted	权限不足	sudo 或加入 dialout 组
AttributeError: 'NoneType' object has no attribute 'x'	SDK 未正确初始化	检查 Piper() 构造函数参数, 确认 can_interface 正确
HDF5 文件读取慢	磁盘 I/O 瓶颈	迁移到 SSD, 使用数据子集训练
机器人动作幅度太小	动作归一化范围问题	检查 action_mean / action_std 是否正确计算
环境噪音导致误触发	工作室噪音/振动	改善环境光照, 固定机械臂底座, 减少背景变化

C.9 获取更多帮助

1. CAN 总线调试工具推荐:

2. `can-utils`: Linux 标准 SocketCAN 工具集 (candump/cansend/cangen)

3. `python-can`: Python CAN 接口库, 可用于编写自定义调试脚本

4. `busmaster`: 开源 CAN 总线分析工具 (带 GUI)

5. **日志收集**: 在提交 Issue 前, 请收集以下信息: `bash # 环境信息` `uname -a # 内核版本` `python3 --version # Python 版本` `pip list | grep -E "piper|lerobot|torch|can" # 依赖版本` `nvidia-smi # GPU 信息 (如适用)` `lsusb # USB 设备列表` `ip link show # 网络接口 (含 CAN)`

6. 联系渠道:

7. PiPER SDK Issue: 查看 SDK 项目仓库的 Issues 页面

8. LeRobot Discord: HuggingFace Discord 中的 `#lerobot` 频道

9. PiPER 技术支持: 联系设备供应商获取固件更新和技术文档